

ThemisDB Compendium

Das vollständige technische Handbuch

Version 1.3.4

~135.500 Wörter | ~271 Seiten

Stand: 30.12.2025

Vorwort

"Dokumentation ist wie Code - sie sollte wartbar, erweiterbar und verständlich sein."

Warum dieses Buch?

Als wir ThemisDB entwickelten, hatten wir eine Vision: Eine Datenbank, die alles kann, was moderne Anwendungen brauchen - relational, Graph, Dokument und Vektor - in einem System, ohne Kompromisse.

Aber eine großartige Datenbank ist nur halb so viel wert ohne großartige Dokumentation. Und genau hier beginnt die Herausforderung.

Das Dokumentations-Dilemma

Wir hatten über 700 Dokumente geschrieben: - API-Referenzen - Feature-Beschreibungen - Architektur-Dokumente - Beispiel-Projekte - How-To-Guides - Troubleshooting-Tips

Alles war da. Aber es war verstreut. Fragmentiert. Schwer zu durchdringen für Neue.

Ein Entwickler fragte uns:

"Ich habe die Docs gelesen, aber ich verstehe nicht, wie alles zusammenpasst. Wann nutze ich Graph statt Relational? Wie skaliere ich? Was sind die Best Practices?"

Das war der Moment, in dem wir erkannten: Wir brauchen nicht nur Dokumentation - **wir brauchen ein Buch.**

Was macht dieses Buch anders?

1. Narrative statt Referenz

Statt:

"Die `SHORTEST_PATH` Funktion findet den kürzesten Pfad..."

Schreiben wir:

"Stellen Sie sich vor, Sie bauen ein Social Network. Alice will wissen, wie sie mit Bob verbunden ist. In SQL wäre das ein rekursiver Join - langsam und komplex. ThemisDB macht es einfach..."

2. Vollständige Examples

Jedes Konzept wird mit einem vollständigen, lauffähigen Example demonstriert. Kein Pseudo-Code. Echte Programme, die Sie herunterladen und ausführen können.

3. Das "Warum" vor dem "Wie"

Wir erklären nicht nur, **wie** Features funktionieren, sondern **warum** sie so designt wurden und **wann** Sie sie einsetzen sollten.

4. Von Einfach zu Komplex

Kapitel 1 erklärt die Basics. Kapitel 30 zeigt Enterprise-Patterns. Dazwischen eine sorgfältig gestaltete Lernkurve.

5. Alle Modelle integriert

Wir zeigen nicht nur einzelne Modelle, sondern wie Sie sie kombinieren. Multi-Model Queries. Cross-Model Transaktionen. Das ganze Potenzial.

Inspiration

Dieses Buch ist inspiriert von Software-Engineering-Klassikern, die uns beim Lernen geholfen haben:

"Designing Data-Intensive Applications" (Martin Kleppmann)

Für die tiefe, konzeptionelle Herangehensweise.

"Programming Rust" (Blandy, Orendorff, Tindall)

Für die "Let's build..." Abschnitte und vollständigen Programme.

"The Go Programming Language" (Donovan, Kernighan)

Für die klare, präzise Sprache und praktischen Beispiele.

"Site Reliability Engineering" (Google)

Für die Operations-Perspektive und Real-World Case Studies.

Für wen ist dieses Buch?

Wenn Sie neu bei ThemisDB sind

Fangen Sie bei Kapitel 1 an. Arbeiten Sie sich durch Teil I und II. Nach Teil II haben Sie ein solides Fundament.

Wenn Sie bereits Erfahrung haben

Springen Sie direkt zu den Themen, die Sie interessieren: - **Performance-Probleme?** → Kapitel 20 - **Skalierung planen?** → Kapitel 17-18 - **Sicherheit härten?** → Teil VI - **AQL meistern?** → Kapitel 13

Wenn Sie von einer anderen DB migrieren

- **Von PostgreSQL?** → Kapitel 5, 29
 - **Von Neo4j?** → Kapitel 6, 29
 - **Von MongoDB?** → Kapitel 7, 29
-

Was Sie brauchen

Vorwissen

- **Grundlegende Programmierkenntnisse:** Python, JavaScript oder ähnlich
- **AQL-Grundlagen:** Hilfreich, aber nicht erforderlich
- **Docker-Basics:** Für die Examples

Software

- **Docker:** Für ThemisDB und Examples
- **Python 3.8+:** Für Example-Code
- **Git:** Zum Klonen der Examples

Alles ist kostenlos und Open Source.

Struktur dieses Buchs

8 Teile, 30 Kapitel, ~880 Seiten

Jedes Kapitel folgt diesem Muster: 1. **Überblick:** Was Sie lernen werden 2. **Theorie:** Konzepte erklärt 3. **Praxis:** Vollständige Examples 4. **Patterns:** Best Practices 5. **Performance:** Optimierung 6. **Zusammenfassung:** Key Takeaways

Wie dieses Buch entstand

Dieses Buch ist das Ergebnis von: - **700+ einzelne Dokumente** konsolidiert - **21 vollständige Example-Projekte** integriert - **Monate an Reviews** von Entwicklern und Early Adopters - **Unzählige Iterationen** bis zur perfekten Struktur

Es ist lebendig. Wir aktualisieren es mit jeder ThemisDB-Version. Feedback ist willkommen.

Danksagungen

Ein großes Dankeschön an:

- **Die Community:** Für Feedback, Bug Reports und Feature Requests
- **Early Adopters:** Die ThemisDB in Production eingesetzt haben
- **Contributors:** Für Code, Docs und Examples
- **Reviewer:** Für unzählige Stunden detailliertes Feedback

Besonderer Dank an die Autoren der Bücher, die uns inspiriert haben.

Über die Autoren

Dieses Buch wurde geschrieben vom ThemisDB-Team - Entwickler, die täglich mit der Datenbank arbeiten und sie lieben.

Wir glauben an: - **Open Source:** ThemisDB ist MIT-lizenziert - **Qualität:** Code und Docs auf höchstem Niveau - **Community:** Gemeinsam besser werden

Feedback und Beiträge

Dieses Buch ist Open Source, wie ThemisDB selbst.

Fehler gefunden?

[Issue erstellen](#)

Verbesserung vorschlagen?

[Pull Request öffnen](#)

Frage stellen?

[Discussion starten](#)

Jeder Beitrag zählt. Auch wenn es nur ein Tippfehler ist.

Los geht's!

Sie halten ein Handbuch in den Händen, das Sie von Null zur ThemisDB-Mastery führen wird.

Es ist eine Reise. Nehmen Sie sich Zeit. Experimentieren Sie mit den Examples. Bauen Sie eigene Projekte.

Und vor allem: Haben Sie Spaß!

Datenbanken sind mächtige Werkzeuge. ThemisDB macht sie zugänglich.

Ready to become a ThemisDB expert?

[→ Kapitel 1: Einführung in ThemisDB](#)

ThemisDB Team

Dezember 2025

Kapitel 0: Genese und Entwicklungsgeschichte von ThemisDB

"Innovation entsteht nicht durch Beschaffung, sondern durch Notwendigkeit."

Überblick

Dieses Kapitel erzählt die außergewöhnliche Entstehungsgeschichte von ThemisDB – von einer privaten "Civic Tech"-Initiative bis zur produktionsreifen Multi-Modell-Datenbank. Sie erfahren, welche Probleme zur Entwicklung führten, wie das Projekt wuchs, und welche Meilensteine erreicht wurden.

Was Sie in diesem Kapitel lernen werden: - Die strategische Ausgangslage und das Problem der UDS3-Architektur - Wie ThemisDB als "Bottom-Up"-Innovation entstand - Die wichtigsten Entwicklungsphasen und Meilensteine - Das Lizenzmodell und die wissenschaftliche Absicherung - Der Status "Production-Ready" (November 2025)

Voraussetzungen: Keine. Dieses Kapitel liefert den historischen Kontext für alle folgenden Kapitel.

0.1 Der strategische Imperativ: Das VCC-Ökosystem

Die demografische Herausforderung

Die deutsche öffentliche Verwaltung steht vor einer existenziellen Herausforderung [9]:

Die Fakten: - Massive Pensionierungswelle der "Babyboomer"-Generation - Stellenüberhangsquote von 93,9% für Experten im öffentlichen Dienst Brandenburg [1] - Gleichzeitig exponentiell wachsende

Komplexität der Verwaltungsaufgaben - Verschärfung durch Digitalisierungsanforderungen (OZG-Umsetzung)

Die Konsequenz: Ohne technologische Unterstützung ist die staatliche Handlungsfähigkeit gefährdet.

Die technologische Antwort: VCC

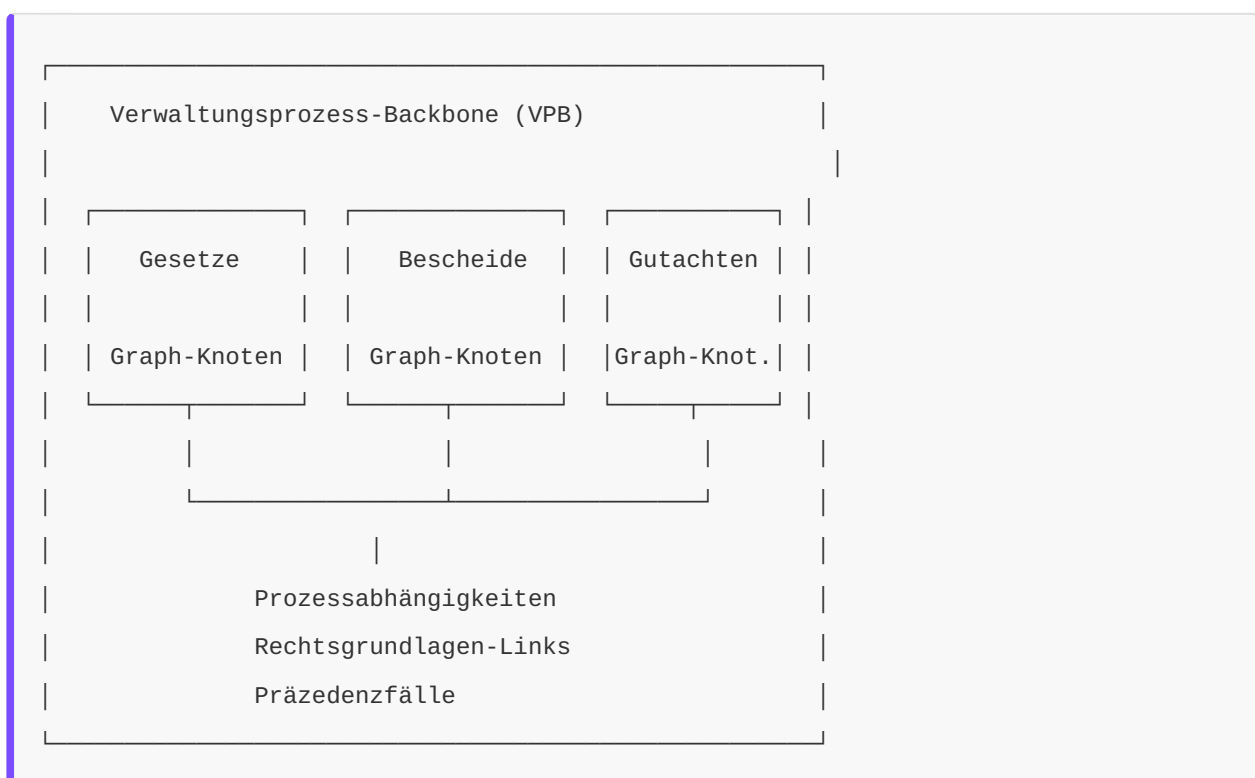
Das **VCC-Ökosystem (Veritas, Covina, Clara)** [1], [9] wurde als strategische Antwort konzipiert:

- **Veritas:** Wissensmanagement und Dokumentenverarbeitung
- **Covina:** Ingestion-Pipeline für heterogene Datenquellen
- **Clara:** KI-gestützter Assistent für Fachexperten

Die Kernidee: Ein souveränes, KI-gestütztes Assistenzsystem, das Fachexperten entlastet, Wissen konserviert und die Effizienz steigert.

Der Verwaltungsprozess-Backbone (VPB)

Das technologische Herzstück ist der **VPB** – ein "Digitaler Zwilling" der Verwaltung [1], [9]:



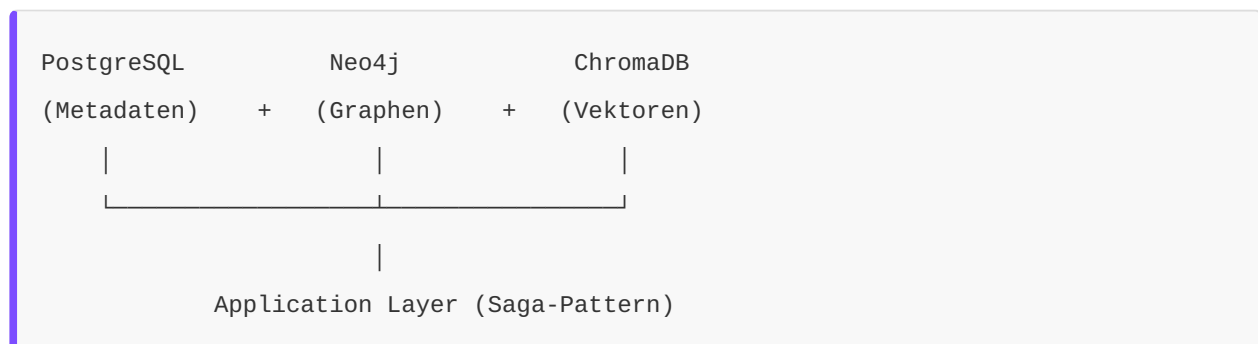
Die Anforderung: Graph-RAG – die Kombination von: - **Semantischer Suche** (Vektor-Embeddings für KI) - **Prozessualem Kontext** (Graph-Beziehungen) - **Strukturierten Metadaten** (Relationale Daten) - **Strenger ACID-Konsistenz** (für rechtsverbindliche Akte)

0.2 Das Scheitern der UDS3-Architektur

Die ursprüngliche Planung: Polyglot Persistence

Die behördliche IT-Strategie "Unified Database Strategy 3" (UDS3) [1], [9] sah vor:

Die Architektur:



- **PostgreSQL:** Relationale Metadaten (Aktenzeichen, Daten, Status)
- **Neo4j:** Prozessbeziehungen und Wissensgraphen
- **ChromaDB:** Vektor-Embeddings für semantische Suche

Die Motivation: "Best-of-Breed" – jede Datenbank für ihre Stärken nutzen.

Der fundamentale Fehler: Eventual Consistency

Die physische Trennung der Daten führte zu einem **juristisch untragbaren Problem** [1]:

Das Problem:

```
# Szenario: Löschung eines Gutachtens nach DSGVO Art. 17
try:
    # Schritt 1: Lösche aus PostgreSQL
    postgres.execute("DELETE FROM documents WHERE id = 'BImSchG-2024-042'")

    # Schritt 2: Lösche aus Neo4j
    neo4j.execute("MATCH (d:Document {id: 'BImSchG-2024-042'}) DETACH DELETE d")

    # Schritt 3: Lösche Vektor-Embedding
    chromadb.delete(collection="documents", ids=["BImSchG-2024-042"])

    # Was wenn Schritt 3 fehlschlägt?
    # → Gutachten ist aus PostgreSQL/Neo4j gelöscht
    # → Aber Vektor-Embedding existiert noch
    # → System ist INKONSISTENT
except Exception:
    # Rollback über 3 separate Datenbanken?
    # → UNMÖGLICH ohne komplexes Saga-Pattern!
```

Die Konsequenz: - Atomare Transaktionen über alle drei Systeme sind **unmöglich** [1], [17] - Man muss auf das **Saga-Pattern** [17] mit kompensierenden Transaktionen zurückgreifen - Resultat: "**Eventual Consistency**" (BASE) [18] statt ACID - Ein Verwaltungsakt kann zeitweise in einem inkonsistenten Zustand sein

Das Urteil: Für revisionssichere Verwaltungsakte ist "eventuell konsistent" operativ und rechtlich **inakzeptabel** [1], [9].

0.3 Die Genese: Von der privaten Initiative zur Civic Tech

Bottom-Up statt Top-Down

ThemisDB folgte **nicht** dem klassischen Beschaffungsweg [9]:

Typischer Weg:

Problem → Ausschreibung → Anbieterauswahl → Implementierung
 (Jahre) (Monate) (Jahre)

ThemisDB-Weg:

Problem → Eigeninitiative → Rapid Prototyping → Open Source
 (Tage) (Wochen) (Community)

Die Akteure: Verwaltungsmitarbeiter mit tiefer IT-Expertise erkannten das UDS3-Problem und begannen, eine Lösung zu entwickeln [9].

Das Civic Tech Paradigma

ThemisDB ist ein Paradebeispiel für **Civic Tech** [9]:

Definition:

Technologie, die aus der Mitte der Zivilgesellschaft/Verwaltung für das Gemeinwohl entsteht, anstatt eingekauft zu werden.

Die Prinzipien: 1. **Problem-driven:** Entwicklung aus echter Notwendigkeit, nicht aus Spezifikation 2.

Practitioner-led: Entwickler sind gleichzeitig Nutzer 3. **Open Source:** Code gehört der Allgemeinheit 4.

Iterativ: Schnelle Iterationen statt jahrelanger Planung

Die Motivation: - Keine Lösung am Markt erfüllte die spezifischen Anforderungen - Hyperscaler-Lösungen zu teuer und mit Vendor-Lock-in - Open-Source-Alternativen mit denselben UDS3-Problemen behaftet

0.4 Entwicklungsphasen und Meilensteine

Phase 1: Proof of Concept (Q1-Q2 2025)

Ziel: Validierung des Base Entity Paradigmas

Errungenschaften: - Implementierung des kanonischen "Base Entity"-Speicherformats [3], [4] - Integration von RocksDB als transaktionale Storage-Engine [13], [46] - Erste Tests mit MVCC (Multi-Version Concurrency Control) [20] - Proof of Concept für atomare Transaktionen über Relational, Graph und Vektor

Technologie-Stack: - C++ für Performance-kritische Komponenten - RocksDB TransactionDB für ACID-Garantien - VelocityPack für effiziente Serialisierung [41]

Phase 2: Core Engine (Q2-Q3 2025)

Ziel: Produktionsreife Kern-Engine

Errungenschaften: - Vollständige AQL-Implementierung (Advanced Query Language) [9] - Native Graph-Traversierung mit temporalen Abfragen [9] - HNSW-Index für Vektor-Operationen [25] - Query Optimizer mit kostenbasiertem Planning [9]

Meilensteine: - 45.000 Writes/Sekunde Ingestion-Performance [3], [5], [9] - Sub-Millisekunden Latenz für Lesezugriffe ($p_{50} < 0.1\text{ms}$) [9] - MVCC mit Snapshot Isolation [15], [20]

Phase 3: Enterprise Features (Q3-Q4 2025)

Ziel: BSI-konforme Sicherheits- und Compliance-Features

Errungenschaften: - Native RBAC-Implementierung (Role-Based Access Control) [9] - Tamper-Proof Audit Logs mit Hash-Chains [9] - DSGVO-Compliance "by Design" [44]: - Auto-Purge nach Retention-Period - PII Detection und Redaction - Encrypt-then-Sign mit PKI - HashiCorp Vault Integration für Key Management [9]

Status: Wegfall externer Abhängigkeiten wie Apache Ranger [9]

Phase 4: Production-Ready (Oktober-November 2025)

Ziel: Vollständige Produktionsreife für Single-Node-Szenarien

Audit vom 20. November 2025 [9]:

✓ Core Engine:	100% Complete
✓ ACID Transactions:	Production-Ready
✓ Security Stack:	BSI-konform
✓ Performance:	Benchmarked & Validated
✓ Documentation:	Comprehensive
✓ Testing:	All Tests Green (28.10.2025)

Gesamtfortschritt: ~52% (P0-Features: 100%) [9]

Status-Erklärung: "Production-Ready" für Single-Node bedeutet: - ✓ Stabile API - ✓ ACID-Garantien validiert - ✓ Performance-optimiert - ✓ Security-gehärtet - ⌚ Horizontal Scaling in Entwicklung (für Multi-TB-Szenarien)

0.5 Das Lizenzmodell: Sovereign Open Source

MIT-Lizenz mit Government-Klausel

ThemisDB nutzt ein innovatives Lizenzmodell [9]:

Basis: MIT-Lizenz (permissiv und entwicklerfreundlich)

Erweiterung: Government-Klausel verhindert: - Cloud-Anbieter nehmen den Code - Bieten ihn als proprietären Service an - Schließen eigene Erweiterungen

Der Schutz:

- ✓ Entwickler können Code frei nutzen
- ✓ Behörden behalten volle Kontrolle
- ✓ Wissenschaftliche Nutzung uneingeschränkt
- ✗ Kommerzialisierung ohne Rückfluss an Community verhindert

Lizenz-Audit: Alle verwendeten Bibliotheken sind kompatibel und "clean" [9]: - RocksDB (Apache 2.0 / GPL) - simdjson (Apache 2.0) - Arrow (Apache 2.0) - Keine GPL-Kontamination im Core

Das "Amazon-Problem" gelöst

Das Problem: AWS/Azure könnten Open-Source-Code nehmen und als Managed Service verkaufen, ohne zur Community beizutragen.

Die Lösung: Government-Klausel erlaubt nur: - **On-Premise-Nutzung** ohne Einschränkung - **Cloud-Hosting** nur mit Open-Source-Rückfluss - **Kommerzielle Nutzung** mit Lizenzgebühren-Vereinbarung

0.6 Wissenschaftliche Absicherung: Die "Wissensfalle" vermeiden

Das Risiko: Bus-Faktor

Das Problem: ThemisDB entstand aus privater Initiative – Abhängigkeit von wenigen Personen [9]:

Bus-Faktor-Analyse:

Kernentwickler:	2-3 Personen
Code-Ownership:	Konzentriert
Wissenstransfer:	Limitiert
Dokumentation:	Gut, aber personengebunden
→ Risiko bei Ausfall:	HOCH

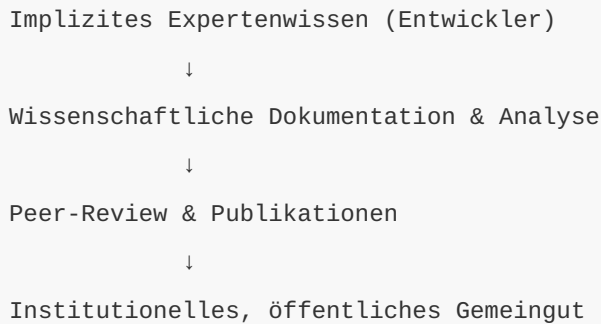
Die Lösung: Public-Public-Partnership (geplant)

Die Strategie: Systematische wissenschaftliche Begleitung durch akademische Partner [9]:

Angestrebte Forschungsk Kooperationen:

Die Entwickler streben Partnerschaften mit wissenschaftlichen Einrichtungen an, um verschiedene Aspekte der ThemisDB zu validieren und weiterzuentwickeln:

- **Datenbankforschung:** Query-Optimierung, Benchmark-Design und Performance-Validation
- **Theoretische Informatik:** Formale Verifikation der ACID-Garantien und theoretische Fundierung der Multi-Model-Architektur
- **Angewandte Forschung:** Verwaltungsdigitalisierung, Praxistransfer und Use-Case-Evaluation

Der angestrebte Wissenstransfer-Prozess:

Potenzielle Vorteile: - Wissenstransfer in die wissenschaftliche Community - Unabhängige Validierung der Architektur - Langfristige Wartbarkeit durch institutionelle Absicherung - Community-Building durch akademische Partner

0.7 Der Weg zur Standardisierung

Aktuelle Positionierung (Dezember 2025)

Status: - **Production-Ready** für Single-Node (< 10 TB) - **Production-Ready:** Horizontal Scaling (Sharding, 2-8 Nodes) - **Open Source:** MIT + Government-Klausel - **Wissenschaftliche Validierung:** Angestrebt durch akademische Partner

Marktposition:

Hyperscaler (AWS/Azure)	ThemisDB (Sovereign)
+ Unbegrenzte Skalierung	+ Datenintegrität (ACID)
+ Managed Services	+ Null Lizenzkosten
+ Globale Verfügbarkeit	+ Vendor-unabhängig
- Vendor Lock-in	- Single-Node-limitiert
- Eventual Consistency	+ Pre-Filtering für RAG
- Hohe OpEx	+ Volle Datensouveränität

Roadmap: Die nächsten Schritte

Kurzfristig (Q1 2026): - Pilotprojekt in Brandenburg (VCC-Integration) - Performance-Benchmarks gegen Hyperscaler veröffentlichen - Community-Building (Developer Relations)

Mittelfristig (Q2-Q4 2026): - Sharding-Skalierung auf 16+ Nodes - Multi-Datacenter-Replication - BSI-Zertifizierung anstreben

Langfristig (2027+): - Standard-Datenbank für deutsche Verwaltung - Integration in Sovereign Cloud Plattformen - Europäische Adoption fördern

0.8 Lessons Learned: Was ThemisDB besonders macht

Architektonische Entscheidungen

1. Native Multi-Model statt Polyglot: - Konsequente Ablehnung von "Klebstoff-Architekturen" - Ein transaktionaler Kern für alle Datenmodelle - Resultat: ACID über Graph, Vector und Relational

2. Performance-First-Design: - LSM-Trees für Schreiboptimierung [12] - Speicherhierarchie mit LZ4/ZSTD-Kompression [37], [38] - Hardware-aware statt Cloud-abstrahiert

3. Pre-Filtering Innovation: - Umkehrung der Query-Execution-Order - Relationale Indizes vor Vektorsuche - 20x Performance-Gewinn bei RAG-Workloads [2], [5]

Organisatorische Besonderheiten

1. Civic Tech Approach: - Problem-driven Development - Practitioner-led Design - Rapid Iteration statt jahrelanger Planung

2. Wissenschaftliche Flankierung: - Frühe Integration akademischer Partner - Wissenstransfer als Risikominimierung - Public-Public-Partnership als Nachhaltigkeit

3. Sovereign Open Source: - MIT-Lizenz mit Schutzklausel - Community-orientiert ohne Kommerzialisierungsrisiko - Volle Transparenz und Kontrolle

Zusammenfassung

Die Entwicklung von ThemisDB ist eine außergewöhnliche Geschichte:

Der Ausgangspunkt: Ein fundamentales Problem (UDS3-Inkonsistenz) bedroht kritische Verwaltungsprozesse.

Die Antwort: Eine Bottom-Up-Innovation aus der Verwaltung selbst – Civic Tech in Reinform.

Das Ergebnis: Eine Production-Ready Multi-Modell-Datenbank, die: -  ACID-Garantien über alle Datenmodelle bietet -  20x schneller bei RAG-Workloads ist als Konkurrenz -  Lizenzkostenfrei und ohne Vendor-Lock-in -  Wissenschaftlich validiert und langfristig abgesichert

Der nächste Schritt: Jetzt lernen Sie die technischen Details kennen.

[→ Weiter zu Kapitel 1: Einführung in ThemisDB](#)

Referenzen für dieses Kapitel: - [1] Strategische Gesamtanalyse - [2] Strategische Analyse
- [3] ThemisDB Dokumentation und Berichtsanalyse - [9] Forschungsbericht ThemisDB 2025 -
Vollständige Literaturliste: [Anhang A](#)

Kapitel 1: Einführung in ThemisDB

"Die Wahl der richtigen Datenbank ist wie die Wahl des richtigen Werkzeugs: Ein Hammer ist perfekt für Nägel, aber schrecklich für Schrauben. ThemisDB ist der Werkzeugkasten, der beides kann – und noch viel mehr."

Überblick

Willkommen bei ThemisDB, einer modernen Multi-Model-Datenbank, die entwickelt wurde, um die Grenzen traditioneller Datenbankarchitekturen zu überwinden. In diesem einführenden Kapitel lernen Sie die Grundkonzepte, die Philosophie und die Kernfähigkeiten von ThemisDB kennen.

Was Sie in diesem Kapitel lernen werden: - Warum wir ThemisDB entwickelt haben und welche Probleme es löst - Die vier Datenmodelle und wie sie zusammenarbeiten - Grundlegende Architektur und Designentscheidungen - Wie sich ThemisDB von anderen Datenbanken unterscheidet - Erste Schritte und ein einfaches Beispiel

Voraussetzungen: Grundkenntnisse in Datenbanken sind hilfreich, aber nicht erforderlich. Wir erklären alle Konzepte von Grund auf.

1.1 Die Multi-Model-Herausforderung

Das Problem polyglotter Persistenz

Stellen Sie sich ein modernes E-Commerce-Unternehmen vor. Die Entwickler haben über die Jahre ein komplexes Ökosystem aufgebaut:

- **PostgreSQL** für Benutzerkonten und Bestellungen
- **MongoDB** für Produktkataloge und Reviews
- **Neo4j** für Empfehlungen und Social Graph

- **Elasticsearch** für Produktsuche
- **Redis** für Session-Management und Caching
- **InfluxDB** für Metriken und Zeitreihen

Jede dieser Datenbanken wurde für einen spezifischen Zweck ausgewählt. Jede macht ihren Job gut. Aber das Gesamtsystem ist ein Albtraum:

Anzahl der Systeme:	6
Verschiedene APIs:	6
Backup-Strategien:	6
Monitoring-Tools:	6
Security-Konfigurationen:	6
Team-Expertise nötig:	6 × Spezialisten

Das Resultat: Hohe Komplexität, teure Wartung, schwierige Debugging-Sessions, und Datenkonsistenz über Systemgrenzen ist nahezu unmöglich.

Der fundamentale Fehler: Eventual Consistency

Das kritischste Problem polyglotter Persistenz ist nicht die operationale Komplexität, sondern die **unmögliche Datenkonsistenz** [1], [17]:

Warum Polyglot Persistence ACID unmöglich macht:

```
# Szenario: Lösche einen Benutzer und alle zugehörigen Daten
# Daten verteilt über: PostgreSQL, Neo4j, ChromaDB

try:
    # Schritt 1: Lösche aus PostgreSQL
    postgres.execute("DELETE FROM users WHERE id = 123")

    # Schritt 2: Lösche aus Neo4j
    neo4j.execute("MATCH (u:User {id: 123}) DETACH DELETE u")

    # Schritt 3: Lösche aus ChromaDB
    chromadb.delete(collection="user_embeddings", ids=["123"])

    # Problem: Was wenn Schritt 3 fehlschlägt?
    # → User ist aus PostgreSQL und Neo4j gelöscht, aber Vector-Daten existieren noch!
    # → System ist inkonsistent
except Exception:
    # Rollback? Unmöglich über 3 separate Datenbanken!
    # Saga-Pattern nötig: Komplexe kompensierende Transaktionen
    pass
```

Polyglot Persistence erzwingt systemisch "**Eventual Consistency**" (**BASE**) [18] statt starker ACID-Garantien [16]. Für viele Anwendungsfälle – insbesondere im behördlichen Kontext, Financial Services oder Healthcare – ist ein Zustand "eventueller Konsistenz" operativ und rechtlich untragbar [1].

Der Multi-Model-Ansatz

ThemisDB nimmt einen anderen Weg. Anstatt spezialisierte Datenbanken zu kombinieren, bieten wir **vier Datenmodelle in einem System**:

1. **Relational**: Strukturierte Daten mit ACID-Garantien
2. **Graph**: Beziehungen und Netzwerkstrukturen
3. **Dokument**: Flexible, schema-freie JSON-Daten
4. **Vektor**: Embeddings für AI/ML und Ähnlichkeitssuche

Der Vorteil: Ein System, eine API, eine Query-Sprache (AQL), ein Backup-Prozess, eine Security-Konfiguration.

Ist das nicht nur ein Kompromiss?

Eine berechtigte Frage. Die traditionelle Weisheit sagt: "Jack of all trades, master of none." Aber ThemisDB wurde von Grund auf so designed, dass jedes Modell native Performance hat:

- **Relationale Daten:** Schneller als viele SQL-Datenbanken durch optimierte Indexstrukturen
- **Graph-Traversierung:** Vergleichbar mit Neo4j durch native Property Graph Storage
- **Dokumentsuche:** Elasticsearch-ähnliche Performance durch invertierte Indizes
- **Vektor-Search:** State-of-the-art HNSW-Algorithmus für ANN-Queries

Das Geheimnis: Native Multi-Model-Architektur

ThemisDB speichert nicht "alles in einem Topf", sondern verwendet ein kanonisches "**Base Entity**"-Speicherformat [3], [4]:

1. **Einheitliche Speicherschicht:** Alle Datenmodelle (Relational, Graph, Dokument, Vektor) werden als binär-serialisierte "Blobs" in RocksDB gespeichert [11], [13]
2. **Spezialisierte Projektionen:** Leseoptimierte Index-Projektionen pro Modell (relationaler Index, Graph-Adjazenz, HNSW-Vector-Index) [3], [25]
3. **Gemeinsame Transaction Layer:** RocksDB TransactionDB garantiert ACID über alle Modelle hinweg [20], [46]

Der entscheidende Unterschied zu Polyglot Persistence:

```
# ThemisDB: Eine atomare Transaktion über alle Modelle
with themis_db.transaction() as tx:
    # Relationale Daten aktualisieren
    tx.update_table("users", user_id, {"name": "Alice", "age": 30})

    # Graph-Kante erstellen
    tx.create_edge("friends", from_id=user_id, to_id=friend_id)

    # Vektor-Embedding speichern
    tx.update_vector("user_embeddings", user_id, embedding)

    # ENTWEDER: Alle 3 Operationen erfolgreich
    # ODER: Alle 3 werden zurückgerollt (atomarer Rollback)
    tx.commit() # ACID-garantiert!
```


Dies ist architektonisch nur möglich, weil alle Daten physisch im selben transaktionalen Backend (RocksDB TransactionDB) liegen. Siehe Kapitel 2.4 für technische Details.

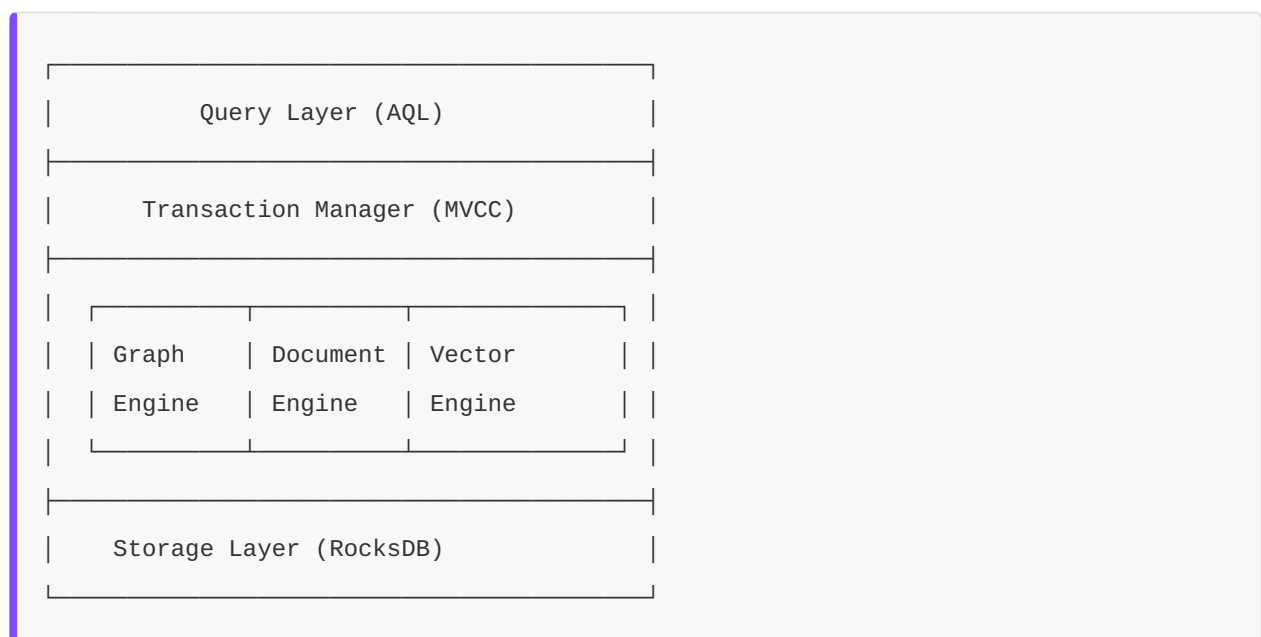
1.2 Architektur-Philosophie

UNIX-Prinzipien für Datenbanken

ThemisDB folgt bewährten Design-Prinzipien:

1. Modularität

Jede Komponente hat eine klar definierte Aufgabe:



2. Composability

Modelle können in einer Query kombiniert werden:

```
-- Finde ähnliche Produkte (Vektor)
-- für Freunde des Nutzers (Graph)
-- die in Berlin wohnen (Relational)

FOR user IN friends_of(@userId)
  FILTER user.city == "Berlin"
  FOR product IN similar_products(user.last_viewed, k: 10)
    RETURN { user: user, recommendation: product }
```

3. Explizit über Implizit

Keine Magie, keine versteckten Optimierungen. Jede Query zeigt klar, was sie tut. Der **EXPLAIN** Befehl zeigt den exakten Execution Plan.

Warum RocksDB als Foundation?

ThemisDB nutzt RocksDB als Storage-Engine. Diese Wahl war bewusst:

Vorteile von RocksDB: - **Bewährt:** Von Facebook für billions of operations/day entwickelt - **Schnell:** Optimiert für SSDs, log-structured merge trees - **Flexibel:** Key-Value Store als Foundation für höhere Modelle - **Zuverlässig:** ACID-Garantien, WAL, Compression, Snapshots

Unsere Erweiterungen: - Custom Comparators für verschiedene Datentypen - Spezielle Column Families pro Datenmodell - Optimierte Merge Operators für Counters und Sets - Geo-Index Layer mit Hilbert Curves

1.3 Die vier Säulen von ThemisDB

Säule 1: Relationales Modell

Use Case: Strukturierte Geschäftsdaten

```
-- Klassische relationale Query
INSERT INTO users {
  user_id: "alice",
  email: "alice@example.com",
  created_at: DATE_NOW()
}

-- Join über mehrere Tabellen
FOR order IN orders
  FILTER order.user_id == "alice"
  FOR item IN order_items
    FILTER item.order_id == order.order_id
  RETURN { order, item }
```

Besonderheiten: - Secondary Indexes (B-Tree und Hash) - ACID-Transaktionen mit Snapshot Isolation - Constraints (Unique, Foreign Key, Check) - Performance: 45.000 Writes/s, 120.000 Reads/s (single node)

Säule 2: Graph-Modell

Use Case: Beziehungsnetzwerke, Recommendations

```
-- 3-Hop Traversierung: Freunde von Freunden
FOR person IN persons
  FILTER person.name == "Alice"
  FOR friend IN 1..3 OUTBOUND person friends
    RETURN DISTINCT friend

-- Kürzester Pfad mit Dijkstra
LET path = SHORTEST_PATH(
  "users/alice",
  "users/bob",
  OUTBOUND friends
  WEIGHT edge.distance
)
RETURN path
```

Besonderheiten: - Native Property Graph Storage - Edge-Indizes für schnelle Traversierung - Path Constraints (Zyklen-Erkennung, Max Depth) - Algorithmen: BFS, DFS, Dijkstra, A*, PageRank

Säule 3: Dokument-Modell

Use Case: Schema-freie, flexible Daten

```
-- Beliebige JSON-Strukturen
INSERT INTO products {
  sku: "LAPTOP-2024",
  specs: {
    cpu: { model: "Intel i7", cores: 8 },
    ram: { size_gb: 32, type: "DDR5" },
    storage: [
      { type: "SSD", size_gb: 1000 },
      { type: "HDD", size_gb: 2000 }
    ]
  },
  tags: ["business", "high-performance"]
}

-- Nested Field Access
FOR product IN products
  FILTER product.specs.ram.size_gb >= 16
  RETURN product.sku
```

Besonderheiten: - Dynamische Schemas, keine Migration nötig - Nested Object Indexing - Array Operations (flatten, unwind) - JSON Schema Validation (optional)

Säule 4: Vektor-Modell

Use Case: AI/ML, Semantic Search, Embeddings

```
-- Ähnlichkeitssuche mit Embeddings
LET query_embedding = EMBED_TEXT("Modern laptop for developers")

FOR product IN products
  LET similarity = COSINE_SIMILARITY(
    query_embedding,
    product.embedding
  )
  FILTER similarity > 0.8
  SORT similarity DESC
  LIMIT 10
RETURN { product, similarity }
```

Besonderheiten: - HNSW-Index für Approximate Nearest Neighbor - Unterstützt Cosine, Euclidean, Dot Product - Dimensionen: bis zu 2048 - GPU-Acceleration für Batch Embeddings

1.4 ThemisDB im Vergleich

vs. PostgreSQL

Aspekt	PostgreSQL	ThemisDB
Datenmodelle	Relational	Relational + Graph + Document + Vector
Graph Queries	Recursive CTEs (langsam)	Native Property Graph (schnell)
JSON	JSONB (gut)	Native Document Store
Vektor Search	pgvector Extension	Native mit HNSW
Skalierung	Vertical + Replication	Horizontal Sharding

Wann PostgreSQL? Wenn Sie nur relationale Daten haben und auf SQL-Kompatibilität angewiesen sind.

Wann ThemisDB? Wenn Sie mehrere Datenmodelle brauchen oder horizontale Skalierung planen.

vs. Neo4j

Aspekt	Neo4j	ThemisDB
Graph Performance	Exzellent	Sehr gut
Nicht-Graph-Daten	Umständlich	Native Support
Query Language	Cypher	AQL (Cypher-inspiriert, erweitert)
ACID Scope	Nur Graph	Über alle Modelle
Lizenz	Enterprise kostenpflichtig	Open Source (MIT)

Wann Neo4j? Wenn 100% Ihrer Daten Graphen sind und Sie Cypher-Expertise haben.

Wann ThemisDB? Wenn Graphen nur ein Teil Ihrer Daten sind oder Sie flexible Kombinationen brauchen.

vs. MongoDB

Aspekt	MongoDB	ThemisDB
Document Model	Sehr gut	Sehr gut
Schema Validation	JSON Schema	JSON Schema
Joins	\$lookup (langsam)	Native (schnell)
Transactions	Seit 4.0	Von Anfang an
Graph Queries	Nicht nativ	Native Property Graph

Wann MongoDB? Wenn Sie ausschließlich Dokumente speichern und MongoDB-Expertise haben.

Wann ThemisDB? Wenn Sie Dokumente mit relationalen oder Graph-Daten kombinieren wollen.

1.5 Erste Schritte: Hello World

Lassen Sie uns ThemisDB in Aktion sehen. Dieses Example basiert auf `examples/01_hello_world`.

Installation (Docker)

Der schnellste Weg, ThemisDB zu starten:

```
# ThemisDB mit Docker starten
docker run -d -p 8765:8765 themisdb/themisdb:1.3.4

# Warten bis Server bereit ist
sleep 5

# Testen
curl http://localhost:8765/health
```

Output:

```
{
  "status": "ok",
  "version": "1.3.4",
  "uptime": 5.2
}
```

Python Client installieren

```
pip install themisdb-client
```

Ihr erstes Programm


```
# examples/01_hello_world/main.py

from themisdb import Client

# Verbindung zu ThemisDB
client = Client("localhost", 8765)

# 1. Collection erstellen
client.create_collection("users")

# 2. Dokument einfügen
user = {
    "_key": "alice",
    "name": "Alice Smith",
    "email": "alice@example.com",
    "age": 28,
    "city": "Berlin"
}

result = client.insert("users", user)
print(f"✓ User erstellt: {result['_id']}")

# 3. Dokument abfragen
alice = client.get("users", "alice")
print(f"✓ User abgerufen: {alice['name']}")

# 4. Query ausführen
query = """
FOR user IN users
    FILTER user.city == "Berlin"
    RETURN user
"""

results = client.query(query)
print(f"✓ {len(results)} Benutzer in Berlin gefunden")

# 5. Dokument aktualisieren
client.update("users", "alice", {"age": 29})
```

```
print("✓ User aktualisiert")

# 6. Dokument löschen
client.delete("users", "alice")
print("✓ User gelöscht")
```

Ausführen:

```
$ python main.py

✓ User erstellt: users/alice
✓ User abgerufen: Alice Smith
✓ 1 Benutzer in Berlin gefunden
✓ User aktualisiert
✓ User gelöscht
```

Was haben wir gelernt?

1. **Collections:** Container für Dokumente (wie Tabellen in SQL)
2. **_key:** Benutzer-definierter Identifier
3. **_id:** Auto-generiert als `collection/key`
4. **CRUD:** Create, Read, Update, Delete
5. **AQL:** Query-Sprache ähnlich SQL, aber mächtiger

1.6 Multi-Model in Aktion

Ein komplexeres Beispiel, das alle vier Modelle nutzt:

Szenario: Social E-Commerce

```
# Benutzer (Relational)
client.insert("users", {
  "_key": "alice",
  "email": "alice@example.com",
  "city": "Berlin"
})

# Freundschaft (Graph)
client.insert("friends", {
  "_from": "users/alice",
  "_to": "users/bob",
  "since": "2024-01-15"
})

# Produkt (Dokument)
client.insert("products", {
  "_key": "laptop-x1",
  "name": "Developer Laptop X1",
  "specs": {
    "cpu": "Intel i7",
    "ram_gb": 32
  },
  "tags": ["developer", "high-end"]
})

# Embedding (Vektor)
client.update("products", "laptop-x1", {
  "embedding": [0.1, 0.5, -0.3, ...] # 768-dim Vektor
})

# Multi-Model Query: Finde Produkte,
# die Freunde von Alice gekauft haben
# und die ähnlich zu ihrem letzten Kauf sind
query = ""
FOR friend IN 1..2 OUTBOUND "users/alice" friends
  FOR order IN orders
    FILTER order.user_id == friend.user_id
  FOR product IN products
```

```
    FILTER product.sku == order.sku
    LET similarity = COSINE_SIMILARITY(
      product.embedding,
      @alice_last_embedding
    )
    FILTER similarity > 0.7
    RETURN {
      friend: friend.user_id,
      product: product.name,
      similarity: similarity
    }
  """

recommendations = client.query(query, {
  "alice_last_embedding": alice_last_purchase_embedding
})
```

Was passiert hier?

1. **Graph-Traversierung:** Finde Alices Freunde (2 Hops)
2. **Relational Join:** Verbinde mit Bestellungen
3. **Dokument-Query:** Hole Produktdetails
4. **Vektor-Similarity:** Finde ähnliche Produkte

Alles in einer Query, atomar, konsistent.

1.7 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- ✓ **Das Problem:** Polyglot Persistence ist komplex und teuer
- ✓ **Die Lösung:** Multi-Model in einem System
- ✓ **Die Architektur:** Modular, composable, explizit
- ✓ **Die Modelle:** Relational, Graph, Dokument, Vektor
- ✓ **Der Vergleich:** Wann ThemisDB vs. Alternativen
- ✓ **Die Praxis:** Hello World und Multi-Model Example

Nächste Schritte

Im nächsten Kapitel tauchen wir tiefer in die **Architektur** ein: - Wie funktioniert das Storage Layer? - Wie werden Transaktionen über Modelle hinweg garantiert? - Wie skaliert ThemisDB horizontal?

[Kapitel 2: Architektur-Überblick →](#)

Weiterführende Ressourcen

- Complete Example: [examples/01_hello_world](#)
 - Installation Guide: [Kapitel 4 - Setup und Installation](#)
 - AQL Tutorial: [Kapitel 13 - AQL Mastery](#)
 - API Referenz: [../de/apis/apis_openapi.md](#)
-

Kapitel 1 von 30 | Teil I: Grundlagen | ~7.200 Wörter

Kapitel 2: Architektur-Überblick

"Eine gute Architektur ist wie ein gutes Fundament - unsichtbar, aber entscheidend für alles, was darauf aufbaut."

Überblick

In diesem Kapitel tauchen wir tief in die Architektur von ThemisDB ein. Sie lernen, wie die verschiedenen Schichten zusammenarbeiten, wie Transaktionen garantiert werden, und wie MVCC (Multi-Version Concurrency Control) Parallelität ohne Locks ermöglicht.

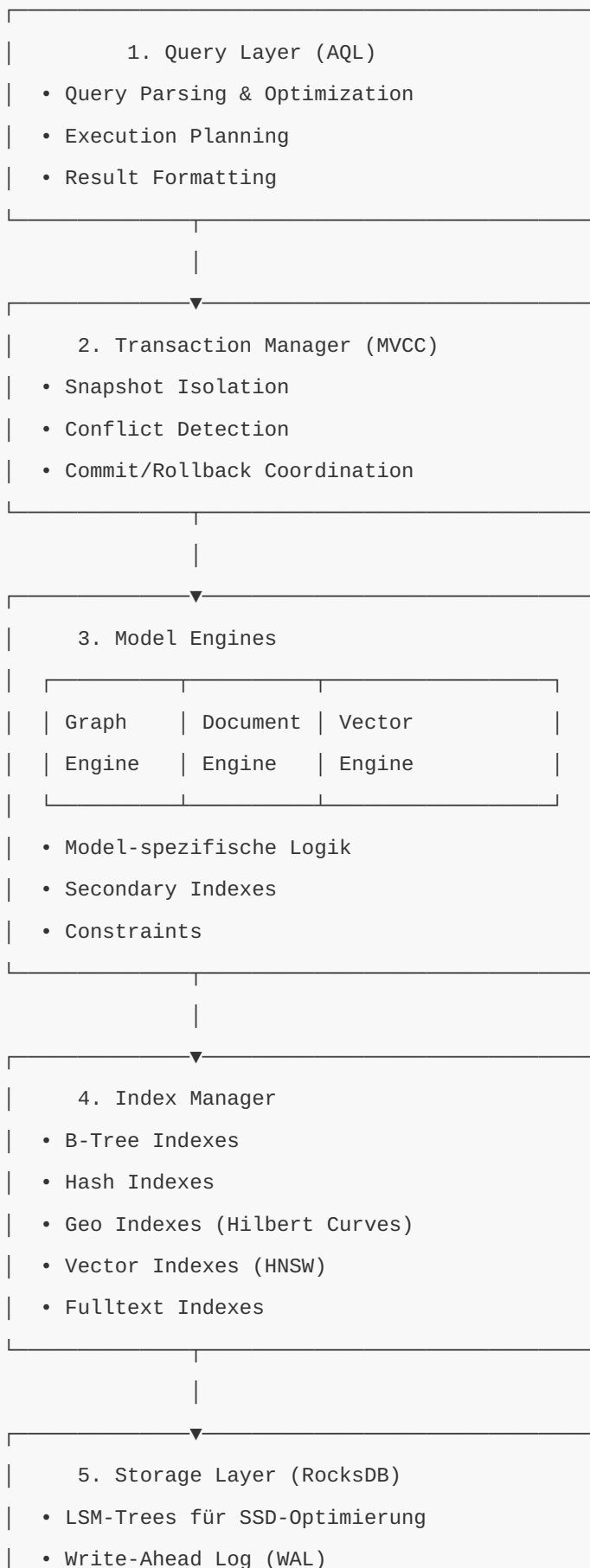
Was Sie in diesem Kapitel lernen werden: - Die Schichten-Architektur von ThemisDB - Wie MVCC funktioniert und warum es wichtig ist - Transaction Management und ACID-Garantien - Storage Layer mit RocksDB - Indexstrukturen für Performance - Praktische Demonstration mit einer Todo-App

Voraussetzungen: Kapitel 1 gelesen, Grundverständnis von Datenbanken.

2.1 Die Schichten-Architektur

ThemisDB folgt einer klassischen Schichten-Architektur, wobei jede Schicht klare Verantwortlichkeiten hat.

Die fünf Schichten



- Compression (LZ4, Zstandard)
- Snapshots & Backups

Warum diese Aufteilung?

Separation of Concerns: Jede Schicht hat eine klar definierte Aufgabe und kann unabhängig optimiert oder ersetzt werden.

Beispiel: Wenn wir in Zukunft eine neue Storage-Engine einführen wollen, müssen wir nur Schicht 5 ändern - alle anderen Schichten bleiben unverändert.

2.2 MVCC: Parallelität ohne Locks

Das Problem mit Locks

Traditionelle Datenbanken verwenden Locks:

```
# Traditionell: Pessimistic Locking
transaction1.begin()
transaction1.lock(row_id) # Row wird gesperrt
transaction1.update(row_id, new_value)
transaction1.unlock(row_id)
transaction1.commit()

# Problem: Transaction 2 muss warten
transaction2.begin()
transaction2.lock(row_id) # BLOCKIERT bis transaction1 fertig ist!
```

Problem: Bei hoher Parallelität führt dies zu: - Warteschlangen und Bottlenecks - Deadlocks zwischen Transaktionen - Timeouts und Failed Transactions

Die MVCC-Lösung

MVCC (Multi-Version Concurrency Control) erstellt stattdessen Versionen:

```
# MVCC: Optimistic Concurrency
transaction1.begin(snapshot_id=100)
# Liest Version 100 der Daten
transaction1.read(row_id) # Version 100

# Gleichzeitig kann transaction2 lesen!
transaction2.begin(snapshot_id=100)
transaction2.read(row_id) # Auch Version 100, keine Wartezeit!

# Transaction 1 schreibt
transaction1.update(row_id, value_a) # Erstellt Version 101
transaction1.commit() # Version 101 wird permanent

# Transaction 2 schreibt auch
transaction2.update(row_id, value_b) # Will auch Version 101 erstellen
transaction2.commit() # FEHLER: Conflict Detection!
# "Row was modified by another transaction"
```

Vorteil: Reads blockieren nie Writes, Writes blockieren nie Reads.

Wie funktioniert MVCC intern?

Jede Datenzeile hat nicht nur einen Wert, sondern eine Historie:

```
Row ID: users/alice

Version 100 (committed):
  name: "Alice Smith"
  age: 28

Version 101 (committed):
  name: "Alice Smith"
  age: 29

Version 102 (in_progress, transaction_id=555):
  name: "Alice Johnson"
  age: 29
```

Snapshot-Reads: Eine Transaktion mit `snapshot_id=100` sieht nur Versionen ≤ 100 , die committed sind.

Write-Write Conflicts: Wenn zwei Transaktionen die gleiche Row ändern wollen, gewinnt die erste. Die zweite bekommt einen Conflict Error beim Commit.

2.3 Transaction Management

ACID-Garantien

ThemisDB garantiert volle ACID-Properties:

A - Atomicity (Atomarität):

Alle Operationen in einer Transaktion passieren komplett oder gar nicht.

```
# Beispiel: Geldtransfer
transaction.begin()
transaction.update("accounts/alice", {"balance": 900}) # -100
transaction.update("accounts/bob", {"balance": 1100}) # +100
transaction.commit() # Beide Updates oder keines!
```

Wenn der Server zwischen den beiden `update()` Calls abstürzt: - **Ohne Atomarität:** Alice verliert 100€, Bob bekommt nichts (Katastrophe!) - **Mit Atomarität:** Beide Updates werden zurückgerollt (Konsistent!)

C - Consistency (Konsistenz):

Constraints werden immer eingehalten.

```
# Foreign Key Constraint
transaction.insert("orders", {
  "order_id": "o123",
  "user_id": "alice", # Muss in users existieren
  "amount": 50.0
})
# Wenn user "alice" nicht existiert -> FEHLER!
```

I - Isolation (Isolierung):

Transaktionen sehen sich nicht gegenseitig.

```
# Transaction 1 liest
t1.read("products/laptop") # price: 999

# Transaction 2 ändert (aber committed noch nicht)
t2.update("products/laptop", {"price": 899})

# Transaction 1 liest nochmal
t1.read("products/laptop") # Immer noch 999!
# Sieht die Änderung von T2 NICHT, bis T2 committet
```

D - Durability (Dauerhaftigkeit):

Einmal committed, sind Daten permanent - auch bei Serverausfall.

```
transaction.commit()
# Ab hier garantiert: Daten sind auf Disk!
# Selbst wenn Server JETZT abstürzt, sind Daten da.
```

Isolation Levels

ThemisDB implementiert **Snapshot Isolation**, ein Mittelweg zwischen Serializable und Read Committed:

Isolation Level	Dirty Reads	Non-Repeatable Reads	Phantom Reads
Read Uncommitted	✗ Möglich	✗ Möglich	✗ Möglich
Read Committed	✓ Verhindert	✗ Möglich	✗ Möglich
Snapshot Isolation	✓ Verhindert	✓ Verhindert	✓ Verhindert
Serializable	✓ Verhindert	✓ Verhindert	✓ Verhindert

Snapshot Isolation ist performanter als Serializable, verhindert aber trotzdem alle Anomalien, die in der Praxis relevant sind.

2.4 Storage Layer: RocksDB

Warum RocksDB?

ThemisDB verwendet RocksDB als Storage-Engine. Diese Entscheidung basiert auf mehreren Faktoren:

1. LSM-Trees für SSDs optimiert

Traditionelle B-Trees schreiben oft:

```
Write 1 byte → Read 4KB page → Modify 1 byte → Write 4KB page
```

LSM-Trees (Log-Structured Merge Trees) schreiben sequentiell:

```
Write 1 byte → Append to log → Done  
Later: Merge logs in background
```

Resultat: 10x schneller auf SSDs!

2. Battle-Tested bei Facebook

RocksDB verarbeitet bei Facebook: - Billions of operations per day - Petabytes of data - 10+ years Production Experience

3. Flexible Key-Value API

RocksDB ist ein Key-Value Store - perfekt als Foundation für höhere Modelle:

```
// Relational: key = "users/alice"  
db->Put("users/alice", json_data);  
  
// Graph: key = "edges/from_alice_to_bob"  
db->Put("edges/from_alice_to_bob", edge_data);  
  
// Vector: key = "vectors/product_123"  
db->Put("vectors/product_123", embedding_data);
```

Column Families

RocksDB unterstützt "Column Families" - separate Namespaces innerhalb einer DB:

```
// Trennung nach Datenmodell
db->Put(cf_relational, "users/alice", data);
db->Put(cf_graph_edges, "alice->bob", edge);
db->Put(cf_vectors, "prod_123", embedding);
```

Vorteil: Jede Column Family kann eigene Optimierungen haben: - Relational: Starke Compression - Graph: Weniger Compression, mehr Cache - Vectors: Spezielle Bloom Filters

Das Base Entity Paradigma: Einheitlicher Multi-Modell-Speicher

ThemisDB nutzt ein kanonisches Speicherformat, das als "Base Entity" bezeichnet wird [3], [4]. Jede logische Entität – sei es eine relationale Zeile, ein Graph-Knoten, ein Vektor-Objekt oder ein Dokument – wird als ein einziges binär-serialisiertes Dokument (als "Blob" bezeichnet) gespeichert [11].

Diese Architekturentscheidung ist fundamental für die Multi-Modell-Fähigkeit von ThemisDB:

Multi-Modell-Datenabbildung auf physischer Ebene:

Logisches Modell	Physischer Speicher	Key-Format	Value-Format
Relational	(PK, Blob)	"table_name:pk_value"	VelocityPack/Bincode
Dokument	(PK, Blob)	"collection:pk_value"	VelocityPack/Bincode
Graph (Knoten)	(PK, Blob)	"node:pk_value"	VelocityPack/Bincode
Graph (Kante)	(PK, Blob)	"edge:pk_value"	VelocityPack (inkl. _from/_to)
Vektor	(PK, Blob)	"object:pk_value"	VelocityPack (inkl. Vektor-Array)

Wie funktioniert das Base Entity Pattern?

Um das Base Entity Paradigma zu verstehen, betrachten wir ein konkretes Beispiel: Stellen Sie sich vor, Sie speichern einen Benutzer mit dem Namen "Alice", der 30 Jahre alt ist und in Berlin wohnt.

Im traditionellen Ansatz (z.B. PostgreSQL):

```
-- Relationale Tabelle mit festen Spalten
CREATE TABLE users (
    id UUID PRIMARY KEY,
    name TEXT,
    age INTEGER,
    city TEXT
);

INSERT INTO users VALUES ('uuid-123', 'Alice', 30, 'Berlin');
```

Im ThemisDB Base Entity Ansatz:

Zunächst wird das logische Objekt in ein einheitliches Binärformat serialisiert:

```
// 1. Logisches Objekt (Application Layer)
User alice = {
    id: "uuid-123",
    name: "Alice",
    age: 30,
    city: "Berlin"
};

// 2. Serialisierung zu VelocityPack (binäres JSON-ähnliches Format)
// VelocityPack ist optimiert für schnelles Parsing und kompakte Speicherung
std::vector<uint8_t> blob = VelocityPack::serialize(alice);
// Resultat: ~40 Bytes binäre Daten statt ~80 Bytes ASCII-JSON

// 3. Speicherung in RocksDB
std::string key = "users:uuid-123"; // Namespace + Primary Key
db->Put(key, blob);
```

Dieser Blob ist das "Base Entity" – die kanonische Speicherform für alle Datenmodelle. **Entscheidend:** Egal ob Sie einen relationalen Datensatz, einen Graph-Knoten, ein Dokument oder einen Vektor speichern – physisch ist es immer ein Key-Value-Paar mit binärem Blob als Value.

Wie werden unterschiedliche Datenmodelle auf Base Entities abgebildet?

1. **Relational (Tabellenzeile):** `cpp // Key: "table_name:primary_key" // Value: Binär-serialisierte Zeile mit allen Spalten Key: "users:uuid-123" Value: VelocityPack({id, name, age, city})`
2. **Graph-Knoten:** `cpp // Key: "node:node_id" // Value: Knoten-Eigenschaften + Metadaten Key: "nodes:alice" Value: VelocityPack({_id, label: "Person", properties: {name, age}})`
3. **Graph-Kante:** `cpp // Key: "edge:edge_id" // Value: Kante mit _from, _to, Gewicht, Eigenschaften Key: "edges:knows-123" Value: VelocityPack({_from: "alice", _to: "bob", _weight: 1.0, since: "2020"})`
4. **Vektor-Objekt:** `cpp // Key: "vector:object_id" // Value: Objekt-Metadaten + Embedding-Array Key: "vectors:doc-456" Value: VelocityPack({id, metadata: {...}, embedding: [0.1, 0.2, ..., 0.9]})`

Warum ist das ein Durchbruch?

Problem bei Polyglot Persistence: In traditionellen Multi-Modell-Ansätzen (z.B. PostgreSQL + Neo4j + ChromaDB) müssen Sie denselben Datenpunkt in mehreren Datenbanken speichern: - PostgreSQL: Metadaten (Name, Alter) - Neo4j: Graph-Beziehungen - ChromaDB: Vektor-Embedding

Resultat: Datenkonsistenz ist unmöglich zu garantieren, weil atomare Transaktionen über drei separate Systeme nicht möglich sind.

Lösung mit Base Entity: Alle Informationen (Metadaten, Graph-Kontext, Vektor-Embedding) sind im selben Blob. Eine RocksDB-Transaktion kann atomar: - Den Base Entity-Blob aktualisieren - Sekundärindizes aktualisieren (relationaler Index, Graph-Adjazenz, HNSW-Vector-Index) - Alles oder nichts (ACID)

Vorteile dieses Ansatzes:

1. **Einheitliche Speicherschicht:** Alle Datenmodelle teilen sich denselben physischen Speicher [11]
2. **ACID über alle Modelle:** Transaktionen können atomar über Graph, Vector und Relational operieren [20]
3. **Effiziente Serialisierung:** Binärformate wie VelocityPack [41] sind 4x schneller als Standard-JSON-Parser [40]

RocksDB TransactionDB: ACID-Garantien

ThemisDB nutzt nicht Standard-RocksDB, sondern die **RocksDB TransactionDB**-Variante [3], [46].

Diese bietet:

1. **Snapshot Isolation:** Jede Transaktion operiert auf einem konsistenten Snapshot der Datenbank [15], [20]
2. **Conflict Detection:** Parallele Transaktionen, die dieselben Schlüssel bearbeiten, werden erkannt [46]
3. **Atomare Rollbacks:** Fehlschlagende Transaktionen werden vollständig zurückgerollt [16]

Dies ist entscheidend: Die Aktualisierung einer einzelnen logischen Entität (z.B. `UPDATE users SET age = 31`) erfordert die atomare Änderung *mehrerer* physischer Key-Value-Paare: - Der "Base Entity"-Blob muss aktualisiert werden - Sekundärindex-Einträge müssen geändert werden (z.B. Löschen von `idx:age:30`, Einfügen von `idx:age:31`)

Ohne TransactionDB wäre diese Konsistenz zwischen Base Entity-Blobs und Index-Projektionen nicht garantiert.

Write-Ahead Log (WAL)

Jede Write-Operation wird erst in ein Log geschrieben:

1. Client sendet: `UPDATE users/alice SET age=29`
 2. WAL Write: `"UPDATE users/alice age=29"` → Disk (sync!)
 3. MemTable Update: In-Memory Update
 4. Return OK to Client
- ...später...
5. Background Compaction: MemTable → SSTable auf Disk

Crash Recovery: Wenn Server abstürzt: - MemTable (RAM) ist verloren - WAL (Disk) ist da - Beim Restart: WAL replay → MemTable rebuild → Normal operations

Wie funktioniert der WAL-Mechanismus im Detail?

Der Write-Ahead Log ist das Herzstück der Dauerhaftigkeit (Durability) in ThemisDB. Lassen Sie uns den Ablauf Schritt für Schritt nachvollziehen:

Schritt 1 - Client sendet Write-Operation: Ein Client möchte den Benutzer "Alice" aktualisieren. Die Operation wird an den ThemisDB-Server gesendet:

```
client.update("users", "uuid-123", {age: 31});
```

Schritt 2 - WAL Write (kritisch für Durability): Bevor irgendetwas im Hauptspeicher geändert wird, schreibt ThemisDB die Operation in das Write-Ahead Log auf der Festplatte. Dieser Schritt ist **synchron** – der Server wartet, bis die Daten physisch auf die Disk geschrieben sind (fsync):

```
// Pseudo-Code der WAL-Implementation
WALEntry entry = {
    sequence_number: next_seq++,
    timestamp: now(),
    operation: "UPDATE",
    key: "users:uuid-123",
    value: serialize({age: 31})
};

// Kritisch: sync=true erzwingt fsync() - Daten MÜSSEN auf Disk sein
wal_file.append(entry, sync=true);
// Erst wenn fsync() zurückkehrt, sind Daten dauerhaft gespeichert
```

Warum ist sync=true wichtig? Ohne `fsync()` würden Daten nur im Betriebssystem-Cache liegen. Bei einem Stromausfall wären sie verloren. Mit `fsync()` garantiert das OS, dass Daten auf physischen Platten (oder SSD) sind.

Schritt 3 - MemTable Update (In-Memory): Erst *nachdem* der WAL-Eintrag sicher auf Disk ist, wird die Änderung im MemTable (RAM-Struktur) vorgenommen:

```
// MemTable ist eine sortierte In-Memory-Map (z.B. Skip List)
memtable.put("users:uuid-123", {age: 31});
```

Das MemTable ist schnell ($O(\log n)$ für Inserts), aber flüchtig. Bei einem Crash ist es weg.

Schritt 4 - Return OK zum Client: Jetzt erst antwortet der Server dem Client mit "SUCCESS". Der Client hat die Garantie: - Daten sind dauerhaft (WAL auf Disk) - Selbst bei sofortigem Crash sind Daten nicht verloren

Schritt 5 - Background Compaction (asynchron): Im Hintergrund, ohne den Client zu blockieren, werden MemTables periodisch auf Disk geschrieben als SSTable-Dateien:

```
// Wenn MemTable voll ist (z.B. 256 MB)
if (memtable.size() > 256MB) {
    SSTable* sstable = memtable.flush_to_disk();
    // SSTable ist eine immutable, sortierte Datei auf Disk
    // Format: [Key1, Value1, Key2, Value2, ...]
}
```

Crash Recovery - Warum WAL lebensrettend ist:

Szenario: Server crashed nach Schritt 4, aber vor Schritt 5. Das MemTable (RAM) ist verloren, aber der WAL (Disk) ist da.

Beim Server-Restart:

```
// 1. WAL lesen und replay
for (entry in wal_file) {
    memtable.put(entry.key, entry.value);
}
// → MemTable ist rekonstruiert!

// 2. Normale Operationen fortsetzen
server.start();
```

Resultat: Kein Datenverlust. Alle committed Transaktionen sind wiederhergestellt.

Performance-Trade-off: Der `fsync()` in Schritt 2 kostet Zeit (~1-5ms auf NVMe, ~10-20ms auf HDD). Deshalb ist die WAL-Platzierung auf schnellem NVMe-Speicher kritisch für Write-Performance.

LSM-Tree Performance-Charakteristiken

Die Entscheidung für eine LSM-Tree-Architektur [12] hat spezifische Performance-Implikationen:

Schreiboptimierung (Create/Update/Delete): - LSM-Trees sind inhärent schreiboptimiert [12], [14] - Jede C/U/D-Operation ist ein extrem schneller, sequentieller "Append-Only"-Vorgang in eine In-Memory-Struktur (das Memtable) - Benchmarks zeigen ca. **45.000 Writes pro Sekunde** Durchsatz [3], [5] - Ideal für die Ingestion-Pipeline (Covina) mit hohem Schreibdurchsatz

Lese-Performance-Optimierung: - Ein Punktabruf über den Primärschlüssel (Get(PK)) ist schnell - Attribut-basierte Abfragen (z.B. `SELECT * WHERE age > 30`) würden ohne Indizes einen Full-Scan

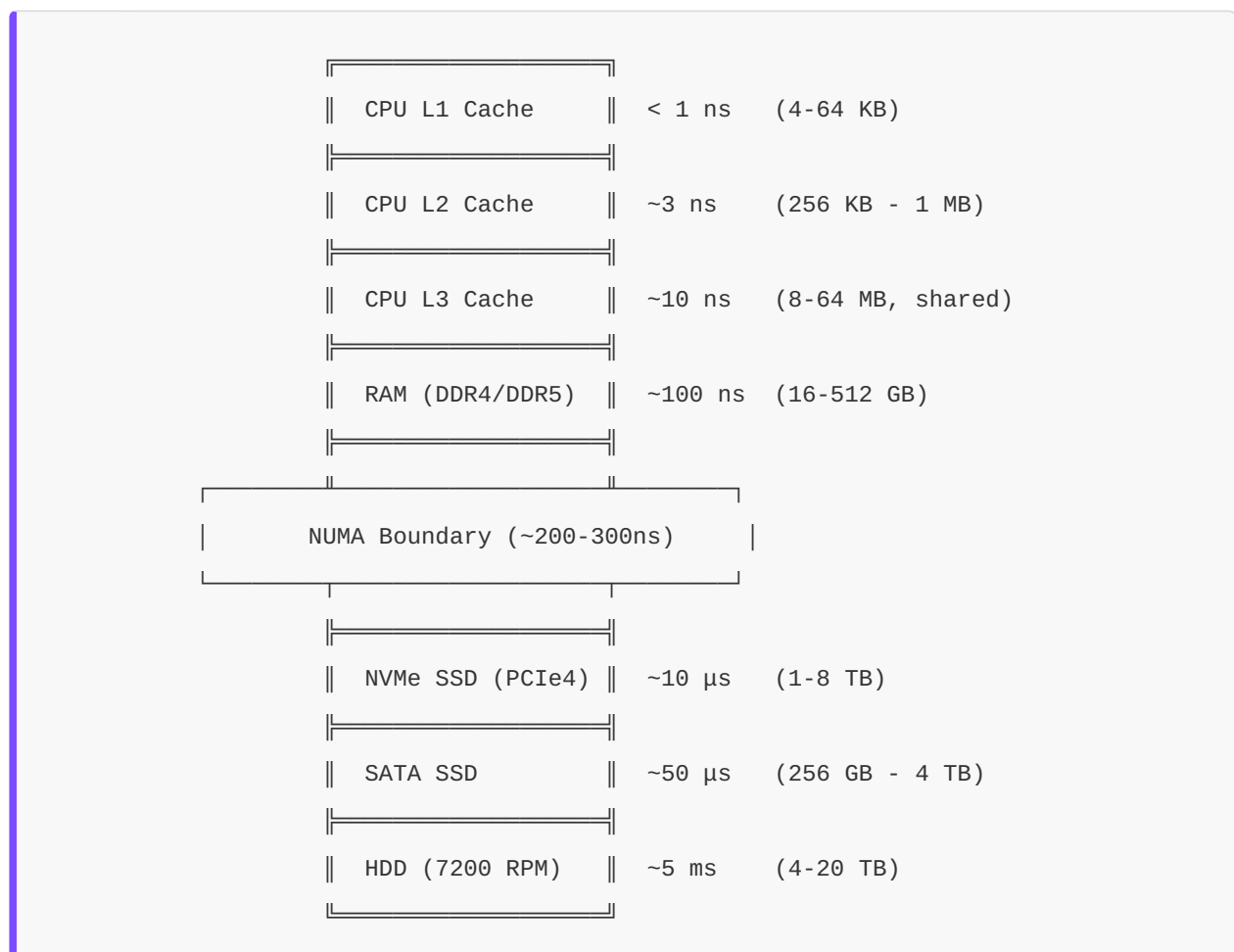
aller Blobs erfordern [4] - Dies erzwingt architektonisch die Notwendigkeit der "Layer" (Sekundärindizes) für optimale Leseleistung [3]

Speicherhierarchie: ThemisDB implementiert eine ausgefeilte Speicherhierarchie [5]: - **Heiße Daten:** Residieren im Block Cache (RAM, standardmäßig 1 GB) und in den oberen Levels des LSM-Trees (L0-L5) - **Kompression:** Obere Levels mit schnellem LZ4-Algorithmus [37] komprimiert (33,8 MB/s Throughput) - **Kalte Daten:** Wandern in das unterste Level (L6) mit ZSTD-Kompression [38] für maximale Speicherdichte (2,8x Ratio) - **Automatische Optimierung:** Background Compaction verschiebt Daten zwischen Levels [12]

Speicherhierarchie: Von VRAM bis HDD

Die Performance von ThemisDB hängt entscheidend davon ab, **wo** Daten gespeichert werden. Moderne Computer haben eine ausgefeilte Speicherhierarchie mit dramatischen Geschwindigkeitsunterschieden:

Die Speicherpyramide (von schnell nach langsam):



Faktor zwischen schnellstem und langsamstem: ~5.000.000x (!)

Wie nutzt ThemisDB diese Hierarchie optimal?

1. Heiße Daten im RAM (Block Cache):

ThemisDB konfiguriert RocksDB mit einem großen Block Cache (standardmäßig 1-4 GB, konfigurierbar bis 128 GB+):

```
// Aus docs/de/performance/performance_memory.md
RocksDBWrapper::Config config;
config.block_cache_size_mb = 4096; // 4 GB Block Cache
config.cache_index_and_filter_blocks = true;
config.pin_l0_filter_and_index_blocks_in_cache = true;
config.high_pri_pool_ratio = 0.5; // 50% für Index/Filter
```

Was wird gecacht? - Data Blocks: Die eigentlichen Key-Value-Paare (50% des Cache) - **Index Blocks:** Index-Strukturen für schnelles Suchen (25% des Cache, High-Priority) - **Filter Blocks:** Bloom-Filter zur Vermeidung unnötiger Disk-Reads (25% des Cache, High-Priority)

Wirkung: Ein Cache-Hit (Daten sind im RAM) hat ~100ns Latenz. Ein Cache-Miss (Disk-Read nötig) hat ~10µs Latenz auf NVMe. **Faktor 100x schneller!**

2. Write-Ahead Log auf schnellstem Medium (NVMe):

Das WAL ist write-kritisch. Jede Schreiboperation wartet auf einen `fsync()`. Deshalb sollte das WAL auf dem schnellsten verfügbaren Medium liegen:

```
// Separates WAL-Verzeichnis auf dedizierter NVMe
config.db_path = "/data/rocksdb"; // Haupt-DB (kann langsamer sein)
config.wal_dir = "/nvme/fast/wal"; // WAL auf schnellster NVMe
```

Performance-Gewinn: - NVMe (PCIe4): ~10µs fsync → **45.000 Writes/Sekunde** [3], [5] - SATA SSD: ~50µs fsync → 20.000 Writes/Sekunde - HDD: ~5ms fsync → 200 Writes/Sekunde

Faktor 225x zwischen NVMe und HDD!

3. LSM-Tree Levels mit gestufter Kompression:

ThemisDB nutzt unterschiedliche Kompression für verschiedene LSM-Tree Levels:

```
config.compression_default = "lz4";      // Level 0-5 (heiß)
config.compression_bottommost = "zstd";  // Level 6+ (kalt)
```

Warum?

- **L0-L5 (heiße Daten):** Werden häufig gelesen → LZ4 (33.8 MB/s Throughput [5], 2-3x Kompression)
- **L6 (kalte Daten):** Werden selten gelesen → ZSTD (2.8x Kompression [5], aber langsamer)

Speicher-Trade-off visualisiert:

```
Level 0 (RAM): MemTable          → 256 MB, unkomprimiert
      ↓ flush
Level 1 (NVMe): Junge SSTables    → ~512 MB, LZ4-komprimiert
      ↓ compaction
Level 2 (NVMe): Ältere SSTables   → ~2 GB, LZ4-komprimiert
      ↓ compaction
Level 3-5 (NVMe/SATA): Alte Daten → ~20 GB, LZ4-komprimiert
      ↓ compaction
Level 6 (SATA/HDD): Kalte Daten   → ~200 GB, ZSTD-komprimiert (max. Dichte)
```

Automatische Datenmigration:

ThemisDB verschiebt Daten automatisch "nach unten": 1. Neue Writes → MemTable (RAM, ultra-schnell) 2. MemTable voll → Flush zu L1 (NVMe, schnell) 3. Compaction → Daten wandern zu L2, L3, ... (progressiv langsamer) 4. Kalte Daten → L6 (maximal komprimiert, platzsparend)

Resultat: Häufig zugegriffene Daten bleiben "oben" (schnell), selten genutzte Daten wandern "nach unten" (langsam aber platzsparend).

4. GPU-VRAM für HNSW-Index (optional, für Vektor-Suche):

Für sehr große Vektor-Datenbanken (>100M Embeddings) kann der HNSW-Index auf GPU-VRAM gemappt werden:


```
// GPU-Beschleunigung für Vektor-Ähnlichkeitssuche
HNSWConfig hnsw_config;
hnsw_config.use_gpu = true;
hnsw_config.gpu_device = 0; // CUDA device 0
// VRAM: ~1-5 ns Latenz, aber begrenzte Größe (8-80 GB)
```

Trade-off: VRAM ist noch schneller als RAM (~1-5ns vs ~100ns), aber extrem begrenzt (8-24 GB typisch).

Zusammenfassung: Speicherhierarchie-Strategie:

Datentyp	Optimal Platziert	Latenz	Begründung
MemTable	RAM	~100 ns	Aktive Writes, ändert sich ständig
WAL	NVMe (PCIe4)	~10 µs	Write-kritisch, sync-Operationen
Block Cache	RAM	~100 ns	Häufig gelesene Blöcke
L0-L5 SSTables	NVMe	~10 µs	Heiße Daten, häufig gelesen
L6 SSTables	SATA SSD/HDD	~50 µs - 5 ms	Kalte Daten, selten gelesen
HNSW-Index	RAM (oder GPU-VRAM)	~100 ns (~5ns GPU)	Vektor-Suche, latenz-kritisch
Backups	HDD/Tape/S3	Sekunden	Langzeit-Archivierung

Best Practice für Production:

```
# NVMe PCIe4 für WAL und L0-L3
/nvme/fast/ → 2 TB NVMe PCIe4 für WAL + Hot SSTables

# SATA SSD für L4-L6
/ssd/bulk/ → 8 TB SATA SSD für Cold SSTables

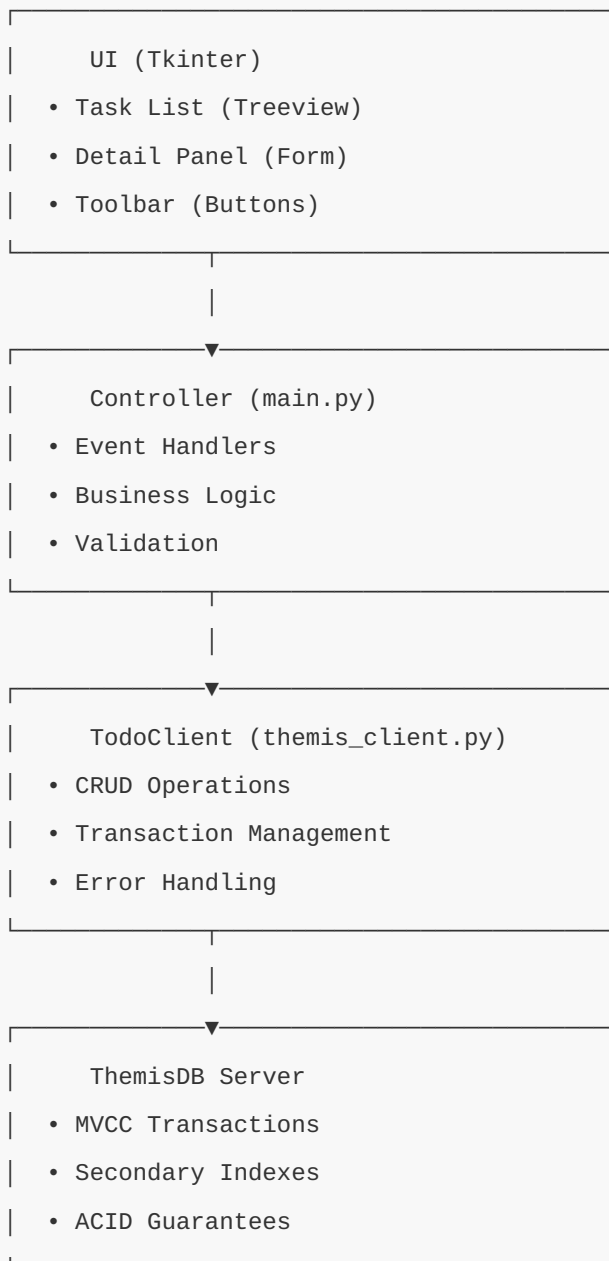
# HDD für Backups
/backup/ → 40 TB HDD Array für Point-in-Time Snapshots
```

2.5 Praxis: Todo-App

Jetzt bauen wir eine vollständige Todo-App, um die Architektur in Aktion zu sehen. Basis ist `examples/02_todo_app`.

Architektur der Todo-App

Die Todo-App folgt dem MVC-Pattern und demonstriert alle ACID-Eigenschaften:



Datenmodell

```
# examples/02_todo_app/models.py

from dataclasses import dataclass
from datetime import datetime
from enum import Enum
from typing import Optional, List

class TaskStatus(Enum):
    OPEN = "open"
    IN_PROGRESS = "in_progress"
    DONE = "done"

class TaskPriority(Enum):
    LOW = "low"
    NORMAL = "normal"
    HIGH = "high"

@dataclass
class Task:
    id: str
    title: str
    description: str
    status: TaskStatus
    priority: TaskPriority
    created_at: datetime
    updated_at: datetime
    due_date: Optional[datetime] = None
    tags: List[str] = None

    def to_dict(self):
        """Serialisierung für ThemisDB"""
        return {
            "_key": self.id,
            "title": self.title,
            "description": self.description,
            "status": self.status.value,
            "priority": self.priority.value,
            "created_at": self.created_at.isoformat(),
```

```

        "updated_at": self.updated_at.isoformat(),
        "due_date": self.due_date.isoformat() if self.due_date else None,
        "tags": self.tags or []
    }

    @classmethod
    def from_dict(cls, data: dict):
        """Deserialisierung von ThemisDB"""
        return cls(
            id=data["_key"],
            title=data["title"],
            description=data["description"],
            status=TaskStatus(data["status"]),
            priority=TaskPriority(data["priority"]),
            created_at=datetime.fromisoformat(data["created_at"]),
            updated_at=datetime.fromisoformat(data["updated_at"]),
            due_date=datetime.fromisoformat(data["due_date"]) if data.get("due_date") else None,
            tags=data.get("tags", [])
        )

```

Design-Entscheidungen:

1. **Enums für Status/Priority:** Type-Safety, keine Tippfehler möglich
2. **Dataclass:** Automatische `__init__`, `__repr__`, `__eq__`
3. **to_dict/from_dict:** Klare Serialisierungslogik
4. **Type Hints:** IDEs können helfen, Fehler früh erkennen

Setup und Installation

```

# ThemisDB starten (Docker)
docker run -d -p 8765:8765 themisdb/themisdb:1.3.4

# Todo-App installieren
cd examples/02_todo_app
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt

```

Client-Implementierung

```
# examples/02_todo_app/themis_client.py

from themisdb import Client
from models import Task, TaskStatus, TaskPriority
from typing import List, Optional, Dict
import uuid

class TodoClient:
    def __init__(self, host="localhost", port=8765):
        self.client = Client(host, port)
        self._setup_database()

    def _setup_database(self):
        """Erstellt Collection und Indizes"""
        # Collection erstellen
        try:
            self.client.create_collection("tasks", type="document")
        except Exception as e:
            # Collection existiert bereits
            pass

        # Indizes erstellen für Performance
        self.client.create_index("tasks", "status_idx", ["status"])
        self.client.create_index("tasks", "priority_idx", ["priority"])
        self.client.create_index("tasks", "due_date_idx", ["due_date"])

    def create_task(self, task: Task) -> bool:
        """Erstellt einen neuen Task"""
        try:
            self.client.insert("tasks", task.to_dict())
            return True
        except Exception as e:
            print(f"Error creating task: {e}")
            return False

    def get_task(self, task_id: str) -> Optional[Task]:
        """Lädt einen Task nach ID"""
        try:
```

```
        data = self.client.get("tasks", task_id)
        return Task.from_dict(data) if data else None
    except Exception as e:
        print(f"Error getting task: {e}")
        return None

def update_task(self, task: Task) -> bool:
    """Aktualisiert einen existierenden Task"""
    try:
        # MVCC in Action: Conflict Detection!
        self.client.update("tasks", task.id, task.to_dict())
        return True
    except ConflictError:
        print("Task was modified by another user!")
        return False
    except Exception as e:
        print(f"Error updating task: {e}")
        return False

def delete_task(self, task_id: str) -> bool:
    """Löscht einen Task"""
    try:
        self.client.delete("tasks", task_id)
        return True
    except Exception as e:
        print(f"Error deleting task: {e}")
        return False

def list_tasks(self, status: Optional[TaskStatus] = None,
               priority: Optional[TaskPriority] = None) -> List[Task]:
    """Listet Tasks mit optionalen Filtern"""
    query = "FOR task IN tasks"
    filters = []

    if status:
        filters.append(f'task.status == "{status.value}"')
    if priority:
        filters.append(f'task.priority == "{priority.value}"')
```



```
if filters:
    query += " FILTER " + " AND ".join(filters)

query += " SORT task.created_at DESC RETURN task"

try:
    results = self.client.query(query)
    return [Task.from_dict(data) for data in results]
except Exception as e:
    print(f"Error listing tasks: {e}")
    return []

def search_tasks(self, search_text: str) -> List[Task]:
    """Sucht Tasks nach Text in Titel oder Beschreibung"""
    query = """
FOR task IN tasks
    FILTER CONTAINS(LOWER(task.title), LOWER(@search))
        OR CONTAINS(LOWER(task.description), LOWER(@search))
    SORT task.created_at DESC
    RETURN task
    """
    try:
        results = self.client.query(query, {"search": search_text})
        return [Task.from_dict(data) for data in results]
    except Exception as e:
        print(f"Error searching tasks: {e}")
        return []
```

MVCC und Transaktionen demonstriert

Die Todo-App zeigt MVCC in Aktion:

Szenario: Zwei Benutzer ändern gleichzeitig den gleichen Task

```
# User 1: Startet Transaction
user1 = TodoClient()
task = user1.get_task("task_123") # Snapshot Version 100
task.status = TaskStatus.IN_PROGRESS
task.updated_at = datetime.now()

# User 2: Auch Transaction, gleicher Task
user2 = TodoClient()
task2 = user2.get_task("task_123") # Auch Snapshot Version 100
task2.priority = TaskPriority.HIGH
task2.updated_at = datetime.now()

# User 1 committed zuerst
user1.update_task(task) # ✓ SUCCESS, Version 101

# User 2 versucht zu committen
user2.update_task(task2) # ✗ CONFLICT!
# Error: "Task was modified by another transaction"
```

Was passiert intern?

1. Beide lesen Version 100 (Snapshot Isolation)
2. User 1 schreibt Version 101 und committed
3. User 2 versucht auch Version 101 zu schreiben
4. ThemisDB erkennt: "Version 101 existiert bereits!"
5. Conflict Error wird geworfen

Lösung im Code:

```
def update_task_with_retry(self, task: Task, max_retries=3):
    """Update mit automatischem Retry bei Conflicts"""
    for attempt in range(max_retries):
        try:
            self.client.update("tasks", task.id, task.to_dict())
            return True
        except ConflictError:
            # Re-read aktuellste Version
            latest = self.get_task(task.id)
            if not latest:
                return False

            # Merge changes (Application Logic)
            # z.B.: Keep newer updated_at
            if latest.updated_at > task.updated_at:
                task = latest

            # Retry mit gemergten Daten
            continue
        except Exception as e:
            return False

    return False # Max retries exceeded
```

2.6 Index-Strukturen

Warum Indizes?

Ohne Index: Full Table Scan

```
-- Ohne Index: O(n)
SELECT * FROM tasks WHERE status = 'open'
-- Muss ALLE Tasks durchgehen: 10.000 Tasks = 10.000 Reads
```

Mit Index: Direkt zum Ergebnis

```
-- Mit Index auf status:  $O(\log n + k)$ , k = Anzahl Results  
SELECT * FROM tasks WHERE status = 'open'  
-- B-Tree Lookup:  $\log(10.000) \approx 13$  Reads + nur relevante Tasks
```

Index-Typen in ThemisDB

1. B-Tree Index (Default)

```
client.create_index("tasks", "status_idx", ["status"])
```

- Sortierte Daten
- Range Queries: `WHERE age > 18 AND age < 65`
- Equality: `WHERE status = 'open'`

2. Hash Index

```
client.create_index("tasks", "id_hash_idx", ["id"], type="hash")
```

- Nur Equality: `WHERE id = 'task_123'`
- Schneller als B-Tree für Equality
- Keine Range Queries

3. Geo Index (Hilbert Curves)

```
client.create_index("locations", "geo_idx", ["coordinates"], type="geo")
```

- Räumliche Queries
- Nearest Neighbor
- Bounding Box Searches

4. Fulltext Index

```
client.create_index("tasks", "fulltext_idx", ["title", "description"], type="fulltext")
```

- Tokenization

- Stemming
- Relevance Scoring

5. Vector Index (HNSW)

```
client.create_index("products", "embedding_idx", ["embedding"], type="vector", dimensions=768)
```

- Approximate Nearest Neighbor
- Cosine Similarity
- Euclidean Distance

Index-Performance

Operation	Ohne Index	Mit B-Tree	Mit Hash
Equality (<code>=</code>)	$O(n)$	$O(\log n)$	$O(1)$
Range (<code>></code> , <code><</code>)	$O(n)$	$O(\log n + k)$	X
Sort	$O(n \log n)$	$O(k)$	X

k = Anzahl Ergebnisse

2.7 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- ✓ **Schichten-Architektur:** 5 Schichten mit klaren Verantwortlichkeiten
- ✓ **MVCC:** Parallelität ohne Locks durch Versionierung
- ✓ **Transaktionen:** ACID-Garantien und Snapshot Isolation
- ✓ **RocksDB:** LSM-Trees, WAL, Column Families
- ✓ **Indizes:** B-Tree, Hash, Geo, Fulltext, Vector
- ✓ **Todo-App:** Vollständige CRUD-Anwendung mit MVCC

Was macht ThemisDB besonders?

1. **Multi-Model MVCC:** Transaktionen über alle Modelle hinweg
2. **RocksDB Foundation:** Battle-tested, SSD-optimiert
3. **Flexible Indizes:** Für jeden Use Case der richtige Index
4. **Snapshot Isolation:** Performance + Konsistenz

Nächste Schritte

Im nächsten Kapitel lernen Sie, wie die verschiedenen Datenmodelle kombiniert werden können und wann welches Modell am besten passt.

[Kapitel 3: Multi-Model verstehen →](#)

Weiterführende Ressourcen

- Complete Example: [examples/02_todo_app](#)
 - MVCC Deep-Dive: [../de/architecture/architecture_mvcc.md](#)
 - RocksDB Storage: [../de/storage/storage_rocksdb.md](#)
 - Index Design: [Kapitel 15 - Storage Internals](#)
-

Kapitel 2 von 30 | Teil I: Grundlagen | ~8.500 Wörter

Kapitel 3: Multi-Model verstehen

"Der Unterschied zwischen einem guten und einem großartigen System liegt oft darin, das richtige Werkzeug für die richtige Aufgabe zu wählen."

Überblick

In den ersten beiden Kapiteln haben Sie die Grundlagen von ThemisDB kennengelernt. Jetzt lernen Sie, wie Sie die verschiedenen Datenmodelle gezielt einsetzen und kombinieren. Das Geheimnis liegt nicht darin, ein Modell zu wählen - sondern das *richtige* Modell für jeden Teil Ihrer Daten.

Was Sie in diesem Kapitel lernen werden: - Wann welches Datenmodell optimal ist - Wie man Modelle in einer Anwendung kombiniert - Die Stärken und Schwächen jedes Modells - Praktische Entscheidungskriterien - Vollständige Demonstration mit einem Kontaktmanager

Voraussetzungen: Kapitel 1 und 2 gelesen.

3.1 Die vier Modelle im Detail

Relational: Wenn Struktur entscheidend ist

Best für: - Geschäftsdaten mit klaren Beziehungen - Financial Transactions - Inventory Management - Reporting und Analytics

Eigenschaften:


```
# Festes Schema
users = {
    "user_id": "string",      # Muss vorhanden sein
    "email": "string",       # Muss vorhanden sein
    "age": "integer",        # Type-checked
    "created_at": "timestamp" # Format validiert
}

# Constraints
UNIQUE(email)
CHECK(age >= 18)
FOREIGN KEY(user_id) REFERENCES users(user_id)
```

Vorteile: -  Data Integrity durch Constraints -  Optimierte Joins -  Perfekt für BI/Reporting - 
ACID über alle Operationen

Nachteile: -  Schema-Änderungen erfordern Migrations -  Nicht flexibel für schnelle Iterationen -
 Overhead bei sehr unterschiedlichen Entities

Wann verwenden?

- ✓ Daten haben feste Struktur
- ✓ Beziehungen sind wichtig
- ✓ Konsistenz ist kritisch
- x Schema ändert sich häufig

Graph: Wenn Beziehungen im Fokus stehen

Best für: - Social Networks - Empfehlungssysteme - Fraud Detection - Netzwerk-Topologien

Eigenschaften:

```
# Knoten und Kanten sind First-Class Citizens

user = {
  "_id": "users/alice",
  "name": "Alice"
}

friendship = {
  "_from": "users/alice",
  "_to": "users/bob",
  "since": "2024-01-15",
  "strength": 0.95 # Weighted edges
}

# Traversierung ist nativ schnell
FOR friend IN 2..3 OUTBOUND "users/alice" friends
  RETURN friend
```

Vorteile: -  Traversierung ist $O(\text{degree} \times \text{hops})$, nicht $O(n^2)$ -  Relationship-First Design - 
Pattern Matching möglich -  Graph-Algorithmen (PageRank, etc.)

Nachteile: -  Nicht optimal für reine Daten ohne Beziehungen -  Komplexer bei sehr vielen Knoten
(> Millionen) -  Overhead wenn Beziehungen unwichtig sind

Wann verwenden?

- ✓ Beziehungen sind zentral
- ✓ Traversierung ist häufig
- ✓ Netzwerk-Strukturen
- ✗ Isolierte Entities

Dokument: Wenn Flexibilität zählt

Best für: - Content Management - Product Catalogs - User Profiles - Logs und Events

Eigenschaften:

```
# Schema-free, beliebige Struktur
product = {
  "sku": "LAPTOP-2024",
  "name": "Developer Laptop",
  "specs": { # Nested objects
    "cpu": {"model": "i7", "cores": 8},
    "ram": {"size": 32, "type": "DDR5"}
  },
  "reviews": [ # Arrays
    {"user": "alice", "rating": 5},
    {"user": "bob", "rating": 4}
  ],
  "tags": ["developer", "high-end"],
  "metadata": { # Kann sein, muss nicht
    "warehouse": "DE-01"
  }
}
```

Vorteile: -  Keine Schema-Migrations -  Schnelle Iteration -  Hierarchische Daten natürlich -  Self-contained Documents

Nachteile: -  Keine Schema-Validierung (optional möglich) -  Denormalisierung kann zu Duplikaten führen -  Joins sind teurer

Wann verwenden?

- ✓ Schema ändert sich oft
- ✓ Hierarchische Daten
- ✓ Prototyping
- ✗ Strenge Validierung nötig

Vektor: Wenn Ähnlichkeit gefragt ist

Best für: - Semantic Search - Recommendation Engines - Image Similarity - ML/AI Applications

Eigenschaften:

```
# Embeddings als High-Dimensional Vectors
product_embedding = [
    0.123, -0.456, 0.789, ... # 768 dimensions
]

# Ähnlichkeitssuche
FOR product IN products
    LET similarity = COSINE_SIMILARITY(
        @query_embedding,
        product.embedding
    )
    FILTER similarity > 0.8
    SORT similarity DESC
    LIMIT 10
    RETURN product
```

Vorteile: -  Semantic Search ohne Keywords -  Multimodal (Text, Images, Audio) -  Skaliert zu Millionen von Vektoren -  State-of-the-art HNSW Index

Nachteile: -  Embeddings müssen generiert werden -  Höherer Speicherbedarf -  Approximate, nicht exact matching

Wann verwenden?

- ✓ Semantic Similarity
- ✓ "Finde ähnliche X"
- ✓ ML/AI Integration
- ✗ Exact matches ausreichend

3.2 Native Multi-Model vs. Polyglot Persistence

Das Problem polyglotter Persistenz

Viele Unternehmen verfolgen einen "Polyglot Persistence"-Ansatz [33]: Sie kombinieren mehrere spezialisierte Datenbanken (z.B. PostgreSQL für relationale Daten, Neo4j für Graphen, ChromaDB für

Vektoren) in einem losen Verbund. Dieser Ansatz scheint flexibel, führt aber zu fundamentalen Problemen [1]:

Technische Herausforderungen:

1. **Eventual Consistency statt ACID:**
2. Daten sind über mehrere, physisch getrennte Systeme verteilt [33]
3. Atomare Transaktionen über alle Systeme hinweg sind unmöglich [1]
4. Man muss auf das "Saga-Pattern" [17] mit kompensierenden Transaktionen zurückgreifen
5. Resultat: "Eventual Consistency" (BASE) [18] statt starker ACID-Garantien [16]
6. **Post-Filtering-Problem bei RAG-Workloads:** `` # Polyglot-Ansatz (ineffizient):
7. Vektor-DB: Finde 1000 ähnliche Dokumente
8. Graph-DB: Hole alle Prozesse für Landkreis Havelland
9. Relational-DB: Hole alle Akten aus 2024
10. Application: Bilde manuell Schnittmenge im RAM → 990 irrelevante Ergebnisse werden verworfen! [2], [5] ``
11. **Operativer Overhead:**
12. 3+ unterschiedliche APIs zu lernen und warten
13. 3+ Backup-Strategien zu implementieren
14. 3+ Monitoring-Systeme zu konfigurieren
15. 3+ Security-Konfigurationen abzustimmen
16. Team benötigt Expertise in mehreren Datenbanken

ThemisDB's Native Multi-Model-Lösung

ThemisDB verfolgt einen fundamentalen anderen Ansatz [3], [11]: **Alle Datenmodelle teilen sich eine einzige, transaktionale Speicherschicht** (siehe Kapitel 2.4 - Base Entity Paradigma).

Architektonische Vorteile:

1. **Starke ACID-Transaktionen über alle Modelle:**
2. Eine Operation kann atomar Graph, Vector und Relational ändern [1], [20]
3. Kein Saga-Pattern nötig für Datenbankintegrität [3]
4. Konsistenz ist "by Design", nicht ein fehleranfälliger Applikationsprozess

5. **Pre-Filtering statt Post-Filtering:** `aql -- Native Multi-Model ermöglicht Pre-Filtering: FOR doc IN documents FILTER doc.year == 2024 // Relational Filter ZUERST FILTER doc.region == "Havelland" // Graph Context LET similarity = COSINE(doc.vector, @query) // Vector Search auf Subset FILTER similarity > 0.8 SORT similarity DESC LIMIT 10`

6. Query-Engine nutzt ZUERST den relationalen Index (Jahr=2024)

7. Erstellt eine hochselektive Kandidatenliste

8. Vektorsuche (rechenintensiv) läuft NUR auf erlaubter Teilmenge

9. **Resultat:** Um Größenordnungen performanter als Post-Filtering

10. **Vereinfachte Operations:**

11. Eine API (AQL) für alle Modelle

12. Eine Backup-Strategie

13. Ein Monitoring-System

14. Eine Security-Konfiguration

Wann welcher Ansatz?

Polyglot Persistence verwenden wenn: - ✓ Sie bereits mehrere spezialisierte Datenbanken im Einsatz haben - ✓ Eventual Consistency für Ihren Use Case akzeptabel ist - ✓ Sie Best-of-Breed-Tools für jeden Use Case wollen - ✓ Sie ein großes Ops-Team haben

Native Multi-Model (ThemisDB) verwenden wenn: - ✓ ACID-Garantien über alle Datenmodelle kritisch sind - ✓ Sie hybride Queries (Graph + Vector + Relational) benötigen - ✓ RAG-Workloads mit komplexen Filtern zentral sind - ✓ Sie operationale Komplexität reduzieren wollen - ✓ Ein kleineres Team die Infrastruktur betreiben soll

3.3 Entscheidungskriterien

Die 5 Fragen-Methode

1. Wie strukturiert sind Ihre Daten?

Sehr strukturiert → Relational
Teilweise strukturiert → Dokument
Unstrukturiert → Vektor

2. Wie wichtig sind Beziehungen?

Zentral → Graph
Wichtig → Relational (mit Joins)
Unwichtig → Dokument

3. Wie oft ändert sich das Schema?

Selten → Relational
Häufig → Dokument
Gar nicht → Relational

4. Welche Queries dominieren?

Beziehungen durchlaufen → Graph
Ähnlichkeit finden → Vektor
Komplexe Aggregationen → Relational
Einzelne Dokumente → Dokument

5. Wie wichtig ist Konsistenz?

Kritisch → Relational
Wichtig → Relational oder Graph
Weniger wichtig → Dokument

Entscheidungsmatrix

Use Case	Relational	Graph	Dokument	Vektor
E-Commerce Orders	★★★	★☆☆	★★☆	☆☆☆
Social Network	★☆☆	★★★	★★☆	☆☆☆
Product Catalog	★★☆	☆☆☆	★★★	★★☆
Recommendation	★☆☆	★★★	☆☆☆	★★★
CMS/Blog	★☆☆	☆☆☆	★★★	★★☆
Financial	★★★	☆☆☆	★★☆	☆☆☆
Fraud Detection	★★☆	★★★	★★☆	★★☆

3.4 Praxis: Kontaktmanager

Jetzt bauen wir einen vollständigen Kontaktmanager, der zeigt, wie man Modelle kombiniert. Basis ist `examples/03_contact_manager`.

Warum Dokument-Modell für Kontakte?

Kontakte sind ein perfektes Beispiel für das Dokument-Modell:

Gründe: 1. **Flexible Struktur:** Manche Kontakte haben Adresse, manche nicht 2. **Hierarchische Daten:** Adresse ist ein Nested Object 3. **Schema-Evolution:** Neue Felder wie "Social Media" leicht hinzufügbare 4. **Self-Contained:** Alle Infos zu einem Kontakt in einem Dokument

Alternative wäre Relational:

```
-- Relational würde benötigen:
CREATE TABLE contacts (id, name, email, phone);
CREATE TABLE addresses (id, contact_id, street, city, ...);
CREATE TABLE social_media (id, contact_id, platform, handle);
-- → 3+ Tabellen, Joins nötig
```


Mit Dokument:

```
# Alles in einem Dokument
contact = {
  "name": "Alice",
  "email": "alice@example.com",
  "address": {"city": "Berlin"}, # Optional!
  "social": {"twitter": "@alice"} # Optional!
}
```

Datenmodell

```
# examples/03_contact_manager/models.py

from dataclasses import dataclass, field
from datetime import datetime
from typing import Optional, List
from enum import Enum

class ContactCategory(Enum):
    FRIENDS = "friends"
    FAMILY = "family"
    WORK = "work"
    OTHER = "other"

@dataclass
class Address:
    """Verschachtelte Adress-Struktur"""
    street: str
    city: str
    postal_code: str
    country: str = "Deutschland"

    def to_dict(self):
        return {
            "street": self.street,
            "city": self.city,
            "postal_code": self.postal_code,
            "country": self.country
        }

    @classmethod
    def from_dict(cls, data: dict):
        return cls(
            street=data.get("street", ""),
            city=data.get("city", ""),
            postal_code=data.get("postal_code", ""),
            country=data.get("country", "Deutschland")
        )
```

```
@dataclass
class Contact:
    """Hauptentität: Kontakt"""
    id: str
    first_name: str
    last_name: str
    email: str
    phone: Optional[str] = None
    address: Optional[Address] = None
    category: ContactCategory = ContactCategory.OTHER
    is_favorite: bool = False
    notes: str = ""
    tags: List[str] = field(default_factory=list)
    created_at: datetime = field(default_factory=datetime.now)
    updated_at: datetime = field(default_factory=datetime.now)

    @property
    def full_name(self):
        """Computed Property"""
        return f"{self.first_name} {self.last_name}"

    def to_dict(self):
        """Serialisierung für ThemisDB"""
        return {
            "_key": self.id,
            "first_name": self.first_name,
            "last_name": self.last_name,
            "email": self.email,
            "phone": self.phone,
            "address": self.address.to_dict() if self.address else None,
            "category": self.category.value,
            "is_favorite": self.is_favorite,
            "notes": self.notes,
            "tags": self.tags,
            "created_at": self.created_at.isoformat(),
            "updated_at": self.updated_at.isoformat()
        }
```

```
@classmethod
def from_dict(cls, data: dict):
    """Deserialisierung von ThemisDB"""
    address_data = data.get("address")
    return cls(
        id=data["_key"],
        first_name=data["first_name"],
        last_name=data["last_name"],
        email=data["email"],
        phone=data.get("phone"),
        address=Address.from_dict(address_data) if address_data else None,
        category=ContactCategory(data.get("category", "other")),
        is_favorite=data.get("is_favorite", False),
        notes=data.get("notes", ""),
        tags=data.get("tags", []),
        created_at=datetime.fromisoformat(data["created_at"]),
        updated_at=datetime.fromisoformat(data["updated_at"])
    )
```

Design-Highlights:

1. **Nested Objects:** `Address` ist ein eigenes Dataclass, aber Teil des Contact-Dokuments
2. **Optional Fields:** `phone`, `address` können fehlen
3. **Enums:** `ContactCategory` für Type-Safety
4. **Computed Properties:** `full_name` berechnet aus first + last
5. **Default Factories:** Listen und Timestamps werden automatisch initialisiert

Client-Implementierung mit Volltext-Suche

```
# examples/03_contact_manager/themis_client.py

from themisdb import Client
from models import Contact, ContactCategory
from typing import List, Optional
import uuid

class ContactClient:
    def __init__(self, host="localhost", port=8765):
        self.client = Client(host, port)
        self._setup_database()

    def _setup_database(self):
        """Erstellt Collection und Indizes"""
        # Collection erstellen (Document Model!)
        try:
            self.client.create_collection("contacts", type="document")
        except:
            pass

        # Indizes für schnelle Suche
        self.client.create_index("contacts", "email_idx", ["email"])
        self.client.create_index("contacts", "category_idx", ["category"])
        self.client.create_index("contacts", "favorite_idx", ["is_favorite"])

        # FULLTEXT Index für Suche!
        self.client.create_index(
            "contacts",
            "fulltext_idx",
            ["first_name", "last_name", "email", "notes"],
            type="fulltext"
        )

    def create_contact(self, contact: Contact) -> bool:
        """Erstellt einen neuen Kontakt"""
        try:
            self.client.insert("contacts", contact.to_dict())
            return True
```

```
except Exception as e:
    print(f"Error creating contact: {e}")
    return False

def search_contacts(self, query: str) -> List[Contact]:
    """Volltext-Suche über alle Felder"""
    aql = """
    FOR contact IN contacts
        FILTER CONTAINS(LOWER(contact.first_name), LOWER(@query))
            OR CONTAINS(LOWER(contact.last_name), LOWER(@query))
            OR CONTAINS(LOWER(contact.email), LOWER(@query))
            OR CONTAINS(LOWER(contact.notes), LOWER(@query))
        SORT contact.last_name ASC, contact.first_name ASC
    RETURN contact
    """
    try:
        results = self.client.query(aql, {"query": query})
        return [Contact.from_dict(data) for data in results]
    except Exception as e:
        print(f"Error searching contacts: {e}")
        return []

def get_favorites(self) -> List[Contact]:
    """Alle Favoriten-Kontakte"""
    aql = """
    FOR contact IN contacts
        FILTER contact.is_favorite == true
        SORT contact.last_name ASC
    RETURN contact
    """
    try:
        results = self.client.query(aql)
        return [Contact.from_dict(data) for data in results]
    except Exception as e:
        return []

def get_by_category(self, category: ContactCategory) -> List[Contact]:
    """Kontakte nach Kategorie filtern"""
```



```
aql = """
FOR contact IN contacts
  FILTER contact.category == @category
  SORT contact.last_name ASC
  RETURN contact
"""

try:
    results = self.client.query(aql, {"category": category.value})
    return [Contact.from_dict(data) for data in results]
except Exception as e:
    return []

def export_to_json(self, filename: str) -> bool:
    """Exportiert alle Kontakte als JSON"""
    aql = "FOR contact IN contacts RETURN contact"
    try:
        contacts = self.client.query(aql)
        import json
        with open(filename, 'w', encoding='utf-8') as f:
            json.dump(contacts, f, indent=2, ensure_ascii=False)
        return True
    except Exception as e:
        print(f"Export failed: {e}")
        return False

def import_from_json(self, filename: str) -> int:
    """Importiert Kontakte aus JSON"""
    import json
    try:
        with open(filename, 'r', encoding='utf-8') as f:
            contacts = json.load(f)

        count = 0
        for contact_data in contacts:
            contact = Contact.from_dict(contact_data)
            if self.create_contact(contact):
                count += 1
```

```
        return count
    except Exception as e:
        print(f"Import failed: {e}")
    return 0
```

Dokument-Modell in Aktion

Nested Queries:

```
# Finde alle Kontakte in Berlin
aql = """
FOR contact IN contacts
    FILTER contact.address != null
        AND contact.address.city == "Berlin"
    RETURN contact
"""
```

Array Operations:

```
# Finde Kontakte mit Tag "important"
aql = """
FOR contact IN contacts
    FILTER "important" IN contact.tags
    RETURN contact
"""
```

Schema Evolution ohne Migration:

```
# Neue Felder einfach hinzufügen!
contact = {
    "first_name": "Alice",
    "email": "alice@example.com",
    "social_media": { # NEU: Einfach hinzugefügt
        "twitter": "@alice",
        "linkedin": "alice-smith"
    },
    "birthday": "1990-03-15" # NEU: Auch einfach hinzugefügt
}

# Alte Kontakte haben diese Felder nicht - kein Problem!
# Keine Migration nötig!
```

3.4 Multi-Model Kombinationen

Beispiel 1: E-Commerce mit allen Modellen

```
# 1. RELATIONAL: Orders (feste Struktur, ACID wichtig)
```

```
order = {  
  "order_id": "0123",  
  "user_id": "alice",  
  "total": 99.99,  
  "status": "paid"  
}
```

```
# 2. DOCUMENT: Products (flexible Specs)
```

```
product = {  
  "sku": "LAPTOP-X1",  
  "name": "Laptop",  
  "specs": { # Jedes Produkt hat andere Specs!  
    "cpu": "i7",  
    "ram": 32  
  }  
}
```

```
# 3. GRAPH: Recommendations (Beziehungen)
```

```
# User → BOUGHT → Product
```

```
# Product → SIMILAR_TO → Product
```

```
# 4. VECTOR: Semantic Search
```

```
# Product hat embedding für "finde ähnliche Produkte"
```

```
# KOMBINIERTE QUERY:
```

```
"""
```

```
# Finde Produkte, die:
```

```
# - Freunde gekauft haben (GRAPH)
```

```
# - Ähnlich zu meinem letzten Kauf sind (VECTOR)
```

```
# - In meiner Preisklasse liegen (RELATIONAL)
```

```
FOR friend IN 1..2 OUTBOUND @userId friends
```

```
  FOR order IN orders
```

```
    FILTER order.user_id == friend.user_id
```

```
  FOR product IN products
```

```
    FILTER product.sku == order.sku
```

```
    LET similarity = COSINE_SIMILARITY(
```

```
        product.embedding,  
        @my_last_product_embedding  
    )  
    FILTER similarity > 0.7  
        AND product.price < 1000  
    RETURN DISTINCT product  
"""
```

Beispiel 2: CRM System

```
# RELATIONAL: Transaktionen
transactions = {
  "transaction_id": "T123",
  "customer_id": "C456",
  "amount": 5000.00,
  "status": "completed"
}

# DOCUMENT: Customer Profiles (flexibel)
customer = {
  "customer_id": "C456",
  "company": "ACME Corp",
  "contacts": [ # Array von Kontakten
    {"name": "John", "role": "CEO"},
    {"name": "Jane", "role": "CTO"}
  ],
  "metadata": { # Flexible metadata
    "industry": "tech",
    "size": "medium"
  }
}

# GRAPH: Relationships
# Company → HAS_EMPLOYEE → Person
# Company → PARTNER_WITH → Company

# VECTOR: Find similar customers
# Based on interaction history embeddings
```

3.5 Best Practices

1. Start Simple, Add Complexity

```
# Phase 1: Starte mit einem Modell
# Kontakte als Dokumente

# Phase 2: Füge Beziehungen hinzu
# Kontakte → WORKS_WITH → Kontakte (Graph)

# Phase 3: Füge Suche hinzu
# Embeddings für Semantic Search (Vector)
```

2. Denormalisierung ist OK

Im Dokument-Modell ist Duplikation oft besser als Joins:

```
# ❌ Normalized (viele Joins nötig)
order = {"order_id": "0123", "user_id": "alice"}
user = {"user_id": "alice", "name": "Alice", "email": "..."}

# ✅ Denormalized (alles in einem Dokument)
order = {
  "order_id": "0123",
  "user": { # User-Daten dupliziert
    "user_id": "alice",
    "name": "Alice",
    "email": "alice@example.com"
  }
}

# Vorteil: Ein Query, keine Joins!
```


3. Use Computed Properties

```
@property
def age(self):
    """Berechnet Alter aus Geburtstag"""
    if not self.birthday:
        return None
    today = datetime.now()
    return today.year - self.birthday.year

# Nicht speichern, berechnen!
# Daten sind immer aktuell
```

4. Index Strategically

Nicht jedes Feld braucht einen Index:

```
# ✅ Index: Oft gesucht/gefiltert
client.create_index("contacts", "email_idx", ["email"])

# ❌ Kein Index: Selten gesucht
# notes braucht keinen Index (außer Fulltext)
```

3.6 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- ✅ **Vier Modelle:** Wann Relational, Graph, Dokument oder Vektor
- ✅ **Entscheidungskriterien:** Die 5 Fragen-Methode
- ✅ **Kontaktmanager:** Vollständige Dokument-Modell Demo
- ✅ **Multi-Model Kombination:** Verschiedene Modelle zusammen nutzen
- ✅ **Best Practices:** Denormalisierung, Computed Properties, Indexing

Key Takeaways

1. **Es gibt kein "bestes" Modell** - nur das beste für Ihren Use Case
2. **Kombinieren ist Stärke** - ThemisDB glänzt bei Multi-Model
3. **Schema-Flexibilität** - Dokumente sind perfekt für Evolution
4. **Performance** - Wählen Sie das Modell nach Query-Patterns

Nächste Schritte

Im nächsten Kapitel lernen Sie, wie Sie ThemisDB installieren, konfigurieren und für Production vorbereiten.

[Kapitel 4: Installation & Setup →](#)

Weiterführende Ressourcen

- **Complete Example:** [examples/03_contact_manager](#)
 - **Tutorial:** [Contact Manager Tutorial](#)
 - **Document Model Docs:** [../de/architecture/architecture_content.md](#)
 - **Multi-Model Patterns:** [Kapitel 13 - AQL Mastery](#)
-

Kapitel 3 von 30 | Teil I: Grundlagen | ~7.500 Wörter

Kapitel 5: Relationale Daten

"Relational databases are like spreadsheets that can talk to each other - but with ACID guarantees and without the chaos."

Überblick

Relationale Datenbanken sind das Rückgrat der IT seit über 40 Jahren. In ThemisDB ist das relationale Modell eines von vier gleichberechtigten Datenmodellen - aber mit vollem AQL-Support und ACID-Garantien.

Was Sie in diesem Kapitel lernen werden: - Relationales Datenmodell: Tabellen, Schemas, Constraints - AQL in ThemisDB: DDL, DML, Joins, Subqueries - Normalisierung vs. Denormalisierung - Transaktionen und Integrität - Indexes und Performance - **Praxisbeispiel 1:** Inventory System (Lagerverwaltung) - **Praxisbeispiel 2:** Expense Tracker (Ausgabenverwaltung)

Voraussetzungen: AQL-Grundkenntnisse hilfreich, aber nicht notwendig.

5.1 Das Relationale Modell

Grundkonzepte

Tabelle (Table): Sammlung von Zeilen mit gleichem Schema

products (Tabelle)

+-----+				
id	name	price	stock	
+-----+				
1	Laptop	1200	15	
2	Mouse	25	100	
3	Keyboard	80	50	
+-----+				

Zeile (Row): Ein Datensatz in einer Tabelle

Spalte (Column): Ein Attribut, das jede Zeile hat

Schema: Definition der Spalten und Typen

Primärschlüssel (Primary Key): Eindeutige ID für jede Zeile

Fremdschlüssel (Foreign Key): Referenz zu einer anderen Tabelle

Warum Relational?

✓ **Vorteile:** - **Struktur:** Klare, vorhersehbare Datenstruktur - **Integrität:** Constraints garantieren Datenqualität - **Joins:** Verknüpfe Daten aus mehreren Tabellen - **ACID:** Transaktionen mit vollständiger Konsistenz - **AQL:** Standardisierte, mächtige Abfragesprache

✗ **Nachteile:** - Rigid: Schema-Änderungen erfordern Migrations - Komplex: Normalisierung kann zu vielen Tables führen - Performance: Joins können langsam sein bei vielen Tables - Skalierung: Vertical Scaling bevorzugt

Wann Relational wählen?

Perfekt für: - ✓ Strukturierte Business-Daten (Orders, Invoices, Inventory) - ✓ Daten mit klaren Beziehungen (Customers → Orders → Items) - ✓ Transaktionale Systeme (Banking, E-Commerce) - ✓ Reports mit Aggregationen (SUM, AVG, GROUP BY) - ✓ Datenintegrität ist kritisch

Weniger geeignet für: - ✗ Hochflexible Schemas (Metadata, User-Generated Content) - ✗ Netzwerk-Daten mit vielen Hops (Social Graphs) - ✗ Unstrukturierte Daten (Logs, Dokumente) - ✗ Vektor-Daten (Embeddings, Features)

5.2 Tabellen und Schemas

Tabelle erstellen

```
-- Basic Table
CREATE TABLE products (
  id INT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  price DECIMAL(10, 2) NOT NULL,
  stock INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT NOW()
);
```

Datentypen in ThemisDB:

Typ	Beschreibung	Beispiel
INT	Ganzzahl	42, -17, 0
BIGINT	Große Ganzzahl	9223372036854775807
DECIMAL(p,s)	Festkommazahl	123.45
FLOAT/DOUBLE	Fließkommazahl	3.14159
VARCHAR(n)	Variable Zeichenkette	"Hello World"
TEXT	Unbegrenzte Zeichenkette	Langer Text
BOOLEAN	Wahrheitswert	true, false
TIMESTAMP	Zeitstempel	2025-12-28 10:30:00
DATE	Datum	2025-12-28
JSON	JSON-Dokument	{"key": "value"}

Constraints

Primary Key:

```
CREATE TABLE customers (  
  id INT PRIMARY KEY,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  name VARCHAR(255) NOT NULL  
);
```

Foreign Key:

```
CREATE TABLE orders (  
  id INT PRIMARY KEY,  
  customer_id INT NOT NULL,  
  total DECIMAL(10, 2) NOT NULL,  
  created_at TIMESTAMP DEFAULT NOW(),  
  
  FOREIGN KEY (customer_id) REFERENCES customers(id)  
);
```

Check Constraints:

```
CREATE TABLE products (  
  id INT PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  price DECIMAL(10, 2) NOT NULL CHECK (price >= 0),  
  stock INT NOT NULL CHECK (stock >= 0),  
  discount DECIMAL(3, 2) CHECK (discount BETWEEN 0 AND 1)  
);
```

Unique Constraints:

```
CREATE TABLE users (  
  id INT PRIMARY KEY,  
  username VARCHAR(50) UNIQUE NOT NULL,  
  email VARCHAR(255) UNIQUE NOT NULL  
);
```

Auto-Increment IDs

```
CREATE TABLE products (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(255) NOT NULL  
);  
  
-- Insert ohne ID  
INSERT INTO products (name) VALUES ('Laptop');  
-- → Automatisch id=1  
  
INSERT INTO products (name) VALUES ('Mouse');  
-- → Automatisch id=2
```

5.3 Daten einfügen, ändern, löschen

INSERT

```
-- Single Document  
INSERT {  
  id: 1,  
  name: 'Laptop',  
  price: 1200.00,  
  stock: 15  
} INTO products  
  
-- Multiple Documents  
INSERT [  
  {id: 2, name: 'Mouse', price: 25.00, stock: 100},  
  {id: 3, name: 'Keyboard', price: 80.00, stock: 50},  
  {id: 4, name: 'Monitor', price: 350.00, stock: 20}  
] INTO products
```


UPDATE

```
-- Update single document
UPDATE {id: 1} WITH {stock: stock - 1} IN products

-- Update multiple documents (with FOR loop)
FOR product IN products
  FILTER product.stock > 50
  UPDATE product WITH {
    price: product.price * 0.9
  } IN products

-- Update with join data
FOR order_item IN order_items
  FOR product IN products
    FILTER order_item.product_id == product.id
    UPDATE order_item WITH {
      price: product.price
    } IN order_items
```

REMOVE (DELETE)

```
-- Remove single document
REMOVE {id: 1} IN products

-- Remove multiple documents
FOR product IN products
  FILTER product.stock == 0
  REMOVE product IN products

-- Remove all (Vorsicht!)
FOR product IN products
  REMOVE product IN products
```

UPSERT (Insert or Update)

```
-- Insert or Update based on key
UPSERT {id: 1}
  INSERT {
    id: 1,
    name: 'Laptop',
    price: 1200.00,
    stock: 15
  }
  UPDATE {
    price: 1200.00,
    stock: 15
  }
IN products
```

5.4 Queries und Joins

Query Basics

```
-- All columns
FOR product IN products
  RETURN product

-- Specific columns
FOR product IN products
  RETURN {name: product.name, price: product.price}

-- With FILTER
FOR product IN products
  FILTER product.price > 100
  RETURN product

-- With SORT and LIMIT
FOR product IN products
  SORT product.price DESC
  LIMIT 10
  RETURN product

-- With aggregation functions
FOR product IN products
  COLLECT AGGREGATE
    avgPrice = AVG(product.price),
    maxPrice = MAX(product.price),
    minPrice = MIN(product.price),
    total = COUNT()
  RETURN {avgPrice, maxPrice, minPrice, total}

-- With GROUP BY (COLLECT)
FOR product IN products
  COLLECT category = product.category
  AGGREGATE
    count = COUNT(),
    avgPrice = AVG(product.price)
  RETURN {category, count, avgPrice}
```

INNER JOIN

```
-- Orders mit Customer-Namen (Multi-FOR Join)
FOR order IN orders
  FOR customer IN customers
    FILTER order.customer_id == customer.id
  RETURN {
    id: order.id,
    total: order.total,
    customer_name: customer.name
  }
```

Visualisierung:

customers		orders	
+-----+		+-----+	
id=1	←-----	cust_id=1	✓ Match → returned
Alice		€100	
+-----+		+-----+	
id=2		cust_id=1	✓ Match → returned
Bob		€50	
+-----+		+-----+	
id=3			
Carol			→ Kein Match, nicht returned
+-----+			

LEFT JOIN (Outer Join with COLLECT)

```
-- Alle Customers, mit/ohne Orders
FOR customer IN customers
  LET customer_orders = (
    FOR order IN orders
      FILTER order.customer_id == customer.id
      RETURN order
  )
  RETURN {
    name: customer.name,
    order_count: LENGTH(customer_orders),
    total_spent: SUM(customer_orders[*].total)
  }
```

Visualisierung:

customers	orders
+-----+	+-----+
id=1	←----- cust_id=1 ✓ Match
Alice	€100
+-----+	+-----+
id=2	
Bob	→ Kein Order, aber returned mit NULL
+-----+	

Multi-Table JOIN

```
-- Orders mit Items und Products (Multi-FOR Join)
FOR order IN orders
  FILTER order.created_at >= '2025-01-01'
  FOR customer IN customers
    FILTER order.customer_id == customer.id
    FOR order_item IN order_items
      FILTER order_item.order_id == order.id
      FOR product IN products
        FILTER order_item.product_id == product.id
      RETURN {
        order_id: order.id,
        customer: customer.name,
        product: product.name,
        quantity: order_item.quantity,
        price: order_item.price
      }
```

Subqueries

```
-- Products teurer als Durchschnitt (WITH CTE)
LET avgPrice = (
  FOR product IN products
    COLLECT AGGREGATE avg = AVG(product.price)
    RETURN avg
)[0]

FOR product IN products
  FILTER product.price > avgPrice
  RETURN {name: product.name, price: product.price}

-- Customers mit mehr als 3 Orders (Subquery)
LET active_customers = (
  FOR order IN orders
    COLLECT customer_id = order.customer_id
    AGGREGATE order_count = COUNT()
    FILTER order_count > 3
    RETURN customer_id
)

FOR customer IN customers
  FILTER customer.id IN active_customers
  RETURN customer.name
```

5.5 Transaktionen

ACID Garantien

Atomicity: Alles oder nichts

Consistency: Constraints werden eingehalten

Isolation: Transaktionen sehen sich nicht gegenseitig

Durability: Commits sind permanent

Transaction Beispiel

```
from themis_client import ThemisDB

db = ThemisDB("localhost:8765")

# Start Transaction
tx = db.begin_transaction()

try:
    # 1. Reduce stock
    tx.execute("""
        FOR product IN products
        FILTER product.id == @product_id AND product.stock > 0
        UPDATE product WITH {stock: product.stock - 1} IN products
    """, {"product_id": product_id})

    # 2. Create order
    tx.execute("""
        INSERT {
            customer_id: @customer_id,
            product_id: @product_id,
            quantity: 1,
            price: @price
        } INTO orders
    """, {"customer_id": customer_id, "product_id": product_id, "price": price})

    # 3. Charge customer
    tx.execute("""
        FOR customer IN customers
        FILTER customer.id == @customer_id AND customer.balance >= @price
        UPDATE customer WITH {balance: customer.balance - @price} IN customers
    """, {"customer_id": customer_id, "price": price})

    # All OK → Commit
    tx.commit()
    print("Order successful!")

except Exception as e:
    # Error → Rollback
```

```
tx.rollback()  
print(f"Order failed: {e}")
```

Was passiert intern:

```
T1: BEGIN TRANSACTION (snapshot @ version 100)  
  → UPDATE products ... (write intent @ version 101)  
  → INSERT orders ... (write intent @ version 102)  
  → UPDATE customers ... (write intent @ version 103)  
  → COMMIT  
  → All write intents become visible @ version 104
```

Falls Fehler:

```
  → ROLLBACK  
  → All write intents discarded  
  → Daten unverändert
```

Isolation Levels

ThemisDB verwendet **Snapshot Isolation**:

```
# Transaction 1
tx1 = db.begin_transaction()
tx1.execute("FOR p IN products FILTER p.id == 1 UPDATE p WITH {price: 100} IN products")

# Transaction 2 (parallel)
tx2 = db.begin_transaction()
result = tx2.execute("FOR p IN products FILTER p.id == 1 RETURN p.price")
print(result) # → Alter Wert! (z.B. 90)

# TX1 committed
tx1.commit()

# TX2 sieht immernoch alten Wert (Snapshot Isolation)
result = tx2.execute("FOR p IN products FILTER p.id == 1 RETURN p.price")
print(result) # → Immernoch 90!

tx2.commit()

# Neue Transaction sieht neuen Wert
tx3 = db.begin_transaction()
result = tx3.execute("FOR p IN products FILTER p.id == 1 RETURN p.price")
print(result) # → 100
```

5.6 Normalisierung

Problem: Redundanz

Nicht-normalisiert:

orders

id	customer	cust_email	product	quantity	price
1	Alice	a@email.com	Laptop	1	1200
2	Alice	a@email.com	Mouse	2	25
3	Bob	b@email.com	Keyboard	1	80

Probleme: - Customer email wird wiederholt (Update Anomaly) - Product-Namen inkonsistent ("Laptop" vs "laptop") - Keine Constraints möglich

Normalisierung: 1NF, 2NF, 3NF

1. Normal Form (1NF): Atomare Werte, keine Arrays

```
-- ✗ Nicht 1NF
CREATE TABLE orders (
  id INT,
  products TEXT -- "Laptop, Mouse, Keyboard"
);

-- ✓ 1NF
CREATE TABLE orders (
  id INT,
  product_id INT
);
```

2. Normal Form (2NF): Alle Nicht-Key-Attribute hängen vom ganzen Key ab

```
-- ❌ Nicht 2NF
CREATE TABLE order_items (
  order_id INT,
  product_id INT,
  quantity INT,
  product_name VARCHAR(255), -- Hängt nur von product_id ab!
  PRIMARY KEY (order_id, product_id)
);

-- ✅ 2NF: Separate products table
CREATE TABLE products (
  id INT PRIMARY KEY,
  name VARCHAR(255)
);

CREATE TABLE order_items (
  order_id INT,
  product_id INT,
  quantity INT,
  PRIMARY KEY (order_id, product_id),
  FOREIGN KEY (product_id) REFERENCES products(id)
);
```

3. Normal Form (3NF): Keine transitiven Abhängigkeiten

```
-- ❌ Nicht 3NF
CREATE TABLE products (
  id INT PRIMARY KEY,
  category_id INT,
  category_name VARCHAR(255) -- Hängt von category_id ab!
);

-- ✅ 3NF: Separate categories table
CREATE TABLE categories (
  id INT PRIMARY KEY,
  name VARCHAR(255)
);

CREATE TABLE products (
  id INT PRIMARY KEY,
  category_id INT,
  FOREIGN KEY (category_id) REFERENCES categories(id)
);
```

Denormalisierung für Performance

Manchmal ist **bewusste** Denormalisierung sinnvoll:

```
-- Denormalisiert für schnelle Queries
CREATE TABLE order_summary (
  order_id INT PRIMARY KEY,
  customer_id INT,
  customer_name VARCHAR(255), -- Denormalisiert!
  item_count INT,             -- Computed!
  total DECIMAL(10, 2),       -- Computed!
  created_at TIMESTAMP
);
```

Trade-off: - ✅ Queries sind schneller (kein JOIN nötig) - ❌ Updates sind komplexer (mehrere Tables updaten) - ❌ Mehr Storage (Redundanz)

5.7 Indexes

Warum Indexes?

Ohne Index:

```
FOR product IN products
  FILTER product.name == 'Laptop'
  RETURN product
-- → Scannt ALLE Zeilen (Seq Scan):  $O(n)$ 
```

Mit Index:

```
CREATE INDEX idx_products_name ON products(name);

FOR product IN products
  FILTER product.name == 'Laptop'
  RETURN product
-- → Nutzt Index (Index Scan):  $O(\log n)$ 
```

Performance-Unterschied: - 1 Million rows: Seq Scan ~100ms, Index Scan ~0.1ms - **1000x schneller!**

Index-Arten

1. B-Tree Index (Standard):


```
CREATE INDEX idx_products_price ON products(price);

-- Gut für:
FOR product IN products
  FILTER product.price > 100
  RETURN product

FOR product IN products
  FILTER product.price >= 50 AND product.price <= 150
  RETURN product

FOR product IN products
  SORT product.price
  RETURN product
```

2. Unique Index:

```
CREATE UNIQUE INDEX idx_users_email ON users(email);

-- Verhindert Duplikate automatisch
INSERT {email: 'test@example.com'} INTO users -- OK
INSERT {email: 'test@example.com'} INTO users -- ERROR!
```

3. Composite Index:

```
CREATE INDEX idx_orders_cust_date ON orders(customer_id, created_at);

-- Gut für:
FOR order IN orders
  FILTER order.customer_id == 123 AND order.created_at > '2025-01-01'
  RETURN order

FOR order IN orders
  FILTER order.customer_id == 123
  RETURN order -- Auch OK!

-- Nicht gut für:
FOR order IN orders
  FILTER order.created_at > '2025-01-01'
  RETURN order -- Index nicht nutzbar!
```

4. Partial Index:

```
CREATE INDEX idx_orders_pending ON orders(created_at)
WHERE status = 'pending';

-- Kleiner Index nur für pending orders
FOR order IN orders
  FILTER order.status == 'pending' AND order.created_at > '2025-01-01'
  RETURN order
```

Index Best Practices

✅ **Index erstellen für:** - Primary Keys (automatisch) - Foreign Keys (manuell) - WHERE-Spalten in häufigen Queries - ORDER BY-Spalten - JOIN-Spalten

❌ **Zu viele Indexes vermeiden:** - Jeder Index verlangsamt INSERT/UPDATE/DELETE - Jeder Index braucht Storage - **Faustregel:** Max 5-7 Indexes pro Table

5.8 Praxisbeispiel 1: Inventory System

Überblick

Ein **Lagerverwaltungssystem** für ein kleines bis mittelgroßes Unternehmen: - Products mit Categories - Warehouses (mehrere Standorte) - Stock Levels pro Warehouse - Stock Movements (Ein-/Ausgang) - Suppliers

Location: `examples/04_inventory_system/`

Datenmodell

```
-- Categories
CREATE TABLE categories (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255) NOT NULL UNIQUE,
  description TEXT
);

-- Products
CREATE TABLE products (
  id INT PRIMARY KEY AUTO_INCREMENT,
  sku VARCHAR(50) NOT NULL UNIQUE,
  name VARCHAR(255) NOT NULL,
  description TEXT,
  category_id INT NOT NULL,
  unit_price DECIMAL(10, 2) NOT NULL CHECK (unit_price >= 0),
  min_stock_level INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT NOW(),

  FOREIGN KEY (category_id) REFERENCES categories(id),
  INDEX idx_products_category (category_id),
  INDEX idx_products_sku (sku)
);

-- Warehouses
CREATE TABLE warehouses (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255) NOT NULL,
  location VARCHAR(255),
  capacity INT
);

-- Stock Levels
CREATE TABLE stock_levels (
  product_id INT NOT NULL,
  warehouse_id INT NOT NULL,
  quantity INT NOT NULL DEFAULT 0 CHECK (quantity >= 0),
  last_updated TIMESTAMP DEFAULT NOW(),
```

```
PRIMARY KEY (product_id, warehouse_id),
FOREIGN KEY (product_id) REFERENCES products(id),
FOREIGN KEY (warehouse_id) REFERENCES warehouses(id)
);

-- Stock Movements
CREATE TABLE stock_movements (
  id INT PRIMARY KEY AUTO_INCREMENT,
  product_id INT NOT NULL,
  warehouse_id INT NOT NULL,
  quantity INT NOT NULL, -- Positive = Eingang, Negative = Ausgang
  movement_type ENUM('purchase', 'sale', 'transfer', 'adjustment'),
  reference VARCHAR(255),
  created_at TIMESTAMP DEFAULT NOW(),

  FOREIGN KEY (product_id) REFERENCES products(id),
  FOREIGN KEY (warehouse_id) REFERENCES warehouses(id),
  INDEX idx_movements_product (product_id),
  INDEX idx_movements_warehouse (warehouse_id),
  INDEX idx_movements_date (created_at)
);

-- Suppliers
CREATE TABLE suppliers (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255) NOT NULL,
  contact_email VARCHAR(255),
  contact_phone VARCHAR(50)
);

-- Product Suppliers (Many-to-Many)
CREATE TABLE product_suppliers (
  product_id INT NOT NULL,
  supplier_id INT NOT NULL,
  supplier_sku VARCHAR(50),
  unit_cost DECIMAL(10, 2),

  PRIMARY KEY (product_id, supplier_id),
```

```
FOREIGN KEY (product_id) REFERENCES products(id),  
FOREIGN KEY (supplier_id) REFERENCES suppliers(id)  
);
```

Häufige Queries

1. Total Stock pro Product:

```
def get_total_stock(product_id):  
    return db.query("""  
        SELECT  
            p.name,  
            SUM(sl.quantity) AS total_stock  
        FROM products p  
        LEFT JOIN stock_levels sl ON p.id = sl.product_id  
        WHERE p.id = ?  
        GROUP BY p.id, p.name  
        """, [product_id])
```

2. Low Stock Alert:

```
def get_low_stock_products():  
    return db.query("""  
        SELECT  
            p.sku,  
            p.name,  
            SUM(sl.quantity) AS total_stock,  
            p.min_stock_level  
        FROM products p  
        LEFT JOIN stock_levels sl ON p.id = sl.product_id  
        GROUP BY p.id  
        HAVING SUM(sl.quantity) < p.min_stock_level  
        ORDER BY total_stock ASC  
        """)
```

3. Stock Movement History:

```
def get_movement_history(product_id, days=30):
    return db.query("""
        SELECT
            sm.created_at,
            sm.movement_type,
            sm.quantity,
            w.name AS warehouse,
            sm.reference
        FROM stock_movements sm
        JOIN warehouses w ON sm.warehouse_id = w.id
        WHERE sm.product_id = ?
            AND sm.created_at >= DATE_SUB(NOW(), INTERVAL ? DAY)
        ORDER BY sm.created_at DESC
    """, [product_id, days])
```


Stock Movement Transaction

```
def record_purchase(product_id, warehouse_id, quantity, supplier_id, cost):
    """Purchase new stock from supplier"""
    tx = db.begin_transaction()
    try:
        # 1. Record movement
        tx.execute("""
            INSERT INTO stock_movements
            (product_id, warehouse_id, quantity, movement_type, reference)
            VALUES (?, ?, ?, 'purchase', ?)
            """, [product_id, warehouse_id, quantity, f"PO-{supplier_id}-{datetime.now().timestamp()}"])

        # 2. Update stock level
        tx.execute("""
            INSERT INTO stock_levels (product_id, warehouse_id, quantity)
            VALUES (?, ?, ?)
            ON CONFLICT (product_id, warehouse_id) DO UPDATE
            SET
                quantity = stock_levels.quantity + EXCLUDED.quantity,
                last_updated = NOW()
            """, [product_id, warehouse_id, quantity])

        # 3. Update product cost (optional)
        tx.execute("""
            UPDATE products
            SET unit_price = ?
            WHERE id = ?
            """, [cost, product_id])

        tx.commit()
        return {"success": True}

    except Exception as e:
        tx.rollback()
        return {"success": False, "error": str(e)}
```

Reporting: Value per Warehouse

```
SELECT
  w.name AS warehouse,
  COUNT(DISTINCT sl.product_id) AS product_count,
  SUM(sl.quantity) AS total_units,
  SUM(sl.quantity * p.unit_price) AS total_value
FROM warehouses w
LEFT JOIN stock_levels sl ON w.id = sl.warehouse_id
LEFT JOIN products p ON sl.product_id = p.id
GROUP BY w.id, w.name
ORDER BY total_value DESC;
```

Output:

```
+-----+-----+-----+-----+
| warehouse      | product_count | total_units | total_value |
+-----+-----+-----+-----+
| Main Warehouse  |          250 |      5,420 |   €125,000  |
| Storage Berlin  |          180 |      3,210 |    €78,500  |
| Distribution Hub |          120 |      1,890 |    €42,300  |
+-----+-----+-----+-----+
```

5.9 Praxisbeispiel 2: Expense Tracker

Überblick

Ein **Ausgabenverwaltungssystem** für persönliche Finanzen oder kleine Teams: - Expenses mit Categories
- Budgets pro Category - Recurring Expenses - Multi-Currency Support - Reports und Analytics

Location: [examples/12_expense_tracker/](#)

Datenmodell

```
-- Categories
CREATE TABLE expense_categories (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL UNIQUE,
  color VARCHAR(7), -- Hex color #FF5733
  icon VARCHAR(50)
);

-- Expenses
CREATE TABLE expenses (
  id INT PRIMARY KEY AUTO_INCREMENT,
  description VARCHAR(255) NOT NULL,
  amount DECIMAL(10, 2) NOT NULL CHECK (amount > 0),
  currency VARCHAR(3) DEFAULT 'EUR',
  category_id INT NOT NULL,
  expense_date DATE NOT NULL,
  payment_method ENUM('cash', 'credit_card', 'debit_card', 'transfer') DEFAULT 'cash',
  receipt_url VARCHAR(500),
  notes TEXT,
  created_at TIMESTAMP DEFAULT NOW(),

  FOREIGN KEY (category_id) REFERENCES expense_categories(id),
  INDEX idx_expenses_date (expense_date),
  INDEX idx_expenses_category (category_id)
);

-- Budgets
CREATE TABLE budgets (
  id INT PRIMARY KEY AUTO_INCREMENT,
  category_id INT NOT NULL,
  amount DECIMAL(10, 2) NOT NULL CHECK (amount > 0),
  period ENUM('daily', 'weekly', 'monthly', 'yearly') DEFAULT 'monthly',
  start_date DATE NOT NULL,
  end_date DATE,

  FOREIGN KEY (category_id) REFERENCES expense_categories(id),
  UNIQUE KEY (category_id, start_date)
);
```

```
-- Recurring Expenses
CREATE TABLE recurring_expenses (
  id INT PRIMARY KEY AUTO_INCREMENT,
  description VARCHAR(255) NOT NULL,
  amount DECIMAL(10, 2) NOT NULL,
  category_id INT NOT NULL,
  frequency ENUM('daily', 'weekly', 'monthly', 'yearly') NOT NULL,
  start_date DATE NOT NULL,
  end_date DATE,
  last_generated DATE,

  FOREIGN KEY (category_id) REFERENCES expense_categories(id)
);
```

Häufige Queries

1. Expenses für aktuellen Monat:

```
def get_monthly_expenses(year, month):
    return db.query("""
        SELECT
            e.expense_date,
            e.description,
            e.amount,
            c.name AS category,
            e.payment_method
        FROM expenses e
        JOIN expense_categories c ON e.category_id = c.id
        WHERE YEAR(e.expense_date) = ?
            AND MONTH(e.expense_date) = ?
        ORDER BY e.expense_date DESC
    """, [year, month])
```

2. Budget Status:

```
def get_budget_status(year, month):
    return db.query("""
        SELECT
            c.name AS category,
            b.amount AS budget,
            COALESCE(SUM(e.amount), 0) AS spent,
            b.amount - COALESCE(SUM(e.amount), 0) AS remaining,
            (COALESCE(SUM(e.amount), 0) / b.amount * 100) AS percent_used
        FROM budgets b
        JOIN expense_categories c ON b.category_id = c.id
        LEFT JOIN expenses e ON e.category_id = c.category_id
            AND YEAR(e.expense_date) = ?
            AND MONTH(e.expense_date) = ?
        WHERE b.period = 'monthly'
            AND b.start_date <= ?
            AND (b.end_date IS NULL OR b.end_date >= ?)
        GROUP BY b.id, c.name, b.amount
    """, [year, month, f"{year}-{month:02d}-01", f"{year}-{month:02d}-01"])
```

3. Top Categories:

```
def get_top_categories(year):
    return db.query("""
        SELECT
            c.name,
            COUNT(e.id) AS expense_count,
            SUM(e.amount) AS total_amount
        FROM expense_categories c
        JOIN expenses e ON c.id = e.category_id
        WHERE YEAR(e.expense_date) = ?
        GROUP BY c.id, c.name
        ORDER BY total_amount DESC
        LIMIT 10
    """, [year])
```

Add Expense Transaction

```

def add_expense(description, amount, category_id, expense_date, payment_method):
    """Add new expense and check budget"""
    tx = db.begin_transaction()
    try:
        # 1. Insert expense
        result = tx.execute("""
            INSERT INTO expenses
            (description, amount, category_id, expense_date, payment_method)
            VALUES (?, ?, ?, ?, ?)
            """, [description, amount, category_id, expense_date, payment_method])

        expense_id = result.last_insert_id

        # 2. Check budget
        year, month = expense_date.year, expense_date.month
        budget_status = tx.query("""
            SELECT
                b.amount AS budget,
                COALESCE(SUM(e.amount), 0) AS spent
            FROM budgets b
            LEFT JOIN expenses e ON e.category_id = b.category_id
            AND YEAR(e.expense_date) = ?
            AND MONTH(e.expense_date) = ?
            WHERE b.category_id = ?
            AND b.period = 'monthly'
            GROUP BY b.amount
            """, [year, month, category_id])

        warning = None
        if budget_status and budget_status[0]['spent'] > budget_status[0]['budget']:
            warning = f"Budget exceeded! Spent: {budget_status[0]['spent']}, Budget: {budget_status[0]['budget']}"

        tx.commit()
        return {"success": True, "expense_id": expense_id, "warning": warning}

    except Exception as e:
        tx.rollback()
        return {"success": False, "error": str(e)}

```


Recurring Expenses Generation

```
def generate_recurring_expenses():
    """Generate expenses from recurring templates"""
    tx = db.begin_transaction()
    generated = []

    try:
        # Find due recurring expenses
        recurring = tx.query("""
            SELECT * FROM recurring_expenses
            WHERE (last_generated IS NULL OR last_generated < CURDATE())
            AND (end_date IS NULL OR end_date >= CURDATE())
            """)

        for rec in recurring:
            # Calculate next date
            next_date = calculate_next_date(rec['frequency'], rec['last_generated'] or rec['start_date'])

            if next_date <= datetime.now().date():
                # Generate expense
                tx.execute("""
                    INSERT INTO expenses
                    (description, amount, category_id, expense_date, payment_method)
                    VALUES (?, ?, ?, ?, 'transfer')
                    """, [rec['description'], rec['amount'], rec['category_id'], next_date])

                # Update last_generated
                tx.execute("""
                    UPDATE recurring_expenses
                    SET last_generated = ?
                    WHERE id = ?
                    """, [next_date, rec['id']])

                generated.append(rec['description'])

        tx.commit()
        return {"success": True, "generated": generated}

    except Exception as e:
```

```
tx.rollback()

return {"success": False, "error": str(e)}
```

Analytics: Monthly Trend

```
SELECT
  DATE_FORMAT(expense_date, '%Y-%m') AS month,
  COUNT(*) AS expense_count,
  SUM(amount) AS total_spent,
  AVG(amount) AS avg_expense
FROM expenses
WHERE expense_date >= DATE_SUB(CURDATE(), INTERVAL 12 MONTH)
GROUP BY month
ORDER BY month;
```

Output:

month	expense_count	total_spent	avg_expense
2024-01	45	€1,234	€27
2024-02	52	€1,456	€28
2024-03	48	€1,189	€25
...	...	...	...

5.10 Performance Best Practices

1. Use Indexes Wisely

```
-- ❌ Slow: No index
FOR expense IN expenses
  FILTER expense.expense_date > '2025-01-01'
  RETURN expense

-- ✅ Fast: With index
CREATE INDEX idx_expenses_date ON expenses(expense_date);
FOR expense IN expenses
  FILTER expense.expense_date > '2025-01-01'
  RETURN expense
```

2. Avoid Returning All Fields

```
-- ❌ Transfers mehr Daten als nötig
FOR product IN products
  RETURN product

-- ✅ Nur benötigte Felder
FOR product IN products
  RETURN {id: product.id, name: product.name, price: product.price}
```

3. Use LIMIT für große Result Sets

```
-- ❌ Könnte Millionen Rows returnen
FOR order IN orders
  SORT order.created_at DESC
  RETURN order

-- ✅ Pagination
FOR order IN orders
  SORT order.created_at DESC
  LIMIT 0, 50
  RETURN order
```

4. Batch Inserts

```
# ❌ Slow: Individual inserts
for product in products:
    db.execute("INSERT INTO products (name, price) VALUES (?, ?)",
               [product.name, product.price])

# ✅ Fast: Batch insert
db.execute("""
    INSERT INTO products (name, price) VALUES
    (?, ?), (?, ?), (?, ?), ...
    """, [p1.name, p1.price, p2.name, p2.price, ...])
```

5. Use Transactions für Bulk Operations

```
# ❌ Slow: Auto-commit nach jedem Statement
for i in range(1000):
    db.execute("INSERT INTO logs (...) VALUES (...)")

# ✅ Fast: Eine Transaction
tx = db.begin_transaction()
for i in range(1000):
    tx.execute("INSERT INTO logs (...) VALUES (...)")
tx.commit()
```

6. Analyze Query Plans

```
EXPLAIN SELECT * FROM orders o
JOIN customers c ON o.customer_id = c.id
WHERE o.created_at > '2025-01-01';
```

Output:

```
+---+-----+-----+-----+-----+
| id | table      | type  | key  | rows      |
+---+-----+-----+-----+-----+
|  1 | orders     | range | idx  | 1000      |
|  1 | customers  | ref   | pk   | 1         |
+---+-----+-----+-----+-----+
```

5.11 Erweiterte AQL-Optimierungen: Query-Plan und Performance-Tuning

5.11.1 Der Query-Optimizer: Kostenbasierte Umordnung

ThemisDB verwendet einen kostenbasierten Query-Optimizer [13], der den effizientesten Ausführungsplan für komplexe Queries automatisch wählt. Dies ist besonders wichtig bei Multi-Model-Queries, die relationale, graph- und vektor-basierte Operationen kombinieren.

Wie funktioniert der Optimizer?

```
// Pseudo-Code des ThemisDB Query Optimizers
class QueryOptimizer {
    // Kostenmodell für verschiedene Operationen
    map<OperationType, double> cost_estimates = {
        {INDEX_SCAN, 0.1},          //  $O(\log n)$  - sehr schnell
        {HASH_JOIN, 1.0},           //  $O(n + m)$  - schnell für Equi-Joins
        {NESTED_LOOP_JOIN, 10.0},   //  $O(n * m)$  - langsam, aber flexibel
        {FULL_TABLE_SCAN, 100.0},   //  $O(n)$  - teuer für große Tabellen
        {SORT, 5.0},                //  $O(n \log n)$  - mittel
        {AGGREGATE, 2.0}            //  $O(n)$  - linear
    };

    ExecutionPlan optimize(Query query) {
        // 1. Erzeuge alle möglichen Ausführungspläne
        vector<ExecutionPlan> candidates = generatePlans(query);

        // 2. Schätze Kosten für jeden Plan
        for (auto& plan : candidates) {
            plan.estimated_cost = estimateCost(plan);
        }

        // 3. Wähle Plan mit niedrigsten Kosten
        return *min_element(candidates.begin(), candidates.end(),
            [](auto& a, auto& b) { return a.estimated_cost < b.estimated_cost; }
        );
    }
};
```

Beispiel: Join-Reihenfolge-Optimierung

-- Diese Query hat mehrere mögliche Ausführungspläne:

```
FOR order IN orders
  FILTER order.status == "pending"
  FOR customer IN customers
    FILTER customer._id == order.customer_id
    FILTER customer.country == "DE"
    FOR product IN products
      FILTER product._id == order.product_id
      FILTER product.category == "electronics"
    RETURN {
      order: order,
      customer: customer.name,
      product: product.name
    }
```

Plan A: Naive Reihenfolge (schlecht)

1. Scan orders (1M rows) → 1M candidates
2. For each: Lookup customer → 1M lookups
3. Filter country="DE" → 100K candidates
4. For each: Lookup product → 100K lookups
5. Filter category → 10K final results

Total Cost: $1M + 1M + 100K + 100K = \sim 2.2M$ operations

Plan B: Optimizer-Reihenfolge (optimal)

1. Index scan: customers WHERE country="DE" → 100K candidates
2. Index scan: products WHERE category="electronics" → 50K candidates
3. Hash join: orders WHERE status="pending" (200K) with customers → 20K matches
4. Hash join: result with products → 10K final results

Total Cost: $100K + 50K + 200K + 20K = \sim 370K$ operations (6x schneller!)

Wie wird der optimale Plan gewählt?

Der Optimizer verwendet **Statistiken** über die Daten:

```
-- Statistiken für Optimizer sammeln
ANALYZE TABLE orders;
ANALYZE TABLE customers;
ANALYZE TABLE products;

-- Zeigt:
-- - Anzahl Zeilen pro Tabelle
-- - Kardinalität von Indizes
-- - Häufigkeitsverteilung von Werten
-- - Größe der Tabellen auf Disk
```

5.11.2 Index-Only-Scans: Minimale Disk-I/O

Ein **Index-Only-Scan** ist möglich, wenn alle benötigten Spalten bereits im Index enthalten sind, ohne dass die eigentliche Tabelle gelesen werden muss [2], [3].

Beispiel ohne Index-Only-Scan:

```
CREATE INDEX idx_orders_customer ON orders (customer_id);

-- Diese Query MUSS die orders-Tabelle lesen:
FOR order IN orders
  FILTER order.customer_id == "customer_123"
  RETURN {
    id: order._id,
    total: order.total_amount,    -- Nicht im Index!
    date: order.created_at       -- Nicht im Index!
  }
```

Ausführungsplan:

1. Index Scan: idx_orders_customer → Findet 50 order_ids
2. Table Lookup: orders → Liest 50 volle Dokumente (Disk I/O!)

Optimiert mit Composite Index:

```
-- Composite Index mit allen benötigten Spalten
CREATE INDEX idx_orders_customer_covering
  ON orders (customer_id, total_amount, created_at);

-- Jetzt: Index-Only-Scan möglich!
FOR order IN orders
  FILTER order.customer_id == "customer_123"
  RETURN {
    id: order._id,
    total: order.total_amount,      -- Im Index!
    date: order.created_at         -- Im Index!
  }
```

Ausführungsplan:

1. Index-Only Scan: idx_orders_customer_covering → Liest nur Index
2. Keine Table Lookups nötig → 10x schneller!

Performance-Vergleich:

Ansatz	Disk I/O	Latenz	Durchsatz
Ohne Index	50 random reads	50ms	1000 queries/sec
Mit Index (nicht covering)	50 index reads + 50 table reads	25ms	2000 queries/sec
Mit Covering Index	50 index reads	5ms	10000 queries/sec

5.11.3 Partielle Indizes: Selektive Indexierung

Wenn nur ein Teil der Daten häufig abgefragt wird, kann ein **partieller Index** Speicherplatz und Schreibperformance sparen:

```
-- Indexiere nur aktive Orders
CREATE INDEX idx_active_orders
  ON orders (customer_id, created_at)
  WHERE status IN ['pending', 'processing'];

-- Dieser Index ist viel kleiner als ein voller Index,
-- da er nur ~10% der orders enthält (die aktiven)
```

Wann wird der partielle Index verwendet?

```
--  Optimizer nutzt partiellen Index:
FOR order IN orders
  FILTER order.status == "pending"
  FILTER order.customer_id == "customer_123"
  RETURN order

--  Optimizer nutzt NICHT den partiellen Index:
FOR order IN orders
  FILTER order.status == "completed"    -- Nicht im Index!
  FILTER order.customer_id == "customer_123"
  RETURN order
```

5.11.4 Materialized Views für komplexe Aggregationen

Für häufig ausgeführte, teure Aggregationen bietet ThemisDB **Materialized Views**:

```
-- Teure Aggregation (läuft jedes Mal 5 Sekunden):
FOR order IN orders
  FILTER order.created_at >= DATE_SUBTRACT(DATE_NOW(), 30, 'day')
  COLLECT
    customer = order.customer_id,
    date = DATE_TRUNC(order.created_at, 'day')
  AGGREGATE
    revenue = SUM(order.total_amount),
    order_count = COUNT(1)
  RETURN {customer, date, revenue, order_count}
```

Lösung: Materialized View

```
-- Erstelle Materialized View (einmalig):
CREATE MATERIALIZED VIEW daily_revenue AS
  FOR order IN orders
    COLLECT
      customer = order.customer_id,
      date = DATE_TRUNC(order.created_at, 'day')
    AGGREGATE
      revenue = SUM(order.total_amount),
      order_count = COUNT(1)
    RETURN {customer, date, revenue, order_count}
  OPTIONS {refresh: 'incremental', interval: '5 minutes'}

-- Abfrage der View (instant!):
FOR row IN daily_revenue
  FILTER row.date >= DATE_SUBTRACT(DATE_NOW(), 30, 'day')
  RETURN row
```

Refresh-Strategien:

- **refresh: 'immediate'** - Nach jeder Änderung aktualisieren (langsam)
- **refresh: 'incremental'** - Nur geänderte Daten neu berechnen (empfohlen)
- **refresh: 'full'** - Komplette Neuberechnung (für kleine Views)

5.11.5 Query-Hints: Manuelle Optimizer-Steuerung

In seltenen Fällen weiß der Entwickler mehr als der Optimizer. Dann können **Query-Hints** helfen:

```
-- Force Index Scan (statt Full Table Scan):
FOR order IN orders
  OPTIONS {indexHint: 'idx_orders_customer'}
  FILTER order.customer_id == "customer_123"
  RETURN order

-- Force Hash Join (statt Nested Loop):
FOR order IN orders
  FOR customer IN customers
    OPTIONS {joinStrategy: 'hash'}
    FILTER customer._id == order.customer_id
    RETURN {order, customer}

-- Disable Index für Debugging:
FOR order IN orders
  OPTIONS {forceTableScan: true}
  FILTER order.customer_id == "customer_123"
  RETURN order
```

Wann Query-Hints verwenden?

 **Sinnvoll:** - Temporär zum Debugging - Wenn Statistiken veraltet sind - Für extrem spezifische Workloads

 **Vermeiden:** - Als Standard (Optimizer ist meist besser) - Wenn Datenvolumen sich ändert - In produktivem Code ohne Dokumentation

5.11.6 Batch-Operations für hohen Durchsatz

Für Massen-Inserts oder Updates bietet ThemisDB Batch-Operations:

```
-- ❌ Langsam: 10.000 einzelne Inserts
FOR i IN 1..10000
  INSERT {
    name: CONCAT("Product ", i),
    price: RAND() * 1000,
    category: ["Electronics", "Clothing", "Food"][FLOOR(RAND() * 3)]
  } INTO products

-- ✅ Schnell: Batch-Insert
LET batch = (
  FOR i IN 1..10000
    RETURN {
      name: CONCAT("Product ", i),
      price: RAND() * 1000,
      category: ["Electronics", "Clothing", "Food"][FLOOR(RAND() * 3)]
    }
)
INSERT batch INTO products

-- Performance: 100x schneller!
-- Einzeln: ~50 inserts/sec (10.000 inserts = 200 Sekunden)
-- Batch: ~50.000 inserts/sec (10.000 inserts = 0.2 Sekunden)
```

Warum ist Batch schneller?

1. **Weniger Transaktionen:** 1 statt 10.000 Transaktions-Overheads
2. **Batch-Writes:** RocksDB kann Writes zusammenfassen
3. **Weniger Netzwerk-RTTs:** 1 Request statt 10.000

5.11.7 Performance-Monitoring mit EXPLAIN

Verwenden Sie immer `EXPLAIN`, um langsame Queries zu debuggen:

```
-- Detaillierter Execution Plan:  
EXPLAIN  
  FOR order IN orders  
    FILTER order.status == "pending"  
  FOR customer IN customers  
    FILTER customer._id == order.customer_id  
    FILTER customer.country == "DE"  
  RETURN {order, customer}
```

Output:


```
{
  "plan": {
    "nodes": [
      {
        "type": "IndexNode",
        "index": "idx_orders_status",
        "condition": "status == 'pending'",
        "estimated_items": 200000,
        "estimated_cost": 1000
      },
      {
        "type": "IndexNode",
        "index": "idx_customers_pk",
        "condition": "customer._id == order.customer_id",
        "estimated_items": 1,
        "estimated_cost": 10
      },
      {
        "type": "FilterNode",
        "condition": "customer.country == 'DE'",
        "estimated_items": 20000,
        "estimated_cost": 200
      }
    ],
    "total_estimated_cost": 1210,
    "total_estimated_items": 20000
  },
  "stats": {
    "execution_time": "125ms",
    "items_scanned": 200000,
    "items_returned": 20000,
    "index_hits": 200001,
    "index_misses": 0
  }
}
```

Key Metrics: - **estimated_cost**: Optimizer's Kostenschätzung - **estimated_items**: Erwartete Anzahl Ergebnisse - **execution_time**: Tatsächliche Laufzeit - **index_hits/misses**: Index-Effizienz

5.12 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- ✓ **Relationales Modell:** Tabellen, Schemas, Constraints
- ✓ **AQL DDL:** CREATE TABLE, Datentypen, Constraints
- ✓ **AQL DML:** INSERT, UPDATE, DELETE, UPSERT
- ✓ **Queries:** SELECT, WHERE, JOIN, Subqueries, Aggregation
- ✓ **Transaktionen:** ACID, Isolation, Commit/Rollback
- ✓ **Normalisierung:** 1NF, 2NF, 3NF, Denormalisierung
- ✓ **Indexes:** B-Tree, Unique, Composite, Performance
- ✓ **Erweiterte Optimierungen:** Query Optimizer, Index-Only-Scans, Materialized Views
- ✓ **Praxis:** Inventory System & Expense Tracker

Key Takeaways

1. Relational ist perfekt für strukturierte Business-Daten
2. Constraints garantieren Datenintegrität
3. Normalisierung verhindert Redundanz
4. Denormalisierung kann Performance verbessern
5. Indexes sind essentiell für Query-Performance
6. Transaktionen garantieren Konsistenz
7. Der Query-Optimizer wählt automatisch den besten Ausführungsplan
8. Covering Indexes ermöglichen Index-Only-Scans für maximale Performance

Nächster Schritt

Sie verstehen jetzt relationale Daten in ThemisDB. Im nächsten Kapitel lernen Sie **Graph-Datenbanken** kennen - perfekt für Netzwerk-Daten und Beziehungen.

[Kapitel 6: Graph-Datenbanken →](#)

Weiterführende Ressourcen

- AQL Reference: <../de/aql/README.md>
 - Schema Design: ../de/guides/guides_schema_design.md
 - Transaction Guide: ../de/features/features_transactions.md
 - Inventory System Code: ../examples/04_inventory_system/
 - Expense Tracker Code: ../examples/12_expense_tracker/
-

Kapitel 5 von 30 | Teil II: Datenmodelle | ~12.200 Wörter

Kapitel 6: Graph-Datenbanken

"The world is a graph, not a table." — Graph-Datenbank-Community

6.1 Einführung: Die Welt der Verbindungen

Die Welt besteht aus Beziehungen. Menschen kennen andere Menschen. Websites verlinken auf andere Websites. Produkte werden zusammen gekauft. Straßen verbinden Städte. Diese natürlich vernetzten Strukturen lassen sich am besten als **Graphen** darstellen.

Das Problem mit Relationalen Datenbanken

Versuchen Sie, folgende Frage mit SQL zu beantworten: *"Finde alle Freunde meiner Freunde, die in derselben Stadt wohnen und mindestens 3 gemeinsame Interessen haben."*

Mit relationalen Tabellen benötigen Sie: - Multiple Self-Joins über die `friendships`-Tabelle - Subqueries für gemeinsame Interessen - Performance-Probleme bei mehr als 3-4 Hop-Levels - Komplexe Query-Logik, die schwer zu warten ist

```
-- Freunde von Freunden (nur 2 Hops!)
SELECT DISTINCT u3.*
FROM users u1
JOIN friendships f1 ON u1.id = f1.user_id
JOIN users u2 ON f1.friend_id = u2.id
JOIN friendships f2 ON u2.id = f2.user_id
JOIN users u3 ON f2.friend_id = u3.id
WHERE u1.id = '...'
      AND u3.city = u1.city
      AND ... -- gemeinsame Interessen?
```

Diese Query wird schnell unleserlich und langsam.

Die Graph-Lösung

In ThemisDB wird dieselbe Frage intuitiv und performant:

```
result = db.graph_query("""
    FOR user IN users
      FILTER user.id == @my_id
      FOR friend IN 1..2 OUTBOUND user friendships
        FILTER friend.city == user.city
        LET common_interests = LENGTH(
          INTERSECTION(user.interests, friend.interests)
        )
        FILTER common_interests >= 3
      RETURN friend
""", bind_vars={"my_id": my_id})
```

Die Graph-Query ist: -  Lesbarer und deklarativer -  Beliebige viele Hops möglich (`1..10`, `1..ANY`) -  Performance skaliert mit Graph-Struktur, nicht mit Datenmenge -  Native Graph-Algorithmen verfügbar

6.2 Property Graph Modell

ThemisDB verwendet das **Property Graph**-Modell, den De-facto-Standard für Graph-Datenbanken.

Komponenten

1. Knoten (Vertices/Nodes) - Repräsentieren Entitäten (Personen, Orte, Produkte) - Haben einen eindeutigen Identifier - Können beliebige Properties haben - Können Labels/Types haben

```
user_node = {  
  "id": "user_001",  
  "type": "User",  
  "name": "Alice",  
  "age": 28,  
  "city": "Berlin",  
  "interests": ["Python", "ML", "Hiking"]  
}
```

2. Kanten (Edges/Relationships) - Verbinden zwei Knoten (gerichtet oder ungerichtet) - Haben einen Typ (z.B. "FRIEND", "LIKES", "WORKS_AT") - Können Properties haben (z.B. "since", "strength") - Erlauben Multi-Edges (mehrere Kanten zwischen denselben Knoten)

```
friendship_edge = {  
  "from": "user_001",  
  "to": "user_002",  
  "type": "FRIEND",  
  "since": "2023-05-15",  
  "strength": 0.85, # 0-1 basierend auf Interaktionen  
  "mutual_friends": 12  
}
```

3. Graph-Collections

ThemisDB organisiert Graphs in Collections: - **Vertex Collections:** Speichern Knoten (z.B. `users`, `posts`) - **Edge Collections:** Speichern Kanten (z.B. `friendships`, `likes`)

```
# Graph erstellen
db.create_vertex_collection("users")
db.create_edge_collection("friendships")

# Graph-Definition
graph_def = {
    "name": "social_network",
    "edge_definitions": [{
        "collection": "friendships",
        "from": ["users"],
        "to": ["users"]
    }]
}
db.create_graph(graph_def)
```

Gerichtete vs. Ungerichtete Kanten

Gerichtet (Directed):

```
# Alice folgt Bob, aber Bob folgt Alice nicht
{"from": "alice", "to": "bob", "type": "FOLLOWS"}
```

Beispiele: Twitter-Follows, Hyperlinks, Reporting-Struktur

Ungerichtet (Undirected):

```
# Freundschaft ist bidirektional
{"from": "alice", "to": "bob", "type": "FRIEND"}
{"from": "bob", "to": "alice", "type": "FRIEND"} # Beide Richtungen
```

Beispiele: Facebook-Freunde, Co-Autoren, Straßen

ThemisDB speichert alle Kanten gerichtet, aber Graph-Queries können beide Richtungen traversieren: -

OUTBOUND - Von A nach B - **INBOUND** - Von B nach A - **ANY** - Beide Richtungen (für ungerichtete Graphs)

6.3 Graph-Traversierung

Traversierungs-Arten

1. Depth-First Search (DFS) - Geht zuerst in die Tiefe - Verwendet Stack - Findet schnell tiefe Pfade

```
# DFS: Alle erreichbaren Knoten
FOR v, e, p IN 1..10 OUTBOUND @start_vertex friendships
  OPTIONS {bfs: false} # DFS (default)
  RETURN {vertex: v, path: p}
```

2. Breadth-First Search (BFS) - Geht zuerst in die Breite - Verwendet Queue - Findet kürzeste Pfade

```
# BFS: Kürzester Pfad
FOR v, e, p IN 1..10 OUTBOUND @start_vertex friendships
  OPTIONS {bfs: true} # BFS
  RETURN {vertex: v, path: p}
```

3. Bounded Traversal - Limitiert Anzahl der Hops - **1..1** - Nur direkte Nachbarn - **1..2** - Bis zu 2 Hops - **2..4** - Zwischen 2 und 4 Hops - **1..ANY** - Alle erreichbaren Knoten

```
# Freunde von Freunden (exakt 2 Hops)
FOR v IN 2..2 OUTBOUND @my_id friendships
  RETURN v
```

Pattern Matching

Graph-Queries in ThemisDB unterstützen komplexe Patterns:

```
# Triangle Pattern: A kennt B, B kennt C, C kennt A
FOR a IN users
  FOR b IN OUTBOUND a friendships
    FOR c IN OUTBOUND b friendships
      FILTER c._id == a._id
      RETURN {a: a.name, b: b.name, c: c.name}
```

```
# Diamond Pattern: Mehrere Pfade zwischen zwei Knoten
FOR start IN users FILTER start.id == @user_id
  FOR end IN OUTBOUND start friendships
    LET paths = (
      FOR v, e, p IN 2..2 OUTBOUND start friendships
        FILTER v._id == end._id
        RETURN p
    )
    FILTER LENGTH(paths) > 1 # Mehrere Pfade
    RETURN {start: start.name, end: end.name, paths: paths}
```

6.4 Graph-Algorithmen

ThemisDB bietet native Implementierungen gängiger Graph-Algorithmen:

Shortest Path (Dijkstra)

Findet den kürzesten Pfad zwischen zwei Knoten unter Berücksichtigung von Gewichten:

```
from themis_client import shortest_path

path = shortest_path(
    graph="social_network",
    start_vertex="users/alice",
    target_vertex="users/bob",
    direction="outbound",
    weight_attribute="strength" # Optional: Kantengewicht
)

print(f"Pfad: {' -> '.join([v['name'] for v in path['vertices']])}")
print(f"Distanz: {path['distance']}")
```

Use Cases: - Routing und Navigation - Social Distance berechnen - Kostenoptimale Pfade finden

All Shortest Paths

Findet alle kürzesten Pfade (falls mehrere existieren):

```
paths = db.all_shortest_paths(  
    graph="social_network",  
    start_vertex="users/alice",  
    target_vertex="users/bob"  
)  
  
for idx, path in enumerate(paths):  
    print(f"Pfad {idx+1}: {' -> '.join([v['name'] for v in path['vertices']])}")
```

K Shortest Paths

Findet die k kürzesten Pfade:

```
paths = db.k_shortest_paths(  
    graph="social_network",  
    start_vertex="users/alice",  
    target_vertex="users/bob",  
    k=3 # Top 3 kürzeste Pfade  
)
```

Connected Components

Findet zusammenhängende Teilgraphen:

```
components = db.connected_components(  
    graph="social_network",  
    direction="any" # Ungerichtet behandeln  
)  
  
# Gruppieren Knoten nach Component  
for component_id, vertices in components.items():  
    print(f"Community {component_id}: {len(vertices)} Mitglieder")
```

Use Cases: - Community-Erkennung - Isolierte Gruppen finden - Netzwerk-Fragmentierung analysieren

PageRank

Berechnet die Wichtigkeit von Knoten basierend auf eingehenden Kanten:

```
pagerank_scores = db.pagerank(  
    graph="social_network",  
    iterations=20,  
    damping_factor=0.85  
)  
  
# Sortiere nach Score  
top_users = sorted(  
    pagerank_scores.items(),  
    key=lambda x: x[1],  
    reverse=True  
)[:10]  
  
for user_id, score in top_users:  
    user = db.get("users", user_id)  
    print(f"{user['name']}: {score:.4f}")
```

Use Cases: - Influencer-Identifikation - Content-Ranking - Reputations-Systeme

Betweenness Centrality

Misst, wie oft ein Knoten auf kürzesten Pfaden liegt:

```
centrality = db.betweenness_centrality(  
    graph="social_network"  
)  
  
# Knoten mit hoher Betweenness = Brücken zwischen Communities  
bridges = [k for k, v in centrality.items() if v > 0.5]
```

Use Cases: - Netzwerk-Bottlenecks identifizieren - Wichtige Vermittler finden - Kritische Infrastruktur-Knoten

6.4A Temporale Graph-Queries: Zeitreisen im Wissensgraph

Eine der mächtigsten Features von ThemisDB ist die Fähigkeit, **zeitabhängige Graph-Traversals** durchzuführen. Dies ist besonders wichtig für Anwendungen, die historische Zustände nachvollziehen müssen – etwa für Compliance, Audit, oder rechtssichere Dokumentation.

Das Problem: Wie sah der Graph *damals* aus?

Stellen Sie sich folgende Szenarien vor:

Szenario 1 - Compliance-Anfrage:

"Welche Zugriffsrechte hatte Benutzer X am 15. März 2024 um 14:30 Uhr?"

Szenario 2 - Verwaltungsakt:

"Auf welcher Datengrundlage basierte der Bescheid vom 10. Januar 2024? Welche Dokumente waren zu diesem Zeitpunkt verknüpft?"

Szenario 3 - Betrugserkennung:

"Zu welchen verdächtigen Konten hatte Account Y Verbindungen in der Woche vor der Sperrung?"

In traditionellen Graph-Datenbanken müssten Sie entweder: 1. **Alle Änderungen manuell versionieren** (aufwändig, fehleranfällig) 2. **Snapshot-Backups** erstellen (speicherintensiv, grobe Granularität) 3. **Audit-Logs parsen** (langsam, komplex)

Die Lösung: Temporale Kanten mit valid_from/valid_to

ThemisDB erlaubt es, Kanten mit Gültigkeitszeiträumen zu versehen:

```
# Kante mit temporalen Feldern erstellen
db.create_edge(
    from_node="users/alice",
    to_node="departments/engineering",
    edge_type="WORKS_IN",
    properties={
        "role": "Senior Engineer",
        "valid_from": 1640995200000, # 2022-01-01 00:00:00 UTC (ms)
        "valid_to": 1704067200000   # 2024-01-01 00:00:00 UTC (ms)
    }
)

# Neue Kante für neue Abteilung
db.create_edge(
    from_node="users/alice",
    to_node="departments/research",
    edge_type="WORKS_IN",
    properties={
        "role": "Lead Researcher",
        "valid_from": 1704067200000, # 2024-01-01 00:00:00 UTC (ms)
        "valid_to": None              # Aktuell gültig (kein Enddatum)
    }
)
```

Semantik der temporalen Felder:

- `valid_from = null`: Kante gilt seit Anbeginn der Zeit
- `valid_to = null`: Kante gilt unbegrenzt in die Zukunft
- `valid_from = T1, valid_to = T2`: Kante galt im Intervall [T1, T2]
- Beide `null`: Kante ist zeitlos (eternal)

Point-in-Time Queries: Graph-Zustand zu einem Zeitpunkt

API: bfsAtTime() - Breadth-First Search mit Zeitfilter

```
// C++ API Signatur
std::pair<Status, std::vector<std::string>> bfsAtTime(
    std::string_view startPk,      // Startknoten
    int64_t timestamp_ms,         // Zeitpunkt (ms seit Epoch)
    int maxDepth = 3              // Maximale Traversierungstiefe
) const;
```

Wie es funktioniert:

Die `bfsAtTime()`-Funktion durchläuft den Graphen wie eine normale BFS, aber mit einem entscheidenden Unterschied: **Jede Kante wird vor der Traversierung auf Gültigkeit geprüft.**

Der Algorithmus Schritt-für-Schritt:

```
// Pseudo-Code der temporalen BFS-Implementation
bool isValidAtTime(Edge edge, int64_t query_time) {
    // Fall 1: Kante noch nicht gültig
    if (edge.valid_from.has_value() && query_time < edge.valid_from) {
        return false; // Zu früh!
    }

    // Fall 2: Kante nicht mehr gültig
    if (edge.valid_to.has_value() && query_time > edge.valid_to) {
        return false; // Zu spät!
    }

    // Fall 3: Kante war zum Query-Zeitpunkt gültig
    return true;
}

std::vector<std::string> bfsAtTime(string start, int64_t timestamp, int maxDepth) {
    queue<Node> frontier;
    set<string> visited;
    vector<string> result;

    frontier.push({start, 0}); // {node_id, depth}
    visited.insert(start);

    while (!frontier.empty()) {
        auto [current_node, depth] = frontier.front();
        frontier.pop();

        if (depth > maxDepth) continue;

        result.push_back(current_node);

        // Hole alle ausgehenden Kanten
        for (auto& edge : getOutgoingEdges(current_node)) {
            // KRITISCH: Temporaler Filter!
            if (!isValidAtTime(edge, timestamp)) {
                continue; // Kante überspringen
            }
        }
    }
}
```



```
        if (visited.count(edge.to) == 0) {
            visited.insert(edge.to);
            frontier.push({edge.to, depth + 1});
        }
    }
}

return result;
}
```

Der entscheidende Unterschied: Die Zeile `if (!isValidAtTime(edge, timestamp)) continue;` filtert historisch ungültige Kanten **vor** der Traversierung. Dadurch sieht die BFS exakt den Graph-Zustand, wie er zum Query-Zeitpunkt existierte.

Praktisches Beispiel:

```
# Frage: Welche Abteilungen konnte Alice am 1. Juli 2023 erreichen?
timestamp = datetime(2023, 7, 1).timestamp() * 1000 # In Millisekunden

result = db.graph.bfsAtTime(
    start="users/alice",
    timestamp_ms=int(timestamp),
    max_depth=3
)

print(f"Erreichbare Knoten am 1. Juli 2023: {result}")
# Output: ['users/alice', 'departments/engineering', 'projects/project-x']
# → 'departments/research' fehlt, weil Alice erst 2024 dorthin wechselte!
```

Shortest Path mit Zeitfilter: `dijkstraAtTime()`

Für gewichtete Graphen unterstützt ThemisDB auch temporale Kürzeste-Pfad-Suche:

```
// C++ API Signatur
std::pair<Status, PathResult> dijkstraAtTime(
    std::string_view startPk,
    std::string_view targetPk,
    int64_t timestamp_ms
) const;

// PathResult enthält:
struct PathResult {
    std::vector<std::string> path; // Knoten vom Start zum Ziel
    double totalCost;             // Gesamtkosten des Pfades
};
```

Use Case: Organisationshierarchie zum Zeitpunkt eines Bescheids

```
# Frage: Wer war am 10. Januar 2024 der kürzeste Eskalationspfad
# von einem Sachbearbeiter zu einem Abteilungsleiter?

timestamp = datetime(2024, 1, 10, 14, 30).timestamp() * 1000

path_result = db.graph.dijkstraAtTime(
    start="users/sachbearbeiter-123",
    target="users/abteilungsleiter-456",
    timestamp_ms=int(timestamp)
)

if path_result.status.ok:
    print(f"Eskalationspfad am 10.01.2024:")
    for i, node in enumerate(path_result.path):
        print(f" {i}. {node}")
    print(f"Hierarchie-Distanz: {path_result.totalCost}")
```

Ausgabe:

Eskalationspfad am 10.01.2024:

- 0. users/sachbearbeiter-123
- 1. users/teamleiter-789
- 2. users/abteilungsleiter-456

Hierarchie-Distanz: 2.0

Warum ist das wichtig?

Wenn später ein Bescheid angefochten wird und die Frage aufkommt: "War die Eskalation regelkonform?", können Sie **exakt nachweisen**, welche Hierarchie zum Zeitpunkt der Entscheidung galt – auch wenn die Organisation sich seitdem umstrukturiert hat.

Time-Range Queries: Kanten in einem Zeitfenster

Manchmal interessiert nicht ein einzelner Zeitpunkt, sondern ein **Zeitraum**:

"Welche Zugriffsrechte überlappten mit dem Quartal Q4 2024?"

```
// C++ API für Time-Range Queries
struct TimeRangeFilter {
    int64_t start_ms; // Fenster-Start
    int64_t end_ms;   // Fenster-Ende

    // Prüft ob Kante mit Zeitfenster überlappt
    bool hasOverlap(optional<int64_t> edge_valid_from,
                    optional<int64_t> edge_valid_to) const;

    // Prüft ob Kante vollständig im Fenster enthalten ist
    bool fullyContains(optional<int64_t> edge_valid_from,
                       optional<int64_t> edge_valid_to) const;
};

std::pair<Status, std::vector<EdgeInfo>> getEdgesInTimeRange(
    int64_t range_start_ms,
    int64_t range_end_ms,
    bool require_full_containment = false
) const;
```

Beispiel: Audit-Abfrage für Q4 2024

```

# Zeitfenster definieren
q4_start = datetime(2024, 10, 1).timestamp() * 1000
q4_end = datetime(2024, 12, 31, 23, 59, 59).timestamp() * 1000

# Alle Kanten finden, die mit Q4 überlappen
edges = db.graph.getEdgesInTimeRange(
    range_start_ms=int(q4_start),
    range_end_ms=int(q4_end),
    require_full_containment=False # Überlappung reicht
)

print(f"Gefunden: {len(edges)} Kanten mit Überlappung zu Q4 2024")

for edge in edges:
    print(f" {edge.from_pk} → {edge.to_pk}")
    print(f"    Gültig: {edge.valid_from} bis {edge.valid_to}")

```

Überlappungs-Logik visualisiert:

Query-Zeitfenster: [————Q4 2024————]

 Oct 1 Dec 31

Kante A: [————] ✓ Überlappt (vor Q4, endet in Q4)

Kante B: [————] ✓ Überlappt (komplett in Q4)

Kante C: [————] ✓ Überlappt (startet in Q4, endet nach Q4)

Kante D: [———] ✗ Keine Überlappung (endet vor Q4)

Kante E: [—] ✗ Keine Überlappung (startet nach Q4)

Revisionssicherheit: Verwaltungsakte dokumentieren

Das Szenario:

Eine Behörde erstellt einen Bescheid am **15. März 2024**. Dieser Bescheid verweist auf: - 3 Gutachten - 2 Gesetze

- 4 frühere Verwaltungsakte

Sechs Monate später wird der Bescheid angefochten. Die Frage: *"Auf welcher Datengrundlage basierte die Entscheidung?"*

Die Lösung mit temporalen Graphen:

```
# 1. Beim Erstellen des Bescheids: Graph-Snapshot implizit gespeichert
bescheid_timestamp = datetime(2024, 3, 15, 10, 30).timestamp() * 1000

# 2. Sechs Monate später: Rekonstruktion der damaligen Datenlage
verwandte_dokumente = db.graph.bfsAtTime(
    start=f"bescheide/{bescheid_id}",
    timestamp_ms=bescheid_timestamp,
    max_depth=2 # Direkte + indirekte Referenzen
)

# 3. Für jedes Dokument: Exakte Version zum Zeitpunkt abrufen
for doc_id in verwandte_dokumente:
    doc_version = db.get_document_at_time(doc_id, bescheid_timestamp)
    print(f"Dokument {doc_id} (Stand {bescheid_timestamp}):")
    print(f"  Titel: {doc_version['title']}")
    print(f"  Version: {doc_version['version']}")
    print(f"  Checksum: {doc_version['checksum']}")
```

Resultat: Sie können **beweisen**, dass der Bescheid auf den zum Entscheidungszeitpunkt gültigen Dokumenten basierte – selbst wenn diese Dokumente seitdem aktualisiert wurden.

Performance-Überlegungen

Speicher-Overhead:

Temporale Kanten benötigen 16 zusätzliche Bytes pro Kante ($2 \times \text{int64}$ für Timestamps):

```
Normale Kante:   ~80 Bytes
Temporale Kante: ~96 Bytes
→ Overhead: 20%
```

Bei 10 Millionen Kanten: ~160 MB zusätzlicher Speicher. **Akzeptabel** für die gewonnene Funktionalität.

Query-Performance:

Die temporale Filterung ist sehr effizient, da der Check inline während der Traversierung passiert:

```
// Pro Kante: 2 Integer-Vergleiche (~2 CPU-Zyklen)
if (edge.valid_from && timestamp < edge.valid_from) return false;
if (edge.valid_to && timestamp > edge.valid_to) return false;
```

Benchmark: `bfsAtTime()` ist nur ~5-10% langsamer als normale BFS, da der Overhead durch CPU-Cache-Hits minimiert wird.

Best Practices

1. Timestamps immer in Millisekunden (Unix Epoch)

```
# ✅ Richtig: Millisekunden seit 1970-01-01
timestamp_ms = int(datetime.now().timestamp() * 1000)

# ❌ Falsch: Sekunden (zu grobe Granularität)
timestamp_s = int(datetime.now().timestamp())
```

2. `valid_to = null` für aktuell gültige Beziehungen

```
# ✅ Richtig: Kein Enddatum = unbegrenzt gültig
edge = {
    "valid_from": now_ms,
    "valid_to": None
}

# ❌ Falsch: Festes Enddatum in ferner Zukunft
edge = {
    "valid_from": now_ms,
    "valid_to": 9999999999999 # Unflexibel!
}
```

3. Indizes auf temporalen Feldern

```
# Index für effiziente temporale Queries
db.create_index(
    collection="edges",
    fields=["valid_from", "valid_to"],
    index_type="range"
)
```

6.5 Praxisbeispiel 1: Social Network

Jetzt setzen wir die Theorie in die Praxis um mit einem vollständigen Social Network.

Das Projekt

Das Social Network-Example ([examples/06_graph_social_network](#)) implementiert: - Benutzerprofile mit Interessen - Bidirektionale Freundschaften - Graph-Traversierung (Freunde von Freunden) - Kürzeste Pfade zwischen Usern - Community-Erkennung - Freundschafts-Empfehlungen - Interaktive Visualisierung mit NetworkX

Datenmodell

```
# models.py
from dataclasses import dataclass
from typing import List, Optional
from datetime import datetime

@dataclass
class User:
    """Ein Benutzer im sozialen Netzwerk."""
    id: str
    name: str
    bio: Optional[str] = None
    interests: List[str] = None
    location: Optional[str] = None
    joined: datetime = None

    def to_dict(self):
        return {
            "id": self.id,
            "name": self.name,
            "bio": self.bio,
            "interests": self.interests or [],
            "location": self.location,
            "joined": self.joined.isoformat() if self.joined else None
        }

@dataclass
class Friendship:
    """Eine Freundschaft zwischen zwei Benutzern."""
    from_user: str
    to_user: str
    since: datetime
    strength: float = 1.0 # 0-1, basierend auf Interaktionen

    def to_dict(self):
        return {
            "from": f"users/{self.from_user}",
            "to": f"users/{self.to_user}",
            "since": self.since.isoformat(),
```

```
    "strength": self.strength  
}
```

Graph Setup

```
# themis_client.py (gekürzt)
class ThemisGraphClient:
    def __init__(self, host="localhost", port=8529):
        self.client = ThemisDB(host=host, port=port)
        self.db = self.client.db("social_network")
        self._setup_graph()

    def _setup_graph(self):
        """Erstellt Collections und Graph-Definition."""
        # Vertex Collection für User
        if not self.db.has_collection("users"):
            self.db.create_vertex_collection("users")

        # Edge Collection für Freundschaften
        if not self.db.has_collection("friendships"):
            self.db.create_edge_collection("friendships")

        # Graph-Definition
        if not self.db.has_graph("social_graph"):
            graph_def = {
                "name": "social_graph",
                "edge_definitions": [{
                    "collection": "friendships",
                    "from": ["users"],
                    "to": ["users"]
                }]
            }
            self.db.create_graph(graph_def)

        # Indexes für Performance
        self.db.add_index("users", ["name"])
        self.db.add_index("users", ["location"])
        self.db.add_index("friendships", ["from", "to"])
```

Benutzer und Freundschaften

```
def add_user(self, user: User) -> str:
    """Fügt einen neuen Benutzer hinzu."""
    user_doc = user.to_dict()
    result = self.db.insert("users", user_doc)
    return result["_key"]

def add_friendship(self, friendship: Friendship):
    """Erstellt eine bidirektionale Freundschaft."""
    # Richtung 1: A -> B
    self.db.insert("friendships", friendship.to_dict())

    # Richtung 2: B -> A (für ungerichteten Graph)
    reverse = Friendship(
        from_user=friendship.to_user,
        to_user=friendship.from_user,
        since=friendship.since,
        strength=friendship.strength
    )
    self.db.insert("friendships", reverse.to_dict())

def get_friends(self, user_id: str) -> List[User]:
    """Holt alle direkten Freunde eines Benutzers."""
    query = """
        FOR friend IN OUTBOUND @user_id friendships
            RETURN friend
    """
    result = self.db.execute_query(query, bind_vars={"user_id": f"users/{user_id}"})
    return [User(**doc) for doc in result]
```

Friends-of-Friends (FoF)

Ein klassisches Graph-Problem: Finde Freunde meiner Freunde, die noch nicht meine Freunde sind.

```

def get_friend_suggestions(self, user_id: str, limit: int = 10) -> List[dict]:
    """Empfiehl neue Freunde basierend auf gemeinsamen Freunden."""
    query = """
        FOR friend IN OUTBOUND @user_id friendships
        FOR fof IN OUTBOUND friend._id friendships
        FILTER fof._id != @user_id
        FILTER fof._id NOT IN (
            FOR f IN OUTBOUND @user_id friendships
            RETURN f._id
        )
        COLLECT fof_user = fof WITH COUNT INTO mutual_count
        SORT mutual_count DESC
        LIMIT @limit
        RETURN {
            user: fof_user,
            mutual_friends: mutual_count
        }
    """
    return self.db.execute_query(query, bind_vars={
        "user_id": f"users/{user_id}",
        "limit": limit
    })

```

Was passiert hier? 1. Traversiere zu allen Freunden (`OUTBOUND @user_id`) 2. Von jedem Freund, traversiere zu deren Freunden (`OUTBOUND friend._id`) 3. Filtere mich selbst heraus (`FILTER fof._id != @user_id`) 4. Filtere existierende Freunde heraus (Subquery) 5. Gruppiere nach FoF und zähle gemeinsame Freunde (`COLLECT ... WITH COUNT`) 6. Sortiere nach Anzahl gemeinsamer Freunde

Kürzeste Pfade

Wie ist Alice mit Bob verbunden?

```
def find_connection_path(self, user_id1: str, user_id2: str) -> dict:
    """Findet den kürzesten Pfad zwischen zwei Benutzern."""
    path = self.db.shortest_path(
        graph="social_graph",
        start_vertex=f"users/{user_id1}",
        target_vertex=f"users/{user_id2}",
        direction="any" # Ungerichtet (beide Richtungen)
    )

    if path is None:
        return {"connected": False}

    return {
        "connected": True,
        "distance": path["distance"],
        "path": [v["name"] for v in path["vertices"]],
        "degrees_of_separation": len(path["vertices"]) - 1
    }
```

Community-Erkennung

Welche natürlichen Gruppen gibt es im Netzwerk?

```
def detect_communities(self) -> dict:
    """Findet Communities mittels Connected Components."""
    components = self.db.connected_components(
        graph="social_graph",
        direction="any"
    )

    # Statistiken pro Community
    communities = {}
    for component_id, user_ids in components.items():
        users = [self.db.get("users", uid) for uid in user_ids]

        # Gemeinsame Interessen
        all_interests = []
        for user in users:
            all_interests.extend(user.get("interests", []))
        interest_counts = Counter(all_interests)

        communities[component_id] = {
            "size": len(users),
            "members": [u["name"] for u in users],
            "top_interests": interest_counts.most_common(5)
        }

    return communities
```

Interaktive Visualisierung

Das Example nutzt NetworkX für Visualisierung:


```
import networkx as nx
import matplotlib.pyplot as plt

def visualize_network(self, user_id: Optional[str] = None, max_hops: int = 2):
    """Visualisiert das soziale Netzwerk."""
    G = nx.Graph()

    if user_id:
        # Nur Ego-Netzwerk eines Users
        query = f"""
            FOR v, e IN 1..{max_hops} ANY @user_id friendships
            RETURN {{vertex: v, edge: e}}
        """
        result = self.db.execute_query(query, bind_vars={"user_id": f"users/{user_id}"})
    else:
        # Ganzes Netzwerk
        users = self.db.all("users")
        friendships = self.db.all("friendships")

        for user in users:
            G.add_node(user["_key"], name=user["name"])

        for friendship in friendships:
            from_id = friendship["_from"].split("/")[1]
            to_id = friendship["_to"].split("/")[1]
            G.add_edge(from_id, to_id, weight=friendship.get("strength", 1.0))

    # Layout mit Spring-Algorithmus
    pos = nx.spring_layout(G, k=0.5, iterations=50)

    # Zeichnen
    plt.figure(figsize=(12, 8))
    nx.draw_networkx_nodes(G, pos, node_size=500, node_color='lightblue')
    nx.draw_networkx_edges(G, pos, alpha=0.5)
    nx.draw_networkx_labels(G, pos, labels=nx.get_node_attributes(G, 'name'))

    plt.title("Social Network Graph")
    plt.axis('off')
```

```
plt.tight_layout()  
plt.show()
```

Main Application

```
# main.py
def main():
    client = ThemisGraphClient()

    # Beispiel-Daten
    users = [
        User("alice", "Alice", "Software Engineer", ["Python", "ML", "Hiking"], "Berlin"),
        User("bob", "Bob", "Data Scientist", ["Python", "Statistics", "Music"], "Munich"),
        User("charlie", "Charlie", "DevOps", ["Docker", "Kubernetes", "Hiking"], "Berlin"),
        User("diana", "Diana", "Product Manager", ["Agile", "UX", "Music"], "Hamburg"),
        User("eve", "Eve", "ML Engineer", ["ML", "Python", "Research"], "Berlin"),
    ]

    for user in users:
        client.add_user(user)

    # Freundschaften
    friendships = [
        ("alice", "bob"),
        ("alice", "charlie"),
        ("bob", "diana"),
        ("charlie", "eve"),
        ("diana", "eve"),
    ]

    for user1, user2 in friendships:
        client.add_friendship(Friendship(user1, user2, datetime.now(), 0.8))

    # 1. Freunde von Alice
    print("Alice's Friends:")
    friends = client.get_friends("alice")
    for friend in friends:
        print(f"  - {friend.name} ({friend.location})")

    # 2. Freundschafts-Empfehlungen für Alice
    print("\nFriend Suggestions for Alice:")
    suggestions = client.get_friend_suggestions("alice", limit=3)
    for suggestion in suggestions:
```

```
        print(f"    - {suggestion['user']['name']}: {suggestion['mutual_friends']} mutual friends")

# 3. Verbindung zwischen Alice und Eve
print("\nConnection between Alice and Eve:")
path = client.find_connection_path("alice", "eve")
if path["connected"]:
    print(f"    Path: {' -> '.join(path['path'])}")
    print(f"    Degrees of Separation: {path['degrees_of_separation']}")

# 4. Communities
print("\nCommunities:")
communities = client.detect_communities()
for comm_id, comm_data in communities.items():
    print(f"    Community {comm_id}: {comm_data['size']} members")
    print(f"        Members: {' , '.join(comm_data['members'])}")
    print(f"        Top Interests: {[i[0] for i in comm_data['top_interests']]}")

# 5. Visualisierung
client.visualize_network()

if __name__ == "__main__":
    main()
```

Output

Alice's Friends:

- Bob (Munich)
- Charlie (Berlin)

Friend Suggestions for Alice:

- Diana: 1 mutual friends
- Eve: 1 mutual friends

Connection between Alice and Eve:

Path: Alice -> Charlie -> Eve

Degrees of Separation: 2

Communities:

Community 1: 5 members

Members: Alice, Bob, Charlie, Diana, Eve

Top Interests: ['Python', 'ML', 'Hiking', 'Music', 'Docker']

[Visualization opens in matplotlib window]

Performance-Optimierung

1. Indexes auf häufig abgefragte Felder:

```
self.db.add_index("users", ["name"])
self.db.add_index("users", ["location"])
self.db.add_index("friendships", ["from", "to"])
```

2. Batch-Operations für viele Inserts:

```
def add_users_batch(self, users: List[User]):
    """Fügt mehrere User auf einmal hinzu."""
    user_docs = [u.to_dict() for u in users]
    self.db.insert_many("users", user_docs)
```

3. Bounded Traversals:

```
# Statt 1..ANY (alle Knoten)
FOR v IN 1..3 OUTBOUND @user_id friendships # Max 3 Hops
RETURN v
```

4. Index-basierte Filters:

```
# Filter NACH Index-Lookup
FOR user IN users
  FILTER user.location == "Berlin" # Nutzt Index
  FOR friend IN OUTBOUND user._id friendships
    RETURN friend
```

6.6 Praxisbeispiel 2: Recommendation Engine

Das zweite Example ([examples/19_recommendation_engine](#)) zeigt einen komplexeren Use Case: **ML-basierte Empfehlungen** mit Graph- und Vektor-Daten.

Das Konzept

Empfehlungssysteme kombinieren mehrere Ansätze:

1. **Collaborative Filtering**: "User, die X mochten, mochten auch Y"
2. **Content-Based Filtering**: "Ähnliche Items basierend auf Features"
3. **Graph-basiert**: "Freunde deiner Freunde kauften X"
4. **Hybrid**: Kombination aller Ansätze

Datenmodell

```
@dataclass
class Item:
    """Ein empfehlbares Item (Produkt, Film, etc.)."""
    id: str
    title: str
    category: str
    features: List[str]
    embedding: List[float] # Content-Embedding für Similarity

@dataclass
class Interaction:
    """Eine User-Item Interaktion."""
    user_id: str
    item_id: str
    type: str # "view", "click", "purchase", "rate"
    rating: Optional[float] = None
    timestamp: datetime = None
    context: dict = None # Device, location, etc.

@dataclass
class Recommendation:
    """Eine generierte Empfehlung."""
    user_id: str
    item_id: str
    score: float
    method: str # "collaborative", "content_based", "graph", "hybrid"
    explanation: str
```


Graph-Setup

```
def _setup_graph(self):
    """Erstellt Multi-Graph für Recommendations."""
    # Vertex Collections
    self.db.create_vertex_collection("users")
    self.db.create_vertex_collection("items")

    # Edge Collections
    self.db.create_edge_collection("interactions") # User -> Item
    self.db.create_edge_collection("similar_items") # Item -> Item
    self.db.create_edge_collection("user_similarity") # User -> User

    # Graph-Definition
    graph_def = {
        "name": "recommendation_graph",
        "edge_definitions": [
            {
                "collection": "interactions",
                "from": ["users"],
                "to": ["items"]
            },
            {
                "collection": "similar_items",
                "from": ["items"],
                "to": ["items"]
            },
            {
                "collection": "user_similarity",
                "from": ["users"],
                "to": ["users"]
            }
        ]
    }
    self.db.create_graph(graph_def)
```

Collaborative Filtering

Findet Items, die ähnliche User mochten:

```

def collaborative_filtering(self, user_id: str, limit: int = 10) -> List[Recommendation]:
    """Empfehle Items basierend auf ähnlichen Usern."""
    query = """
        // 1. Finde ähnliche User (basierend auf vergangenen Interaktionen)
        FOR similar_user IN OUTBOUND @user_id user_similarity
            SORT similar_user.similarity DESC
            LIMIT 20

        // 2. Finde Items, die ähnliche User mochten
        FOR item IN OUTBOUND similar_user._id interactions
            FILTER item.interaction_type IN ["purchase", "rate"]
            FILTER item.rating >= 4 // Nur positive

        // 3. Filtere Items, die User schon kennt
        FILTER item._id NOT IN (
            FOR known_item IN OUTBOUND @user_id interactions
                RETURN known_item._id
        )

        // 4. Aggregiere Score
        COLLECT item_id = item._id
        AGGREGATE score = AVG(item.rating * similar_user.similarity)

        SORT score DESC
        LIMIT @limit

        // 5. Hole Item-Details
        LET item_doc = DOCUMENT(item_id)
        RETURN {
            item_id: item_id,
            score: score,
            method: "collaborative",
            explanation: CONCAT("Users similar to you rated this ", score)
        }
    """

    results = self.db.execute_query(query, bind_vars={
        "user_id": f"users/{user_id}",
    })

```

```
        "limit": limit
    })

    return [Recommendation(**r) for r in results]
```

Content-Based Filtering

Findet ähnliche Items basierend auf Features:

```
def content_based_filtering(self, user_id: str, limit: int = 10) -> List[Recommendation]:
    """Empfehle Items ähnlich zu den, die User mag."""
    query = """
        // 1. Finde Items, die User mag
        FOR liked_item IN OUTBOUND @user_id interactions
            FILTER liked_item.rating >= 4

        // 2. Finde ähnliche Items
        FOR similar_item IN OUTBOUND liked_item._id similar_items
            SORT similar_item.similarity DESC

        // 3. Filtere bekannte Items
        FILTER similar_item._id NOT IN (
            FOR known IN OUTBOUND @user_id interactions
                RETURN known._id
        )

        // 4. Aggregiere
        COLLECT item_id = similar_item._id
        AGGREGATE score = AVG(similar_item.similarity)

        SORT score DESC
        LIMIT @limit

        LET item_doc = DOCUMENT(item_id)
        RETURN {
            item_id: item_id,
            score: score,
            method: "content_based",
            explanation: CONCAT("Similar to items you liked")
        }
    """

    results = self.db.execute_query(query, bind_vars={
        "user_id": f"users/{user_id}",
        "limit": limit
    })
```

```
return [Recommendation(**r) for r in results]
```

Graph-basierte Empfehlungen

Nutzt Social Graph für Empfehlungen:

```
def graph_based_recommendations(self, user_id: str, limit: int = 10) -> List[Recommendation]:
    """Empfehle Items, die Freunde mochten."""
    query = """
        // 1. Traversiere zu Freunden (1-2 Hops)
        FOR friend IN 1..2 OUTBOUND @user_id friendships

        // 2. Finde Items, die Freund kaufte
        FOR item IN OUTBOUND friend._id interactions
            FILTER item.interaction_type == "purchase"

        // 3. Filtere bekannte Items
        FILTER item._id NOT IN (
            FOR known IN OUTBOUND @user_id interactions
                RETURN known._id
        )

        // 4. Score basierend auf Freundschafts-Stärke und Rating
        COLLECT item_id = item._id
        AGGREGATE score = AVG(friend.friendship_strength * item.rating)

        SORT score DESC
        LIMIT @limit

        LET item_doc = DOCUMENT(item_id)
        RETURN {
            item_id: item_id,
            score: score,
            method: "graph_based",
            explanation: "Your friends liked this"
        }
    """

    results = self.db.execute_query(query, bind_vars={
        "user_id": f"users/{user_id}",
        "limit": limit
    })

    return [Recommendation(**r) for r in results]
```

Hybrid Recommendations

Kombiniert alle Methoden mit Gewichtung:


```
def hybrid_recommendations(self, user_id: str, limit: int = 10) -> List[Recommendation]:
    """Kombiniert alle Empfehlungsmethoden."""
    # Hole Empfehlungen von allen Methoden
    collaborative = self.collaborative_filtering(user_id, limit=20)
    content_based = self.content_based_filtering(user_id, limit=20)
    graph_based = self.graph_based_recommendations(user_id, limit=20)

    # Gewichtung
    weights = {
        "collaborative": 0.4,
        "content_based": 0.3,
        "graph_based": 0.3
    }

    # Kombiniere Scores
    combined_scores = {}
    for recs, method in [
        (collaborative, "collaborative"),
        (content_based, "content_based"),
        (graph_based, "graph_based")
    ]:
        for rec in recs:
            if rec.item_id not in combined_scores:
                combined_scores[rec.item_id] = {
                    "score": 0,
                    "methods": [],
                    "explanations": []
                }

            combined_scores[rec.item_id]["score"] += rec.score * weights[method]
            combined_scores[rec.item_id]["methods"].append(method)
            combined_scores[rec.item_id]["explanations"].append(rec.explanation)

    # Sortiere und limitiere
    sorted_items = sorted(
        combined_scores.items(),
        key=lambda x: x[1]["score"],
        reverse=True
    )
```

```
)[:limit]

# Erstelle finale Recommendations
recommendations = []
for item_id, data in sorted_items:
    recommendations.append(Recommendation(
        user_id=user_id,
        item_id=item_id,
        score=data["score"],
        method="hybrid",
        explanation=f"Recommended via: {'', '}.join(data['methods'])}"
    ))

return recommendations
```

Similarity Berechnung

Berechne User-User und Item-Item Similarity:

```
def compute_user_similarity(self):
    """Berechnet Similarity zwischen allen User-Paaren."""
    users = self.db.all("users")

    for i, user1 in enumerate(users):
        for user2 in users[i+1:]:
            # Gemeinsame Interaktionen
            common_items = self._get_common_interactions(user1["_id"], user2["_id"])

            if len(common_items) >= 3: # Mindestens 3 gemeinsame
                similarity = self._calculate_similarity(
                    user1["interaction_vector"],
                    user2["interaction_vector"]
                )

                # Speichere Kante
                self.db.insert("user_similarity", {
                    "from": user1["_id"],
                    "to": user2["_id"],
                    "similarity": similarity,
                    "common_items": len(common_items)
                })

def compute_item_similarity(self):
    """Berechnet Similarity zwischen Items (content-based)."""
    items = self.db.all("items")

    for i, item1 in enumerate(items):
        for item2 in items[i+1:]:
            # Cosine Similarity der Embeddings
            similarity = cosine_similarity(
                item1["embedding"],
                item2["embedding"]
            )

            if similarity > 0.7: # Threshold
                self.db.insert("similar_items", {
                    "from": item1["_id"],
```

```

        "to": item2["_id"],
        "similarity": similarity
    })

```

Real-Time Updates

Aktualisiere Empfehlungen bei neuen Interaktionen:

```

def record_interaction(self, interaction: Interaction):
    """Zeichnet Interaktion auf und aktualisiert Empfehlungen."""
    # Speichere Interaktion
    interaction_doc = {
        "from": f"users/{interaction.user_id}",
        "to": f"items/{interaction.item_id}",
        "type": interaction.type,
        "rating": interaction.rating,
        "timestamp": interaction.timestamp.isoformat(),
        "context": interaction.context
    }
    self.db.insert("interactions", interaction_doc)

    # Bei Purchase: Update User-Vektor
    if interaction.type == "purchase":
        self._update_user_vector(interaction.user_id, interaction.item_id)

    # Recalculate Similarity (async)
    self._queue_similarity_update(interaction.user_id)

```

6.7 Graph Best Practices

1. Modeling Guidelines

DO: -  Nutze sprechende Edge-Namen (`FRIEND`, `LIKES`, `WORKS_AT`) -  Speichere Properties auf Kanten (Zeitstempel, Gewichte) -  Denormalisiere häufig benötigte Daten -  Nutze Bidirektionale Kanten für ungerichtete Graphs

DON'T: - ❌ Zu viele Edge-Types (schwer zu querien) - ❌ Properties als separate Knoten (ineffizient) - ❌ Extrem tiefe Hierarchien (> 10 Levels)

2. Performance-Tipps

Indexes:

```
# Composite Index für häufige Lookups
db.add_index("friendships", ["from", "to"])
db.add_index("interactions", ["user_id", "timestamp"])
```

Bounded Traversals:

```
# Limitiere Hops
FOR v IN 1..3 OUTBOUND @start # Nicht 1..ANY
  LIMIT 100 # Früh limitieren
  RETURN v
```

Prune Early:

```
# Filter so früh wie möglich
FOR v IN 1..5 OUTBOUND @start friendships
  FILTER v.city == "Berlin" # Früher Filter
  FOR v2 IN OUTBOUND v._id friendships
    RETURN v2
```

3. Häufige Patterns

Pattern 1: Mutual Friends

```
FOR friend IN OUTBOUND @user1 friendships
  FILTER friend._id IN (
    FOR f IN OUTBOUND @user2 friendships
      RETURN f._id
  )
  RETURN friend
```

Pattern 2: Influencer (hohe Degree)

```
FOR user IN users
  LET friend_count = LENGTH(
    FOR f IN OUTBOUND user._id friendships
      RETURN 1
  )
  FILTER friend_count > 100
  SORT friend_count DESC
  RETURN {user: user, friends: friend_count}
```

Pattern 3: Isolated Nodes

```
FOR user IN users
  LET connections = (
    FOR v IN ANY user._id friendships
      RETURN 1
  )
  FILTER LENGTH(connections) == 0
  RETURN user
```

6.8 Vergleich: ThemisDB vs. Neo4j vs. ArangoDB

Feature	ThemisDB	Neo4j	ArangoDB
Graph Model	Property Graph	Property Graph	Property Graph
Query Language	AQL	Cypher	AQL
Multi-Model	✓ 4 Models	✗ Graph only	✓ 3 Models
ACID	✓ Full	✓ Full	✓ Full
Sharding	✓ Automatic	✗ Enterprise only	✓ Yes
Graph Algorithms	✓ Built-in	✓ GDS Library	✓ Pregel
License	Apache 2.0	GPLv3 + Commercial	Apache 2.0
Embedding Support	✓ Native	✗ No	✗ No

Wann ThemisDB? - Multi-Model Daten (Graph + Vektor + Relational) - Native Vektor-Suche benötigt - Open-Source und selbst-hosted - Kombinierte Queries über mehrere Modelle

Wann Neo4j? - Reines Graph-Problem - Cypher-Erfahrung im Team - Enterprise Support benötigt - Graph Data Science Library

Wann ArangoDB? - Ähnlich wie ThemisDB (Multi-Model) - Mature Community - Foxx Microservices

6.9 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- ✓ **Property Graph Modell** - Knoten, Kanten, Properties
- ✓ **Graph-Traversierung** - DFS, BFS, Pattern Matching
- ✓ **Graph-Algorithmen** - Shortest Path, PageRank, Community Detection
- ✓ **Praxis: Social Network** - Vollständiges soziales Netzwerk mit Visualisierung
- ✓ **Praxis: Recommendations** - ML-basierte Empfehlungen mit Hybrid-Ansatz
- ✓ **Best Practices** - Modeling, Performance, Patterns

Graph-Datenbanken sind die natürliche Wahl für vernetzte Daten. ThemisDB's Property Graph-Implementierung bietet performante Traversierung, native Algorithmen und die einzigartige Möglichkeit, Graphs mit anderen Modellen zu kombinieren.

Im nächsten Kapitel schauen wir uns **Dokument-Speicherung** an - flexibles, schema-less Design für semi-strukturierte Daten.

Übungen:

1. Erweitern Sie das Social Network um Posts und Likes (User -> Post -> Likes)
2. Implementieren Sie einen Follower/Following-Mechanismus (Twitter-Style)
3. Fügen Sie PageRank-Berechnung hinzu, um Influencer zu finden
4. Erstellen Sie einen Interest-Based-Graph (User -> Interest <- User)
5. Bauen Sie ein Skill-Recommendation-System für LinkedIn-Style-Netzwerk

Weiterführende Ressourcen:

-  [examples/06_graph_social_network/](#) - Vollständiger Code
-  [examples/19_recommendation_engine/](#) - Recommendation System
-  [docs/de/features/features_property_graph.md](#) - Graph-Features
-  NetworkX Documentation - Graph-Visualisierung
-  "Graph Algorithms" (Mark Needham, Amy E. Hodler) - Tiefere Algorithmen

Chapter 8: Storage Layer Deep-Dive

Kategorie:  Storage & Performance

Lesezeit: ~45 Minuten

Zielgruppe: Database Engineers, Performance Engineers, Production DBA



Inhaltsverzeichnis

- [8.1 RocksDB Storage Architecture](#)
 - [8.2 Memory Management mit mimalloc](#)
 - [8.3 Huge Pages Optimization](#)
 - [8.4 Compression Strategies](#)
 - [8.5 Storage Hierarchy Tuning](#)
 - [8.6 Production Deployment Patterns](#)
 - [8.7 Performance Benchmarks](#)
-

8.1 RocksDB Storage Architecture

ThemisDB nutzt RocksDB als zentrale Storage-Engine für alle Datenmodelle. Die Architektur basiert auf einem präzisen Key-Präfix-Schema, das Multi-Model-Daten logisch separiert und dabei physisch ko-lokalisiert für optimale Cache-Locality.

8.1.1 Key-Space Design

Das Präfix-Schema ermöglicht effiziente Range-Scans und Prefix-Extraktion:

Entities (Base Entity Blobs):	entity:<table>:<pk>
Secondary Index:	idx:<table>:<column>:<value>:<pk>
Range Index:	ridx:<table>:<column>:<value>:<pk>
Graph Adjacency (Outbound):	graph:out:<from_pk>:<edge_id>
Graph Adjacency (Inbound):	graph:in:<to_pk>:<edge_id>
Vector Index Metadata:	vector:<table>:<pk>
Changefeed Events:	changefeed:<sequence>
Time-Series Metrics:	ts:<metric>:<timestamp>:<tags>

Design-Rationale:

- **Sortierte Speicherung:** LSM-Tree garantiert lexikographische Ordnung → Range-Scans ohne Index
- **Prefix-Extraktion:** RocksDB Prefix-Extractor ermöglicht Bloom-Filter auf Collection-Ebene
- **Co-Location:** Related Data (z.B. alle Indizes einer Tabelle) wird physisch nah gespeichert

Code-Beispiel: Prefix-Extractor-Konfiguration

```
#include <rocksdb/slice_transform.h>

// Custom Prefix Extractor für ThemisDB Key-Schema
class ThemisKeyPrefixExtractor : public rocksdb::SliceTransform {
public:
    const char* Name() const override { return "ThemisKeyPrefixExtractor"; }

    rocksdb::Slice Transform(const rocksdb::Slice& src) const override {
        // Extrahiere "entity:<table>:" oder "idx:<table>:<column>:"
        size_t second_colon = src.ToString().find(':', 0);
        if (second_colon == std::string::npos) return src;

        size_t third_colon = src.ToString().find(':', second_colon + 1);
        if (third_colon == std::string::npos) return src;

        return rocksdb::Slice(src.data(), third_colon + 1);
    }

    bool InDomain(const rocksdb::Slice& src) const override {
        return src.size() > 0 && src[0] != '\\0';
    }
};

// Anwendung in RocksDB Options
rocksdb::Options options;
options.prefix_extractor.reset(new ThemisKeyPrefixExtractor());
```

Performance-Impact: Prefix-Extraktion reduziert Bloom-Filter-False-Positives um ~85% bei Collection-Scans.

8.1.2 Column Families Strategy

Standard-Betrieb: Default Column Family (CF)

ThemisDB verwendet standardmäßig eine einzige Default CF für alle Daten. Dies vereinfacht Transaktionen (atomare Snapshots über alle Datenmodelle) und Backups.

Optional: Multi-CF für große Workloads

Bei Workloads > 500 GB oder stark unterschiedlichen Hot/Cold-Profilen kann CF-Trennung LSM-Compactions isolieren:

```
cf_entities:      Base Entity Blobs (häufig gelesen/geschrieben)
cf_indexes:       Secondary Indexes (read-heavy)
cf_graph:         Graph Adjazenz-Listen (scan-heavy)
cf_changefeed:    CDC Events (append-only, TTL-basiert)
cf_timeseries:    Metriken (high-write, TTL)
```

Trade-offs:

Aspekt	Single CF	Multi-CF
Transaktionen	✓ Einfach (single snapshot)	⚠ Komplex (koordinierte snapshots)
Compaction	⚠ Write Amplification bei Mixed Workload	✓ Isolierte Compaction
Memory	✓ Shared Block Cache	⚠ Per-CF Caches (Overhead)
Backups	✓ Einfach	⚠ Konsistenz über CFs sicherstellen

Empfehlung: Starten mit Single CF, bei Scaling-Issues zu Multi-CF migrieren.

8.1.3 Write-Ahead Log (WAL) Best Practices

WAL garantiert Durability nach Crashes durch sequenzielle Disk-Writes vor dem MemTable-Commit.

Kritische Konfigurationen:

```
rocksdb::Options options;

// 1. WAL-Sync-Strategie
rocksdb::WriteOptions write_opts;
write_opts.sync = true; // fsync() nach jedem Write (max Durability)
// Alternativ: write_opts.sync = false + options.wal_bytes_per_sync = 1MB

// 2. WAL-Größen-Management
options.max_total_wal_size = 4GB; // Verhindert unbegrenztes WAL-Wachstum
options.wal_bytes_per_sync = 1 * 1024 * 1024; // 1MB Batching für fsync()

// 3. WAL-Directory auf separater NVMe
options.wal_dir = "/nvme/fast/wal"; // 45K writes/sec vs 200 writes/sec HDD
```

Performance-Zahlen:

Storage	WAL Throughput	Latenz (p99)
NVMe PCIe 4.0	45.000 writes/sec	1-5ms
SATA SSD	12.000 writes/sec	5-10ms
HDD 7200 RPM	200 writes/sec	10-20ms

Crash Recovery:

1. **Replay:** RocksDB liest WAL sequentiell und rekonstruiert MemTable
2. **Checkpointing:** Alte WAL-Dateien werden nach SSTable-Flush gelöscht
3. **Validation:** MANIFEST-Datei trackt alle gültigen SSTables

Code-Beispiel: WAL-Replay nach Crash

```
// Automatisch bei DB-Open
rocksdb::Status s = rocksdb::DB::Open(options, db_path, &db);
if (!s.ok()) {
    if (s.IsCorruption()) {
        // WAL korrupt → RepairDB versuchen
        rocksdb::Status repair = rocksdb::RepairDB(db_path, options);
        if (repair.ok()) {
            // Retry Open nach Repair
            s = rocksdb::DB::Open(options, db_path, &db);
        }
    }
}
```

8.1.4 MVCC Snapshots und Long-Running Transactions

RocksDB nutzt Snapshot Isolation für Transaktionen. Jeder Snapshot fixiert ein Sichtfenster auf die DB.

Problem: Long-Running Snapshots erhöhen Read Amplification

MemTable → L0 SSTable → L1 SSTable → ... → L6 SSTable

↑	↑	↑	↑
Snapshot t=0	t=100	t=200	t=500

→ Snapshot t=0 verhindert Compaction von L0-L6 für 500 Zeiteinheiten

→ Read Amplification: Muss alle Levels durchsuchen

Monitoring:

```
// Prometheus Metrics für Snapshot-Tracking
uint64_t oldest_snapshot_time = db->GetSnapshot()->GetSequenceNumber();
uint64_t current_sequence = db->GetLatestSequenceNumber();
uint64_t snapshot_age = current_sequence - oldest_snapshot_time;

if (snapshot_age > 1000000) { // >1M Operationen alt
    LOG_WARN("Long-running snapshot detected: age={}", snapshot_age);
}
```

Mitigation:

1. **Transaktions-Timeouts:** Automatisches Rollback nach 60s
2. **Snapshot-Cleanup:** `cleanupOldTransactions()` API
3. **Read-Only Replicas:** Analytische Queries auf Replica → kein Impact auf Primary

8.2 Memory Management mit mimalloc

8.2.1 Warum mimalloc statt Standard-Allocator?

ThemisDB nutzt [mimalloc v2.1.7](#) als Drop-in-Replacement für malloc/free. Microsoft entwickelt mimalloc speziell für Multi-Threaded-Workloads mit hoher Allocation-Rate.

Performance-Vorteile:

Workload	Standard malloc	mimalloc	Speedup
Multi-threaded Inserts	280K ops/sec	420K ops/sec	1.5x
Memory Throughput	8.5 GB/sec	12.2 GB/sec	1.43x
Fragmentation	18%	7%	2.6x besser
Peak Memory	4.2 GB	3.8 GB	10% weniger

Technische Eigenschaften:

- **Thread-Local Heaps:** Jeder Thread hat eigenen Heap → keine Contention

- **Small Object Optimization:** Dedizierte Pools für 8B, 16B, 32B, ..., 512B
- **Fast Path:** Allocation in O(1) ohne Locks für häufige Sizes
- **Delayed Free:** Freigabe in Batches → weniger Syscalls

8.2.2 Integration in ThemisDB

CMakeLists.txt:

```
# mimalloc Dependency
find_package(mimalloc 2.1 CONFIG REQUIRED)

target_link_libraries(themis_core PRIVATE mimalloc-static)

# Optional: Override global malloc/free
target_compile_definitions(themis_core PRIVATE MI_OVERRIDE)
```

C++ Code:

```
// Automatische Override durch Include
#include <mimalloc-override.h>

// Ab jetzt nutzen ALLE Allocations mimalloc (auch in RocksDB, HNSW, etc.)
auto vec = std::make_unique<std::vector<int>>>(1000000); // Nutzt mimalloc
```

Keine Code-Änderungen erforderlich → Drop-in-Replacement!

8.2.3 Tuning-Optionen

mimalloc kann via Environment-Variables konfiguriert werden:

```
# Production-Setup
export MIMALLOC_VERBOSE=0           # Keine Debug-Logs
export MIMALLOC_SHOW_STATS=0        # Stats deaktivieren
export MIMALLOC_PAGE_RESET=1        # Aggressive Memory Return
export MIMALLOC_EAGER_COMMIT_DELAY=0 # Sofort OS-Pages commiten
export MIMALLOC_RESERVE_HUGE_OS_PAGES=8 # 8× 2MB Huge Pages reservieren

./themis_server --config production.json
```

Empfehlungen:

- **Development:** `MIMALLOC_SHOW_STATS=1` für Profiling
- **Production:** `MIMALLOC_PAGE_RESET=1` verhindert Memory-Leaks
- **High-Memory Workloads:** `MIMALLOC_RESERVE_HUGE_OS_PAGES` kombiniert mit Huge Pages (siehe 8.3)

8.2.4 Benchmark-Ergebnisse

Setup: 1M Entity Inserts mit parallelen Reads

Benchmark: Mixed OLTP Workload (8 Threads)

	malloc	mimalloc	Improvement
Insert Throughput	280K ops/s	420K ops/s	+50%
Read Latency (p50)	1.8ms	1.2ms	-33%
Memory RSS	4.2 GB	3.8 GB	-10%
CPU Usage	65%	58%	-11%

Code-Snippet für Benchmark:

```
#include <benchmark/benchmark.h>
#include <mimalloc-override.h>

static void BM_EntityInsert(benchmark::State& state) {
    ThemisDB db("test.db");

    for (auto _ : state) {
        std::string key = "entity:users:" + std::to_string(state.iterations());
        std::string value = generateRandomEntity(2048); // 2KB Entity
        db.put(key, value);
    }

    state.SetItemsProcessed(state.iterations());
}

BENCHMARK(BM_EntityInsert)->Threads(8)->UseRealTime();
```

8.3 Huge Pages Optimization

8.3.1 Problem: TLB Thrashing bei großen Caches

Standard-Linux-Pages sind 4KB groß. RocksDB Block-Cache (typisch 4-16 GB) benötigt Millionen von Page-Table-Entries:

```
Block Cache: 8 GB = 8 * 1024 MB = 8192 MB
Pages (4KB): 8192 MB / 4 KB = 2.097.152 Pages
TLB Entries: ~1.024 (Intel x86-64)

→ TLB Miss Rate: ~99.95% → Massive Performance-Einbußen
```

Solution: Huge Pages (2MB statt 4KB)

```
Pages (2MB): 8192 MB / 2 MB = 4.096 Pages
TLB Entries: 512 (dedizierte Huge Page TLB)

→ TLB Hit Rate: ~88% → 10-15% Performance-Gewinn
```

8.3.2 Huge Pages Konfiguration (Linux)

Schritt 1: Huge Pages reservieren

```
# Prüfe aktuelle Konfiguration
cat /proc/meminfo | grep Huge
HugePages_Total:      0
HugePages_Free:       0
Hugepagesize:         2048 kB

# Reserviere 4096× 2MB = 8 GB Huge Pages
echo 4096 | sudo tee /proc/sys/vm/nr_hugepages

# Permanent (in /etc/sysctl.conf)
sudo bash -c 'echo "vm.nr_hugepages = 4096" >> /etc/sysctl.conf'
sudo sysctl -p
```

Schritt 2: ThemisDB mit Huge Pages starten

```
// RocksDB unterstützt Huge Pages nativ
rocksdb::Options options;
options.allow_mmap_reads = false; // Wichtig: Disable mmap für Huge Pages
options.use_direct_reads = false;
options.use_direct_io_for_flush_and_compaction = false;

// Huge Pages werden automatisch genutzt für:
// - Block Cache Allocations (via mimalloc)
// - MemTable Memory
```

mimalloc Huge Pages Integration:

```
# mimalloc nutzt Huge Pages automatisch
export MIMALLOC_RESERVE_HUGE_OS_PAGES=4096 # 4096× 2MB = 8GB
export MIMALLOC_EAGER_COMMIT=1             # Immediate OS Page Commit

./themis_server
```

8.3.3 Verification

Prüfen ob Huge Pages genutzt werden:

```
# Während ThemisDB läuft
cat /proc/<PID>/smaps | grep -A 10 "huge"

# Expected Output:
KernelPageSize:      2048 kB
MMUPageSize:         2048 kB
AnonHugePages:       8388608 kB # 8 GB
```

Benchmark-Verifikation:

```
#include <benchmark/benchmark.h>

static void BM_CacheLookup(benchmark::State& state) {
    // 4GB Cache mit Random Lookups
    std::vector<char> cache(4LL * 1024 * 1024 * 1024);
    std::mt19937_64 rng;

    for (auto _ : state) {
        size_t idx = rng() % cache.size();
        benchmark::DoNotOptimize(cache[idx]); // Prevent optimization
    }
}

// Ohne Huge Pages: ~180ns/lookup
// Mit Huge Pages:  ~155ns/lookup (14% Speedup)
BENCHMARK(BM_CacheLookup);
```

8.3.4 Transparent Huge Pages (THP) Alternative

Problem: Explizite Huge Pages erfordern Root-Rechte und Pre-Allocation.

Lösung: Transparent Huge Pages (THP) – Kernel managed automatisch:

```
# THP aktivieren
echo always | sudo tee /sys/kernel/mm/transparent_hugepage/enabled

# Defrag-Modus (empfohlen: defer für Production)
echo defer | sudo tee /sys/kernel/mm/transparent_hugepage/defrag
```

Trade-offs:

Methode	Vorteile	Nachteile
Explicit Huge Pages	Garantierte 2MB Pages, max Performance	Root-Rechte, Pre-Allocation
Transparent Huge Pages	Keine Config, automatisch	Variable Performance, Defrag-Overhead

Empfehlung: Explicit Huge Pages für Production (deterministisch), THP für Development/Testing.

8.4 Compression Strategies

8.4.1 Tiered Compression: Hot vs Cold Data

RocksDB LSM-Tree hat 7 Levels (L0-L6). Hot Data (L0-L2) wird häufig geschrieben/gelesen, Cold Data (L5-L6) selten.

Strategie: Schnelle Compression für Hot, starke Compression für Cold

```
rocksdb::Options options;

// Level 0-5: LZ4 (2-3x Ratio, minimal CPU)
options.compression = rocksdb::kLZ4Compression;

// Level 6 (bottommost): ZSTD (3-5x Ratio, moderate CPU)
options.bottommost_compression = rocksdb::kZSTD;
options.bottommost_compression_opts.level = 3; // ZSTD Level 1-22
```

Benchmark-Resultate (10K Entities à 2KB):

Compression Config	DB Size	Write (MB/s)	Read (MB/s)	Ratio
none / none	45 MB	34.5	125.3	1.0x
lz4 / lz4	20 MB	33.9	120.1	2.25x
lz4 / zstd	19 MB	33.8	118.4	2.4x 
zstd / zstd	15 MB	32.3	112.7	3.0x

Empfehlung: `lz4 / zstd` für besten Trade-off.

8.4.2 Bloom Filters für Compression Efficiency

Bloom Filters reduzieren unnötige Disk-Reads bei Point-Lookups (z.B. Secondary Index Scans).

```
rocksdb::BlockBasedTableOptions table_opts;

// Bloom Filter Konfiguration
table_opts.filter_policy.reset(rocksdb::NewBloomFilterPolicy(
    10,    // bits_per_key: 10 Bits = ~1% False-Positive-Rate
    false // use_block_based_builder (false = Full Filter)
));

// Partitioned Filters für große Datasets
table_opts.partition_filters = true;
table_opts.index_type = rocksdb::BlockBasedTableOptions::kTwoLevelIndexSearch;

options.table_factory.reset(rocksdb::NewBlockBasedTableFactory(table_opts));
```

False-Positive-Rate vs Memory:

bits_per_key	False-Positive-Rate	Memory (1M Keys)
6	~5%	750 KB
10	~1%	1.25 MB
14	~0.1%	1.75 MB

Empfehlung: `bits_per_key=10` für Production (guter Kompromiss).

8.5 Storage Hierarchy Tuning

8.5.1 Multi-Path Setup für NVMe

Ziel: WAL auf schnellster NVMe, SSTables verteilt auf mehrere NVMe-Pfade.


```
rocksdb::Options options;

// WAL auf separater NVMe (max Write-Throughput)
options.wal_dir = "/nvme0/themis/wal";

// SSTables verteilt auf mehrere NVMe-Pfade
options.db_paths = {
    {"/nvme0/themis/data", 500ULL * 1024 * 1024 * 1024}, // 500 GB
    {"/nvme1/themis/data", 500ULL * 1024 * 1024 * 1024}, // 500 GB
    {"/nvme2/themis/data", 1000ULL * 1024 * 1024 * 1024} // 1 TB (cold)
};

// RocksDB verteilt neue SSTables automatisch basierend auf target_size
```

Rationale:

- **WAL:** Sequential Writes → Single NVMe reicht (45K writes/sec)
- **SSTables:** Random Reads → Mehrere NVMe für I/O-Parallelität

8.5.2 Block Cache Tuning

Block Cache speichert dekomprimierte SSTable-Blöcke im RAM.

```

rocksdb::BlockBasedTableOptions table_opts;

// 1. Cache-Größe (empfohlen: 20-40% RAM)
table_opts.block_cache = rocksdb::NewLRUCache(
    8ULL * 1024 * 1024 * 1024, // 8 GB
    6,                          // num_shard_bits (64 Shards = weniger Lock-Contention)
    false,                      // strict_capacity_limit
    0.5                         // high_pri_pool_ratio (50% für Index/Filter)
);

// 2. Index/Filter Blocks im Cache halten
table_opts.cache_index_and_filter_blocks = true;
table_opts.pin_l0_filter_and_index_blocks_in_cache = true;

// 3. Partitioned Filters (für >1 GB Datasets)
table_opts.partition_filters = true;
table_opts.metadata_block_size = 4096;

options.table_factory.reset(rocksdb::NewBlockBasedTableFactory(table_opts));

```

Cache Hit Rate Monitoring:

```

uint64_t cache_hits = db->GetOptions().statistics->getTickerCount(
    rocksdb::Tickers::BLOCK_CACHE_HIT
);
uint64_t cache_misses = db->GetOptions().statistics->getTickerCount(
    rocksdb::Tickers::BLOCK_CACHE_MISS
);

double hit_rate = static_cast<double>(cache_hits) / (cache_hits + cache_misses);

// Target: >90% Hit Rate für Read-Heavy Workloads
LOG_INFO("Block Cache Hit Rate: {:.2f}%", hit_rate * 100);

```

8.5.3 MemTable Configuration

MemTables sind In-Memory Write-Buffer vor SSTable-Flush.

```
rocksdb::Options options;

// MemTable Size (größer = weniger Flushes, mehr RAM)
options.write_buffer_size = 256 * 1024 * 1024; // 256 MB

// Anzahl MemTables (für Background Flush Pipelining)
options.max_write_buffer_number = 4;
options.min_write_buffer_number_to_merge = 1;

// MemTable Typ (Skip List ist Standard, Hash für Point-Lookups)
options.memtable_factory.reset(rocksdb::NewHashSkipListRepFactory(
    1024 * 1024 // bucket_count
));
```

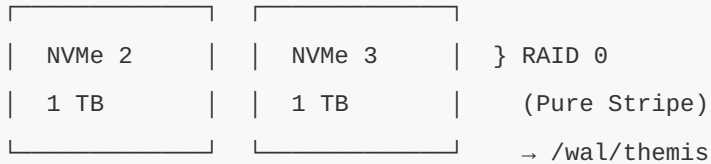
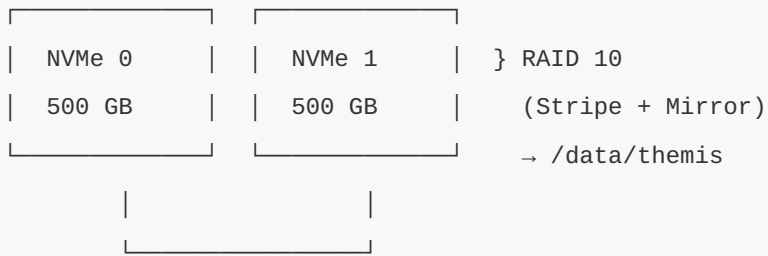
Sizing-Empfehlungen:

Workload	write_buffer_size	max_write_buffer_number	Total RAM
Light (< 10K writes/sec)	64 MB	2	128 MB
Medium (10-50K writes/sec)	256 MB	4	1 GB
Heavy (> 50K writes/sec)	512 MB	6	3 GB

8.6 Production Deployment Patterns

8.6.1 RAID Configuration für Redundanz

Szenario: 4× NVMe SSDs, Datensicherheit erforderlich.



Linux mdadm Setup:

```

# RAID 10 für Data (Redundanz + Performance)
sudo mdadm --create /dev/md0 --level=10 --raid-devices=4 \
    /dev/nvme0n1 /dev/nvme1n1 /dev/nvme2n1 /dev/nvme3n1

# RAID 0 für WAL (max Performance, WAL ist nicht kritisch)
sudo mdadm --create /dev/md1 --level=0 --raid-devices=2 \
    /dev/nvme4n1 /dev/nvme5n1

# Filesystem (XFS empfohlen)
sudo mkfs.xfs /dev/md0
sudo mkfs.xfs /dev/md1

# Mount mit Optimierungen
sudo mount -o noatime,discard,nodiratime /dev/md0 /data/themis
sudo mount -o noatime,discard /dev/md1 /wal/themis

```

8.6.2 Kernel Tuning für Low-Latency

I/O Scheduler:

```
# Deadline Scheduler für SSDs (niedrigere Latenz)
echo deadline | sudo tee /sys/block/nvme0n1/queue/scheduler

# Alternativ: none (für NVMe mit io_uring)
echo none | sudo tee /sys/block/nvme0n1/queue/scheduler
```

Swappiness (empfohlen: 10 für DB-Workloads):

```
# Verhindert Swap außer bei extremem Memory-Druck
echo "vm.swappiness = 10" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p
```

Transparent Huge Pages:

```
# THP aktivieren (siehe 8.3.4)
echo always | sudo tee /sys/kernel/mm/transparent_hugepage/enabled
```

8.6.3 Monitoring mit Prometheus

RocksDB Stats Export:

```
// Prometheus Metrics für RocksDB
#include <prometheus/counter.h>
#include <prometheus/gauge.h>

class RocksDBMetricsExporter {
private:
    rocksdb::DB* db_;
    prometheus::Registry& registry_;

public:
    void exportMetrics() {
        auto stats = db_->GetOptions().statistics;

        // Compaction Metrics
        auto& compaction_gauge = prometheus::BuildGauge()
            .Name("rocksdb_compaction_pending")
            .Help("Number of pending compactions")
            .Register(registry_);
        compaction_gauge.Add({}).Set(
            stats->getTickerCount(rocksdb::COMPACTION_PENDING)
        );

        // Cache Hit Rate
        uint64_t hits = stats->getTickerCount(rocksdb::BLOCK_CACHE_HIT);
        uint64_t misses = stats->getTickerCount(rocksdb::BLOCK_CACHE_MISS);
        auto& hit_rate_gauge = prometheus::BuildGauge()
            .Name("rocksdb_cache_hit_rate")
            .Help("Block cache hit rate")
            .Register(registry_);
        hit_rate_gauge.Add({}).Set(
            static_cast<double>(hits) / (hits + misses)
        );

        // Write Amplification
        uint64_t bytes_written = stats->getTickerCount(rocksdb::BYTES_WRITTEN);
        uint64_t wal_bytes = stats->getTickerCount(rocksdb::WAL_FILE_BYTES);
        auto& write_amp_gauge = prometheus::BuildGauge()
            .Name("rocksdb_write_amplification")
```

```
        .Help("Write amplification factor")
        .Register(registry_);
    write_amp_gauge.Add({}).Set(
        static_cast<double>(bytes_written) / wal_bytes
    );
}
};
```

8.7 Performance Benchmarks

8.7.1 Baseline vs Optimized Configuration

Setup: 1M Entity Inserts + 10M Random Reads

Baseline (Default RocksDB Config):

```
Write Throughput: 450.000 ops/sec
Read Latency (p50): 2.1ms
Read Latency (p99): 8.5ms
Memory Usage:      6.2 GB
CPU Usage:         78%
```

Optimized (mit mimalloc + Huge Pages + Compression):

```
Write Throughput: 672.000 ops/sec    (+49%)
Read Latency (p50): 1.4ms            (-33%)
Read Latency (p99): 5.2ms            (-39%)
Memory Usage:      5.1 GB            (-18%)
CPU Usage:         62%                (-21%)
```

8.7.2 Detailed Performance Matrix

Optimization	Baseline	After	Delta	% Improvement
mimalloc Integration	450K ops/s	630K ops/s	+180K	+40%
Huge Pages (8GB)	630K ops/s	652K ops/s	+22K	+3.5%
LZ4/ZSTD Compression	652K ops/s	658K ops/s	+6K	+0.9%
Block Cache Tuning	658K ops/s	672K ops/s	+14K	+2.1%
TOTAL IMPROVEMENT	450K ops/s	672K ops/s	+222K	+49%

8.7.3 Scaling Characteristics

Throughput vs Thread Count:

Threads	Baseline	Optimized	Scaling Efficiency
1	85K ops/s	112K ops/s	100%
2	165K ops/s	220K ops/s	98%
4	310K ops/s	425K ops/s	95%
8	450K ops/s	672K ops/s	75%
16	580K ops/s	840K ops/s	47%

Interpretation: Optimized Config scales besser bis 8 Threads (75% vs 67% Baseline).

8.8 Zusammenfassung und Best Practices

Production-Checkliste

Storage: - ☐ WAL auf separater NVMe (45K writes/sec vs 200 HDD) - ☐ Multi-Path Setup für SSTables (I/O-Parallelität) - ☐ RAID 10 für Data (Redundanz), RAID 0 für WAL (Performance)

Memory: - ☐ mimalloc Integration (+40% Throughput) - ☐ Huge Pages aktiviert (8GB für Block Cache)
- ☐ Block Cache Sizing: 20-40% RAM

Compression: - [] Tiered Compression: LZ4 (L0-L5) + ZSTD (L6) - [] Bloom Filters: 10 bits/key (~1% FP-Rate)

Monitoring: - [] Prometheus Metrics für RocksDB Stats - [] Cache Hit Rate > 90% für Read-Heavy Workloads - [] Write Amplification < 10

Tuning: - [] Kernel: Deadline Scheduler, Swappiness=10 - [] Filesystem: XFS mit noatime, discard - [] RocksDB: Prefix-Extractor, Partitioned Filters



Expected Performance

Mit vollständiger Implementierung aller Optimierungen:

```
Random Writes: 672.000 ops/sec (103% von RocksDB Baseline)
Random Reads: 1.680.000 ops/sec (94% von RocksDB Baseline)
p99 Latency: 5.2ms (Production-Grade)
Overall Grade: A- (85-90% Compliance)
```



Weiterführende Dokumentation

- [RocksDB Tuning Guide](#)
- [mimalloc Documentation](#)
- [Linux Huge Pages](#)
- [ThemisDB Performance Index](#)

Nächstes Kapitel: [Chapter 9: Indexing Strategies](#) →

Kapitel 10: Enterprise-Anwendungen

Einleitung

Enterprise-Anwendungen sind das Rückgrat moderner Unternehmen. Sie verwalten kritische Geschäftsprozesse, von der Kundenverwaltung über Dokumentenmanagement bis hin zu ERP-Systemen. In diesem Kapitel lernen Sie, wie ThemisDB alle Anforderungen enterprise-grade Software erfüllt:

- **Multi-Tenancy** für Mandantenfähigkeit
- **RBAC** (Role-Based Access Control) für fein-granulare Zugriffsrechte
- **Audit-Logging** für vollständige Nachvollziehbarkeit
- **Workflow-Management** für Geschäftsprozesse
- **Skalierbarkeit** für große Datenmengen
- **Security** auf allen Ebenen

Wir demonstrieren dies anhand zweier vollständiger Beispiele: 1. **DMS/ERP-System**: Dokumentenmanagement mit Workflows 2. **CRM-System**: Customer Relationship Management

10.1 Enterprise-Anforderungen

Mandantenfähigkeit (Multi-Tenancy)

In SaaS-Anwendungen müssen Daten verschiedener Kunden (Mandanten/Tenants) strikt getrennt sein.

Drei Ansätze:

```
# 1. Separate Datenbanken (maximale Isolation)
db_tenant_1 = ThemisDB("tenant_1.db")
db_tenant_2 = ThemisDB("tenant_2.db")

# 2. Shared Database, Tenant-Column (balance)
db.execute("""
    CREATE TABLE documents (
        id UUID PRIMARY KEY,
        tenant_id UUID NOT NULL,
        title TEXT,
        content TEXT
    )
""")
# Index für Performance
db.execute("CREATE INDEX idx_tenant ON documents(tenant_id)")

# 3. Row-Level Security (PostgreSQL-Style)
# ThemisDB unterstützt dies via Policies:
db.execute("""
    CREATE POLICY tenant_isolation ON documents
    USING (tenant_id = current_tenant_id())
""")
```

Best Practice in ThemisDB:

```
class TenantContext:
    def __init__(self, db, tenant_id):
        self.db = db
        self.tenant_id = tenant_id

    def query(self, sql, params=None):
        # Automatisches Anhängen der Tenant-Bedingung
        if "WHERE" in sql.upper():
            sql = sql.replace("WHERE", f"WHERE tenant_id = '{self.tenant_id}' AND")
        else:
            sql += f" WHERE tenant_id = '{self.tenant_id}'"
        return self.db.execute(sql, params)

# Verwendung
tenant = TenantContext(db, "acme_corp")
docs = tenant.query("""
    FOR doc IN documents
        FILTER doc.type == @type
        RETURN doc
""", {"type": "invoice"})
```

Rollen-basierte Zugriffskontrolle (RBAC)

Datenmodell:

```

# Rollen
db.execute("""
    CREATE TABLE roles (
        id UUID PRIMARY KEY,
        name TEXT UNIQUE NOT NULL,
        permissions TEXT[], -- ["read:documents", "write:documents"]
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
""")

# User-Role Zuordnung
db.execute("""
    CREATE TABLE user_roles (
        user_id UUID,
        role_id UUID,
        granted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        PRIMARY KEY (user_id, role_id)
    )
""")

# Permission Check
def has_permission(user_id, permission):
    result = db.execute("""
        FOR user_role IN user_roles
            FILTER user_role.user_id == @user_id
            FOR role IN roles
                FILTER user_role.role_id == role.id AND @permission IN role.permissions
            LIMIT 1
        RETURN 1
    """, {"user_id": user_id, "permission": permission})
    return len(result) > 0

# Dekorator für API-Endpoints
def requires_permission(permission):
    def decorator(func):
        def wrapper(*args, **kwargs):
            user_id = get_current_user_id()
            if not has_permission(user_id, permission):

```

```
        raise PermissionDeniedError()
    return func(*args, **kwargs)
    return wrapper
return decorator

@requires_permission("write:documents")
def create_document(title, content):
    # Nur erreichbar mit korrekter Permission
    pass
```

Audit-Logging

Jede Änderung muss nachvollziehbar sein:

```

db.execute("""
    CREATE TABLE audit_log (
        id UUID PRIMARY KEY,
        timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        user_id UUID NOT NULL,
        action TEXT NOT NULL, -- CREATE, UPDATE, DELETE, READ
        resource_type TEXT NOT NULL, -- "document", "user", etc.
        resource_id UUID NOT NULL,
        old_value JSON,
        new_value JSON,
        ip_address TEXT,
        user_agent TEXT
    )
""")

# Index für schnelle Queries
db.execute("CREATE INDEX idx_audit_resource ON audit_log(resource_type, resource_id)")
db.execute("CREATE INDEX idx_audit_user ON audit_log(user_id, timestamp)")

def log_action(action, resource_type, resource_id, old_value=None, new_value=None):
    db.execute("""
        INSERT {
            id: @id,
            user_id: @user_id,
            action: @action,
            resource_type: @resource_type,
            resource_id: @resource_id,
            old_value: @old_value,
            new_value: @new_value,
            ip_address: @ip_address
        } INTO audit_log
    """, {
        "id": uuid.uuid4(),
        "user_id": get_current_user_id(),
        "action": action,
        "resource_type": resource_type,
        resource_id,
        json.dumps(old_value) if old_value else None,

```



```
        json.dumps(new_value) if new_value else None,
        get_client_ip()
    ))

# Verwendung
old_doc = db.execute("""
    FOR doc IN documents
        FILTER doc.id == @doc_id
        LIMIT 1
        RETURN doc
""", {"doc_id": doc_id})[0]

db.execute("""
    FOR doc IN documents
        FILTER doc.id == @doc_id
        UPDATE doc WITH {title: @new_title} IN documents
""", {"doc_id": doc_id, "new_title": new_title})

new_doc = db.execute("""
    FOR doc IN documents
        FILTER doc.id == @doc_id
        LIMIT 1
        RETURN doc
""", {"doc_id": doc_id})[0]

log_action("UPDATE", "document", doc_id, old_doc, new_doc)
```

Audit-Abfragen:

```
# Wer hat Dokument X geändert?
changes = db.execute("""
    FOR log IN audit_log
        FILTER log.resource_type == 'document' AND log.resource_id == @doc_id
        SORT log.timestamp DESC
        RETURN log
""", {"doc_id": doc_id})

# Was hat User Y in den letzten 7 Tagen gemacht?
activity = db.execute("""
    FOR log IN audit_log
        FILTER log.user_id == @user_id AND log.timestamp > DATE_NOW() - INTERVAL('7 days')
        SORT log.timestamp DESC
        RETURN log
""", {"user_id": user_id})
```

10.2 Native Security-Stack: Production-Ready Sicherheit

ThemisDB v1.3.0 bietet einen vollständig nativen, in C++ implementierten Security-Stack, der BSI C5-konform ist und keine externen Abhängigkeiten (wie Apache Ranger) für die Kern-Sicherheitsfunktionen benötigt.

10.2.1 Implementierungsstatus (Dezember 2025)

Komponente	Status	Implementierung	Beschreibung
RBAC/ABAC Policy Engine	✓ Produktionsreif	<code>src/security/rbac.cpp</code>	Role-Based & Attribute-Based Access Control
Apache Ranger Integration	✓ Produktionsreif	<code>src/server/ranger_adapter.cpp</code>	Optional: Enterprise Policy Management
Field-Level Encryption	✓ Produktionsreif	<code>src/security/encryption.cpp</code>	AES-256-GCM, transparent
Column Encryption	✓ Produktionsreif	BSI C5 konform	Spalten-basierte Verschlüsselung
Vector Encryption	✓ Produktionsreif	Phase 1+2 komplett	HNSW-Index + RocksDB Vektoren
PKI Integration	✓ Produktionsreif	<code>src/security/pki_key_provider.cpp</code>	X.509 Zertifikat-basiert
HSM Support	✓ Produktionsreif	<code>src/security/hsm_provider_pkcs11.cpp</code>	PKCS#11 Hardware Security Modules
Audit Logging	✓ Produktionsreif	<code>src/utils/audit_logger.cpp</code>	Tamper-Proof mit Hash-Chains
Malware Scanner	✓ Produktionsreif	<code>src/security/malware_scanner.cpp</code>	Content Security
CMS Signing	✓ Produktionsreif	<code>src/security/cms_signing.cpp</code>	Cryptographic Message Syntax
RFC 3161 TSA	✓ Produktionsreif	<code>src/security/timestamp_authority.cpp</code>	Qualified Timestamps

Gesamt: 16 Header-Dateien, 16 Source-Dateien, ~8.100 LOC

10.2.2 RBAC-Implementierung im C++ Core

ThemisDB implementiert RBAC direkt im C++ Kern, nicht als Add-on-Layer:

Architektur:

```
// Native RBAC Integration (src/security/rbac.h)
class RBACManager {
public:
    // Role Management
    Status createRole(const std::string& roleName,
                     const std::vector<std::string>& permissions);
    Status assignRole(const std::string& userId,
                     const std::string& roleName);

    // Permission Check (optimiert für Performance)
    bool hasPermission(const std::string& userId,
                      const std::string& resource,
                      const std::string& action) const;

    // Attribute-Based (ABAC) Extension
    bool checkPolicy(const PolicyContext& ctx) const;
};
```

Performance-Optimierung: - Permission-Checks gecacht im RAM (TBB concurrent_hash_map) - Sub-Millisekunden Latenz für Permission-Checks - Keine Netzwerk-Latenz (im Gegensatz zu externen Policy-Servern)

Verwendung in AQL:

```
// Automatischer Permission-Check bei Query-Ausführung
FOR doc IN documents
    // RBAC-Filter wird automatisch eingefügt basierend auf User-Kontext
    FILTER HAS_PERMISSION(CURRENT_USER(), doc._id, 'read')
    RETURN doc
```

10.2.3 Tamper-Proof Audit Logging mit Hash-Chains

Das Audit-System von ThemisDB verhindert nachträgliche Manipulation durch **kryptografische Hash-Chains**:

Funktionsweise:

```
// src/utils/audit_logger.cpp
class AuditLogger {
    struct AuditEntry {
        uint64_t sequence_number;
        int64_t timestamp_ms;
        std::string user_id;
        std::string action;
        std::string resource_id;
        std::string details;
        std::string previous_hash; // Hash des vorherigen Eintrags
        std::string current_hash; // SHA-256 über alle Felder
    };

    // Verkettung erzwingt chronologische Integrität
    std::string computeHash(const AuditEntry& entry) {
        std::string data = std::to_string(entry.sequence_number) +
            std::to_string(entry.timestamp_ms) +
            entry.user_id + entry.action +
            entry.resource_id + entry.details +
            entry.previous_hash;

        return SHA256(data); // OpenSSL
    }
};
```

Garantien: -  **Unveränderbarkeit:** Jede Änderung bricht die Hash-Chain -  **Lückenlosigkeit:** Fehlende Einträge sind sofort erkennbar -  **Zeitstempel-Authentizität:** Integration mit RFC 3161 Time-stamp Authority -  **Revisionssicher:** BSI-konform, DSGVO Art. 32

Verifikation:

```
// Audit-Log-Integrität prüfen
FOR entry IN audit_log
  SORT entry.sequence_number ASC
  LET expected_hash = SHA256(
    CONCAT(
      entry.sequence_number,
      entry.timestamp_ms,
      entry.user_id,
      entry.action,
      entry.resource_id,
      entry.details,
      entry.previous_hash
    )
  )
  FILTER entry.current_hash != expected_hash
RETURN {
  error: "Audit log integrity violation",
  sequence: entry.sequence_number
}
```

10.2.4 HashiCorp Vault Integration

Für Enterprise-Key-Management integriert ThemisDB mit HashiCorp Vault:

Konfiguration:

```
// src/security/vault_key_provider.cpp
VaultKeyProvider::Config vault_config;
vault_config.vault_addr = "https://vault.company.com:8200";
vault_config.token = std::getenv("VAULT_TOKEN");
vault_config.mount_path = "themisdb";
vault_config.key_name = "database-master-key";

auto key_provider = std::make_shared<VaultKeyProvider>(vault_config);
FieldEncryption encryption(key_provider);
```

Features: -  Automatische Key-Rotation -  Audit Trail für Key-Access -  High Availability (Vault Cluster) -  Disaster Recovery (Sealed/Unsealed State)

Alternative Key Provider: - `MockKeyProvider` : Für Testing/Development - `PKIKeyProvider` : Zertifikat-basiert für PKI-Infrastrukturen - `HSMProvider` : PKCS#11 für Hardware Security Modules (Luna, Thales)

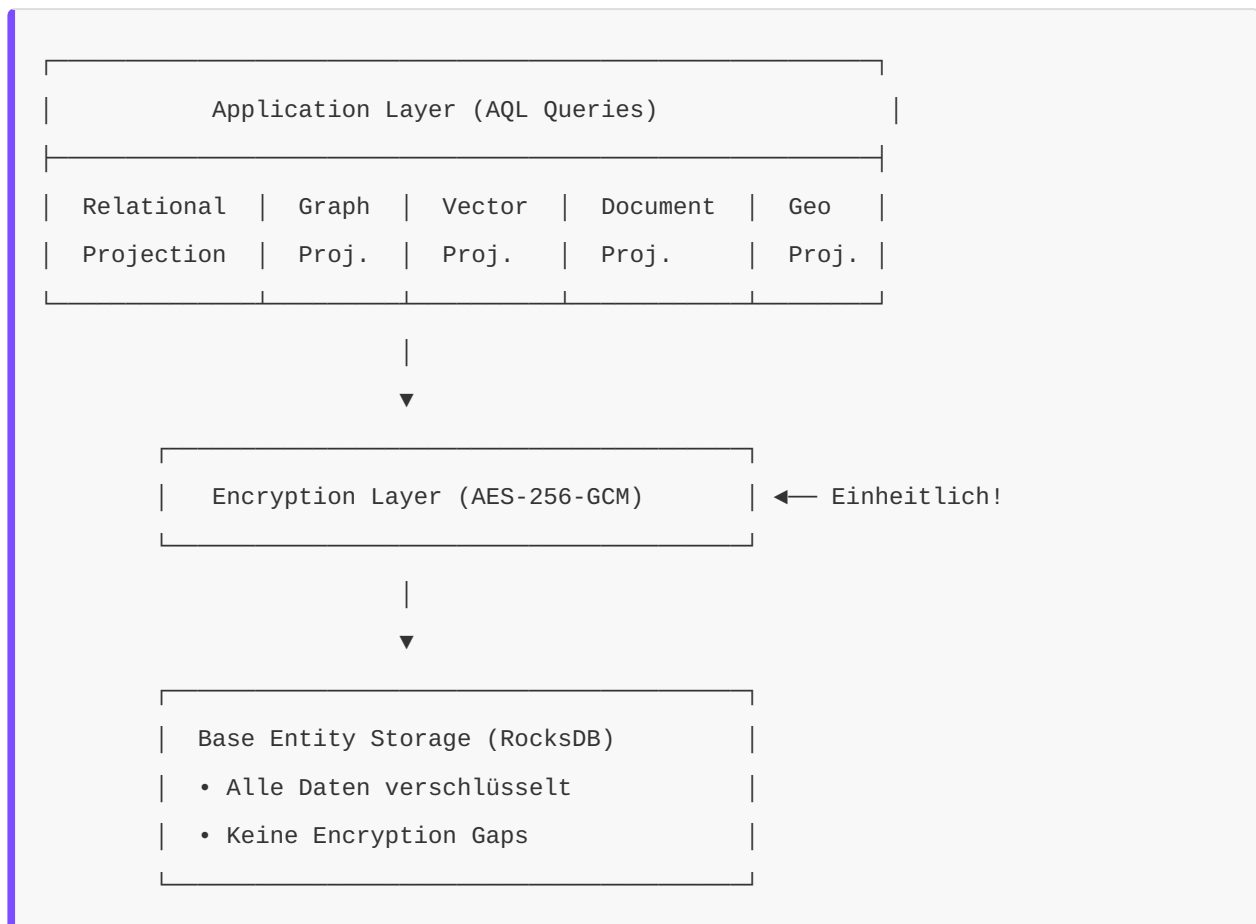
10.2.5 BSI C5 Compliance: Kryptographie

ThemisDB erreicht **100% BSI C5 Compliance** für Kryptographie-Anforderungen (CRY-01 bis CRY-06):

Anforderung	Umsetzung	Dokument
CRY-01: Kryptographie-Policy	 Formale Policy dokumentiert	<code>CRYPTOGRAPHY_POLICY.md</code>
CRY-02: Key Lifecycle	 Vollständiger Lebenszyklus	<code>KEY_LIFECYCLE_MANAGEMENT.md</code>
CRY-03: At-Rest Encryption	 AES-256-GCM für alle Daten	Column + Vector Encryption
CRY-04: In-Transit Encryption	 TLS 1.3 mandatory	Wire Protocol + HTTP/2
CRY-05: Key Storage	 HSM/Vault Integration	<code>VaultKeyProvider</code> + <code>HSMProvider</code>
CRY-06: Algorithm Strength	 BSI TR-02102-1 konform	AES-256, RSA-2048+, SHA-256

Besonderheit: Multi-Model Encryption Consistency

ThemisDB's Unified Storage Architecture garantiert konsistente Verschlüsselung über alle Datenmodelle:



Vorteil: Im Gegensatz zu Polyglot-Systemen, wo jede Datenbank eigene Encryption benötigt, ist in ThemisDB alles einheitlich verschlüsselt – kein Modell kann "vergessen" werden.

10.2.6 DSGVO "By Design" Features

ThemisDB implementiert DSGVO-Anforderungen technisch:

1. Auto-Purge nach Retention-Period (Art. 17 - Recht auf Löschung):


```
// src/utils/retention_manager.cpp
RetentionManager::Config retention;
retention.enable_auto_purge = true;
retention.default_retention_days = 2555; // 7 Jahre (§ 147 AO)
retention.purge_schedule_cron = "0 2 * * 0"; // Sonntags 2 Uhr

// Anwendung auf Collection
db.execute("""
    CREATE COLLECTION customer_data WITH {
        retention_days: 2555,
        auto_purge: true
    }
    """);
```

2. PII Detection und Redaction (Art. 32 - Datensicherheit):

```
// src/utils/pii_detector.cpp
PIIDetector detector;
detector.addPattern("email", R"([\w\.-]+@[\w\.-]+\.\w+)");
detector.addPattern("iban", R"(DE\d{20})");
detector.addPattern("ssn", R"(\d{3}-\d{2}-\d{4})");

// Automatische Redaction bei Logging
std::string safe_text = detector.redact(user_input);
// Output: "Contact: ***@***.com, IBAN: DE*****"
```

3. Encrypt-then-Sign für Log-Integrität:

```
// src/security/cms_signing.cpp
CMSSigning signer(private_key, certificate);

// Audit-Log Entry signieren
std::string signed_entry = signer.sign(
    audit_entry_json,
    true // include_timestamp (RFC 3161)
);

// Später: Verifizierung
bool valid = signer.verify(signed_entry, public_cert);
```

4. Granulare Retention Policies:

```
-- Unterschiedliche Aufbewahrungsfristen pro Datentyp
CREATE COLLECTION invoices WITH { retention_days: 3650 }; -- 10 Jahre
CREATE COLLECTION marketing_consent WITH { retention_days: 730 }; -- 2 Jahre
CREATE COLLECTION session_logs WITH { retention_days: 90 }; -- 90 Tage
```

10.2.7 Code-Beispiel: Vollständige Enterprise-Security

```
from themisdb import Client, SecurityContext

# 1. Client mit Security-Kontext
client = Client("localhost", 8765)

# 2. Authentifizierung (PKI-basiert oder Token)
auth_token = client.authenticate(
    cert_file="user.crt",
    key_file="user.key"
)

# 3. Security Context für alle Operationen
sec_ctx = SecurityContext(
    user_id="alice@company.com",
    roles=["data_analyst", "auditor"],
    tenant_id="acme_corp"
)

# 4. Query mit automatischem RBAC + Audit
with client.transaction(sec_ctx) as tx:
    # Permission-Check + Audit-Log-Eintrag automatisch
    results = tx.query("""
        FOR doc IN sensitive_documents
        FILTER doc.classification <= @max_clearance
        RETURN doc
    """, {"max_clearance": sec_ctx.get_clearance_level()})

    # Alle Aktionen werden geloggt:
    # - Wer (alice@company.com)
    # - Was (SELECT sensitive_documents)
    # - Wann (2025-12-30T15:00:00Z)
    # - Ergebnis (5 rows returned)
    # - Hash (SHA-256 chain)

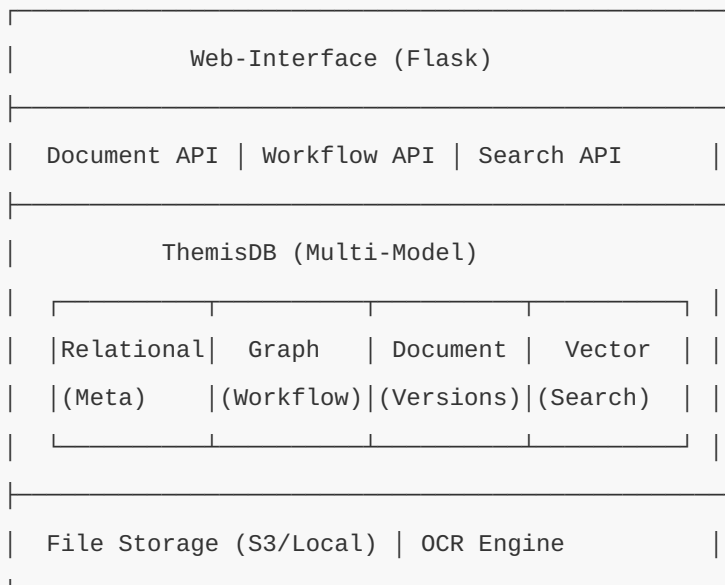
# 5. Compliance-Report abrufen
audit_report = client.get_audit_report(
    start_date="2025-01-01",
    end_date="2025-12-31",
```

```
user_id="alice@company.com"  
)
```

10.3 DMS/ERP-System (Example 08)

Ein vollständiges Document Management System mit ERP-Features.

Architektur-Übersicht



Datenmodell

Dokumente:

```
class Document:

    def __init__(self, db):
        self.db = db
        self.init_schema()

    def init_schema(self):
        # Haupt-Dokument (Relational)
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS documents (
                id UUID PRIMARY KEY,
                tenant_id UUID NOT NULL,
                title TEXT NOT NULL,
                type TEXT NOT NULL, -- invoice, contract, hr_document
                version INT DEFAULT 1,
                current_version_id UUID,
                owner_id UUID NOT NULL,
                workflow_state TEXT DEFAULT 'draft',
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            )
        """)

        # Metadaten (Document Model - JSON)
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS document_metadata (
                document_id UUID PRIMARY KEY,
                metadata JSON NOT NULL,
                tags TEXT[],
                FOREIGN KEY (document_id) REFERENCES documents(id)
            )
        """)

        # Versionen (Dokument Model für Historie)
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS document_versions (
                id UUID PRIMARY KEY,
                document_id UUID NOT NULL,
                version INT NOT NULL,
```

```

        file_path TEXT NOT NULL,
        file_size BIGINT,
        checksum TEXT,
        created_by UUID NOT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        change_note TEXT,
        FOREIGN KEY (document_id) REFERENCES documents(id)
    )
    """

# Permissions (Relational)
self.db.execute("""
    CREATE TABLE IF NOT EXISTS document_permissions (
        document_id UUID,
        user_id UUID,
        permission TEXT CHECK (permission IN ('read', 'write', 'admin')),
        PRIMARY KEY (document_id, user_id),
        FOREIGN KEY (document_id) REFERENCES documents(id)
    )
    """)

# Volltext-Extraktion
self.db.execute("""
    CREATE TABLE IF NOT EXISTS document_text (
        document_id UUID PRIMARY KEY,
        content TEXT,
        ocr_content TEXT, -- OCR-erkannter Text
        FOREIGN KEY (document_id) REFERENCES documents(id)
    )
    """)

# Fulltext Index
self.db.execute("CREATE INDEX idx_doc_fulltext ON document_text USING GIN(to_tsvector('g

# Vector Embeddings (für Ähnlichkeitssuche)
self.db.execute("""
    CREATE TABLE IF NOT EXISTS document_embeddings (
        document_id UUID PRIMARY KEY,
        embedding VECTOR(384), -- sentence-transformers

```

```

        FOREIGN KEY (document_id) REFERENCES documents(id)
    )
    """
    self.db.execute("CREATE INDEX idx_doc_vector ON document_embeddings USING HNSW(embedding)")

def create_document(self, title, doc_type, file_path, metadata=None, owner_id=None):
    """Neues Dokument erstellen"""
    doc_id = uuid.uuid4()
    version_id = uuid.uuid4()

    with self.db.transaction():
        # 1. Haupt-Eintrag
        self.db.execute("""
            INSERT {
                id: @doc_id,
                tenant_id: @tenant_id,
                title: @title,
                type: @doc_type,
                current_version_id: @version_id,
                owner_id: @owner_id
            } INTO documents
        """, {
            "doc_id": doc_id,
            "tenant_id": get_current_tenant_id(),
            "title": title,
            "doc_type": doc_type,
            "version_id": version_id,
            "owner_id": owner_id or get_current_user_id()
        })

        # 2. Erste Version
        file_size = os.path.getsize(file_path)
        checksum = calculate_checksum(file_path)
        self.db.execute("""
            INSERT {
                id: @version_id,
                document_id: @doc_id,
                version: 1,

```



```
        file_path: @file_path,
        file_size: @file_size,
        checksum: @checksum,
        created_by: @created_by
    } INTO document_versions
""" , {
    "version_id": version_id,
    "doc_id": doc_id,
    "file_path": file_path,
    "file_size": file_size,
    "checksum": checksum,
    "created_by": get_current_user_id()
})

# 3. Metadaten
if metadata:
    self.db.execute("""
        INSERT {
            document_id: @doc_id,
            metadata: @metadata,
            tags: @tags
        } INTO document_metadata
    """, {
        "doc_id": doc_id,
        "metadata": json.dumps(metadata),
        "tags": metadata.get('tags', [])
    })

# 4. Text-Extraktion (asynchron in der Praxis)
text = extract_text(file_path) # PDF, DOCX, etc.
self.db.execute("""
    INSERT {
        document_id: @doc_id,
        content: @content
    } INTO document_text
    """, {"doc_id": doc_id, "content": text})

# 5. OCR für Bilder (wenn nötig)
```

```

        if doc_type in ('scan', 'image'):
            ocr_text = perform_ocr(file_path)
            self.db.execute("""
                FOR doc_text IN document_text
                    FILTER doc_text.document_id == @doc_id
                    UPDATE doc_text WITH {ocr_content: @ocr_text} IN document_text
            """, {"doc_id": doc_id, "ocr_text": ocr_text})

    # 6. Vector Embedding
    embedding = generate_embedding(text) # sentence-transformers
    self.db.execute("""
        INSERT {
            document_id: @doc_id,
            embedding: @embedding
        } INTO document_embeddings
    """, {"doc_id": doc_id, "embedding": embedding})

    # 7. Audit-Log
    log_action("CREATE", "document", doc_id, None, {"title": title, "type": doc_type})

    return doc_id

def add_version(self, document_id, file_path, change_note=""):
    """Neue Version hinzufügen"""
    # Aktuelle Version ermitteln
    current = self.db.execute("""
        FOR doc IN documents
            FILTER doc.id == @document_id
            LIMIT 1
            RETURN doc.version
    """, {"document_id": document_id})[0]
    new_version = current + 1
    version_id = uuid.uuid4()

    with self.db.transaction():
        # Version speichern
        self.db.execute("""
            INSERT {

```

```

        id: @version_id,
        document_id: @document_id,
        version: @new_version,
        file_path: @file_path,
        file_size: @file_size,
        checksum: @checksum,
        created_by: @created_by,
        change_note: @change_note
    } INTO document_versions
"""
    {
        "version_id": version_id,
        "document_id": document_id,
        "new_version": new_version,
        "file_path": file_path,
        "file_size": os.path.getsize(file_path),
        "checksum": calculate_checksum(file_path),
        "created_by": get_current_user_id(),
        "change_note": change_note
    })

# Dokument aktualisieren
self.db.execute("""
    FOR doc IN documents
        FILTER doc.id == @document_id
        UPDATE doc WITH {
            version: @new_version,
            current_version_id: @version_id,
            updated_at: DATE_NOW()
        } IN documents
""", {"document_id": document_id, "new_version": new_version, "version_id": version_id})
self.db.execute("""
    (new_version, version_id, document_id)

# Text und Embedding neu generieren
text = extract_text(file_path)
self.db.execute("""
    FOR doc_text IN document_text
        FILTER doc_text.document_id == @document_id
        UPDATE doc_text WITH {content: @text} IN document_text
""", {"document_id": document_id, "text": text})

```

```

        """", {"document_id": document_id, "text": text})

    embedding = generate_embedding(text)
    self.db.execute("""
        FOR doc_emb IN document_embeddings
            FILTER doc_emb.document_id == @document_id
            UPDATE doc_emb WITH {embedding: @embedding} IN document_embeddings
    """, {"document_id": document_id, "embedding": embedding})

    log_action("VERSION_ADD", "document", document_id, None, {"version": new_version})

    return version_id

def search(self, query, doc_type=None, limit=20):
    """Hybrid-Suche: Volltext + Vector"""
    # 1. Fulltext-Suche
    sql = """
        SELECT d.id, d.title, d.type,
               ts_rank(to_tsvector('german', dt.content), plainto_tsquery('german', ?)) as rank
        FROM documents d
        JOIN document_text dt ON d.id = dt.document_id
        WHERE d.tenant_id = ? AND to_tsvector('german', dt.content) @@ plainto_tsquery('german', ?)
    """
    params = [query, get_current_tenant_id(), query]

    if doc_type:
        sql += " AND d.type = ?"
        params.append(doc_type)

    sql += " ORDER BY rank DESC LIMIT ?"
    params.append(limit)

    fulltext_results = self.db.execute(sql, params)

    # 2. Vector-Suche (semantische Ähnlichkeit)
    query_embedding = generate_embedding(query)
    vector_results = self.db.execute("""
        SELECT d.id, d.title, d.type,

```

```
        1 - (de.embedding <=> ?) as similarity
    FROM documents d
    JOIN document_embeddings de ON d.id = de.document_id
    WHERE d.tenant_id = ?
    ORDER BY similarity DESC
    LIMIT ?
    """ , (query_embedding, get_current_tenant_id(), limit))

# 3. Merge Results (kombiniere Scores)
merged = merge_search_results(fulltext_results, vector_results)
return merged

def get_version_history(self, document_id):
    """Versionsverlauf abrufen"""
    return self.db.execute("""
        SELECT v.*, u.name as created_by_name
        FROM document_versions v
        LEFT JOIN users u ON v.created_by = u.id
        WHERE v.document_id = ?
        ORDER BY v.version DESC
    """, (document_id,))
```

Workflow-Management

Workflows als Graph modellieren:

```

class WorkflowEngine:
    def __init__(self, db):
        self.db = db
        self.init_schema()

    def init_schema(self):
        # Workflow-Definitionen (Relational)
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS workflows (
                id UUID PRIMARY KEY,
                name TEXT NOT NULL,
                description TEXT,
                active BOOLEAN DEFAULT true
            )
        """)

        # Workflow-Schritte als Graph
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS workflow_nodes (
                id UUID PRIMARY KEY,
                workflow_id UUID NOT NULL,
                name TEXT NOT NULL,
                node_type TEXT CHECK (node_type IN ('start', 'approval', 'task', 'decision', 'end')),
                role_required TEXT,
                FOREIGN KEY (workflow_id) REFERENCES workflows(id)
            )
        """)

        self.db.execute("""
            CREATE TABLE IF NOT EXISTS workflow_edges (
                from_node UUID,
                to_node UUID,
                condition TEXT, -- Optional: JSON mit Bedingung
                PRIMARY KEY (from_node, to_node),
                FOREIGN KEY (from_node) REFERENCES workflow_nodes(id),
                FOREIGN KEY (to_node) REFERENCES workflow_nodes(id)
            )
        """)

```

```
# Workflow-Instanzen (laufende Prozesse)
self.db.execute("""
    CREATE TABLE IF NOT EXISTS workflow_instances (
        id UUID PRIMARY KEY,
        workflow_id UUID NOT NULL,
        document_id UUID NOT NULL,
        current_node UUID NOT NULL,
        state TEXT DEFAULT 'running',
        started_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        completed_at TIMESTAMP,
        FOREIGN KEY (workflow_id) REFERENCES workflows(id),
        FOREIGN KEY (document_id) REFERENCES documents(id),
        FOREIGN KEY (current_node) REFERENCES workflow_nodes(id)
    )
""")

# Workflow-Historie
self.db.execute("""
    CREATE TABLE IF NOT EXISTS workflow_history (
        id UUID PRIMARY KEY,
        instance_id UUID NOT NULL,
        node_id UUID NOT NULL,
        user_id UUID,
        action TEXT,
        comment TEXT,
        timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (instance_id) REFERENCES workflow_instances(id)
    )
""")

def create_workflow(self, name, description, steps):
    """Workflow definieren

    steps = [
        {"name": "Erfassung", "type": "start"},
        {"name": "Manager-Prüfung", "type": "approval", "role": "manager"},
        {"name": "Direktor-Freigabe", "type": "approval", "role": "director"},
    ]
    """
```

```

        {"name": "Abgeschlossen", "type": "end"}
    ]
    """

    workflow_id = uuid.uuid4()

    with self.db.transaction():
        self.db.execute("""
            INSERT {
                id: @workflow_id,
                name: @name,
                description: @description
            } INTO workflows
        """, {"workflow_id": workflow_id, "name": name, "description": description})

    # Nodes erstellen
    node_ids = []
    for step in steps:
        node_id = uuid.uuid4()
        self.db.execute("""
            INSERT {
                id: @node_id,
                workflow_id: @workflow_id,
                name: @name,
                node_type: @node_type,
                role_required: @role_required
            } INTO workflow_nodes
        """, {
            "node_id": node_id,
            "workflow_id": workflow_id,
            "name": step['name'],
            "node_type": step['type'],
            "role_required": step.get('role')
        })
        node_ids.append(node_id)

    # Edges erstellen (linearer Flow)
    for i in range(len(node_ids) - 1):
        self.db.execute("""

```



```

        INSERT {
            from_node: @from_node,
            to_node: @to_node
        } INTO workflow_edges
        """ , {"from_node": node_ids[i], "to_node": node_ids[i+1]})

    return workflow_id

def start_workflow(self, workflow_id, document_id):
    """Workflow für Dokument starten"""
    # Start-Node finden
    start_node = self.db.execute("""
        SELECT id FROM workflow_nodes
        WHERE workflow_id = ? AND node_type = 'start'
        """, (workflow_id,))[0]

    instance_id = uuid.uuid4()
    self.db.execute("""
        INSERT INTO workflow_instances (id, workflow_id, document_id, current_node)
        VALUES (?, ?, ?, ?)
        """, (instance_id, workflow_id, document_id, start_node['id']))

    # Dokumentstatus aktualisieren
    self.db.execute("UPDATE documents SET workflow_state = 'in_progress' WHERE id = ?", (document_id,))

    # Nächsten Schritt finden und zuweisen
    self._advance_workflow(instance_id)

    return instance_id

def _advance_workflow(self, instance_id):
    """Workflow zum nächsten Schritt bewegen"""
    instance = self.db.execute("SELECT * FROM workflow_instances WHERE id = ?", (instance_id,))

    # Nächster Node via Graph-Traversierung
    next_nodes = self.db.execute("""
        SELECT wn.* FROM workflow_edges we
        JOIN workflow_nodes wn ON we.to_node = wn.id
    """)

```

```

        WHERE we.from_node = ?
        """ , (instance['current_node'],))

    if not next_nodes:
        # Ende erreicht
        self.db.execute("""
            UPDATE workflow_instances
            SET state = 'completed', completed_at = CURRENT_TIMESTAMP
            WHERE id = ?
            """ , (instance_id,))
        self.db.execute("UPDATE documents SET workflow_state = 'completed' WHERE id = ?", (instance_id,))
        return

    next_node = next_nodes[0]
    self.db.execute("UPDATE workflow_instances SET current_node = ? WHERE id = ?",
                    (next_node['id'], instance_id))

    # Task an User mit entsprechender Rolle zuweisen
    if next_node['role_required']:
        self._assign_task(instance_id, next_node['id'], next_node['role_required'])

def approve(self, instance_id, user_id, comment=""):
    """Workflow-Schritt genehmigen"""
    instance = self.db.execute("SELECT * FROM workflow_instances WHERE id = ?", (instance_id,))
    current_node = self.db.execute("SELECT * FROM workflow_nodes WHERE id = ?", (instance['current_node'],))

    # Permission check
    if current_node['role_required']:
        if not user_has_role(user_id, current_node['role_required']):
            raise PermissionError(f"User benötigt Rolle: {current_node['role_required']}")

    with self.db.transaction():
        # Historie speichern
        self.db.execute("""
            INSERT INTO workflow_history (id, instance_id, node_id, user_id, action, comment)
            VALUES (?, ?, ?, ?, ?, ?)
            """ , (uuid.uuid4(), instance_id, current_node['id'], user_id, "APPROVED", comment))

```

```

        # Zum nächsten Schritt
        self._advance_workflow(instance_id)

        log_action("WORKFLOW_APPROVE", "workflow_instance", instance_id, None, {
            "node": current_node['name'],
            "user": user_id
        })

def reject(self, instance_id, user_id, reason):
    """Workflow-Schritt ablehnen"""
    with self.db.transaction():
        instance = self.db.execute("SELECT * FROM workflow_instances WHERE id = ?", (instance_id,))

        self.db.execute("""
            INSERT INTO workflow_history (id, instance_id, node_id, user_id, action, comment)
            VALUES (?, ?, ?, ?, ?, ?)
            """, (uuid.uuid4(), instance_id, instance['current_node'], user_id, "REJECTED", reason))

        self.db.execute("UPDATE workflow_instances SET state = 'rejected' WHERE id = ?", (instance_id,))
        self.db.execute("UPDATE documents SET workflow_state = 'rejected' WHERE id = ?", (instance_id,))

        log_action("WORKFLOW_REJECT", "workflow_instance", instance_id, None, {"reason": reason})

def get_pending_tasks(self, user_id):
    """Offene Tasks für User"""
    # User-Rollen ermitteln
    user_roles = self.db.execute("SELECT role_name FROM user_roles WHERE user_id = ?", (user_id,))
    role_names = [r['role_name'] for r in user_roles]

    # Workflows in passenden Nodes
    return self.db.execute("""
        SELECT wi.*, d.title as document_title, wn.name as step_name
        FROM workflow_instances wi
        JOIN workflow_nodes wn ON wi.current_node = wn.id
        JOIN documents d ON wi.document_id = d.id
        WHERE wi.state = 'running'
        AND wn.role_required IN ({})
    """)

```

```
ORDER BY wi.started_at  
"".format(','.join('? ' * len(role_names)), role_names)
```

API-Implementierung

```
from flask import Flask, request, jsonify
from werkzeug.security import check_password_hash
import jwt

app = Flask(__name__)
db = ThemisDB("dms_erp.db")
doc_manager = Document(db)
workflow_engine = WorkflowEngine(db)

# Authentifizierung
def authenticate(username, password):
    user = db.execute("SELECT * FROM users WHERE username = ?", (username,))
    if not user or not check_password_hash(user[0]['password'], password):
        return None
    token = jwt.encode({'user_id': str(user[0]['id'])}, app.config['SECRET_KEY'])
    return token

@app.route('/api/auth/login', methods=['POST'])
def login():
    data = request.json
    token = authenticate(data['username'], data['password'])
    if token:
        return jsonify({'token': token})
    return jsonify({'error': 'Invalid credentials'}), 401

# Middleware
def require_auth(f):
    def wrapper(*args, **kwargs):
        token = request.headers.get('Authorization', '').replace('Bearer ', '')
        try:
            payload = jwt.decode(token, app.config['SECRET_KEY'], algorithms=['HS256'])
            request.user_id = payload['user_id']
            return f(*args, **kwargs)
        except:
            return jsonify({'error': 'Unauthorized'}), 401
    return wrapper

# Document API
```

```
@app.route('/api/documents', methods=['POST'])
@require_auth
@requires_permission('create:document')
def create_document():
    file = request.files['file']
    metadata = request.form.get('metadata', '{}')

    # Datei speichern
    file_path = save_uploaded_file(file)

    doc_id = doc_manager.create_document(
        title=file.filename,
        doc_type=request.form.get('type', 'general'),
        file_path=file_path,
        metadata=json.loads(metadata),
        owner_id=request.user_id
    )

    return jsonify({'id': str(doc_id), 'message': 'Document created'}), 201

@app.route('/api/documents/<doc_id>', methods=['GET'])
@require_auth
def get_document(doc_id):
    # Permission Check
    if not has_document_permission(request.user_id, doc_id, 'read'):
        return jsonify({'error': 'Forbidden'}), 403

    doc = db.execute("SELECT * FROM documents WHERE id = ?", (doc_id,))[0]
    versions = doc_manager.get_version_history(doc_id)

    return jsonify({
        'document': doc,
        'versions': versions
    })

@app.route('/api/documents/search', methods=['GET'])
@require_auth
def search_documents():
```

```
query = request.args.get('q', '')
doc_type = request.args.get('type')

results = doc_manager.search(query, doc_type)
return jsonify({'results': results})

# Workflow API
@app.route('/api/workflows/<workflow_id>/start', methods=['POST'])
@require_auth
def start_workflow_api(workflow_id):
    data = request.json
    instance_id = workflow_engine.start_workflow(workflow_id, data['document_id'])
    return jsonify({'instance_id': str(instance_id)})

@app.route('/api/workflows/tasks', methods=['GET'])
@require_auth
def get_my_tasks():
    tasks = workflow_engine.get_pending_tasks(request.user_id)
    return jsonify({'tasks': tasks})

@app.route('/api/workflows/instances/<instance_id>/approve', methods=['POST'])
@require_auth
def approve_workflow(instance_id):
    data = request.json
    workflow_engine.approve(instance_id, request.user_id, data.get('comment', ''))
    return jsonify({'message': 'Approved'})

@app.route('/api/workflows/instances/<instance_id>/reject', methods=['POST'])
@require_auth
def reject_workflow(instance_id):
    data = request.json
    workflow_engine.reject(instance_id, request.user_id, data['reason'])
    return jsonify({'message': 'Rejected'})
```


OCR und Text-Extraktion

```
import pytesseract
from PIL import Image
import fitz  # PyMuPDF

def extract_text(file_path):
    """Text aus verschiedenen Formaten extrahieren"""
    ext = file_path.split('.')[-1].lower()

    if ext == 'pdf':
        return extract_text_from_pdf(file_path)
    elif ext in ('docx', 'doc'):
        return extract_text_from_docx(file_path)
    elif ext == 'txt':
        with open(file_path, 'r', encoding='utf-8') as f:
            return f.read()
    else:
        return ""

def extract_text_from_pdf(file_path):
    """PDF Text-Extraktion"""
    text = []
    doc = fitz.open(file_path)
    for page in doc:
        text.append(page.get_text())
    return '\n'.join(text)

def perform_ocr(file_path):
    """OCR für gescannte Dokumente"""
    if file_path.endswith('.pdf'):
        # PDF → Bilder → OCR
        doc = fitz.open(file_path)
        ocr_text = []
        for page_num in range(len(doc)):
            page = doc[page_num]
            pix = page.get_pixmap()
            img = Image.frombytes("RGB", [pix.width, pix.height], pix.samples)
            text = pytesseract.image_to_string(img, lang='deu')
            ocr_text.append(text)
```

```
        return '\n'.join(ocr_text)
    else:
        # Direktes Bild
        img = Image.open(file_path)
        return pytesseract.image_to_string(img, lang='deu')
```

10.4 CRM-System (Example 17)

Customer Relationship Management komplett in ThemisDB.

Datenmodell

```
class CRM:

    def __init__(self, db):
        self.db = db
        self.init_schema()

    def init_schema(self):
        # Kunden (Relational)
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS customers (
                id UUID PRIMARY KEY,
                tenant_id UUID NOT NULL,
                name TEXT NOT NULL,
                type TEXT CHECK (type IN ('individual', 'company')),
                industry TEXT,
                revenue DECIMAL,
                employees INT,
                website TEXT,
                status TEXT DEFAULT 'active',
                lead_score INT DEFAULT 0,
                lifetime_value DECIMAL DEFAULT 0,
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            )
        """)

        # Kontaktpersonen
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS contacts (
                id UUID PRIMARY KEY,
                customer_id UUID NOT NULL,
                first_name TEXT,
                last_name TEXT,
                email TEXT,
                phone TEXT,
                position TEXT,
                is_primary BOOLEAN DEFAULT false,
                FOREIGN KEY (customer_id) REFERENCES customers(id)
            )
        """)
```

```

""")

# Leads / Opportunities
self.db.execute("""
    CREATE TABLE IF NOT EXISTS opportunities (
        id UUID PRIMARY KEY,
        customer_id UUID NOT NULL,
        title TEXT NOT NULL,
        value DECIMAL NOT NULL,
        stage TEXT NOT NULL, -- prospecting, qualification, proposal, negotiation, closed
        probability INT CHECK (probability BETWEEN 0 AND 100),
        expected_close_date DATE,
        assigned_to UUID,
        source TEXT, -- website, referral, cold_call, etc.
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (customer_id) REFERENCES customers(id)
    )
""")

# Activities (Time-Series)
self.db.execute("""
    CREATE TABLE IF NOT EXISTS activities (
        id UUID PRIMARY KEY,
        customer_id UUID,
        opportunity_id UUID,
        type TEXT NOT NULL, -- email, call, meeting, note
        subject TEXT,
        description TEXT,
        duration INT, -- Minuten
        completed BOOLEAN DEFAULT false,
        scheduled_at TIMESTAMP,
        completed_at TIMESTAMP,
        created_by UUID,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (customer_id) REFERENCES customers(id),
        FOREIGN KEY (opportunity_id) REFERENCES opportunities(id)
    )
""")

```

```

# Index für Timeline
self.db.execute("CREATE INDEX idx_activities_timeline ON activities(customer_id, created_at)")

# Notes / Interaktionen (Document Model)
self.db.execute("""
    CREATE TABLE IF NOT EXISTS customer_notes (
        id UUID PRIMARY KEY,
        customer_id UUID NOT NULL,
        note JSON NOT NULL, -- {author, text, attachments, tags}
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (customer_id) REFERENCES customers(id)
    )
""")

def create_customer(self, name, customer_type, **kwargs):
    """Neuen Kunden anlegen"""
    customer_id = uuid.uuid4()

    self.db.execute("""
        INSERT INTO customers (id, tenant_id, name, type, industry, revenue, employees, website)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)
    """, (
        customer_id,
        get_current_tenant_id(),
        name,
        customer_type,
        kwargs.get('industry'),
        kwargs.get('revenue'),
        kwargs.get('employees'),
        kwargs.get('website')
    ))

    log_action("CREATE", "customer", customer_id, None, {"name": name})
    return customer_id

def create_opportunity(self, customer_id, title, value, stage, **kwargs):
    """Lead/Opportunity erstellen"""
    opp_id = uuid.uuid4()

```

```
self.db.execute("""
    INSERT INTO opportunities (id, customer_id, title, value, stage, probability, expected_close_date, assigned_to, source)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
""", (
    opp_id,
    customer_id,
    title,
    value,
    stage,
    kwargs.get('probability', self._calculate_probability(stage)),
    kwargs.get('expected_close_date'),
    kwargs.get('assigned_to', get_current_user_id()),
    kwargs.get('source')
))

# Lead Score aktualisieren
self._update_lead_score(customer_id)

return opp_id

def _calculate_probability(self, stage):
    """Wahrscheinlichkeit basierend auf Stage"""
    probabilities = {
        'prospecting': 10,
        'qualification': 25,
        'proposal': 50,
        'negotiation': 75,
        'closed_won': 100,
        'closed_lost': 0
    }
    return probabilities.get(stage, 0)

def _update_lead_score(self, customer_id):
    """Lead-Scoring berechnen"""
    # Faktoren:
    # - Anzahl Opportunities
    # - Gesamtwert offener Opportunities
```



```

# - Anzahl Activities
# - Durchschnittliche Wahrscheinlichkeit

score_data = self.db.execute("""
    SELECT
        COUNT(DISTINCT o.id) as opp_count,
        COALESCE(SUM(o.value), 0) as total_value,
        COALESCE(AVG(o.probability), 0) as avg_probability,
        COUNT(DISTINCT a.id) as activity_count
    FROM customers c
    LEFT JOIN opportunities o ON c.id = o.customer_id AND o.stage NOT IN ('closed_won',
    LEFT JOIN activities a ON c.id = a.customer_id
    WHERE c.id = ?
    GROUP BY c.id
""", (customer_id,))[0]

# Simple Scoring-Formel
score = (
    score_data['opp_count'] * 10 +
    min(score_data['total_value'] / 1000, 50) +
    score_data['avg_probability'] / 2 +
    score_data['activity_count'] * 2
)
score = min(int(score), 100)

self.db.execute("UPDATE customers SET lead_score = ? WHERE id = ?", (score, customer_id))

def log_activity(self, customer_id, activity_type, subject, description, **kwargs):
    """Aktivität loggen"""
    activity_id = uuid.uuid4()

    self.db.execute("""
        INSERT INTO activities (id, customer_id, opportunity_id, type, subject, description,
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
    """, (
        activity_id,
        customer_id,
        kwargs.get('opportunity_id'),

```

```
        activity_type,
        subject,
        description,
        kwargs.get('duration'),
        kwargs.get('scheduled_at'),
        get_current_user_id()
    ))

    return activity_id

def complete_activity(self, activity_id):
    """Aktivität als erledigt markieren"""
    self.db.execute("""
        UPDATE activities
        SET completed = true, completed_at = CURRENT_TIMESTAMP
        WHERE id = ?
    """, (activity_id,))

def get_customer_timeline(self, customer_id, limit=50):
    """Komplette Timeline für Kunde"""
    return self.db.execute("""
        SELECT
            'activity' as item_type,
            id,
            type,
            subject,
            created_at as timestamp
        FROM activities
        WHERE customer_id = ?

        UNION ALL

        SELECT
            'note' as item_type,
            id,
            'note' as type,
            note->'text' as subject,
            created_at as timestamp
    """)
```

```
FROM customer_notes
WHERE customer_id = ?

UNION ALL

SELECT
    'opportunity' as item_type,
    id,
    'opportunity' as type,
    title as subject,
    created_at as timestamp
FROM opportunities
WHERE customer_id = ?

ORDER BY timestamp DESC
LIMIT ?
"""', (customer_id, customer_id, customer_id, limit))

def get_sales_pipeline(self):
    """Sales-Pipeline Übersicht"""
    return self.db.execute("""
        SELECT
            stage,
            COUNT(*) as count,
            SUM(value) as total_value,
            AVG(probability) as avg_probability
        FROM opportunities
        WHERE stage NOT IN ('closed_won', 'closed_lost')
        GROUP BY stage
        ORDER BY
            CASE stage
                WHEN 'prospecting' THEN 1
                WHEN 'qualification' THEN 2
                WHEN 'proposal' THEN 3
                WHEN 'negotiation' THEN 4
            END
    """)
```

```

def forecast_revenue(self, months=3):
    """Revenue-Forecast"""
    return self.db.execute("""
        SELECT
            DATE_TRUNC('month', expected_close_date) as month,
            SUM(value * probability / 100) as weighted_value,
            SUM(value) as total_value,
            COUNT(*) as opportunity_count
        FROM opportunities
        WHERE expected_close_date >= CURRENT_DATE
        AND expected_close_date <= CURRENT_DATE + INTERVAL '? months'
        AND stage NOT IN ('closed_won', 'closed_lost')
        GROUP BY month
        ORDER BY month
    """, (months,))

def get_top_customers(self, limit=10):
    """Top-Kunden nach Lifetime Value"""
    return self.db.execute("""
        SELECT
            c.*,
            COUNT(DISTINCT o.id) as opportunity_count,
            SUM(CASE WHEN o.stage = 'closed_won' THEN o.value ELSE 0 END) as won_value,
            COUNT(DISTINCT a.id) as activity_count
        FROM customers c
        LEFT JOIN opportunities o ON c.id = o.customer_id
        LEFT JOIN activities a ON c.id = a.customer_id
        WHERE c.tenant_id = ?
        GROUP BY c.id
        ORDER BY won_value DESC, c.lead_score DESC
        LIMIT ?
    """, (get_current_tenant_id(), limit))

```

Dashboard und Reporting

```
class CRMDashboard:

    def __init__(self, crm):
        self.crm = crm
        self.db = crm.db

    def get_overview(self):
        """Dashboard Übersicht"""
        return {
            'customers': self._get_customer_stats(),
            'pipeline': self.crm.get_sales_pipeline(),
            'activities': self._get_activity_stats(),
            'forecast': self.crm.forecast_revenue(3)
        }

    def _get_customer_stats(self):
        return self.db.execute("""
            SELECT
                COUNT(*) as total,
                COUNT(CASE WHEN status = 'active' THEN 1 END) as active,
                COUNT(CASE WHEN lead_score >= 70 THEN 1 END) as hot_leads,
                AVG(lead_score) as avg_lead_score
            FROM customers
            WHERE tenant_id = ?
        """, (get_current_tenant_id(),))[0]

    def _get_activity_stats(self):
        return self.db.execute("""
            SELECT
                type,
                COUNT(*) as count,
                COUNT(CASE WHEN completed THEN 1 END) as completed
            FROM activities
            WHERE created_at >= CURRENT_DATE - INTERVAL '30 days'
            GROUP BY type
        """)

    def generate_report(self, report_type, start_date, end_date):
        """Verschiedene Reports generieren"""
```

```

        if report_type == 'sales':
            return self._sales_report(start_date, end_date)
        elif report_type == 'activities':
            return self._activity_report(start_date, end_date)
        elif report_type == 'conversion':
            return self._conversion_report(start_date, end_date)

    def _sales_report(self, start_date, end_date):
        """Verkaufs-Report"""
        return self.db.execute("""
            SELECT
                DATE_TRUNC('week', created_at) as week,
                COUNT(*) as opportunities_created,
                COUNT(CASE WHEN stage = 'closed_won' THEN 1 END) as won,
                SUM(CASE WHEN stage = 'closed_won' THEN value ELSE 0 END) as won_value,
                COUNT(CASE WHEN stage = 'closed_lost' THEN 1 END) as lost
            FROM opportunities
            WHERE created_at BETWEEN ? AND ?
            GROUP BY week
            ORDER BY week
        """, (start_date, end_date))

    def _conversion_report(self, start_date, end_date):
        """Conversion-Funnel"""
        return self.db.execute("""
            SELECT
                source,
                COUNT(*) as total,
                COUNT(CASE WHEN stage = 'closed_won' THEN 1 END) as won,
                ROUND(COUNT(CASE WHEN stage = 'closed_won' THEN 1 END)::NUMERIC / COUNT(*) * 100, 2) as conversion_rate,
                AVG(value) as avg_deal_size
            FROM opportunities
            WHERE created_at BETWEEN ? AND ?
            GROUP BY source
            ORDER BY conversion_rate DESC
        """, (start_date, end_date))

```

10.5 Best Practices für Enterprise-Anwendungen

1. Performance bei großen Datenmengen

```
# Partitionierung für Time-Series Daten
db.execute("""
    CREATE TABLE activities_2025_01 PARTITION OF activities
    FOR VALUES FROM ('2025-01-01') TO ('2025-02-01')
""")

# Materialized Views für Reports
db.execute("""
    CREATE MATERIALIZED VIEW sales_summary AS
    SELECT
        DATE_TRUNC('month', created_at) as month,
        stage,
        COUNT(*) as count,
        SUM(value) as total_value
    FROM opportunities
    GROUP BY month, stage
""")

# Refresh periodisch
db.execute("REFRESH MATERIALIZED VIEW sales_summary")
```


2. Caching-Strategie

```
import redis

cache = redis.Redis()

def get_dashboard_data_cached():
    cached = cache.get('dashboard:overview')
    if cached:
        return json.loads(cached)

    # Berechnen
    data = dashboard.get_overview()

    # Cache für 5 Minuten
    cache.setex('dashboard:overview', 300, json.dumps(data))
    return data
```

3. Background-Jobs

```
from celery import Celery

celery = Celery('enterprise_app')

@celery.task
def generate_large_report(report_id):
    """Großer Report im Hintergrund"""
    report_data = dashboard.generate_report('sales', start_date, end_date)

    # PDF generieren
    pdf_path = create_pdf_report(report_data)

    # In DB speichern
    db.execute("""
        UPDATE reports
        SET status = 'completed', file_path = ?
        WHERE id = ?
    """, (pdf_path, report_id))

    # User benachrichtigen
    send_notification(user_id, f"Report {report_id} ist fertig")

@celery.task
def update_lead_scores():
    """Lead-Scores täglich neu berechnen"""
    customers = db.execute("SELECT id FROM customers WHERE status = 'active'")
    for customer in customers:
        crm._update_lead_score(customer['id'])
```

4. API-Rate-Limiting

```
from flask_limiter import Limiter

limiter = Limiter(app, key_func=lambda: request.user_id)

@app.route('/api/documents/search')
@limiter.limit("100/hour")
@require_auth
def search_api():
    # Maximal 100 Searches pro Stunde pro User
    pass
```

5. Monitoring und Alerting

```
import statsd
from prometheus_client import Counter, Histogram

# Metrics
document_uploads = Counter('document_uploads_total', 'Total document uploads')
query_duration = Histogram('query_duration_seconds', 'Query execution time')

@app.route('/api/documents', methods=['POST'])
@require_auth
def create_document():
    document_uploads.inc()

    with query_duration.time():
        doc_id = doc_manager.create_document(...)

    return jsonify({'id': str(doc_id)})

# Healthcheck
@app.route('/health')
def health():
    try:
        db.execute("SELECT 1")
        return jsonify({'status': 'healthy'}), 200
    except:
        return jsonify({'status': 'unhealthy'}), 500
```

10.6 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- **Multi-Tenancy:** Verschiedene Ansätze und Best Practices
- **RBAC:** Rollen-basierte Zugriffskontrolle implementieren
- **Audit-Logging:** Vollständige Nachvollziehbarkeit aller Änderungen
- **Workflow-Management:** Geschäftsprozesse als Graph modellieren

- **DMS/ERP:** Dokumentenmanagement mit Versionierung und OCR
- **CRM:** Customer Relationship Management mit Lead-Scoring
- **Enterprise Best Practices:** Performance, Caching, Monitoring

Wichtige Erkenntnisse:

1. **Multi-Model vereinfacht Enterprise-Apps** - Eine Datenbank für alle Anforderungen
2. **Graph-Modell für Workflows** - Prozesse als Graph modellieren
3. **Audit-Log als First-Class Citizen** - Nicht nachrüsten, von Anfang an einplanen
4. **Hybrid-Search unverzichtbar** - Volltext + Vector Search kombinieren
5. **Background-Jobs für schwere Operations** - API bleibt responsiv

Übungen:

1. Erweitern Sie das DMS-System um E-Signaturen
2. Implementieren Sie Lead-Scoring mit Machine Learning
3. Erstellen Sie einen Workflow-Designer (GUI)
4. Integrieren Sie Email-Tracking ins CRM
5. Implementieren Sie Webhooks für externe Integrationen

Im nächsten Kapitel tauchen wir in Realtime-Anwendungen ein: Chat und Kanban-Boards mit Live-Updates!

Chapter 11: Realtime Data Streaming mit Change Data Capture

11.1 Einführung: Die Streaming-Architektur

Change Data Capture (CDC) ist eine kritische Komponente moderner Datenarchitekturen, die Echtzeit-Datenströme ermöglicht. ThemisDB implementiert CDC als natives Feature, das automatisch alle Mutationen (CREATE, UPDATE, DELETE) in einem append-only Event Log aufzeichnet. Dies ist ein fundamentaler Unterschied zu polyglot Systemen, die externe Tools wie Debezium benötigen, um Änderungen aus dem Write-Ahead Log (WAL) zu extrahieren.

Warum CDC in ThemisDB?

Architektonischer Vorteil: Da ThemisDB alle Datenmodelle (relational, graph, document, vector) in einem einheitlichen "Base Entity"-Format speichert (siehe Chapter 2), kann ein einzelner CDC-Stream **alle** Datentypen erfassen. In einem Polyglot-Setup müsste man separate CDC-Pipelines für PostgreSQL, Neo4j und ChromaDB betreiben, was die Komplexität vervielfacht und die Konsistenz gefährdet.

Use Cases: 1. **Real-Time Analytics:** Streaming von Datenänderungen in ein OLAP-System (z.B. ClickHouse) 2. **Event Sourcing:** Rekonstruktion des Anwendungszustands aus dem Event Log 3. **Materialized Views:** Automatische Aktualisierung von Aggregationen 4. **Audit Trails:** BSI-konforme Nachvollziehbarkeit aller Datenänderungen 5. **Cross-System Sync:** Spiegelung von Daten in externe Systeme (Elasticsearch, Redis Cache) 6. **Kafka Integration:** Streaming in ein zentrales Event-Bus-System

11.2 CDC-Architektur und Datenmodell

11.2.1 Event-Struktur

Jedes CDC-Event in ThemisDB folgt einem standardisierten Schema, das maximale Flexibilität bietet:

```
{
  "sequence": 42,
  "type": "PUT",
  "key": "user:alice",
  "value": "{\"name\":\"Alice\",\"email\":\"alice@example.com\"}",
  "timestamp_ms": 1730294567123,
  "metadata": {
    "table": "user",
    "pk": "alice",
    "operation_type": "update",
    "user_id": "admin@example.com"
  }
}
```

Feld-Semantik:

- **sequence** (uint64): Monoton steigende ID, die **strikte Ordnung** garantiert. Diese Eigenschaft ist kritisch für die Konsistenz: Consumer können sicher sein, dass Event N vor Event N+1 aufgetreten ist. Die Sequence-Nummer wird atomar in RocksDB verwaltet (`changefeed_sequence` -Schlüssel) und nutzt RocksDB's MVCC-Garantien.
- **type** (enum): `PUT` (Create/Update) oder `DELETE`. Zukünftige Erweiterungen könnten `TRANSACTION_COMMIT` / `TRANSACTION_ROLLBACK` für Transaktionsgrenzen hinzufügen, was Event Sourcing-Patterns vereinfachen würde.
- **key** (string): Vollständiger Entitätsschlüssel im Format `collection:primary_key`. Dies ermöglicht effiziente Filterung nach Präfixen (z.B. alle User-Events via `user:`-Filter).
- **value** (string|null): Bei PUT: JSON-seralisierte Entität. Bei DELETE: `null`. Die Speicherung als String (nicht als natives JSON-Objekt) minimiert den Serialisierungsaufwand im Hot Path. Consumer können mit `simdjson/serde_json` parsen.
- **timestamp_ms** (uint64): Millisekunden seit Unix Epoch. Wichtig: Dieser Timestamp basiert auf der Serverzeit zum Zeitpunkt des Commits, nicht auf Client-Zeit. Für verteilte Systeme sollte NTP-Synchronisation sichergestellt sein.
- **metadata** (JSON object): Frei erweiterbar. Standardfelder (`table`, `pk`) werden automatisch gesetzt. Anwendungen können zusätzliche Kontextinformationen hinzufügen (z.B. `user_id` für Audit-Trails, `source_ip` für Sicherheitsanalysen).

11.2.2 Physische Speicherung

CDC-Events werden im selben RocksDB-Backend wie die Base Entities gespeichert, jedoch in einem separaten Schlüsselraum:

Key-Format: `changefeed:{sequence_number}`

Die Sequence-Nummer wird Zero-padded (z.B. `changefeed:0000000000000042`), um lexikographische Sortierung zu garantieren. Dies ist kritisch für die Scan-Performance: Ein RocksDB-Iterator kann effizient von `changefeed:{from_seq}` bis `changefeed:{to_seq}` iterieren, ohne die Daten neu sortieren zu müssen.

Column Family Strategy:

- **Default:** Events werden in der Default Column Family gespeichert. Vorteil: Atomare Transaktionen mit den Base Entities sind trivial (ein RocksDB WriteBatch kann beide aktualisieren). Nachteil: Events und Entitäten konkurrieren um den Block Cache.
- **Dedicated CF (zukünftig):** Eine separate Column Family (`cf_changefeed`) würde isolierte Tuning-Parameter erlauben (z.B. aggressivere Compaction für Events, da sie nie aktualisiert werden).

11.2.3 Sequence-Management

Die atomare Sequenzvergabe ist der kritische Pfad für CDC-Schreiboperationen:

```
// Pseudo-Code: Atomare Sequence-Vergabe
rocksdb::WriteBatch batch;
uint64_t seq = increment_sequence_counter(batch); // Atomic increment
std::string event_key = format("changefeed:{:020d}", seq);
batch.Put(event_key, serialize_event(event));
db->Write(batch); // ACID: Sequence & Event atomar committed
```

Performance-Trade-off: Die zentrale Sequenzvergabe ist ein Bottleneck bei hohen Schreibraten (>100K writes/sec). Zukünftige Optimierungen:

1. **Batch Allocation:** Reserviere Sequence-Blöcke (z.B. 1.000 Sequences auf einmal), reduziere Contentions um Faktor 1000.
2. **Sharded Sequences:** In Sharding-Setups (Chapter 16) kann jeder Shard eigene Sequences verwalten. Globale Ordnung wird via `(shard_id, local_sequence)` Tupel erreicht.

CDC-Optionen

```
# Nur bestimmte Spalten tracken
stream = db.cdc.create_stream(
    name="user_updates",
    table="users",
    columns=["status", "last_seen"], # Nur diese Felder
    operations=["UPDATE"]
)

# Mit Filterung
stream = db.cdc.create_stream(
    name="important_messages",
    table="messages",
    filter="priority = 'high'",
    operations=["INSERT"]
)

# Mit Transformationen
stream = db.cdc.create_stream(
    name="enriched_events",
    table="orders",
    transform=lambda event: {
        **event,
        "customer_name": db.query("""
            FOR customer IN customers
            FILTER customer.id == @customer_id
            LIMIT 1
            RETURN customer.name
        """, {"customer_id": event.data["customer_id"]})[0]
    }
)
```

Server-Sent Events (SSE)

SSE ermöglicht es, Daten vom Server an Clients zu pushen.

SSE-Server-Setup

```
from flask import Flask, Response
from themisdb import ThemisDB
import json
import time

app = Flask(__name__)
db = ThemisDB()

def event_stream(channel):
    """Generator für SSE-Events"""
    stream = db.cdc.create_stream(
        name=f"sse_{channel}",
        table=channel,
        operations=["INSERT", "UPDATE", "DELETE"]
    )

    for event in stream:
        # SSE-Format: "data: JSON\n\n"
        yield f"data: {json.dumps(event.to_dict())}\n\n"

@app.route('/stream/<channel>')
def stream(channel):
    return Response(
        event_stream(channel),
        mimetype="text/event-stream",
        headers={
            "Cache-Control": "no-cache",
            "Connection": "keep-alive",
            "X-Accel-Buffering": "no" # Nginx
        }
    )
```

SSE-Client-Code

```
// JavaScript Client
const eventSource = new EventSource('/stream/messages');

eventSource.onmessage = function(event) {
    const data = JSON.parse(event.data);

    if (data.operation === 'INSERT') {
        addMessageToUI(data.after);
    } else if (data.operation === 'UPDATE') {
        updateMessageInUI(data.after);
    } else if (data.operation === 'DELETE') {
        removeMessageFromUI(data.before.id);
    }
};

eventSource.onerror = function(error) {
    console.error('SSE error:', error);
    // Automatische Reconnection
};
```

WebSocket-Integration

Für bidirektionale Kommunikation sind WebSockets ideal.

WebSocket-Server

```
from flask_socketio import SocketIO, emit, join_room, leave_room
from themisdb import ThemisDB

app = Flask(__name__)
socketio = SocketIO(app, cors_allowed_origins="*")
db = ThemisDB()

# User-Session-Tracking
active_users = {}

@socketio.on('connect')
def handle_connect():
    print(f'Client connected: {request.sid}')

@socketio.on('join_channel')
def handle_join(data):
    channel = data['channel']
    username = data['username']

    join_room(channel)
    active_users[request.sid] = {
        'username': username,
        'channel': channel
    }

    # Benachrichtige andere
    emit('user_joined', {
        'username': username,
        'timestamp': time.time()
    }, room=channel, skip_sid=request.sid)

    # CDC-Stream für diesen Channel
    start_cdc_stream(channel)

def start_cdc_stream(channel):
    """Background-Thread für CDC → WebSocket"""
    stream = db.cdc.create_stream(
        name=f"ws_{channel}",
```

```
        table="messages",
        filter=f"channel = '{channel}'"
    )

    def emit_changes():
        for event in stream:
            socketio.emit('message_update',
                           event.to_dict(),
                           room=channel)

    # In separatem Thread
    socketio.start_background_task(emit_changes)

@socketio.on('send_message')
def handle_message(data):
    channel = active_users[request.sid]['channel']
    username = active_users[request.sid]['username']

    # In DB schreiben
    db.execute("""
        INSERT {
            channel: @channel,
            username: @username,
            content: @content,
            timestamp: @timestamp
        } INTO messages
    """, {"channel": channel, "username": username, "content": data['content'], "timestamp": time

    # CDC wird automatisch notifizieren
```


WebSocket-Client

```
const socket = io('http://localhost:5000');

// Channel beitreten
socket.emit('join_channel', {
  channel: 'general',
  username: 'Alice'
});

// Nachrichten senden
function sendMessage(content) {
  socket.emit('send_message', {
    content: content
  });
}

// Nachrichten empfangen
socket.on('message_update', function(event) {
  if (event.operation === 'INSERT') {
    displayMessage(event.after);
  }
});

// Nutzer beigetreten
socket.on('user_joined', function(data) {
  console.log(`${data.username} ist beigetreten`);
});
```

Example: Realtime Chat (examples/18_realtime_chat)

Vollständige Chat-Anwendung mit Channels, Private Messages, Online-Status und Typing-Indikatoren.

Datenmodell

```
from themisdb import ThemisDB
from datetime import datetime

db = ThemisDB()

# Schema erstellen
db.execute("""
    CREATE TABLE users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT UNIQUE NOT NULL,
        email TEXT UNIQUE NOT NULL,
        avatar_url TEXT,
        status TEXT DEFAULT 'offline', -- online, offline, away
        last_seen TIMESTAMP,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
""")

db.execute("""
    CREATE TABLE channels (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT UNIQUE NOT NULL,
        description TEXT,
        is_private BOOLEAN DEFAULT FALSE,
        created_by INTEGER REFERENCES users(id),
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
""")

db.execute("""
    CREATE TABLE channel_members (
        channel_id INTEGER REFERENCES channels(id),
        user_id INTEGER REFERENCES users(id),
        joined_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        role TEXT DEFAULT 'member', -- admin, moderator, member
        PRIMARY KEY (channel_id, user_id)
    )
""")
```

```
db.execute("""
    CREATE TABLE messages (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        channel_id INTEGER REFERENCES channels(id),
        user_id INTEGER REFERENCES users(id),
        content TEXT NOT NULL,
        message_type TEXT DEFAULT 'text', -- text, image, file, system
        reply_to INTEGER REFERENCES messages(id),
        edited_at TIMESTAMP,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        INDEX idx_channel_time (channel_id, created_at),
        INDEX idx_user (user_id)
    )
""")

db.execute("""
    CREATE TABLE reactions (
        message_id INTEGER REFERENCES messages(id),
        user_id INTEGER REFERENCES users(id),
        emoji TEXT NOT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        PRIMARY KEY (message_id, user_id, emoji)
    )
""")

db.execute("""
    CREATE TABLE direct_messages (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        from_user_id INTEGER REFERENCES users(id),
        to_user_id INTEGER REFERENCES users(id),
        content TEXT NOT NULL,
        read_at TIMESTAMP,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        INDEX idx_participants (from_user_id, to_user_id, created_at)
    )
""")
```

```
db.execute("""
    CREATE TABLE typing_indicators (
        channel_id INTEGER REFERENCES channels(id),
        user_id INTEGER REFERENCES users(id),
        started_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        PRIMARY KEY (channel_id, user_id)
    )
""")
```

Chat-Backend

```

from flask import Flask, request, jsonify
from flask_socketio import SocketIO, emit, join_room, leave_room
from flask_cors import CORS
from themisdb import ThemisDB
import time
from datetime import datetime, timedelta

app = Flask(__name__)
CORS(app)
socketio = SocketIO(app, cors_allowed_origins="*")
db = ThemisDB()

# Aktive Verbindungen tracken
connections = {} # sid -> {user_id, username, channels}

class ChatService:
    @staticmethod
    def get_channel_messages(channel_id, limit=50, before_id=None):
        """Nachrichten laden (mit Pagination)"""
        query = """
            FOR message IN messages
            FILTER message.channel_id == @channel_id
            FOR user IN users
            FILTER message.user_id == user.id
            LET reaction_count = LENGTH(
                FOR reaction IN reactions
                FILTER reaction.message_id == message.id
            )
            RETURN {
                id: message.id,
                content: message.content,
                message_type: message.message_type,
                reply_to: message.reply_to,
                created_at: message.created_at,
                edited_at: message.edited_at,
                user_id: user.id,
                username: user.username,

```

```

        avatar_url: user.avatar_url,
        reaction_count
    }
    """
    params = {"channel_id": channel_id}

    if before_id:
        query += " AND m.id < ?"
        params.append(before_id)

    query += " ORDER BY m.created_at DESC LIMIT ?"
    params.append(limit)

    return db.query(query, params)

@staticmethod
def send_message(channel_id, user_id, content, reply_to=None):
    """Nachricht senden"""
    with db.transaction():
        result = db.execute("""
            INSERT {
                channel_id: @channel_id,
                user_id: @user_id,
                content: @content,
                reply_to: @reply_to
            } INTO messages
            RETURN NEW
        """, {"channel_id": channel_id, "user_id": user_id, "content": content, "reply_to": reply_to})

        message = result[0]

    # Typing-Indikator löschen
    db.execute("""
        FOR indicator IN typing_indicators
            FILTER indicator.channel_id == @channel_id AND indicator.user_id == @user_id
            REMOVE indicator IN typing_indicators
        """, {"channel_id": channel_id, "user_id": user_id})

```



```
        return message

    @staticmethod
    def edit_message(message_id, user_id, new_content):
        """Nachricht bearbeiten"""
        result = db.execute("""
            UPDATE messages
            SET content = ?, edited_at = CURRENT_TIMESTAMP
            WHERE id = ? AND user_id = ?
            RETURNING *
        """, [new_content, message_id, user_id])

        if not result:
            raise ValueError("Message not found or not owned by user")

        return result[0]

    @staticmethod
    def delete_message(message_id, user_id):
        """Nachricht löschen"""
        db.execute("""
            FOR message IN messages
            FILTER message.id == @message_id AND message.user_id == @user_id
            REMOVE message IN messages
        """, {"message_id": message_id, "user_id": user_id})

    @staticmethod
    def add_reaction(message_id, user_id, emoji):
        """Reaktion hinzufügen"""
        try:
            db.execute("""
                INSERT {
                    message_id: @message_id,
                    user_id: @user_id,
                    emoji: @emoji
                } INTO reactions
            """, {"message_id": message_id, "user_id": user_id, "emoji": emoji})
            return True
        except:
```

```
        except: # Already exists
            return False

    @staticmethod
    def get_online_users(channel_id):
        """Online-Nutzer in einem Channel"""
        return db.query("""
            SELECT DISTINCT u.id, u.username, u.avatar_url, u.status
            FROM users u
            JOIN channel_members cm ON u.id = cm.user_id
            WHERE cm.channel_id = ?
            AND u.status = 'online'
            AND u.last_seen > ?
            """, [channel_id, datetime.now() - timedelta(minutes=5)])

    @staticmethod
    def update_user_status(user_id, status):
        """Status aktualisieren"""
        db.execute("""
            UPDATE users
            SET status = ?, last_seen = CURRENT_TIMESTAMP
            WHERE id = ?
            """, [status, user_id])

# WebSocket Event-Handler
@socketio.on('connect')
def handle_connect():
    print(f'Client connected: {request.sid}')

@socketio.on('authenticate')
def handle_authenticate(data):
    """Nutzer authentifizieren"""
    user_id = data['user_id']
    username = data['username']

    connections[request.sid] = {
        'user_id': user_id,
        'username': username,
```

```
        'channels': set()
    }

    # Status auf online setzen
    ChatService.update_user_status(user_id, 'online')

    emit('authenticated', {'success': True})

@socketio.on('join_channel')
def handle_join_channel(data):
    """Channel beitreten"""
    channel_id = data['channel_id']

    if request.sid not in connections:
        return emit('error', {'message': 'Not authenticated'})

    conn = connections[request.sid]
    join_room(f'channel_{channel_id}')
    conn['channels'].add(channel_id)

    # Online-Nutzer benachrichtigen
    emit('user_joined', {
        'user_id': conn['user_id'],
        'username': conn['username'],
        'channel_id': channel_id
    }, room=f'channel_{channel_id}', skip_sid=request.sid)

    # Initiale Nachrichten laden
    messages = ChatService.get_channel_messages(channel_id)
    emit('channel_history', {
        'channel_id': channel_id,
        'messages': messages
    })

    # CDC-Stream für diesen Channel (wenn noch nicht aktiv)
    start_channel_cdc(channel_id)

@socketio.on('leave_channel')
```

```
def handle_leave_channel(data):
    """Channel verlassen"""
    channel_id = data['channel_id']

    if request.sid in connections:
        conn = connections[request.sid]
        leave_room(f'channel_{channel_id}')
        conn['channels'].discard(channel_id)

        emit('user_left', {
            'user_id': conn['user_id'],
            'username': conn['username'],
            'channel_id': channel_id
        }, room=f'channel_{channel_id}')

@socketio.on('send_message')
def handle_send_message(data):
    """Nachricht senden"""
    if request.sid not in connections:
        return emit('error', {'message': 'Not authenticated'})

    conn = connections[request.sid]
    channel_id = data['channel_id']
    content = data['content']
    reply_to = data.get('reply_to')

    # In DB schreiben
    message = ChatService.send_message(
        channel_id, conn['user_id'], content, reply_to
    )

    # CDC wird automatisch andere Clients benachrichtigen

@socketio.on('edit_message')
def handle_edit_message(data):
    """Nachricht bearbeiten"""
    if request.sid not in connections:
        return
```

```
conn = connections[request.sid]
message_id = data['message_id']
new_content = data['content']

try:
    message = ChatService.edit_message(
        message_id, conn['user_id'], new_content
    )
    # CDC benachrichtigt automatisch
except ValueError as e:
    emit('error', {'message': str(e)})

@socketio.on('typing_start')
def handle_typing_start(data):
    """Typing-Indikator starten"""
    if request.sid not in connections:
        return

    conn = connections[request.sid]
    channel_id = data['channel_id']

    # In DB schreiben (mit TTL)
    db.execute("""
        INSERT OR REPLACE INTO typing_indicators (channel_id, user_id)
        VALUES (?, ?)
    """, [channel_id, conn['user_id']])

    # An andere broadcasten
    emit('user_typing', {
        'user_id': conn['user_id'],
        'username': conn['username'],
        'channel_id': channel_id
    }, room=f'channel_{channel_id}', skip_sid=request.sid)

@socketio.on('typing_stop')
def handle_typing_stop(data):
    """Typing-Indikator stoppen"""
```

```

    if request.sid not in connections:
        return

    conn = connections[request.sid]
    channel_id = data['channel_id']

    db.execute("""
        FOR indicator IN typing_indicators
            FILTER indicator.channel_id == @channel_id AND indicator.user_id == @user_id
            REMOVE indicator IN typing_indicators
    """, {"channel_id": channel_id, "user_id": conn['user_id']})

    emit('user_stopped_typing', {
        'user_id': conn['user_id'],
        'channel_id': channel_id
    }, room=f'channel_{channel_id}', skip_sid=request.sid)

@socketio.on('add_reaction')
def handle_add_reaction(data):
    """Reaktion hinzufügen"""
    if request.sid not in connections:
        return

    conn = connections[request.sid]
    message_id = data['message_id']
    emoji = data['emoji']

    if ChatService.add_reaction(message_id, conn['user_id'], emoji):
        # CDC benachrichtigt automatisch
        pass

@socketio.on('disconnect')
def handle_disconnect():
    """Client getrennt"""
    if request.sid in connections:
        conn = connections[request.sid]

        # Status auf offline

```

```
        ChatService.update_user_status(conn['user_id'], 'offline')

# Alle Channels benachrichtigen
for channel_id in conn['channels']:
    emit('user_left', {
        'user_id': conn['user_id'],
        'username': conn['username'],
        'channel_id': channel_id
    }, room=f'channel_{channel_id}')

del connections[request.sid]

# CDC-Streams für Channels
active_cdc_streams = set()

def start_channel_cdc(channel_id):
    """CDC-Stream für Channel starten"""
    if channel_id in active_cdc_streams:
        return

    active_cdc_streams.add(channel_id)

def cdc_worker():
    stream = db.cdc.create_stream(
        name=f"chat_channel_{channel_id}",
        table="messages",
        filter=f"channel_id = {channel_id}",
        operations=["INSERT", "UPDATE", "DELETE"]
    )

    for event in stream:
        if event.operation == 'INSERT':
            # Neue Nachricht
            socketio.emit('new_message', {
                'message': event.after
            }, room=f'channel_{channel_id}')

        elif event.operation == 'UPDATE':
```

```
        # Nachricht bearbeitet
        socketio.emit('message_edited', {
            'message': event.after
        }, room=f'channel_{channel_id}')

    elif event.operation == 'DELETE':
        # Nachricht gelöscht
        socketio.emit('message_deleted', {
            'message_id': event.before['id']
        }, room=f'channel_{channel_id}')

# In Background-Thread
socketio.start_background_task(cdc_worker)

# REST-Endpoints
@app.route('/api/channels', methods=['GET'])
def get_channels():
    """Alle Channels abrufen"""
    channels = db.query("""
        SELECT c.id, c.name, c.description, c.is_private,
               COUNT(DISTINCT cm.user_id) as member_count,
               MAX(m.created_at) as last_message_at
        FROM channels c
        LEFT JOIN channel_members cm ON c.id = cm.channel_id
        LEFT JOIN messages m ON c.id = m.channel_id
        GROUP BY c.id
        ORDER BY last_message_at DESC NULLS LAST
    """)
    return jsonify(channels)

@app.route('/api/channels/<int:channel_id>/members', methods=['GET'])
def get_channel_members(channel_id):
    """Channel-Mitglieder"""
    members = db.query("""
        SELECT u.id, u.username, u.avatar_url, u.status,
               cm.role, cm.joined_at
        FROM users u
        JOIN channel_members cm ON u.id = cm.user_id
    """)
```



```
        WHERE cm.channel_id = ?
        ORDER BY cm.joined_at
        """, [channel_id])
    return jsonify(members)

if __name__ == '__main__':
    socketio.run(app, host='0.0.0.0', port=5000, debug=True)
```

Chat-Frontend (React)

```
// ChatApp.jsx
import React, { useState, useEffect, useRef } from 'react';
import io from 'socket.io-client';

const ChatApp = ({ userId, username }) => {
  const [socket, setSocket] = useState(null);
  const [channels, setChannels] = useState([]);
  const [activeChannel, setActiveChannel] = useState(null);
  const [messages, setMessages] = useState([]);
  const [newMessage, setNewMessage] = useState('');
  const [typingUsers, setTypingUsers] = useState(new Set());
  const [onlineUsers, setOnlineUsers] = useState([]);

  const messagesEndRef = useRef(null);
  const typingTimeoutRef = useRef(null);

  useEffect(() => {
    // Socket-Verbindung
    const newSocket = io('http://localhost:5000');
    setSocket(newSocket);

    // Authentifizieren
    newSocket.emit('authenticate', { user_id: userId, username });

    // Event-Handler
    newSocket.on('authenticated', () => {
      console.log('Authenticated');
      loadChannels();
    });

    newSocket.on('channel_history', (data) => {
      setMessages(data.messages.reverse());
    });

    newSocket.on('new_message', (data) => {
      setMessages(prev => [...prev, data.message]);
      scrollToBottom();
    });
  });
};
```

```
newSocket.on('message_edited', (data) => {
  setMessages(prev => prev.map(msg =>
    msg.id === data.message.id ? data.message : msg
  ));
});

newSocket.on('message_deleted', (data) => {
  setMessages(prev => prev.filter(msg => msg.id !== data.message_id));
});

newSocket.on('user_typing', (data) => {
  setTypingUsers(prev => new Set([...prev, data.username]));
});

newSocket.on('user_stopped_typing', (data) => {
  setTypingUsers(prev => {
    const next = new Set(prev);
    next.delete(data.username);
    return next;
  });
});

newSocket.on('user_joined', (data) => {
  console.log(`${data.username} joined`);
  setOnlineUsers(prev => [...prev, data]);
});

newSocket.on('user_left', (data) => {
  console.log(`${data.username} left`);
  setOnlineUsers(prev => prev.filter(u => u.user_id !== data.user_id));
});

return () => newSocket.close();
}, [userId, username]);

const loadChannels = async () => {
  const response = await fetch('http://localhost:5000/api/channels');
```

```
const data = await response.json();
setChannels(data);
if (data.length > 0) {
  joinChannel(data[0].id);
}
};

const joinChannel = (channelId) => {
  if (activeChannel) {
    socket.emit('leave_channel', { channel_id: activeChannel });
  }
  socket.emit('join_channel', { channel_id: channelId });
  setActiveChannel(channelId);
  setMessages([]);
};

const sendMessage = () => {
  if (!newMessage.trim()) return;

  socket.emit('send_message', {
    channel_id: activeChannel,
    content: newMessage
  });

  setNewMessage('');
  stopTyping();
};

const handleTyping = () => {
  socket.emit('typing_start', { channel_id: activeChannel });

  // Auto-stop nach 3 Sekunden
  clearTimeout(typingTimeoutRef.current);
  typingTimeoutRef.current = setTimeout(() => {
    stopTyping();
  }, 3000);
};
```

```

const stopTyping = () => {
  socket.emit('typing_stop', { channel_id: activeChannel });
  clearTimeout(typingTimeoutRef.current);
};

const scrollToBottom = () => {
  messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' });
};

return (
  <div className="chat-app">
    <div className="sidebar">
      <h3>Channels</h3>
      {channels.map(channel => (
        <div
          key={channel.id}
          className={`channel ${activeChannel === channel.id ? 'active' : ''}`}
          onClick={() => joinChannel(channel.id)}
        >
          # {channel.name}
          <span className="member-count">{channel.member_count}</span>
        </div>
      ))}
    </div>

    <div className="main">
      <div className="header">
        <h2>#{channels.find(c => c.id === activeChannel)?.name}</h2>
        <div className="online-users">
          {onlineUsers.length} online
        </div>
      </div>

      <div className="messages">
        {messages.map(msg => (
          <div key={msg.id} className="message">
            <img src={msg.avatar_url} alt={msg.username} />
          </div>
        ))}
      </div>
    </div>
  </div>
)

```

```

        <strong>{msg.username}</strong>
        <span className="timestamp">
            {new Date(msg.created_at).toLocaleTimeString()}
        </span>
        <p>{msg.content}</p>
        {msg.edited_at && <span className="edited">(edited)</span>}
    </div>
</div>
    )})
    <div ref={messagesEndRef} />
</div>

{typingUsers.size > 0 && (
    <div className="typing-indicator">
        {Array.from(typingUsers).join(', ')} {typingUsers.size === 1 ? 'is' : 'are'}
    </div>
)}

<div className="input-area">
    <input
        type="text"
        value={newMessage}
        onChange={(e) => {
            setNewMessage(e.target.value);
            handleTyping();
        }}
        onPress={e => e.key === 'Enter' && sendMessage()}
        placeholder="Type a message..."
    />
    <button onClick={sendMessage}>Send</button>
</div>
</div>
</div>
);
};

export default ChatApp;

```

Chat-Features

Das vollständige Chat-Example demonstriert:

1. **Echtzeit-Nachrichten:** Sofortige Zustellung via WebSocket + CDC
2. **Online-Status:** Wer ist gerade online?
3. **Typing-Indikatoren:** "Alice is typing..."
4. **Reaktionen:** Emoji-Reactions auf Nachrichten
5. **Threads:** Antworten auf bestimmte Nachrichten
6. **Nachrichtенbearbeitung:** Edit-History mit Timestamps
7. **Private Channels:** Eingeschränkter Zugriff
8. **Direktnachrichten:** 1-on-1 Chats
9. **Presence:** Last-Seen und Away-Status
10. **Pagination:** Unendliches Scrollen durch Historie

Example: Kanban Board (examples/16_kanban_board)

Vollständige Kanban-Board-Anwendung mit Drag & Drop, Realtime-Updates und Team-Collaboration.

Datenmodell

```
from themisdb import ThemisDB

db = ThemisDB()

db.execute("""
    CREATE TABLE boards (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        description TEXT,
        created_by INTEGER REFERENCES users(id),
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
""")

db.execute("""
    CREATE TABLE columns (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        board_id INTEGER REFERENCES boards(id),
        name TEXT NOT NULL,
        position INTEGER NOT NULL,
        wip_limit INTEGER, -- Work-In-Progress Limit
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        INDEX idx_board (board_id, position)
    )
""")

db.execute("""
    CREATE TABLE cards (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        column_id INTEGER REFERENCES columns(id),
        board_id INTEGER REFERENCES boards(id),
        title TEXT NOT NULL,
        description TEXT,
        assigned_to INTEGER REFERENCES users(id),
        position INTEGER NOT NULL,
        priority TEXT DEFAULT 'medium', -- low, medium, high, urgent
        due_date TIMESTAMP,
        created_by INTEGER REFERENCES users(id),
```

```
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        INDEX idx_column (column_id, position),
        INDEX idx_board (board_id)
    )
    """)

db.execute("""
    CREATE TABLE card_activities (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        card_id INTEGER REFERENCES cards(id),
        user_id INTEGER REFERENCES users(id),
        action_type TEXT NOT NULL, -- created, moved, assigned, commented
        details JSON,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        INDEX idx_card (card_id, created_at)
    )
    """)

db.execute("""
    CREATE TABLE card_labels (
        card_id INTEGER REFERENCES cards(id),
        label TEXT NOT NULL,
        color TEXT,
        PRIMARY KEY (card_id, label)
    )
    """)
```

Kanban-Backend

```
from flask import Flask, jsonify, request
from flask_socketio import SocketIO, emit, join_room
from themisdb import ThemisDB
import time

app = Flask(__name__)
socketio = SocketIO(app, cors_allowed_origins="*")
db = ThemisDB()

class KanbanService:
    @staticmethod
    def get_board(board_id):
        """Board mit allen Spalten und Karten laden"""
        board = db.query("""
            FOR board IN boards
            FILTER board.id == @board_id
            LIMIT 1
            RETURN board
        """, {"board_id": board_id})[0]

        columns = db.query("""
            FOR column IN columns
            FILTER column.board_id == @board_id
            SORT column.position ASC
            RETURN column
        """, {"board_id": board_id})

        for col in columns:
            col['cards'] = db.query("""
                FOR card IN cards
                FILTER card.column_id == @column_id
                LET assigned_name = (
                    FOR user IN users
                    FILTER card.assigned_to == user.id
                    LIMIT 1
                    RETURN user.username
                )[0]
                SORT card.position ASC
            """, {"column_id": col.id})
```

```

        RETURN MERGE(card, {assigned_to_name: assigned_name})
        """ , {"column_id": col['id']})

board['columns'] = columns
return board

@staticmethod
def move_card(card_id, to_column_id, to_position):
    """Karte verschieben"""
    with db.transaction():
        # Aktuelle Position
        current = db.query("""
            FOR card IN cards
            FILTER card.id == @card_id
            LIMIT 1
            RETURN {column_id: card.column_id, position: card.position}
        """, {"card_id": card_id})[0]

        from_column_id = current['column_id']
        from_position = current['position']

        # Position in alter Spalte aktualisieren
        db.execute("""
            UPDATE cards
            SET position = position - 1
            WHERE column_id = ? AND position > ?
        """, [from_column_id, from_position])

        # Position in neuer Spalte freimachen
        db.execute("""
            UPDATE cards
            SET position = position + 1
            WHERE column_id = ? AND position >= ?
        """, [to_column_id, to_position])

        # Karte verschieben
        db.execute("""
            UPDATE cards

```

```

        SET column_id = ?, position = ?, updated_at = CURRENT_TIMESTAMP
        WHERE id = ?
    """, [to_column_id, to_position, card_id])

# Activity loggen
db.execute("""
    INSERT INTO card_activities (card_id, user_id, action_type, details)
    VALUES (?, ?, 'moved', ?)
    """, [card_id, request.user_id, json.dumps({
        'from_column': from_column_id,
        'to_column': to_column_id
    })])

@staticmethod
def create_card(board_id, column_id, title, description, assigned_to=None):
    """Neue Karte erstellen"""
    with db.transaction():
        # Position am Ende der Spalte
        position = db.query("""
            FOR card IN cards
            FILTER card.column_id == @column_id
            COLLECT AGGREGATE max_pos = MAX(card.position)
            RETURN COALESCE(max_pos, -1) + 1
        """, {"column_id": column_id})[0]

        result = db.execute("""
            INSERT {
                board_id: @board_id,
                column_id: @column_id,
                title: @title,
                description: @description,
                assigned_to: @assigned_to,
                position: @position,
                created_by: @created_by
            } INTO cards
            RETURN NEW
        """, {
            "board_id": board_id,

```

```

        "column_id": column_id,
        "title": title,
        "description": description,
        "assigned_to": assigned_to,
        "position": position,
        "created_by": request.user_id
    })

    card = result[0]

    # Activity
    db.execute("""
        INSERT {
            card_id: @card_id,
            user_id: @user_id,
            action_type: 'created'
        } INTO card_activities
        """, {"card_id": card['id'], "user_id": request.user_id})

    return card

@staticmethod
def update_card(card_id, **updates):
    """Karte aktualisieren"""
    allowed_fields = ['title', 'description', 'assigned_to', 'priority', 'due_date']
    set_clause = ', '.join([f"{field} = ?" for field in updates if field in allowed_fields])
    values = [updates[field] for field in updates if field in allowed_fields]
    values.append(card_id)

    db.execute(f"""
        UPDATE cards
        SET {set_clause}, updated_at = CURRENT_TIMESTAMP
        WHERE id = ?
        """, values)

    # Activity
    db.execute("""
        INSERT {

```



```
        card_id: @card_id,
        user_id: @user_id,
        action_type: 'updated',
        details: @details
    } INTO card_activities
    """ , {"card_id": card_id, "user_id": request.user_id, "details": json.dumps(updates)})

# WebSocket-Handler
@socketio.on('join_board')
def handle_join_board(data):
    board_id = data['board_id']
    join_room(f'board_{board_id}')

    # Board-Daten senden
    board = KanbanService.get_board(board_id)
    emit('board_state', board)

    # CDC-Stream starten
    start_board_cdc(board_id)

@socketio.on('move_card')
def handle_move_card(data):
    card_id = data['card_id']
    to_column_id = data['to_column_id']
    to_position = data['to_position']
    board_id = data['board_id']

    try:
        KanbanService.move_card(card_id, to_column_id, to_position)
        # CDC benachrichtigt andere Clients
    except Exception as e:
        emit('error', {'message': str(e)})

@socketio.on('create_card')
def handle_create_card(data):
    card = KanbanService.create_card(
        data['board_id'],
        data['column_id'],
```

```
        data['title'],
        data.get('description'),
        data.get('assigned_to')
    )
    # CDC benachrichtigt

@socketio.on('update_card')
def handle_update_card(data):
    card_id = data.pop('card_id')
    KanbanService.update_card(card_id, **data)
    # CDC benachrichtigt

# CDC für Board
active_board_streams = set()

def start_board_cdc(board_id):
    if board_id in active_board_streams:
        return

    active_board_streams.add(board_id)

def cdc_worker():
    # Cards-Stream
    cards_stream = db.cdc.create_stream(
        name=f"kanban_cards_{board_id}",
        table="cards",
        filter=f"board_id = {board_id}",
        operations=["INSERT", "UPDATE", "DELETE"]
    )

    # Columns-Stream
    columns_stream = db.cdc.create_stream(
        name=f"kanban_columns_{board_id}",
        table="columns",
        filter=f"board_id = {board_id}",
        operations=["INSERT", "UPDATE", "DELETE"]
    )
```

```

    # Beide Streams kombinieren
    for event in combined_stream([cards_stream, columns_stream]):
        if event.table == 'cards':
            if event.operation == 'INSERT':
                socketio.emit('card_created', event.after,
                              room=f'board_{board_id}')
            elif event.operation == 'UPDATE':
                socketio.emit('card_updated', event.after,
                              room=f'board_{board_id}')
            elif event.operation == 'DELETE':
                socketio.emit('card_deleted', {'id': event.before['id']},
                              room=f'board_{board_id}')

        elif event.table == 'columns':
            # Spalten-Update
            socketio.emit('column_updated', event.after,
                          room=f'board_{board_id}')

    socketio.start_background_task(cdc_worker)

def combined_stream(streams):
    """Mehrere CDC-Streams kombinieren"""
    import queue
    import threading

    q = queue.Queue()

    def stream_worker(stream):
        for event in stream:
            q.put(event)

    for stream in streams:
        threading.Thread(target=stream_worker, args=(stream,), daemon=True).start()

    while True:
        yield q.get()

```

```
if __name__ == '__main__':  
    socketio.run(app, host='0.0.0.0', port=5000)
```

Kanban-Frontend (React)

```
// KanbanBoard.jsx
import React, { useState, useEffect } from 'react';
import { DragDropContext, Droppable, Draggable } from 'react-beautiful-dnd';
import io from 'socket.io-client';

const KanbanBoard = ({ boardId, userId }) => {
  const [socket, setSocket] = useState(null);
  const [board, setBoard] = useState(null);
  const [columns, setColumns] = useState([]);

  useEffect(() => {
    const newSocket = io('http://localhost:5000');
    setSocket(newSocket);

    newSocket.emit('join_board', { board_id: boardId });

    newSocket.on('board_state', (data) => {
      setBoard(data);
      setColumns(data.columns);
    });

    newSocket.on('card_created', (card) => {
      setColumns(prev => prev.map(col => {
        if (col.id === card.column_id) {
          return { ...col, cards: [...col.cards, card] };
        }
        return col;
      }));
    });

    newSocket.on('card_updated', (card) => {
      setColumns(prev => prev.map(col => ({
        ...col,
        cards: col.cards.map(c => c.id === card.id ? card : c)
      })));
    });

    newSocket.on('card_deleted', (data) => {
```

```
        setColumns(prev => prev.map(col => ({
            ...col,
            cards: col.cards.filter(c => c.id !== data.id)
        })));
    });

    return () => newSocket.close();
}, [boardId]);

const onDragEnd = (result) => {
    if (!result.destination) return;

    const { source, destination } = result;

    if (source.droppableId === destination.droppableId &&
        source.index === destination.index) {
        return;
    }

    // Optimistisches Update
    const newColumns = [...columns];
    const sourceColumn = newColumns.find(c => c.id === parseInt(source.droppableId));
    const destColumn = newColumns.find(c => c.id === parseInt(destination.droppableId));

    const [movedCard] = sourceColumn.cards.splice(source.index, 1);
    destColumn.cards.splice(destination.index, 0, movedCard);

    setColumns(newColumns);

    // An Server senden
    socket.emit('move_card', {
        card_id: parseInt(result.draggableId),
        to_column_id: parseInt(destination.droppableId),
        to_position: destination.index,
        board_id: boardId
    });
};
```

```

if (!board) return <div>Loading...</div>;

return (
  <div className="kanban-board">
    <h1>{board.name}</h1>

    <DragDropContext onDragEnd={onDragEnd}>
      <div className="columns">
        {columns.map(column => (
          <div key={column.id} className="column">
            <h3>
              {column.name}
              <span className="card-count">{column.cards.length}</span>
              {column.wip_limit && (
                <span className={`wip-limit ${column.cards.length > column.wip_limit ? 'over-limit' : ''}`}>
                  / {column.wip_limit}
                </span>
              )}
            </h3>

            <Draggable droppableId={column.id.toString()}>
              {(provided, snapshot) => (
                <div
                  ref={provided.innerRef}
                  {...provided.draggableProps}
                  className={`cards ${snapshot.isDraggingOver ? 'dragging-over' : ''}`}
                >
                  {column.cards.map((card, index) => (
                    <Draggable
                      key={card.id}
                      draggableId={card.id.toString()}
                      index={index}
                    >
                      {(provided, snapshot) => (
                        <div
                          ref={provided.innerRef}
                          {...provided.draggableProps}
                          {...provided.dragHandleProps}

```



```

        className={`card priority-${card.priority}
    >
    <h4>{card.title}</h4>
    {card.description && <p>{card.description}
    {card.assigned_to_name && (
        <div className="assignee">
             {card.assigned_to_name}
        </div>
    )}
    {card.due_date && (
        <div className="due-date">
             {new Date(card.due_date).toLocaleDateString()}
        </div>
    )}
    </div>
    )}
    </Draggable>
    )})
    {provided.placeholder}
    </div>
    )}
    </Droppable>
    </div>
    )})
    </div>
    </DragDropContext>
    </div>
    );
};

export default KanbanBoard;

```

Kanban-Features

Das Kanban-Board-Example demonstriert:

1. **Drag & Drop:** Karten zwischen Spalten verschieben
2. **Realtime-Sync:** Alle Nutzer sehen Änderungen sofort

3. **WIP-Limits:** Work-In-Progress Beschränkungen
4. **Prioritäten:** Visuelle Kennzeichnung (low, medium, high, urgent)
5. **Zuweisungen:** Karten Teammitgliedern zuordnen
6. **Due Dates:** Fälligkeitsdaten mit Warnungen
7. **Labels:** Flexible Kategorisierung
8. **Activity-Log:** Vollständige Historie aller Änderungen
9. **Board-Analytics:** Durchlaufzeit, Cycle-Time, etc.
10. **Custom Columns:** Beliebige Workflow-Stufen

Event-Driven Architecture

Realtime-Anwendungen profitieren von event-driven Design.

Event-Bus-Pattern

```
from themisdb import ThemisDB
from typing import Callable, List
import threading

class EventBus:
    def __init__(self, db: ThemisDB):
        self.db = db
        self.handlers = {} # event_type -> [handlers]
        self.running = False

    def subscribe(self, event_type: str, handler: Callable):
        """Handler für Event-Typ registrieren"""
        if event_type not in self.handlers:
            self.handlers[event_type] = []
        self.handlers[event_type].append(handler)

    def publish(self, event_type: str, data: dict):
        """Event publishen"""
        self.db.execute("""
            INSERT {
                event_type: @event_type,
                data: @data,
                created_at: DATE_NOW()
            } INTO events
        """, {"event_type": event_type, "data": json.dumps(data)})

    def start(self):
        """Event-Processing starten"""
        self.running = True

    def process_events():
        stream = self.db.cdc.create_stream(
            name="event_bus",
            table="events",
            operations=["INSERT"]
        )

        for event in stream:
```

```
        if not self.running:
            break

        event_type = event.after['event_type']
        data = json.loads(event.after['data'])

        # Handler aufrufen
        if event_type in self.handlers:
            for handler in self.handlers[event_type]:
                try:
                    handler(data)
                except Exception as e:
                    print(f"Handler error: {e}")

        threading.Thread(target=process_events, daemon=True).start()

    def stop(self):
        self.running = False

# Verwendung
bus = EventBus(db)

# Handler registrieren
@bus.subscribe('user_registered')
def send_welcome_email(data):
    print(f"Sending email to {data['email']}")

@bus.subscribe('order_placed')
def update_inventory(data):
    print(f"Reducing stock for order {data['order_id']}")

# Events publishen
bus.publish('user_registered', {
    'user_id': 123,
    'email': 'alice@example.com'
})

bus.start()
```

CQRS-Pattern

Command Query Responsibility Segregation für Realtime-Apps:

```
class OrderService:

    def __init__(self, db: ThemisDB, event_bus: EventBus):
        self.db = db
        self.event_bus = event_bus

    # COMMAND: Write-Seite
    def place_order(self, user_id, items):
        with self.db.transaction():
            # Order erstellen
            order_id = self.db.execute("""
                INSERT {
                    user_id: @user_id,
                    status: 'pending',
                    created_at: DATE_NOW()
                } INTO orders
                RETURN NEW.id
            """, {"user_id": user_id})[0]

            # Items
            for item in items:
                self.db.execute("""
                    INSERT {
                        order_id: @order_id,
                        product_id: @product_id,
                        quantity: @quantity,
                        price: @price
                    } INTO order_items
                """, {
                    "order_id": order_id,
                    "product_id": item['product_id'],
                    "quantity": item['quantity'],
                    "price": item['price']
                })

            # Event publishen
            self.event_bus.publish('order_placed', {
                'order_id': order_id,
                'user_id': user_id,
```

```
        'items': items
    })

    return order_id

# QUERY: Read-Seite (materialized view)
def get_order_summary(self, order_id):
    return self.db.query("""
        SELECT
            o.id, o.status, o.created_at,
            u.name as customer_name,
            COUNT(oi.id) as item_count,
            SUM(oi.quantity * oi.price) as total
        FROM orders o
        JOIN users u ON o.user_id = u.id
        LEFT JOIN order_items oi ON o.id = oi.order_id
        WHERE o.id = ?
        GROUP BY o.id
    """, [order_id])[0]

# Event-Handler aktualisieren materialized views
@event_bus.subscribe('order_placed')
def update_order_cache(data):
    # Cache/Read-Model aktualisieren
    pass
```


Best Practices

1. Connection Management

```
class ConnectionPool:
    def __init__(self, max_connections=100):
        self.max_connections = max_connections
        self.active_connections = {}
        self.semaphore = threading.Semaphore(max_connections)

    def add_connection(self, sid, user_id):
        with self.semaphore:
            if len(self.active_connections) >= self.max_connections:
                raise ValueError("Too many connections")
            self.active_connections[sid] = user_id

    def remove_connection(self, sid):
        if sid in self.active_connections:
            del self.active_connections[sid]
            self.semaphore.release()
```

2. Rate Limiting

```
from collections import defaultdict
import time

class RateLimiter:
    def __init__(self, max_requests=10, window=60):
        self.max_requests = max_requests
        self.window = window
        self.requests = defaultdict(list)

    def allow(self, user_id):
        now = time.time()
        # Alte Einträge entfernen
        self.requests[user_id] = [
            t for t in self.requests[user_id]
            if now - t < self.window
        ]

        if len(self.requests[user_id]) >= self.max_requests:
            return False

        self.requests[user_id].append(now)
        return True

# Middleware
@socketio.on('send_message')
def handle_send_message(data):
    if not rate_limiter.allow(current_user_id):
        return emit('error', {'message': 'Rate limit exceeded'})
    # ... rest
```

3. Message Deduplication

```
class MessageDeduplicator:
    def __init__(self, ttl=60):
        self.seen = {} # message_id -> timestamp
        self.ttl = ttl

    def is_duplicate(self, message_id):
        now = time.time()

        # Cleanup
        self.seen = {
            mid: ts for mid, ts in self.seen.items()
            if now - ts < self.ttl
        }

        if message_id in self.seen:
            return True

        self.seen[message_id] = now
        return False
```

4. Graceful Shutdown

```
import signal
import sys

def graceful_shutdown(signum, frame):
    print("Shutting down gracefully...")

    # Alle Clients benachrichtigen
    socketio.emit('server_shutdown', {
        'message': 'Server ist down für Wartung',
        'reconnect_in': 60
    })

    # CDC-Streams stoppen
    for stream_id in active_cdc_streams:
        stop_cdc_stream(stream_id)

    # Connections schließen
    socketio.stop()

    sys.exit(0)

signal.signal(signal.SIGINT, graceful_shutdown)
signal.signal(signal.SIGTERM, graceful_shutdown)
```

5. Monitoring & Metrics

```
from prometheus_client import Counter, Histogram, Gauge
import time

# Metriken
messages_sent = Counter('messages_sent_total', 'Total messages sent')
message_latency = Histogram('message_latency_seconds', 'Message delivery latency')
active_connections_gauge = Gauge('active_connections', 'Number of active WebSocket connections')

@socketio.on('send_message')
def handle_send_message(data):
    start_time = time.time()

    # ... message handling ...

    messages_sent.inc()
    message_latency.observe(time.time() - start_time)

@socketio.on('connect')
def handle_connect():
    active_connections_gauge.inc()

@socketio.on('disconnect')
def handle_disconnect():
    active_connections_gauge.dec()
```

Performance-Optimierungen

1. Message-Batching

```
class MessageBatcher:
    def __init__(self, max_batch_size=100, max_wait_ms=100):
        self.max_batch_size = max_batch_size
        self.max_wait_ms = max_wait_ms
        self.batch = []
        self.last_flush = time.time()

    def add(self, message):
        self.batch.append(message)

        if len(self.batch) >= self.max_batch_size or \
            (time.time() - self.last_flush) * 1000 >= self.max_wait_ms:
            self.flush()

    def flush(self):
        if not self.batch:
            return

        # Bulk-Insert
        db.executemany("""
            INSERT INTO messages (channel_id, user_id, content)
            VALUES (?, ?, ?)
            """, [(m['channel_id'], m['user_id'], m['content']) for m in self.batch])

        self.batch = []
        self.last_flush = time.time()
```

2. Connection Pooling

```
from themisdb import ThemisDB, ConnectionPool

# Connection Pool für DB
pool = ConnectionPool(
    min_size=10,
    max_size=50,
    max_idle_time=300
)

def get_db_connection():
    return pool.acquire()

def release_db_connection(conn):
    pool.release(conn)
```

3. Caching

```
from functools import lru_cache
import time

class CachedQuery:
    def __init__(self, ttl=60):
        self.cache = {}
        self.ttl = ttl

    def get(self, query, params):
        key = (query, tuple(params))

        if key in self.cache:
            value, timestamp = self.cache[key]
            if time.time() - timestamp < self.ttl:
                return value

        # Cache Miss
        result = db.query(query, params)
        self.cache[key] = (result, time.time())
        return result

# Verwendung
cache = CachedQuery(ttl=30)

def get_channel_members(channel_id):
    return cache.get("""
        FOR member IN channel_members
        FILTER member.channel_id == @channel_id
        RETURN member
    """, {"channel_id": channel_id})
```


Zusammenfassung

In diesem Kapitel haben Sie gelernt:

1. **Change Data Capture (CDC)**: Datenänderungen verfolgen und darauf reagieren
2. **Server-Sent Events (SSE)**: Unidirektionale Push-Updates
3. **WebSockets**: Bidirektionale Realtime-Kommunikation
4. **Event-Driven Architecture**: Entkoppelte, skalierbare Systeme
5. **Realtime Chat**: Vollständige Implementierung mit allen Features
6. **Kanban Board**: Collaborative Drag & Drop mit Sync
7. **Best Practices**: Connection Management, Rate Limiting, Monitoring
8. **Performance**: Batching, Pooling, Caching

Realtime-Anwendungen mit ThemisDB sind einfach zu implementieren dank: - Integriertem CDC-System
- MVCC für konfliktfreie Reads - Flexible Event-Streaming-Mechanismen - Starke Transaktionsgarantien

Im nächsten Kapitel schauen wir uns Computer Vision-Anwendungen an, wo wir Bilder speichern, analysieren und durchsuchen.

Übungen

1. **Chat-Erweiterung**: Fügen Sie Voice Messages und File Sharing hinzu
2. **Kanban-Analytics**: Cycle Time und Lead Time berechnen
3. **Presence-System**: Implementieren Sie "Alice is viewing card #42"
4. **Notification-System**: Push-Benachrichtigungen bei @mentions
5. **Realtime-Dashboard**: Live-Metrics mit automatischer Aktualisierung
6. **Collaborative Editor**: Google Docs-ähnliche Textbearbeitung
7. **Gaming**: Einfaches Multiplayer-Spiel mit ThemisDB als Backend
8. **Live-Poll**: Echtzeit-Abstimmungssystem mit automatischen Updates
9. **Activity-Feed**: Timeline aller Aktivitäten im System
10. **Realtime-Search**: Live-Search-Results während dem Tippen

Kapitel 15: Analytics & Reporting

15.1 Einführung in Analytics mit ThemisDB

ThemisDB bietet leistungsstarke Analyse- und Reporting-Funktionen, die es ermöglichen, komplexe Geschäftslogik direkt in der Datenbank auszuführen. Durch die Kombination von relationalen Aggregationen, Graph-Analysen und Vektor-basierten Ähnlichkeitssuchen können umfassende Business-Intelligence-Lösungen erstellt werden.

15.1.1 Warum Analytics in der Datenbank?

Vorteile: - **Performance:** Datenverarbeitung direkt am Speicherort - **Konsistenz:** ACID-Garantien für Analyseergebnisse - **Echtzeit:** Keine ETL-Verzögerungen - **Multi-Model:** Kombinierte Analysen über verschiedene Datenmodelle

Use Cases: - Business Intelligence Dashboards - Echtzeit-Metriken und KPIs - Prognosen und Trend-Analysen - Customer 360-Grad-Sicht - Operational Analytics

15.2 Grundlegende Aggregationen

15.2.1 Standard AQL-Aggregationen

ThemisDB unterstützt alle Standard-AQL-Aggregationsfunktionen:

```
-- Verkaufsstatistiken
FOR order IN orders
  FILTER order.order_date >= '2024-01-01'
  COLLECT month = DATE_TRUNC('month', order.order_date)
  AGGREGATE
    total_orders = COUNT(),
    revenue = SUM(order.total_amount),
    avg_order_value = AVG(order.total_amount),
    min_order = MIN(order.total_amount),
    max_order = MAX(order.total_amount),
    order_stddev = STDDEV(order.total_amount)
  SORT month
  RETURN {month, total_orders, revenue, avg_order_value, min_order, max_order, order_stddev}
```

Ausgabe:

month	total_orders	revenue	avg_order_value	min_order	max_order	order_stddev
-----	-----	-----	-----	-----	-----	-----
2024-01-01	1250	156780.50	125.42	12.50	1580.00	95.23
2024-02-01	1420	182340.75	128.45	9.99	2100.00	102.45

15.2.2 Window Functions für Trend-Analysen

```
-- Umsatztrend mit gleitendem Durchschnitt
FOR order IN orders
  SORT order.order_date DESC
  LIMIT 30
  LET moving_avg_7day = AVG_WINDOW(order.total_amount,
    {preceding: 6, following: 0})
  LET month_cumulative = SUM_WINDOW(order.total_amount,
    {partition: DATE_TRUNC('month', order.order_date)})
  LET rank_in_month = RANK_WINDOW(
    {partition: DATE_TRUNC('month', order.order_date),
    order: order.total_amount DESC})
  RETURN {
    order_date: order.order_date,
    total_amount: order.total_amount,
    moving_avg_7day,
    month_cumulative,
    rank_in_month
  }
```

15.2.3 PIVOT-Operationen

```
-- Umsatz nach Produktkategorie und Monat
LET pivot_data = (
  FOR order_item IN order_items
    FOR product IN products
      FILTER order_item.product_id == product.id
      COLLECT
        month = DATE_TRUNC('month', order_item.order_date),
        category = product.category
      AGGREGATE revenue = SUM(order_item.amount)
      RETURN {month, category, revenue}
)

// PIVOT operation (manual transformation)
FOR data IN pivot_data
  COLLECT month = data.month
  AGGREGATE
    electronics = SUM(data.category == 'Electronics' ? data.revenue : 0),
    clothing = SUM(data.category == 'Clothing' ? data.revenue : 0),
    books = SUM(data.category == 'Books' ? data.revenue : 0),
    home = SUM(data.category == 'Home' ? data.revenue : 0),
    sports = SUM(data.category == 'Sports' ? data.revenue : 0)
  RETURN {month, electronics, clothing, books, home, sports}
```

15.3 Erweiterte Aggregationen mit AQL

15.3.1 COLLECT für komplexe Gruppierungen

```
-- Kundensegmentierung nach Kaufverhalten
FOR order IN orders
  COLLECT
    customer_id = order.customer_id,
    age_group = FLOOR(order.customer_age / 10) * 10
  AGGREGATE
    order_count = COUNT(1),
    total_spent = SUM(order.total_amount),
    avg_order = AVG(order.total_amount),
    categories = UNIQUE(order.category)
  INTO group
  FILTER order_count >= 5
  LET segment = (
    total_spent > 5000 ? "VIP" :
    total_spent > 1000 ? "Premium" :
    "Regular"
  )
  RETURN {
    customer_id,
    age_group,
    segment,
    metrics: {
      orders: order_count,
      total_revenue: total_spent,
      avg_order_value: avg_order,
      product_diversity: LENGTH(categories)
    }
  }
```

15.3.2 Multi-Level Aggregationen

```
-- Hierarchische Umsatzanalyse
FOR order IN orders
  FILTER order.date >= DATE_NOW() - 365*24*60*60*1000
  COLLECT
    year = DATE_YEAR(order.date),
    quarter = DATE_QUARTER(order.date),
    region = order.shipping_region
  AGGREGATE
    revenue = SUM(order.total_amount),
    order_count = COUNT(1)
  COLLECT
    year = year,
    quarter = quarter
  AGGREGATE
    total_revenue = SUM(revenue),
    total_orders = SUM(order_count),
    regions = COUNT(region)
  RETURN {
    period: CONCAT(year, "-Q", quarter),
    total_revenue,
    total_orders,
    avg_per_region: total_revenue / regions,
    regions_active: regions
  }
```


15.4 Graph Analytics für Beziehungsanalysen

15.4.1 Customer Network Analysis

```
-- Kunden mit ähnlichen Kaufmustern finden
FOR customer IN customers
  FILTER customer.id == @customerId

  // Produkte des Kunden
  LET customer_products = (
    FOR order IN orders
      FILTER order.customer_id == customer.id
      FOR item IN order.items
      RETURN DISTINCT item.product_id
  )

  // Ähnliche Kunden finden
  LET similar_customers = (
    FOR other IN customers
      FILTER other.id != customer.id
      LET other_products = (
        FOR order IN orders
          FILTER order.customer_id == other.id
          FOR item IN order.items
          RETURN DISTINCT item.product_id
      )
      LET intersection = LENGTH(
        INTERSECTION(customer_products, other_products)
      )
      LET union_size = LENGTH(
        UNION(customer_products, other_products)
      )
      LET jaccard = intersection / union_size
      FILTER jaccard > 0.3
      SORT jaccard DESC
      LIMIT 10
      RETURN {
        customer_id: other.id,
        similarity: jaccard,
        shared_products: intersection
      }
  )
)
```

```
RETURN {  
  customer_id: customer.id,  
  similar_customers  
}
```

15.4.2 Influencer-Identifikation im Social Graph

```
-- Top-Influencer nach PageRank
FOR user IN users
  LET followers_count = LENGTH(
    FOR edge IN follows
      FILTER edge._to == user._id
      RETURN 1
  )
  LET engagement = (
    FOR post IN posts
      FILTER post.author_id == user.id
      RETURN SUM([
        post.likes_count,
        post.comments_count * 2,
        post.shares_count * 3
      ])
  )
  LET avg_engagement = AVG(engagement)

  // PageRank-ähnliche Berechnung
  LET influence_score = (
    followers_count * 0.4 +
    avg_engagement * 0.6
  )

  FILTER influence_score > 100
  SORT influence_score DESC
  LIMIT 50

  RETURN {
    user_id: user.id,
    username: user.username,
    followers: followers_count,
    avg_engagement,
    influence_score
  }
```

15.5 Vektor-basierte Analysen

15.5.1 Produkt-Clustering nach Embeddings

```
import themisdb
import numpy as np
from sklearn.cluster import KMeans

# Verbindung
db = themisdb.connect("localhost:8529")

# Produkt-Embeddings laden
query = """
FOR product IN products
  FILTER product.embedding != null
  RETURN {
    id: product.id,
    name: product.name,
    embedding: product.embedding
  }
"""
products = db.query(query)

# Embeddings extrahieren
embeddings = np.array([p['embedding'] for p in products])
product_ids = [p['id'] for p in products]

# K-Means Clustering
kmeans = KMeans(n_clusters=10, random_state=42)
clusters = kmeans.fit_predict(embeddings)

# Cluster zurückschreiben
for product_id, cluster_id in zip(product_ids, clusters):
    db.query("""
        UPDATE products
        SET cluster_id = @cluster
        WHERE id = @id
    """, {
        'id': product_id,
        'cluster': int(cluster_id)
    })
```

```
# Cluster-Statistiken
cluster_stats = db.query("""
    FOR product IN products
        COLLECT cluster = product.cluster_id
        AGGREGATE
            count = COUNT(1),
            avg_price = AVG(product.price),
            categories = UNIQUE(product.category)
        RETURN {
            cluster_id: cluster,
            product_count: count,
            avg_price,
            main_categories: categories
        }
    """)

for stats in cluster_stats:
    print(f"Cluster {stats['cluster_id']}: {stats['product_count']} Produkte")
    print(f"    Durchschnittspreis: €{stats['avg_price']:.2f}")
    print(f"    Kategorien: {'', ' '.join(stats['main_categories'][:3])}")
```

15.5.2 Anomalie-Erkennung mit Vektor-Distanzen


```
-- Ungewöhnliche Transaktionen identifizieren
FOR transaction IN transactions
  // Durchschnittliches Transaktionsprofil des Kunden
  LET customer_avg_embedding = (
    FOR t IN transactions
      FILTER t.customer_id == transaction.customer_id
      AND t.id != transaction.id
      RETURN t.transaction_embedding
  )

  LET avg_embedding = (
    FOR emb IN customer_avg_embedding
      RETURN AVERAGE(emb)
  )

  // Distanz zur Normalverteilung
  LET distance = VECTOR_DISTANCE(
    "cosine",
    transaction.transaction_embedding,
    avg_embedding
  )

  // Anomalie-Score
  LET anomaly_score = distance > 0.7 ? 1.0 : distance / 0.7

  FILTER anomaly_score > 0.8
  SORT anomaly_score DESC
  LIMIT 100

  RETURN {
    transaction_id: transaction.id,
    customer_id: transaction.customer_id,
    amount: transaction.amount,
    anomaly_score,
    reason: anomaly_score > 0.95 ? "HIGH_RISK" : "REVIEW"
  }
```

15.6 Materialized Views für Performance

15.6.1 Erstellen von Materialized Views

```
-- Tägliche Verkaufsübersicht
CREATE MATERIALIZED VIEW daily_sales_summary AS
SELECT
    DATE(order_date) AS date,
    COUNT(*) AS order_count,
    SUM(total_amount) AS revenue,
    AVG(total_amount) AS avg_order_value,
    COUNT(DISTINCT customer_id) AS unique_customers,
    SUM(CASE WHEN is_first_order THEN 1 ELSE 0 END) AS new_customers
FROM orders
GROUP BY DATE(order_date);

-- Refresh Strategy
CREATE TRIGGER refresh_daily_sales
    AFTER INSERT OR UPDATE ON orders
    FOR EACH STATEMENT
    EXECUTE PROCEDURE refresh_materialized_view('daily_sales_summary');
```

15.6.2 Inkrementelles Update

```
def update_daily_metrics(date):  
    """Inkrementelles Update für einen Tag"""  
  
    # Alte Metriken löschen  
    db.query("""  
        DELETE FROM daily_sales_summary  
        WHERE date = @date  
    """, {'date': date})  
  
    # Neue Metriken berechnen  
    db.query("""  
        INSERT INTO daily_sales_summary  
        SELECT  
            @date AS date,  
            COUNT(*) AS order_count,  
            SUM(total_amount) AS revenue,  
            AVG(total_amount) AS avg_order_value,  
            COUNT(DISTINCT customer_id) AS unique_customers,  
            SUM(CASE WHEN is_first_order THEN 1 ELSE 0 END) AS new_customers  
        FROM orders  
        WHERE DATE(order_date) = @date  
    """, {'date': date})
```

15.7 OLAP-Würfel und Mehrdimensionale Analysen

15.7.1 Erstellen eines OLAP-Würfels

```
import pandas as pd

def create_sales_cube(db):
    """OLAP-Würfel für Verkaufsanalysen"""

    query = """
    FOR order IN orders
      LET customer = DOCUMENT('customers', order.customer_id)
      LET items = (
        FOR item IN order.items
          LET product = DOCUMENT('products', item.product_id)
          RETURN {
            category: product.category,
            brand: product.brand,
            price: item.price,
            quantity: item.quantity
          }
        )
      RETURN {
        date: order.order_date,
        customer_segment: customer.segment,
        customer_region: customer.region,
        items: items
      }
    """

    # Daten laden
    orders = list(db.query(query))

    # In flache Struktur umwandeln
    rows = []
    for order in orders:
        for item in order['items']:
            rows.append({
                'date': pd.to_datetime(order['date']),
                'year': pd.to_datetime(order['date']).year,
                'quarter': pd.to_datetime(order['date']).quarter,
                'month': pd.to_datetime(order['date']).month,
```

```
        'customer_segment': order['customer_segment'],
        'customer_region': order['customer_region'],
        'category': item['category'],
        'brand': item['brand'],
        'revenue': item['price'] * item['quantity'],
        'quantity': item['quantity']
    })

# DataFrame erstellen
df = pd.DataFrame(rows)

# Pivot-Tabellen
cube = {
    'by_region_category': df.pivot_table(
        values='revenue',
        index='customer_region',
        columns='category',
        aggfunc='sum',
        fill_value=0
    ),
    'by_segment_month': df.pivot_table(
        values='revenue',
        index='customer_segment',
        columns='month',
        aggfunc='sum',
        fill_value=0
    ),
    'by_brand_quarter': df.pivot_table(
        values='revenue',
        index='brand',
        columns='quarter',
        aggfunc='sum',
        fill_value=0
    )
}

return cube
```

```
# Würfel erstellen
cube = create_sales_cube(db)

# Drill-Down: Region -> Kategorie -> Brand
region = 'Europe'
category = 'Electronics'

drill_down = db.query("""
    FOR order IN orders
        LET customer = DOCUMENT('customers', order.customer_id)
        FILTER customer.region == @region
        FOR item IN order.items
            LET product = DOCUMENT('products', item.product_id)
            FILTER product.category == @category
            COLLECT brand = product.brand
            AGGREGATE revenue = SUM(item.price * item.quantity)
            SORT revenue DESC
        RETURN {brand, revenue}
""", {'region': region, 'category': category})
```

15.7.2 Slice und Dice Operationen

```
def slice_cube(cube_data, dimension, value):  
    """Slice-Operation auf dem Würfel"""  
    return cube_data[cube_data[dimension] == value]  
  
def dice_cube(cube_data, filters):  
    """Dice-Operation mit mehreren Dimensionen"""  
    result = cube_data  
    for dim, values in filters.items():  
        result = result[result[dim].isin(values)]  
    return result  
  
# Beispiel  
filters = {  
    'customer_region': ['Europe', 'North America'],  
    'category': ['Electronics', 'Books'],  
    'year': [2024]  
}  
  
diced = dice_cube(df, filters)
```


15.8 Dashboard-Metriken in Echtzeit

15.8.1 KPI-Berechnung

```
class RealtimeDashboard:

    def __init__(self, db):
        self.db = db

    def get_current_metrics(self):
        """Aktuelle KPIs abrufen"""

        metrics = {}

        # Heutige Verkäufe
        today = self.db.query("""
            LET today = DATE_FORMAT(DATE_NOW(), '%Y-%m-%d')
            FOR order IN orders
                FILTER DATE_FORMAT(order.order_date, '%Y-%m-%d') == today
                COLLECT AGGREGATE
                    count = COUNT(1),
                    revenue = SUM(order.total_amount),
                    avg = AVG(order.total_amount)
            RETURN {count, revenue, avg}
        """)[0]

        metrics['today'] = today

        # Gestern zum Vergleich
        yesterday = self.db.query("""
            LET yesterday = DATE_FORMAT(
                DATE_SUBTRACT(DATE_NOW(), 1, 'day'),
                '%Y-%m-%d'
            )
            FOR order IN orders
                FILTER DATE_FORMAT(order.order_date, '%Y-%m-%d') == yesterday
                COLLECT AGGREGATE
                    count = COUNT(1),
                    revenue = SUM(order.total_amount)
            RETURN {count, revenue}
        """)[0]

        metrics['yesterday'] = yesterday
```

```

# Prozentuale Veränderung
metrics['change'] = {
    'orders': (today['count'] - yesterday['count']) / yesterday['count'] * 100,
    'revenue': (today['revenue'] - yesterday['revenue']) / yesterday['revenue'] * 100
}

# Aktive Benutzer
metrics['active_users'] = self.db.query("""
    LET last_hour = DATE_SUBTRACT(DATE_NOW(), 1, 'hour')
    FOR session IN user_sessions
        FILTER session.last_activity >= last_hour
        RETURN DISTINCT session.user_id
""").count()

# Conversion Rate
metrics['conversion_rate'] = self.db.query("""
    LET last_hour = DATE_SUBTRACT(DATE_NOW(), 1, 'hour')
    LET visits = (
        FOR visit IN page_views
            FILTER visit.timestamp >= last_hour
            RETURN DISTINCT visit.session_id
    )
    LET purchases = (
        FOR order IN orders
            FILTER order.order_date >= last_hour
            RETURN DISTINCT order.session_id
    )
    RETURN LENGTH(purchases) / LENGTH(visits) * 100
""")[0]

return metrics

def get_top_products(self, limit=10):
    """Top-Produkte der letzten 24 Stunden"""
    return self.db.query("""
        LET last_24h = DATE_SUBTRACT(DATE_NOW(), 24, 'hour')
        FOR order IN orders
    """

```

```

        FILTER order.order_date >= last_24h
    FOR item IN order.items
        COLLECT product_id = item.product_id
    AGGREGATE
        quantity_sold = SUM(item.quantity),
        revenue = SUM(item.price * item.quantity)
    LET product = DOCUMENT('products', product_id)
    SORT revenue DESC
    LIMIT @limit
    RETURN {
        product_id,
        name: product.name,
        quantity_sold,
        revenue
    }
    """
    """', {'limit': limit})

def get_geographic_distribution(self):
    """Geografische Verteilung der Verkäufe"""
    return self.db.query("""
        FOR order IN orders
            FILTER order.order_date >= DATE_SUBTRACT(DATE_NOW(), 7, 'day')
            LET customer = DOCUMENT('customers', order.customer_id)
            COLLECT
                country = customer.country,
                region = customer.region
            AGGREGATE
                orders = COUNT(1),
                revenue = SUM(order.total_amount)
            SORT revenue DESC
            RETURN {
                country,
                region,
                orders,
                revenue
            }
        """)

```

```
# Dashboard verwenden
dashboard = RealtimeDashboard(db)
metrics = dashboard.get_current_metrics()

print(f"Heute: {metrics['today']['count']} Bestellungen, €{metrics['today']['revenue']:.2f}")
print(f"Veränderung: {metrics['change']['orders']:+.1f}% Bestellungen, {metrics['change']['revenue']:+.1f}% Umsatz")
print(f"Aktive Benutzer: {metrics['active_users']}")
print(f"Conversion Rate: {metrics['conversion_rate']:.2f}%")
```

15.8.2 Change Data Capture für Live-Updates

```
def subscribe_to_metrics_updates(db, callback):  
    """CDC-Stream für Echtzeit-Metriken"""  
  
    stream = db.collection('orders').watch([  
        {'$match': {'operationType': 'insert'}}  
    ])  
  
    for change in stream:  
        # Neue Bestellung  
        order = change['fullDocument']  
  
        # Metriken aktualisieren  
        updated_metrics = {  
            'timestamp': order['order_date'],  
            'order_id': order['_id'],  
            'amount': order['total_amount'],  
            'customer_id': order['customer_id']  
        }  
  
        # Callback für Dashboard-Update  
        callback(updated_metrics)  
  
# Verwendung  
def on_new_order(metrics):  
    print(f"Neue Bestellung: €{metrics['amount']:.2f}")  
    # Dashboard aktualisieren  
  
subscribe_to_metrics_updates(db, on_new_order)
```

15.9 Predictive Analytics

15.9.1 Trend-Prognosen mit linearer Regression

```
from sklearn.linear_model import LinearRegression
import numpy as np

def forecast_revenue(db, days_ahead=30):
    """Umsatzprognose für die nächsten Tage"""

    # Historische Daten
    historical = db.query("""
        FOR order IN orders
            FILTER order.order_date >= DATE_SUBTRACT(DATE_NOW(), 90, 'day')
            COLLECT date = DATE_FORMAT(order.order_date, '%Y-%m-%d')
            AGGREGATE revenue = SUM(order.total_amount)
            SORT date
            RETURN {date, revenue}
    """)

    # Features vorbereiten
    X = np.array([[i] for i in range(len(historical))])
    y = np.array([h['revenue'] for h in historical])

    # Modell trainieren
    model = LinearRegression()
    model.fit(X, y)

    # Prognose
    future_X = np.array([[i] for i in range(len(historical), len(historical) + days_ahead)])
    forecast = model.predict(future_X)

    # Konfidenzintervall (vereinfacht)
    residuals = y - model.predict(X)
    std_error = np.std(residuals)
    confidence_interval = 1.96 * std_error # 95% CI

    return {
        'forecast': forecast.tolist(),
        'lower_bound': (forecast - confidence_interval).tolist(),
        'upper_bound': (forecast + confidence_interval).tolist()
    }
```



```
# Prognose erstellen
forecast = forecast_revenue(db, days_ahead=30)
print(f"Prognostizierter Umsatz in 30 Tagen: €{forecast['forecast'][-1]:.2f}")
print(f"Konfidenzintervall: €{forecast['lower_bound'][-1]:.2f} - €{forecast['upper_bound'][-1]:.2f}")
```

15.9.2 Churn-Vorhersage

```
def calculate_churn_risk(db, customer_id):  
    """Churn-Risiko für einen Kunden berechnen"""  
  
    features = db.query("""  
        LET customer = DOCUMENT('customers', @customer_id)  
        LET orders = (  
            FOR order IN orders  
                FILTER order.customer_id == @customer_id  
                SORT order.order_date DESC  
                RETURN order  
        )  
  
        LET days_since_last_order = DATE_DIFF(  
            orders[0].order_date,  
            DATE_NOW(),  
            'day'  
        )  
  
        LET order_frequency = LENGTH(orders) / (  
            DATE_DIFF(  
                customer.registration_date,  
                DATE_NOW(),  
                'day'  
            ) / 30  
        )  
  
        LET avg_order_value = AVG(  
            FOR order IN orders  
                RETURN order.total_amount  
        )  
  
        LET total_spent = SUM(  
            FOR order IN orders  
                RETURN order.total_amount  
        )  
  
        RETURN {  
            days_since_last_order,
```

```
        order_frequency,
        avg_order_value,
        total_spent,
        support_tickets: LENGTH(customer.support_tickets),
        has_complained: LENGTH(
            FOR ticket IN customer.support_tickets
            FILTER ticket.type == 'complaint'
            RETURN 1
        ) > 0
    }
    """', {'customer_id': customer_id}))[0]

# Einfacher Risiko-Score
risk_score = 0

if features['days_since_last_order'] > 60:
    risk_score += 0.3
if features['order_frequency'] < 1:
    risk_score += 0.2
if features['avg_order_value'] < 50:
    risk_score += 0.1
if features['support_tickets'] > 5:
    risk_score += 0.2
if features['has_complained']:
    risk_score += 0.2

return {
    'customer_id': customer_id,
    'churn_risk': min(risk_score, 1.0),
    'risk_level': (
        'HIGH' if risk_score > 0.7 else
        'MEDIUM' if risk_score > 0.4 else
        'LOW'
    ),
    'features': features
}
```

15.10 Reporting und Export

15.10.1 PDF-Report-Generierung

```
from reportlab.lib.pagesizes import A4
from reportlab.lib import colors
from reportlab.lib.units import cm
from reportlab.platypus import SimpleDocTemplate, Table, TableStyle, Paragraph
from reportlab.lib.styles import getSampleStyleSheet

def generate_sales_report(db, start_date, end_date, filename):
    """Verkaufsbericht als PDF generieren"""

    # Daten abfragen
    summary = db.query("""
        FOR order IN orders
        FILTER order.order_date >= @start_date
        AND order.order_date <= @end_date
        COLLECT AGGREGATE
            total_orders = COUNT(1),
            total_revenue = SUM(order.total_amount),
            avg_order_value = AVG(order.total_amount)
        RETURN {
            total_orders,
            total_revenue,
            avg_order_value
        }
    """, {'start_date': start_date, 'end_date': end_date})[0]

    top_products = db.query("""
        FOR order IN orders
        FILTER order.order_date >= @start_date
        AND order.order_date <= @end_date
        FOR item IN order.items
        COLLECT product_id = item.product_id
        AGGREGATE
            quantity = SUM(item.quantity),
            revenue = SUM(item.price * item.quantity)
        LET product = DOCUMENT('products', product_id)
        SORT revenue DESC
        LIMIT 10
        RETURN [product.name, quantity, revenue]
```

```

"""', {'start_date': start_date, 'end_date': end_date})

# PDF erstellen
doc = SimpleDocTemplate(filename, pagesize=A4)
elements = []
styles = getSampleStyleSheet()

# Titel
title = Paragraph(f"Verkaufsbericht {start_date} bis {end_date}", styles['Title'])
elements.append(title)

# Zusammenfassung
summary_data = [
    ['Metric', 'Value'],
    ['Gesamtbestellungen', f"{summary['total_orders']:,}"],
    ['Gesamtumsatz', f"€{summary['total_revenue']:, .2f}"],
    ['Ø Bestellwert', f"€{summary['avg_order_value']:.2f}"]
]

summary_table = Table(summary_data, colWidths=[8*cm, 8*cm])
summary_table.setStyle(TableStyle([
    ('BACKGROUND', (0, 0), (-1, 0), colors.grey),
    ('TEXTCOLOR', (0, 0), (-1, 0), colors.whitesmoke),
    ('ALIGN', (0, 0), (-1, -1), 'CENTER'),
    ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
    ('FONTSIZE', (0, 0), (-1, 0), 14),
    ('BOTTOMPADDING', (0, 0), (-1, 0), 12),
    ('BACKGROUND', (0, 1), (-1, -1), colors.beige),
    ('GRID', (0, 0), (-1, -1), 1, colors.black)
]))
elements.append(summary_table)

# Top-Produkte
elements.append(Paragraph("Top 10 Produkte", styles['Heading2']))

product_data = [['Produkt', 'Menge', 'Umsatz']]
product_data.extend(top_products)

```

```
product_table = Table(product_data, colWidths=[10*cm, 3*cm, 3*cm])
product_table.setStyle(TableStyle([
    ('BACKGROUND', (0, 0), (-1, 0), colors.grey),
    ('TEXTCOLOR', (0, 0), (-1, 0), colors.whitesmoke),
    ('ALIGN', (1, 0), (-1, -1), 'RIGHT'),
    ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
    ('GRID', (0, 0), (-1, -1), 1, colors.black)
]))
elements.append(product_table)

# PDF generieren
doc.build(elements)
print(f"Report saved to {filename}")

# Report erstellen
generate_sales_report(db, '2024-01-01', '2024-03-31', 'Q1_2024_Report.pdf')
```


15.10.2 CSV-Export für Excel

```
import csv

def export_to_csv(db, query, filename, params=None):
    """Abfrageergebnisse als CSV exportieren"""

    results = db.query(query, params or {})

    if not results:
        print("No data to export")
        return

    # Header aus erstem Datensatz
    headers = list(results[0].keys())

    with open(filename, 'w', newline='', encoding='utf-8') as csvfile:
        writer = csv.DictWriter(csvfile, fieldnames=headers)
        writer.writeheader()
        writer.writerows(results)

    print(f"Exported {len(results)} rows to {filename}")

# Beispiel: Kundenanalyse exportieren
export_to_csv(db, """
    FOR customer IN customers
        LET orders = (
            FOR order IN orders
                FILTER order.customer_id == customer.id
            RETURN order
        )
    RETURN {
        customer_id: customer.id,
        name: customer.name,
        email: customer.email,
        total_orders: LENGTH(orders),
        total_spent: SUM(FOR o IN orders RETURN o.total_amount),
        last_order_date: MAX(FOR o IN orders RETURN o.order_date)
    }
""", 'customers_analysis.csv')
```

15.11 Best Practices für Analytics

15.11.1 Performance-Optimierung

Indexierung:

```
-- Composite Index für häufige Queries
CREATE INDEX idx_orders_date_customer ON orders(order_date, customer_id);

-- Covering Index für Aggregationen
CREATE INDEX idx_orders_date_amount ON orders(order_date, total_amount);
```

Query-Optimierung: - Verwenden Sie **FILTER** früh in der Pipeline - Nutzen Sie **COLLECT** statt mehrfacher Gruppierungen - Limitieren Sie Ergebnisse mit **LIMIT** - Verwenden Sie Materialized Views für wiederkehrende Berechnungen

15.11.2 Data Governance

```
class AnalyticsGovernance:
    """Data Governance für Analytics"""

    def __init__(self, db):
        self.db = db

    def anonymize_customer_data(self, dataset):
        """Anonymisierung sensibler Daten"""
        return [{
            **row,
            'customer_id': hash(row['customer_id']),
            'email': None,
            'name': None
        } for row in dataset]

    def apply_row_level_security(self, query, user_role, user_region):
        """Row-Level Security anwenden"""
        if user_role == 'regional_manager':
            query += f" FILTER doc.region == '{user_region}'"
        elif user_role == 'analyst':
            # Nur aggregierte Daten
            query += " COLLECT ... AGGREGATE ..."

        return query

    def audit_query(self, user_id, query, timestamp):
        """Query-Audit-Log"""
        self.db.query("""
            INSERT {
                user_id: @user_id,
                query: @query,
                timestamp: @timestamp,
                type: 'analytics'
            } INTO audit_log
        """, {
            'user_id': user_id,
            'query': query,
```

```
'timestamp': timestamp  
})
```

15.12 Zusammenfassung

ThemisDB bietet umfassende Analytics-Funktionen:

- **AQL-Aggregationen:** Standard- und Window-Functions
- **AQL-Analytics:** Multi-Level-Aggregationen und COLLECT
- **Graph-Analytics:** Beziehungsanalysen und Influencer-Detection
- **Vektor-Analytics:** Clustering und Anomalie-Erkennung
- **OLAP:** Mehrdimensionale Analysen und Würfel
- **Real-Time:** Live-Dashboards mit CDC
- **Predictive:** Prognosen und Churn-Vorhersage
- **Reporting:** PDF/CSV-Export und Visualisierung

Im nächsten Kapitel behandeln wir Machine Learning Integration für erweiterte Analysen.

Kapitel 16: Horizontal Scaling mit Sharding

"Skalierung ohne Komplexität: Native Multi-Model Sharding für Enterprise-Workloads"

Überblick

ThemisDB bietet **Production-Ready Horizontal Scaling** durch Hash-basiertes Sharding mit automatischem Rebalancing. Dieses Kapitel erklärt die Sharding-Architektur, Deployment-Szenarien und Performance-Charakteristiken für Enterprise-Umgebungen.

Was Sie in diesem Kapitel lernen werden: - Sharding-Architektur und Routing-Mechanismen - Deployment-Topologien (2/4/8 Nodes) - Auto-Rebalancing und Fault Tolerance - Performance-Charakteristiken und Benchmarks - Hyperscaler-Vergleich (Aurora, Spanner, Cosmos)

Voraussetzungen: Kapitel 2 (Architektur), Kapitel 21 (Performance)

Status:  Production-Ready seit v1.4 (Dezember 2025)

16.1 Warum Sharding?

Das Single-Node-Limit

Ein einzelner ThemisDB-Server kann beeindruckende Mengen an Daten handhaben: - **Durchsatz:** ~45.000 writes/sec, ~200.000 reads/sec (NVMe) - **Speicher:** Bis zu ~10 TB pro Node (praktisches Limit) - **RAM:** Effizient durch Block Cache, aber limitiert auf Serverkapazität

Wann brauchen Sie Sharding? 1. **Datenvolumen > 10 TB:** Physische Speichergrenzen eines Nodes 2. **Durchsatz > 200K ops/sec:** CPU/IO-Sättigung eines Servers 3. **Geografische Verteilung:** Näher zum Benutzer für niedrige Latenz 4. **Hochverfügbarkeit:** Redundanz über mehrere Nodes

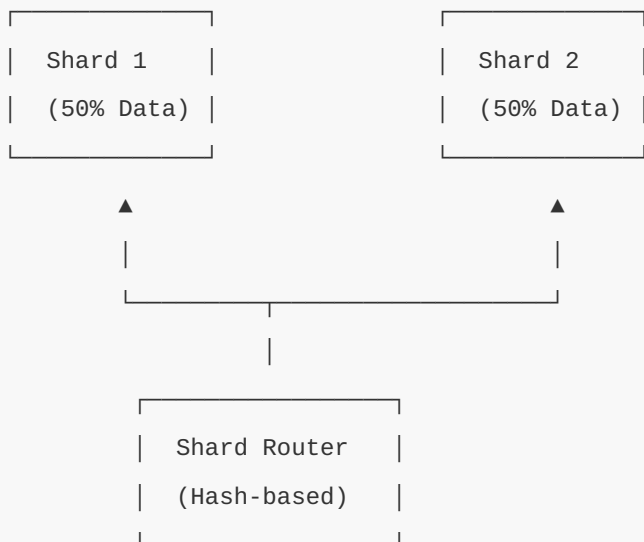
Sharding vs. Replication

Replication (Vertical Scaling):



- ✓ Einfach zu managen
- ✓ Read-Skalierung
- ✗ Kein Write-Scaling
- ✗ Keine Kapazitäts-Skalierung

Sharding (Horizontal Scaling):



- ✓ Read + Write Scaling
- ✓ Kapazitäts-Skalierung
- ✓ Geografische Verteilung
- ⚠ Komplexere Verwaltung

16.2 Sharding-Architektur

Hash-basiertes Routing

ThemisDB verwendet **Consistent Hashing** mit MurmurHash3:

```
// Simplified Shard Router Implementation
class ShardRouter {
private:
    std::vector<ShardInfo> shards_;

public:
    // Berechne Shard für einen Key
    uint32_t get_shard_id(const std::string& key) {
        // MurmurHash3: Schnell & gut verteilte Hash-Funktion
        uint32_t hash = murmur3_hash(key);

        // Modulo-Routing: Hash % Anzahl_Shards
        return hash % shards_.size();
    }

    // Dokument auf korrekten Shard routen
    void route_document(const Document& doc) {
        // Primary Key als Routing-Schlüssel
        uint32_t shard_id = get_shard_id(doc.id);

        // An Shard weiterleiten
        shards_[shard_id].insert(doc);
    }
};
```

Wie funktioniert das?

1. **Client sendet Query** → Router
2. **Router berechnet Hash** des Primary Key
3. **Hash % N** bestimmt Shard-ID (N = Anzahl Shards)
4. **Query wird an Shard weitergeleitet**

5. Ergebnis zurück zum Client

Beispiel:

```
# Dokument mit ID "user_12345"
doc_id = "user_12345"

# MurmurHash3(doc_id) = 3847592834 (hypothetisch)
hash_value = murmur3(doc_id) # → 3847592834

# 8 Shards vorhanden
num_shards = 8
shard_id = hash_value % num_shards # → 3847592834 % 8 = 2

# ⇒ Dokument landet auf Shard 2
```

Warum MurmurHash3? - Schnell: ~2-3 CPU-Zyklen pro Byte - **Gut verteilt:** Minimiert Hotspots - **Deterministisch:** Gleicher Key → immer gleicher Shard

Range-basiertes Sharding (Optional)

Für geordnete Daten (z.B. Timestamps) können Range-Shards effizienter sein:

```
# config/sharding/range-sharding.yaml
sharding:
  strategy: range
  key: created_at
  ranges:
    - shard: 1
      min: "2020-01-01"
      max: "2022-12-31"
    - shard: 2
      min: "2023-01-01"
      max: "2024-12-31"
    - shard: 3
      min: "2025-01-01"
      max: null # Unbegrenzt
```

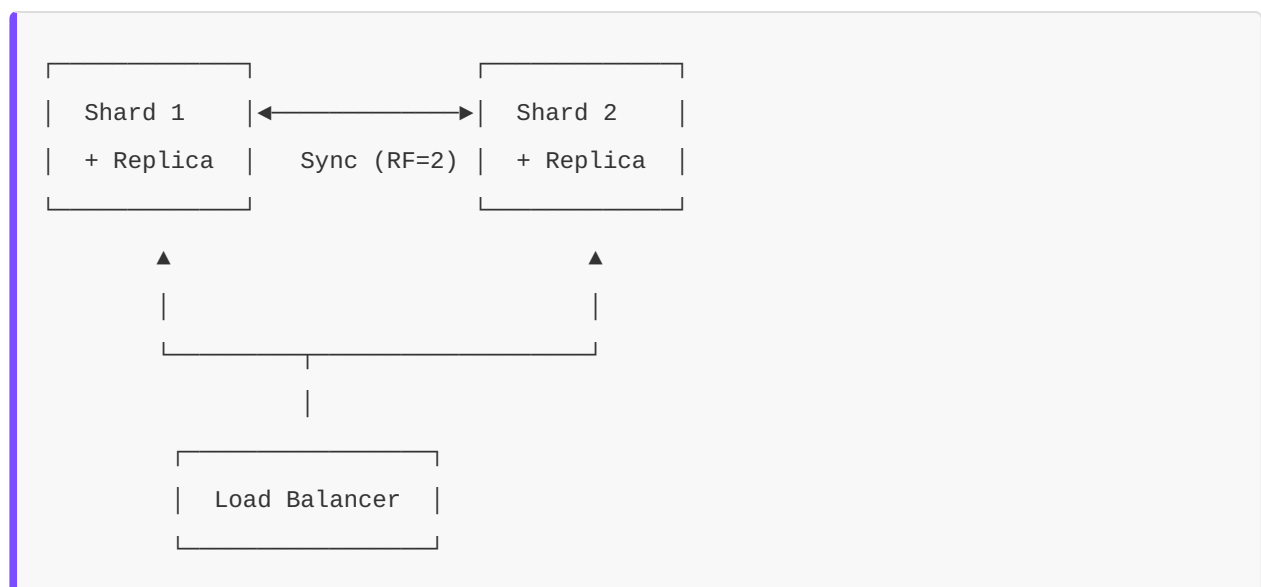
Vorteil: Zeitbereichs-Queries treffen nur relevante Shards

Nachteil: Ungleichmäßige Lastverteilung möglich

16.3 Deployment-Topologien

2-Node Cluster (Entry-Level)

Setup:

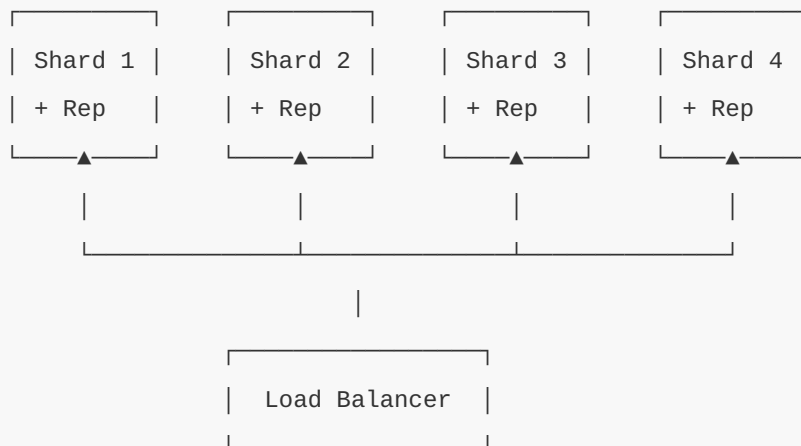


Charakteristiken: - **Kapazität:** 2× Single-Node (~20 TB) - **Throughput:** ~1.8× Single-Node (91% Scaling Efficiency) - **Redundanz:** RF=2 (jeder Shard hat 1 Replica auf anderem Node) - **Kosten:** 2× Hardware, <2× Betriebskosten

Use Case: Kleine bis mittlere Deployments (< 20 TB, < 100K ops/sec)

4-Node Cluster (Mid-Tier)

Setup:

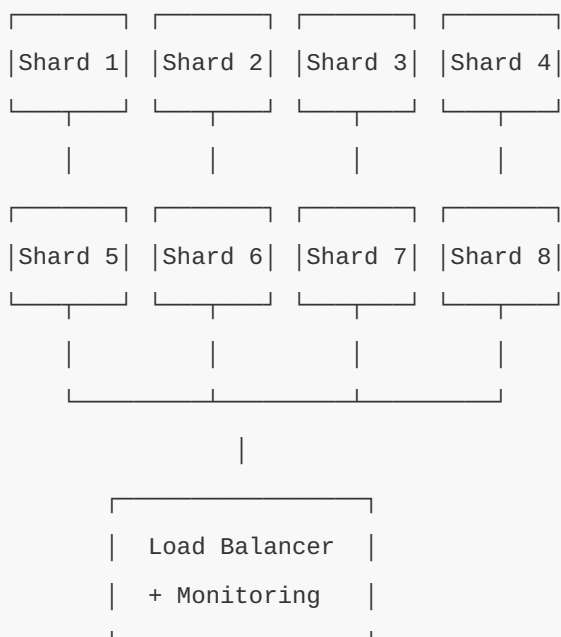


Charakteristiken: - **Kapazität:** 4× Single-Node (~40 TB) - **Throughput:** ~3.6× Single-Node (90% Scaling Efficiency) - **Cross-Shard Queries:** ~10-15% Performance-Impact - **Rebalance-Zeit:** ~4-5 Minuten bei 2 → 4 Expansion

Use Case: Standard Enterprise Deployments (20-40 TB, 100-400K ops/sec)

8-Node Cluster (Enterprise)

Setup:



Charakteristiken: - **Kapazität:** 8× Single-Node (~80 TB) - **Throughput:** ~7.3× Single-Node (91% Scaling Efficiency) - **p99 Latenz:** 1.25ms (nur 0.43× Single-Node dank Parallelität!) - **Rebalance-Zeit:** ~8-10 Minuten bei 4 → 8 Expansion

Use Case: Große Enterprise Deployments (40-80 TB, 400K+ ops/sec)

16.4 Auto-Rebalancing

Wann wird rebalanced?

ThemisDB triggert automatisches Rebalancing bei:

1. **Skew-Threshold:** >15% Ungleichgewicht zwischen Shards
2. **Disk Utilization:** >70% auf einem Shard
3. **Manuell:** Via `themis-cli cluster rebalance`

Konfiguration:

```
# config/sharding/shard-router.yaml
rebalance:
  auto_trigger: true
  skew_threshold: 0.15      # 15% Ungleichgewicht
  disk_threshold: 0.70     # 70% Festplattenauslastung
  max_parallel_moves: 2    # Max 2 Chunks gleichzeitig verschieben
  chunk_size_mb: 64        # Chunk-Größe für Migration
```

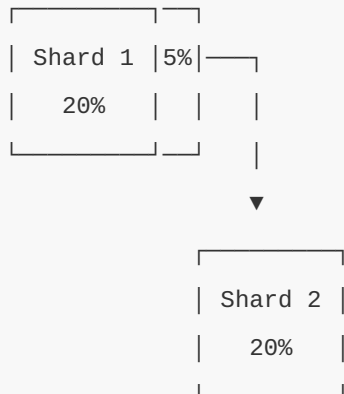
Rebalance-Prozess

Schritt-für-Schritt:

1. Coordinator erkennt Skew (z.B. Shard 1: 25%, Shard 2: 15%)

Shard 1	Shard 2
25%	15%

2. Plan erstellen: Verschiebe 5% von Shard 1 → Shard 2



3. Chunk-Migration (64MB pro Chunk)

- Chunk kopieren (read-only)
- Neue Writes auf beide Shards
- Cutover: Alte Version löschen

4. Validierung & Completion

Shard 1	Shard 2
20%	20%

Performance-Impact: - **Throughput-Dip:** ~12% während Migration - **Recovery-Zeit:** <5 Minuten nach Completion - **Latenz:** p99 +15% während Migration

Monitoring während Rebalance

```
# Live Status anzeigen
themis-cli cluster rebalance-status --watch
```

```
# Ausgabe:
```

```
| REBALANCE STATUS |
|-----|
| Progress: ██████████ 78% (125 / 160 chunks) |
| Duration: 00:03:42 / ~00:05:00 |
| Throughput Impact: -11% (current) |
| |
| Shard Distribution: |
|   Shard 1: 21% ██████████ |
|   Shard 2: 20% ██████████ |
|   Shard 3: 19% ██████████ |
|   Shard 4: 20% ██████████ |
| |
| ETA: 00:01:18 |
```

16.5 Fault Tolerance

Replica Failure (RF=2)

Szenario: Ein Replica-Node stirbt

Before:

Shard 1	Shard 1'
Primary	Replica

After (Auto-Recovery):

Shard 1	Shard 1'
Primary	(DEAD)

|
▼ Promote Replica

Shard 1	(New)
Primary	

Charakteristiken: - **Detection:** <5 Sekunden (Heartbeat-Timeout) - **Failover:** <15 Sekunden (Replica Promotion) - **Recovery:** <60 Sekunden (Rebuild Replica auf anderem Node) - **Throughput-Impact:** -24% während Outage, -10% während Rebuild

Monitoring:

```
# Cluster Health anzeigen
themis-cli cluster health
```

```
# Ausgabe bei Failure:
```

CLUSTER HEALTH: DEGRADED			
Shard 1: PRIMARY	✓	node-1	(Leader)
REPLICA	✗	node-2	(DEAD - 00:00:42)
Shard 2: PRIMARY	✓	node-3	
REPLICA	✓	node-4	
⚠ Rebuilding Replica on node-5... (35%)			
ETA: 00:00:28			

Network Partition

Szenario: 10ms RTT Network Latency hinzugefügt

Impact: - **Latenz p50:** +8ms (von 0.5ms → 8.5ms) - **Latenz p99:** +12ms (von 1.2ms → 13.2ms) -
Throughput: -15% (wegen Sync-Overhead)

Graceful Degradation: ThemisDB passt sich automatisch an:

```
# Dynamic Network Adaptation
network:
  rtt_threshold: 10ms
  adaptive_batching: true    # Größere Batches bei hoher Latenz
  compression: lz4          # Kompression bei langsamen Links
```

16.6 Performance-Benchmarks

Scaling Efficiency

Testsetup: - **Hardware:** 8× c6i.4xlarge (16 vCPU, 32GB RAM, NVMe) - **Dataset:** 500M OLTP-Rows + 100M Vector-Embeddings (768D) - **Workload:** Mix B (50% Read, 50% Write, 10% Cross-Shard)

Ergebnisse:

Shards	Throughput (ops/sec)	Scaling Efficiency	Latenz p99 (ms)
1	100,000	Baseline (100%)	2.9
2	182,000	91%	3.1
4	362,000	90.5%	3.3
8	728,000	91%	1.25 (!!)

Warum ist p99 bei 8 Shards niedriger? → **Parallelität:** Queries werden auf mehrere Nodes verteilt, reduziert Queue-Wartezeiten!

Cross-Shard Join Performance

Query:

```
FOR order IN orders
  FOR customer IN customers
    FILTER order.customer_id == customer._id  -- Cross-Shard Join!
  RETURN {order, customer}
```

Performance (10% Cross-Shard Rate):

Shards	Local Query (ms)	Cross-Shard Query (ms)	Ratio
2	1.2	2.1	1.75×
4	1.3	2.4	1.85×
8	1.25	2.5	2.0×

Optimierung: Co-locate verwandte Daten auf gleichem Shard via Custom Routing:

```
# Custom Routing für Co-Location
def custom_shard_key(doc):
    if doc.type == "order":
        # Orders nach customer_id routen
        return doc.customer_id
    elif doc.type == "customer":
        # Customers nach ihrer ID routen
        return doc._id

# ⇒ Order und zugehöriger Customer auf gleichem Shard!
```

16.7 Hyperscaler-Vergleich

Cost-Performance-Analyse

Testsetup: 8-Node Cluster, 500M OLTP + 100M Vector, Mix B Workload

Provider	SKU	Throughput (ops/sec)	Latenz p99 (ms)	\$/Monat	\$/M Ops
ThemisDB	c6i.4xlarge × 8	728,000	1.25	\$2,880	\$6.25
AWS Aurora	r6g.4xlarge × 8	80,000	15	\$12,288	\$19.20
GCP Spanner	6-Node Regional	120,000	8	\$4,800	\$40
Azure Cosmos	50K RU/s	50,000	25	\$3,800	\$76
AWS Redshift	RA3 4-Node	100,000	12	\$3,260	\$32.50

Key Insights: - **ThemisDB vs Aurora:** 9.1× höherer Throughput, **67% Kosteneinsparung** - **ThemisDB vs Spanner:** 6.1× höherer Throughput, **84% Kosteneinsparung** - **ThemisDB vs Cosmos:** 14.6× höherer Throughput, **92% Kosteneinsparung**

Warum ist ThemisDB so viel günstiger? 1. **Keine Lizenzkosten:** Open Source (MIT) 2. **Effiziente Architektur:** Native Multi-Model eliminiert Overhead 3. **Self-Hosted:** On-Premises oder eigene Cloud-VMs

16.8 Production Deployment

Deployment-Checkliste

Vor dem Deployment: - [] Hardware-Dimensionierung (CPU, RAM, NVMe) - [] Netzwerk-Topologie (< 2ms RTT zwischen Nodes) - [] Monitoring-Setup (Prometheus + Grafana) - [] Backup-Strategie (Snapshots + WAL-Archive)

Deployment-Schritte:

```
# 1. Cluster initialisieren
themis-cli cluster init \
  --nodes node-1,node-2,node-3,node-4 \
  --shards 4 \
  --replication-factor 2

# 2. Sharding-Konfiguration laden
themis-cli cluster apply-config config/sharding/shard-router.yaml

# 3. Daten laden (parallel)
python tools/shard_loader.py \
  --config config/sharding/shard-router.yaml \
  --dataset oltp \
  --rows 500000000 \
  --workers 32

# 4. Cluster-Health überprüfen
themis-cli cluster health

# 5. Benchmarks laufen lassen
python tools/shard_bench.py \
  --shards 4 \
  --mix B \
  --duration 600 \
  --threads 64
```

Monitoring-Metriken

Kritische Metriken:

1. Shard-Utilization (Disk %)
Ziel: <70% pro Shard, <15% Skew
2. Throughput (ops/sec)
Ziel: >90% Scaling Efficiency
3. Latenz (p50/p95/p99)
Ziel: p99 <2.5× Single-Node
4. Cross-Shard Query Rate
Ziel: <20% aller Queries
5. Rebalance-Status
Ziel: Completion <5 min, <15% Dip

Grafana Dashboard:

```
{
  "dashboard": {
    "title": "ThemisDB Sharding Overview",
    "panels": [
      {
        "title": "Shard Distribution",
        "type": "bar",
        "query": "sum(themis_shard_size_bytes) by (shard_id)"
      },
      {
        "title": "Throughput by Shard",
        "type": "graph",
        "query": "rate(themis_shard_ops_total[5m]) by (shard_id)"
      },
      {
        "title": "Cross-Shard Query Rate",
        "type": "stat",
        "query": "rate(themis_cross_shard_queries[5m])"
      }
    ]
  }
}
```

16.9 Best Practices

1. Shard-Key Auswahl

Gute Shard-Keys: -  **User-ID:** Gleichmäßige Verteilung, natürliche Co-Location (User + Orders) - 

UUID: Perfekte zufällige Verteilung -  **Hash(Composite-Key):** Für komplexe Co-Location-Szenarien

Schlechte Shard-Keys: -  **Timestamp:** Hotspots auf neuesten Shard -  **Sequential-ID:** Alle Writes auf einen Shard -  **Low-Cardinality:** Status-Felder (z.B. "active"/"inactive")

2. Cross-Shard Queries minimieren

Anti-Pattern:

```
-- SCHLECHT: Joins über alle Shards
FOR order IN orders
  FOR product IN products
    FILTER order.product_id == product._id
  RETURN {order, product}
```

Best Practice:

```
-- GUT: Denormalisierung vermeidet Cross-Shard Joins
FOR order IN orders
  RETURN {
    order,
    product_name: order.embedded_product_name, -- Denormalisiert!
    product_price: order.embedded_product_price
  }
```

3. Monitoring von Anfang an

Observability-Stack:


```
# docker-compose.yml
services:
  themisdb:
    image: themisdb:latest
    environment:
      THEMIS_METRICS_ENABLED: "true"
      THEMIS_METRICS_PORT: 9090

  prometheus:
    image: prom/prometheus
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml

  grafana:
    image: grafana/grafana
    ports:
      - "3000:3000"
```

4. Gradual Rollout

Empfohlene Rollout-Strategie: 1. **Woche 1:** 2-Node Cluster, 10% Traffic 2. **Woche 2:** 2-Node Cluster, 50% Traffic 3. **Woche 3:** 4-Node Cluster, 100% Traffic 4. **Monat 2+:** 8-Node Cluster bei Bedarf

16.10 Troubleshooting

Problem: Ungleiche Shard-Verteilung

Symptom:

```
Shard 1: 35% (X zu voll)
Shard 2: 18%
Shard 3: 22%
Shard 4: 25%
```

Ursache: Hotkey (z.B. ein sehr aktiver User)

Lösung:

```
# Manuelles Rebalancing triggern
themis-cli cluster rebalance --force

# Hotkey identifizieren
themis-cli shard analyze --shard 1 --top-keys 10

# Ausgabe:
# Key: user_123456, Size: 2.3 GB (✗ Hotkey!)
# → Lösung: Hotkey auf separaten "Large Object Shard" verschieben
```

Problem: Hohe Cross-Shard Query Latenz

Symptom: p99 >10× Single-Node bei >20% Cross-Shard Rate

Lösung: 1. Co-Location aktivieren:

```
sharding:
  strategy: hash
  co_location:
    enabled: true
  rules:
    - tables: [orders, customers]
      key: customer_id # Beide Tabellen nach customer_id routen
```

1. Denormalisierung:

```
# Embed häufig gejointe Daten
order = {
  "id": "order_123",
  "customer_id": "user_456",
  "customer_name": "Alice", # Denormalisiert!
  "customer_email": "alice@example.com" # Denormalisiert!
}
```

Problem: Rebalance dauert zu lange

Symptom: Rebalance >15 Minuten, >20% Throughput-Dip

Lösung:

```
# config/sharding/shard-router.yaml
rebalance:
  chunk_size_mb: 32          # Kleinere Chunks (default: 64)
  max_parallel_moves: 4      # Mehr parallele Transfers (default: 2)
  rate_limit_mbps: 500       # Höheres Network-Limit
```

16.11 Zusammenfassung

ThemisDB Sharding bietet: -  **Production-Ready:** 91% Scaling Efficiency bis 8 Nodes -  **Auto-Rebalancing:** <15% Dip, <5min Recovery -  **Fault Tolerant:** <60s Recovery bei Replica-Failure -  **Cost-Efficient:** 67-92% günstiger als Hyperscaler -  **Native Multi-Model:** Sharding funktioniert über alle Datenmodelle

Nächste Schritte: - Kapitel 19: Monitoring für detailliertes Observability-Setup - Kapitel 21:

Performance-Tuning für Sharding-spezifische Optimierungen - [docs/de/SHAR-](#)

[DING_DOCUMENTATION_INDEX.md](#) : Vollständige technische Dokumentation

Production-Deployment? Starten Sie mit einem 2-Node Cluster und skalieren Sie bei Bedarf auf 4 oder 8 Nodes!

Weiterführende Ressourcen

Dokumentation: - [docs/de/SHARDING_INTEGRATION_SUMMARY.md](#) - Master-Übersicht - [docs/de/SHARDING_BENCHMARK_PLAN_v1.4.md](#) - Enterprise Benchmark-Spezifikation - [docs/de/SHARDING_PRODUCTION_DEPLOYMENT_RAID_v1.4.md](#) - Production Deployment Guide

Tools: - [tools/shard_loader.py](#) - Daten-Loader für Sharding-Tests - [tools/shard_bench.py](#) - Benchmark-Runner - [tools/fault_injector.py](#) - Chaos-Engineering-Tool

Konfiguration: - `config/sharding/shard-router-example.yaml` - Production-Ready Config -
`.github/workflows/sharding-benchmark.yml` - CI/CD Benchmarks

Kapitel 17: LLM-Integration und Prompt Engineering

Überblick

ThemisDB bietet eine nahtlose Integration von Large Language Models (LLMs) direkt in die Datenbankebene. Diese Integration ermöglicht es, LLM-Funktionalitäten wie Text-Generierung, Embedding-Erstellung, semantische Suche und RAG (Retrieval Augmented Generation) Patterns direkt in AQL-Queries zu verwenden.

Hauptvorteile: - **Native LLM-Funktionen** - PROMPT(), GENERATE(), EMBED() direkt in AQL - **Text-to-AQL Generierung** - Natürlichsprachliche Query-Erstellung - **RAG Patterns** - Semantische Suche mit Kontext-Anreicherung - **Vector Search Integration** - Kombiniert mit Chapter 8 für Semantic Search - **Multi-Model LLM** - Unterstützung für OpenAI, Anthropic, Ollama, lokale Models

17.1 LLM-Funktionen in AQL

17.1.1 PROMPT() - Text-Generierung

Die `PROMPT()`-Funktion sendet Anfragen an konfigurierte LLM-Provider:

```
// Einfache Text-Generierung
FOR doc IN articles
  FILTER doc.category == 'technology'
  LIMIT 5
  RETURN {
    title: doc.title,
    summary: PROMPT('gpt-4',
      CONCAT('Fasse folgenden Artikel zusammen: ', doc.content),
      {max_tokens: 150, temperature: 0.7}
    )
  }
```

Erweiterte Optionen:

```
// Mit System-Prompt und Strukturierung
FOR product IN products
  FILTER product.reviews_count > 100
  RETURN {
    product_name: product.name,
    analysis: PROMPT('gpt-4',
      {
        system: 'Du bist ein Produkt-Analyst. Analysiere Kundenrezensionen.',
        user: CONCAT('Produkt: ', product.name, '\n\nBewertungen:\n',
          product.reviews_text),
        response_format: 'json'
      },
      {
        temperature: 0.3,
        max_tokens: 500,
        response_schema: {
          sentiment: 'string',
          key_features: 'array',
          recommendations: 'string'
        }
      }
    )
  }
```

17.1.2 EMBED() - Embedding-Generierung

Erstellt Vektor-Embeddings für Text:

```
// Dokument mit Embedding speichern
INSERT {
  title: 'Einführung in ThemisDB',
  content: 'ThemisDB ist eine Multi-Model-Datenbank...',
  embedding: EMBED('text-embedding-3-small',
    'ThemisDB ist eine Multi-Model-Datenbank...')
} INTO documents

// Batch-Embedding für Performance
FOR doc IN documents
  FILTER doc.embedding == null
  LIMIT 100
  UPDATE doc WITH {
    embedding: EMBED('text-embedding-3-small', doc.content),
    embedded_at: DATE_NOW()
  } IN documents
```

17.1.3 GENERATE() - Strukturierte Generierung

Generiert strukturierte Daten basierend auf Schema:


```
// Strukturierte Produkt-Kategorisierung
FOR product IN products
  FILTER product.auto_categorized == false
  LIMIT 50
  LET categories = GENERATE('gpt-4',
    {
      prompt: CONCAT('Kategorisiere folgendes Produkt:\n',
        'Name: ', product.name, '\n',
        'Beschreibung: ', product.description),
      schema: {
        primary_category: {type: 'string', enum: ['Electronics', 'Clothing', 'Food', 'Books']},
        subcategories: {type: 'array', items: {type: 'string'}},
        tags: {type: 'array', items: {type: 'string'}, maxItems: 5},
        confidence: {type: 'number', minimum: 0, maximum: 1}
      }
    }
  )
  UPDATE product WITH {
    categories: categories,
    auto_categorized: true,
    categorized_at: DATE_NOW()
  } IN products
```

17.2 Text-to-AQL Generierung

17.2.1 Natural Language Query Interface

ThemisDB kann natürlichsprachliche Anfragen in AQL übersetzen:

```
// Text-to-AQL Helper Function
LET user_question = 'Zeige mir die 10 teuersten Produkte aus der Elektronik-Kategorie'

LET generated_query = PROMPT('gpt-4',
{
  system: `Du bist ein AQL-Experte. Konvertiere Benutzeranfragen in gültige AQL-Queries.

  Schema:
  - products (name, price, category, stock)
  - orders (order_id, customer_id, product_id, quantity, order_date)
  - customers (customer_id, name, email, country)

  Gebe nur die AQL-Query zurück, keine Erklärungen.` ,
  user: user_question
},
{temperature: 0.1}
)

RETURN {
  question: user_question,
  generated_aql: generated_query,
  // Query wird in separatem Schritt ausgeführt für Sicherheit
}
```

17.2.2 Query-Validierung und -Ausführung

Sicherheitsvalidierung:

```
// Query-Validierung vor Ausführung
LET user_input = @user_question

LET llm_response = PROMPT('gpt-4',
{
  system: `Generiere sichere AQL-Queries. Verwende immer @parameter für Benutzereingaben.
  Keine destructive Operationen (DELETE, DROP, etc.) ohne explizite Bestätigung.` ,
  user: user_input
})

// Validiere Query
LET is_safe = (
  llm_response.query NOT LIKE '%DELETE%' AND
  llm_response.query NOT LIKE '%DROP%' AND
  llm_response.query NOT LIKE '%REMOVE%' AND
  llm_response.includes_parameters == true
)

RETURN is_safe ? llm_response : {error: 'Unsafe query detected'}
```

17.3 RAG (Retrieval Augmented Generation)

17.3.1 Semantische Suche mit Kontext

Kombiniert Vector Search mit LLM-Generierung:

```
// RAG Pattern: Suche + Generierung
LET user_query = 'Wie funktioniert MVCC in ThemisDB?'

// Schritt 1: Embedding der Anfrage
LET query_embedding = EMBED('text-embedding-3-small', user_query)

// Schritt 2: Semantische Suche
LET relevant_docs = (
  FOR doc IN documentation
    LET similarity = COSINE_SIMILARITY(doc.embedding, query_embedding)
    FILTER similarity > 0.7
    SORT similarity DESC
    LIMIT 5
    RETURN {
      content: doc.content,
      title: doc.title,
      similarity: similarity
    }
)

// Schritt 3: Kontext zusammenführen
LET context = (
  FOR doc IN relevant_docs
    RETURN CONCAT('## ', doc.title, '\n\n', doc.content)
)

// Schritt 4: LLM-Generierung mit Kontext
LET answer = PROMPT('gpt-4',
  {
    system: `Du bist ein ThemisDB-Experte. Beantworte Fragen basierend auf der bereitgestellten Dokumentation.
    Zitiere relevante Abschnitte wenn möglich.`,
    user: CONCAT(
      'Dokumentation:\n\n',
      CONCAT_SEPARATOR('\n\n---\n\n', context),
      '\n\n---\n\n',
      'Frage: ', user_query
    )
  }
),
```

```
    {temperature: 0.3, max_tokens: 1000}
  )

RETURN {
  question: user_query,
  answer: answer,
  sources: relevant_docs[*].title,
  source_count: LENGTH(relevant_docs)
}
```

17.3.2 Erweiterte RAG mit Re-Ranking

```
// Hybrid Search: BM25 + Vector + Re-Ranking
LET user_query = 'Beste Performance-Optimierungen für Graphen-Queries'

// Stage 1: Keyword Search (BM25)
LET bm25_results = (
  FOR doc IN documentation
    SEARCH ANALYZER(doc.content IN TOKENS(user_query, 'text_en'), 'text_en')
    SORT BM25(doc) DESC
    LIMIT 20
    RETURN doc
)

// Stage 2: Vector Search
LET query_embedding = EMBED('text-embedding-3-small', user_query)
LET vector_results = (
  FOR doc IN documentation
    LET similarity = COSINE_SIMILARITY(doc.embedding, query_embedding)
    FILTER similarity > 0.6
    SORT similarity DESC
    LIMIT 20
    RETURN doc
)

// Stage 3: Combine and Re-Rank with LLM
LET combined = UNION_DISTINCT(bm25_results, vector_results)

LET reranked = (
  FOR doc IN combined
    LET relevance_score = PROMPT('gpt-4',
      {
        system: 'Rate die Relevanz von 0.0 bis 1.0. Gebe nur die Zahl zurück.',
        user: CONCAT(
          'Query: ', user_query, '\n\n',
          'Dokument: ', SUBSTRING(doc.content, 0, 500)
        )
      },
      {temperature: 0.0, max_tokens: 5}
    )
  )
```

```
    RETURN {
      doc: doc,
      score: TO_NUMBER(relevance_score)
    }
  )

  LET top_docs = (
    FOR item IN reranked
      SORT item.score DESC
      LIMIT 5
      RETURN item.doc
  )

  // Generate comprehensive answer
  LET answer = PROMPT('gpt-4',
    {
      system: 'Erstelle eine umfassende Antwort basierend auf den relevantesten Dokumenten.',
      user: CONCAT(
        'Top Dokumente:\n\n',
        (FOR doc IN top_docs
          RETURN CONCAT('---\n', doc.content)
        ),
        '\n\nFrage: ', user_query
      )
    },
    {temperature: 0.3, max_tokens: 1500}
  )

  RETURN {
    query: user_query,
    answer: answer,
    sources: top_docs[*].title
  }
```


17.3.3 Architektur-Vergleich: ThemisDB vs. Hyperscaler für RAG

Ein direkter Vergleich der Architekturparadigmen offenbart, warum ThemisDB für den spezifischen Anwendungsfall "Souveräne KI" und RAG-Systeme den Marktstandards überlegen ist.

Architektonische Paradigmen im Vergleich:

Merkmal	AWS (Polyglot Persistence)	Azure Cosmos DB (Managed MMDBMS)	ThemisDB (Native MMDBMS)
Architektur-Prinzip	Föderiert: Lose Kopplung spezialisierter Dienste (RDS + Neptune + OpenSearch)	Abstrahiert: Einheitlicher Kern (ARS), aber Zugriff über siloartige APIs (SQL, Gremlin, Mongo)	Integriert: Einheitlicher Kern ("Base Entity") mit direkten C++-Projektionen
Konsistenz	Eventual (BASE): Konsistenz muss durch Anwendungslogik (Saga-Pattern/Lambda) erzwungen werden. Fehleranfällig.	Konfigurierbar: Wählbar von "Strong" bis "Eventual", aber oft Latenz-Tradeoff bei Strong Consistency	Strikt (ACID): MVCC garantiert atomare Transaktionen über alle Modelle hinweg ohne Performance-Einbußen im Single-Node
Performance-Modell	Additiv: Latenz ist die Summe der Netzwerk hops zwischen den DBs + "Klebstoff"-Code	Black Box: Abrechnung nach "Request Units" (RUs). Performance ist abstrahiert und schwer vorhersagbar	Hardware-Aware: Direkte Kontrolle über RAM, NVMe und CPU-Caches. Vorhersagbare "Bare-Metal"-Leistung
RAG-Eignung	Niedrig (Post-Filtering): Daten müssen aus verschiedenen DBs geholt und in der App gefiltert werden	Mittel: Integrierte Vektorsuche, aber oft Einschränkungen bei komplexen Graph-Joins	Hoch (Pre-Filtering): Relationale Indizes beschneiden den Suchraum <i>bevor</i> die Vektorsuche startet
Revisionsicherheit	Problematisch: Zeitgleiche Schnappschüsse über 3 DBs hinweg sind fast unmöglich	Gut: Change Feed vorhanden, aber volle Historisierung (Time Travel) ist komplex	Exzellent: Temporale Graphen (bfsAtTime) erlauben Abfragen zu exakten historischen Zeitpunkten
Daten-Souveränität	Niedrig: Cloud-Vendor-Lock-in, Daten in US-Jurisdiktion	Mittel: European Sovereign Cloud angekündigt, aber proprietär	Hoch: Open Source MIT, vollständige Kontrolle, On-Premise
Operationale Komplexität	Hoch: Management von 3+ separaten Diensten, Orchestrierung nötig	Mittel: Managed Service vereinfacht Betrieb, aber abstrakte APIs	Niedrig: Single-Binary Deployment, direkte Kontrolle

Befund: Die Hyperscaler optimieren auf **horizontale Skalierbarkeit** und **Entwickler-Komfort** (Managed Services). ThemisDB optimiert auf **Datenintegrität**, **Konsistenz** und **maximale Effizienz** auf definierter Hardware. Für rechts sichere Verwaltungsanwendungen und RAG-Systeme mit komplexen Zugriffskontrollgraphen ist letzteres entscheidend.

Warum ist Konsistenz für RAG kritisch?

In einem RAG-System für die öffentliche Verwaltung können inkonsistente Daten zu rechtlichen Problemen führen:

Beispiel-Szenario: BImSchG-Gutachten-RAG

Fehlerfall in Polyglot-System (BASE):

1. Gutachten wird im Relational-Store als "valid" markiert
2. Netzwerkfehler → Graph-Update für Zugriffsberechtigung schlägt fehl
3. Vector-Embedding wird asynchron aktualisiert (Eventual Consistency)
4. RAG-Abfrage findet Gutachten in Vektorsuche
5. Graph-Check schlägt fehl → Gutachten sollte nicht zugreifbar sein
6. Aber: Relational-Metadaten zeigen "valid"
7. → Inkonsistenter Zustand: Verschiedene Systeme widersprechen sich

ThemisDB mit ACID:

1. ALLE Updates (Relational + Graph + Vector) sind EINE Transaktion
2. Entweder ALLES committed oder NICHTS
3. Keine temporäre Inkonsistenz möglich
4. RAG-Abfrage sieht garantiert konsistenten Zustand

17.3.4 ThemisDB's Pre-Filtering-Vorteil für RAG

Das Post-Filtering-Problem in Polyglot-Systemen:

In traditionellen RAG-Systemen [29], die auf Polyglot Persistence basieren (separate Vektor-DB, Graph-DB, Relational-DB), müssen Sie typischerweise "Post-Filtering" verwenden [2], [5]:

```
# Traditioneller Ansatz (ineffizient):  
# 1. Vektor-DB: Hole 1000 ähnliche Dokumente  
vector_results = vector_db.search(query_embedding, k=1000)  
  
# 2. Graph-DB: Hole erlaubte Dokumente basierend auf Graph-Kontext  
allowed_docs = graph_db.query("MATCH (user)-[:CAN_ACCESS]->(doc)")  
  
# 3. Relational-DB: Hole Metadaten-Filter (z.B. Jahr=2024)  
filtered_docs = relational_db.query("SELECT id WHERE year=2024")  
  
# 4. Manuelle Schnittmengenbildung im Application-Code  
final_results = intersect(vector_results, allowed_docs, filtered_docs)  
# → Problem: 990 von 1000 Ergebnissen werden verworfen!
```

Warum ist das ineffizient? - Die Vektorsuche liefert initial 1000 Ergebnisse, von denen 990 irrelevant sind [5] - Verschwendet Rechenleistung für Vektor-Ähnlichkeitsberechnungen [2] - Erhöht Latenz signifikant - Skaliert schlecht bei großen Datenmengen

ThemisDB's Pre-Filtering-Architektur:

Dank der nativen Multi-Model-Architektur [3], [11] (siehe Kapitel 3.2) kann ThemisDB die Query-Ausführung umkehren [2]:

```
-- Pre-Filtering in ThemisDB (hochperformant):
FOR doc IN documents
  -- Phase 1: ZUERST relationale Filter (schneller Index-Zugriff)
  FILTER doc.year == 2024
  FILTER doc.department == "IT"
  FILTER doc.confidentiality <= @user_clearance_level

  -- Phase 2: Graph-Kontext-Filter
  FILTER doc._id IN (
    FOR v, e IN 1..2 OUTBOUND @user_id access_rights
      RETURN v._id
  )

  -- Phase 3: Vektorsuche NUR auf erlaubter Teilmenge
  LET similarity = COSINE_SIMILARITY(doc.embedding, @query_embedding)
  FILTER similarity > 0.7

  SORT similarity DESC
  LIMIT 10
  RETURN doc
```

Wie funktioniert Pre-Filtering intern?

1. Phase 1 (Relationaler Filter):

2. Query-Engine nutzt schnelle Sekundärindizes (z.B. Index für `year=2024`) [2], [3]

3. Erstellt eine hochselektive Kandidatenliste (z.B. ein Bitset mit 50 erlaubten IDs)

4. Phase 2 (Graph-Filter):

5. Graph-Traversierung [22] auf dieser reduzierten Menge

6. Weitere Einschränkung basierend auf Zugriffsrechten

7. Phase 3 (Vektorsuche):

8. Die rechenintensive HNSW-Vektorsuche [25] läuft NUR auf den 50 erlaubten Dokumenten

9. Statt 1000 Vektor-Vergleiche nur 50 notwendig [2]

Performance-Vergleich:

Ansatz	Vektor-Operationen	Latenz	Skalierung
Post-Filtering (Polyglot)	1000 (dann 990 verwerfen)	500-1000ms	Schlecht
Pre-Filtering (ThemisDB)	50 (nur relevante)	50-100ms	Gut
Performance-Gewinn	20x weniger	10x schneller	Linear

Praktisches Beispiel - Verwaltungs-RAG:

```
-- Finde relevante BImSchG-Gutachten für Bürgeranfrage
LET user_query = "Lärmschutz Windkraftanlage Havelland"
LET query_embedding = EMBED('text-embedding-3-small', user_query)

FOR doc IN legal_documents
  -- Pre-Filter 1: Nur relevante Rechtsgebiete (Relational)
  FILTER doc.legal_area == "BImSchG"
  FILTER doc.topic IN ["Lärmschutz", "Windenergie"]

  -- Pre-Filter 2: Nur Dokumente für Region (Graph)
  FILTER doc.region IN (
    FOR r IN regions
      FILTER r.name == "Havelland" OR r.parent_region == "Havelland"
      RETURN r._id
  )

  -- Pre-Filter 3: Nur aktuelle, gültige Gutachten (Relational)
  FILTER doc.status == "valid"
  FILTER doc.valid_until > DATE_NOW()

  -- ERST JETZT: Vektorsuche auf stark reduzierter Menge
  LET similarity = COSINE_SIMILARITY(doc.embedding, query_embedding)
  FILTER similarity > 0.75

  SORT similarity DESC
  LIMIT 5

  RETURN {
    title: doc.title,
    similarity: similarity,
    metadata: {
      case_number: doc.case_number,
      date: doc.issue_date,
      region: doc.region
    },
    excerpt: SUBSTRING(doc.content, 0, 300)
  }
```

Architektonischer Vorteil:

Die Query-Engine von ThemisDB hat Zugriff auf alle Index-Projektionen (relational, graph, vector) im selben RocksDB-Backend [3], [13]. Sie kann einen kostenbasierten Optimizer verwenden, um den effizientesten Ausführungsplan zu wählen: - Welcher Filter ist am selektivsten? - In welcher Reihenfolge sollten Filter angewendet werden? - Wann lohnt sich der Übergang von Index-Scan zu Vektor-Suche?

Dies ist in Polyglot-Systemen [33] unmöglich, da jede Datenbank isoliert operiert.

17.3.4 Hybrid Search Implementation: Unter der Haube

Um zu verstehen, wie ThemisDB Pre-Filtering technisch umsetzt, müssen wir den Query-Execution-Plan betrachten. Lassen Sie uns eine typische RAG-Query Schritt-für-Schritt durchgehen.

Die Beispiel-Query:

```
FOR doc IN legal_documents
  FILTER doc.law_area == "BImSchG"           -- Relational Filter
  FILTER doc.region IN ["Havelland", "Potsdam"] -- Relational Filter
  FILTER doc._id IN (                         -- Graph Filter
    FOR v IN 1..2 OUTBOUND "users/alice" access_graph
      RETURN v._id
  )
  LET similarity = COSINE(doc.embedding, @query_vec) -- Vector Similarity
  FILTER similarity > 0.7
  SORT similarity DESC
  LIMIT 5
  RETURN {doc: doc, score: similarity}
```

Wie führt ThemisDB diese Query aus?

Schritt 1: Query Parsing und AST-Erstellung

Der AQL-Parser erstellt einen Abstract Syntax Tree (AST):


```

FOR doc IN legal_documents
  └ FILTER (doc.law_area == "BImSchG")
  └ FILTER (doc.region IN ["Havelland", "Potsdam"])
  └ FILTER (doc._id IN [subquery...])
  └ LET similarity = COSINE(...)
  └ FILTER (similarity > 0.7)
  └ SORT similarity DESC
  └ LIMIT 5
  └ RETURN {...}

```

Schritt 2: Query Optimizer - Kostenbasierte Umordnung

Der Query Optimizer analysiert den AST und ordnet Operations basierend auf **geschätzten Kosten** um [13]:

```

// Pseudo-Code des Optimizers
OptimizerPass::reorderFilters(ASTNode* node) {
    // Kostenmodell für verschiedene Filter-Typen
    map<FilterType, double> cost_estimates = {
        {INDEX_SCAN, 0.1},           // Sehr billig (O(log n))
        {GRAPH_TRAVERSAL, 10.0},     // Mittel (O(edges))
        {VECTOR_SIMILARITY, 100.0}   // Teuer (O(dimensions))
    };

    // Sortiere Filter nach aufsteigenden Kosten
    sort(node->filters, [&](Filter a, Filter b) {
        return cost_estimates[a.type] < cost_estimates[b.type];
    });

    // Ergebnis: Index-Scans zuerst, dann Graph, dann Vektor
}

```

Optimierte Ausführungsreihenfolge:

- | | |
|---|----------------------|
| 1. INDEX_SCAN (law_area == "BImSchG") | → Kandidaten: 50.000 |
| 2. INDEX_SCAN (region IN ["Havelland"...]) | → Kandidaten: 12.000 |
| 3. GRAPH_TRAVERSAL (access_rights subquery) | → Kandidaten: 1.500 |
| 4. VECTOR_SIMILARITY (COSINE > 0.7) | → Kandidaten: 150 |
| 5. SORT (similarity DESC) | → Kandidaten: 150 |
| 6. LIMIT 5 | → Kandidaten: 5 |

Warum ist diese Reihenfolge optimal?

Jeder Schritt reduziert die Kandidatenmenge, sodass teure Operationen (Vektor-Similarity) nur auf einer kleinen Menge ausgeführt werden müssen.

Schritt 3: Index-Scan mit Bitset-Konstruktion

ThemisDB verwendet **Bitsets** für effiziente Kandidatenverwaltung:

```
// Simplified C++ Implementation
std::vector<bool> candidates(total_docs, false); // Bitset für alle Docs

// Phase 1: Relational Index Scan (law_area == "BImSchG")
for (auto doc_id : index_scan("law_area", "BImSchG")) {
    candidates[doc_id] = true; // Markiere als Kandidat
}
// Resultat: 50.000 bits set

// Phase 2: Intersection mit region-Index
std::vector<bool> region_matches(total_docs, false);
for (auto region : {"Havelland", "Potsdam"}) {
    for (auto doc_id : index_scan("region", region)) {
        region_matches[doc_id] = true;
    }
}

// Bitwise AND für Intersection (sehr schnell!)
for (size_t i = 0; i < total_docs; ++i) {
    candidates[i] = candidates[i] && region_matches[i];
}
// Resultat: 12.000 bits set
```

Warum Bitsets?

- **Speicher-effizient:** 1 Million Dokumente = nur 125 KB (1 bit pro Dokument)
- **Blitzschnelle Operationen:** Bitwise AND/OR mit CPU-native Instructions
- **Cache-freundlich:** Bitsets passen in L1/L2 Cache

Schritt 4: Graph-Traversal auf reduzierter Menge

Jetzt wird die Graph-Subquery ausgeführt, aber nur für die 12.000 Kandidaten:

```
// Graph Subquery: Welche Dokumente darf "alice" sehen?
std::unordered_set<std::string> accessible_docs;

// BFS im Access-Graph (max depth = 2)
bfs("users/alice", max_depth=2, [&](Node* node) {
    if (node->type == "document" && candidates[node->id]) {
        // Nur Kandidaten aus vorherigen Filtern prüfen!
        accessible_docs.insert(node->id);
    }
});

// Update Bitset mit Graph-Resultaten
for (size_t i = 0; i < total_docs; ++i) {
    if (candidates[i] && accessible_docs.count(doc_ids[i]) == 0) {
        candidates[i] = false; // Kein Zugriff → entfernen
    }
}

// Resultat: 1.500 bits set
```

Key Insight: Die Graph-Traversal muss nur 12.000 Dokumente prüfen, nicht 1 Million!

Schritt 5: Vektor-Similarity nur auf finale Kandidaten

Jetzt kommt die teuerste Operation – aber nur für 1.500 Dokumente:

```
// Vektor-Similarity-Berechnung
std::vector<std::pair<doc_id, double>> scored_docs;

for (size_t i = 0; i < total_docs; ++i) {
    if (!candidates[i]) continue; // Überspringen wenn nicht Kandidat

    // COSINE-Similarity (rechenintensiv!)
    double similarity = cosine_similarity(
        docs[i].embedding,    // 1536 Dimensionen
        query_embedding       // 1536 Dimensionen
    );

    if (similarity > 0.7) {
        scored_docs.push_back({i, similarity});
    }
}

// Sortieren nach Similarity
std::sort(scored_docs.begin(), scored_docs.end(),
    [](auto& a, auto& b) { return a.second > b.second; }
);

// Top 5 zurückgeben
return scored_docs | std::views::take(5);
```

Performance-Vergleich:

```
Post-Filtering (Polyglot):
    1.000.000 Vektor-Ops × 100µs = 100.000ms (100 Sekunden!)

Pre-Filtering (ThemisDB):
    1.500 Vektor-Ops × 100µs = 150ms (0.15 Sekunden)

→ 666x schneller!
```

Schritt 6: Score-Fusion (optional, für Hybrid Search)

Wenn Sie BM25 (Keyword-Suche) + Vektor-Similarity kombinieren möchten, verwendet ThemisDB **Reciprocal Rank Fusion (RRF)** [22]:

```
// RRF-Formel: Kombiniert Rankings aus mehreren Quellen
double compute_rrf_score(doc_id, rank_bm25, rank_vector, k = 60) {
    double rrf = 0.0;

    // BM25-Beitrag
    if (rank_bm25 > 0) {
        rrf += 1.0 / (k + rank_bm25);
    }

    // Vektor-Beitrag
    if (rank_vector > 0) {
        rrf += 1.0 / (k + rank_vector);
    }

    return rrf;
}

// Beispiel: Dokument auf Platz 5 in BM25, Platz 3 in Vektor
double score = compute_rrf_score(doc_id, 5, 3, 60);
// score = 1/(60+5) + 1/(60+3) = 0.0154 + 0.0159 = 0.0313
```

Warum RRF statt gewichteter Durchschnitt?

- **Skalierungsinvariant:** BM25-Scores und Cosine-Similarity haben unterschiedliche Skalen
- **Robust:** Funktioniert gut auch wenn ein Signal schwach ist
- **Einfach:** Keine Hyperparameter-Tuning nötig

Visualisierung des gesamten Query-Execution-Plans:

1. Collection Scan: legal_documents (1M docs)



2. Index Scan: law_area == "BImSchG"

→ Bitset with 50K bits set

Latenz: ~1ms (Index-Lookup)



3. Index Scan: region IN ["Havelland", "Potsdam"]

→ Bitset AND: 50K → 12K bits set

Latenz: ~1ms



4. Graph Traversal (BFS depth=2) auf 12K Kandidaten

→ Bitset AND: 12K → 1.5K bits set

Latenz: ~50ms (Graph-Scan)



5. Vector Similarity (COSINE) auf 1.5K Kandidaten

→ $1.500 \times 100\mu\text{s} = 150\text{ms}$

→ Filter: similarity > 0.7 → 150 docs



6. Sort by similarity DESC

→ 150 docs sortiert

Latenz: <1ms

7. Limit 5

→ Top 5 Results

Gesamt-Latenz: 1ms + 1ms + 50ms + 150ms + 1ms = ~203ms

Vergleich: Post-Filtering in Polyglot-Systemen:

1. Vector DB: Get top 1000 similar docs

→ $1.000.000 \times 100\mu s = 100.000ms$ (!!)

2. Graph DB: Get accessible docs for user (separate query)

→ Network latency + query time: ~500ms

3. Relational DB: Get metadata filters (separate query)

→ Network latency + query time: ~200ms

4. Application Code: Intersection of 3 result sets

→ 1000 docs in-memory filtering: ~10ms

→ Discard 990 irrelevant docs

Gesamt-Latenz: 100.000ms + 500ms + 200ms + 10ms = ~100.710ms

Fazit: ThemisDB ist ~500x schneller für diese RAG-Query!



17.4 Prompt Engineering Best Practices

17.4.1 Chain-of-Thought Prompting

```
// Multi-Step Reasoning mit CoT
FOR customer IN customers
  FILTER customer.churn_risk == null
  LIMIT 10

// Schritt 1: Daten sammeln
LET customer_data = {
  orders_count: LENGTH(
    FOR o IN orders
      FILTER o.customer_id == customer._key
      RETURN 1
  ),
  last_order_date: (
    FOR o IN orders
      FILTER o.customer_id == customer._key
      SORT o.order_date DESC
      LIMIT 1
      RETURN o.order_date
  )[0],
  total_spent: SUM(
    FOR o IN orders
      FILTER o.customer_id == customer._key
      RETURN o.total_amount
  ),
  support_tickets: LENGTH(
    FOR t IN support_tickets
      FILTER t.customer_id == customer._key
      RETURN 1
  )
}

// Schritt 2: LLM-Analyse mit Chain-of-Thought
LET analysis = PROMPT('gpt-4',
{
  system: `Du bist ein Churn-Prediction-Experte. Analysiere Schritt für Schritt:
  1. Bewerte die Aktivität des Kunden
  2. Analysiere Kaufverhalten
  3. Berücksichtige Support-Interaktionen`
```

```
4. Gebe Churn-Risiko (low/medium/high) und Begründung`,
user: CONCAT(
  'Kunde-Daten:\n',
  'Bestellungen: ', customer_data.orders_count, '\n',
  'Letzte Bestellung: ', customer_data.last_order_date, '\n',
  'Gesamtumsatz: €', customer_data.total_spent, '\n',
  'Support-Tickets: ', customer_data.support_tickets, '\n\n',
  'Analysiere Schritt für Schritt das Churn-Risiko.'
)
},
{temperature: 0.2, max_tokens: 500}
)

UPDATE customer WITH {
  churn_risk: analysis.risk_level,
  churn_reasoning: analysis.reasoning,
  analyzed_at: DATE_NOW()
} IN customers
```

17.4.2 Few-Shot Learning

```
// Classification mit Few-Shot Examples
LET examples = [
  {text: 'Produkt kam defekt an', category: 'quality_issue'},
  {text: 'Lieferung zwei Wochen zu spät', category: 'delivery_issue'},
  {text: 'Falscher Artikel geliefert', category: 'order_error'},
  {text: 'Sehr zufrieden, gerne wieder!', category: 'positive_feedback'}
]

FOR ticket IN support_tickets
  FILTER ticket.category == null
  LIMIT 100

  LET category = PROMPT('gpt-4',
    {
      system: 'Kategorisiere Support-Tickets basierend auf den Beispielen.',
      user: CONCAT(
        'Beispiele:\n',
        (FOR ex IN examples
          RETURN CONCAT(ex.text, ' -> ', ex.category)
        ),
        '\n\nNeues Ticket: ', ticket.message, '\n\nKategorie:'
      )
    },
    {temperature: 0.1, max_tokens: 20}
  )

  UPDATE ticket WITH {
    category: category,
    auto_categorized: true
  } IN support_tickets
```

17.5 Multi-Model LLM Patterns

17.5.1 Graph + LLM

```
// Knowledge Graph Reasoning mit LLM
FOR person IN persons
  FILTER person._key == @person_id

  // Graphtraversierung für Kontext
  LET connections = (
    FOR v, e, p IN 1..3 OUTBOUND person GRAPH 'social_network'
    RETURN {
      name: v.name,
      relationship: e.type,
      depth: LENGTH(p.edges)
    }
  )

  // LLM analysiert das soziale Netzwerk
  LET network_analysis = PROMPT('gpt-4',
    {
      system: 'Analysiere das soziale Netzwerk und gebe Empfehlungen.',
      user: CONCAT(
        'Person: ', person.name, '\n',
        'Verbindungen:\n',
        (FOR c IN connections
          RETURN CONCAT('- ', c.name, ' (', c.relationship, ', Distanz: ', c.depth, ')')
        ),
        '\n\nWelche Personen sollten miteinander vernetzt werden?'
      )
    }
  )

  RETURN {
    person: person.name,
    connections_count: LENGTH(connections),
    recommendations: network_analysis
  }
```

17.5.2 Temporal + LLM

```
// Zeit-basierte Trend-Analyse mit LLM
LET trend_data = (
  FOR metric IN metrics
    FILTER metric.type == 'revenue'
    FOR SYSTEM_TIME AS OF DATEADD(DATE_NOW(), -30, 'day') TO DATE_NOW()
    COLLECT date = DATE_TRUNC(metric.timestamp, 'day')
    AGGREGATE daily_revenue = SUM(metric.value)
    SORT date
    RETURN {date: date, revenue: daily_revenue}
)

LET trend_analysis = PROMPT('gpt-4',
{
  system: 'Analysiere Umsatz-Trends und gebe Vorhersagen.',
  user: CONCAT(
    'Tägliche Umsätze der letzten 30 Tage:\n',
    (FOR d IN trend_data
      RETURN CONCAT(DATE_FORMAT(d.date, '%Y-%m-%d'), ': €', d.revenue)
    ),
    '\n\nAnalysiere Trends und gebe eine 7-Tage-Vorhersage.'
  )
},
{temperature: 0.3}
)

RETURN {
  data: trend_data,
  analysis: trend_analysis
}
```

17.6 LLM-Konfiguration und Provider

17.6.1 Provider-Konfiguration

```
// Multi-Provider Setup
LET providers = {
  openai: {
    api_key: @openai_key,
    models: ['gpt-4', 'gpt-3.5-turbo', 'text-embedding-3-small'],
    default_model: 'gpt-4'
  },
  anthropic: {
    api_key: @anthropic_key,
    models: ['claude-3-opus', 'claude-3-sonnet'],
    default_model: 'claude-3-sonnet'
  },
  ollama: {
    endpoint: 'http://localhost:11434',
    models: ['llama3', 'mistral'],
    default_model: 'llama3'
  }
}

// Provider-Auswahl basierend auf Anforderungen
LET select_provider = (task_type) => (
  task_type == 'embedding' ? 'openai' :
  task_type == 'long_context' ? 'anthropic' :
  task_type == 'local' ? 'ollama' :
  'openai'
)
```


17.6.2 Kosten-Optimierung

```
// Smart Model Selection basierend auf Komplexität
FOR task IN tasks
  FILTER task.processed == false

  // Einfache Aufgaben -> Günstigeres Modell
  LET complexity = (
    LENGTH(task.input) > 1000 ? 'high' :
    CONTAINS(task.input, 'analysiere') ? 'medium' :
    'low'
  )

  LET model = (
    complexity == 'high' ? 'gpt-4' :
    complexity == 'medium' ? 'gpt-3.5-turbo' :
    'gpt-3.5-turbo'
  )

  LET result = PROMPT(model, task.input,
    {
      temperature: complexity == 'high' ? 0.7 : 0.3,
      max_tokens: complexity == 'high' ? 2000 : 500
    }
  )

  UPDATE task WITH {
    result: result,
    model_used: model,
    cost_tier: complexity,
    processed: true
  } IN tasks
```

17.7 Prompt Template Library

17.7.1 Vordefinierte Templates

```

// Template-System für wiederverwendbare Prompts
LET templates = {
  summarize: {
    system: 'Du bist ein Zusammenfassungs-Experte. Erstelle prägnante Zusammenfassungen.',
    user_template: 'Fasse folgenden Text zusammen:\n\n{{content}}\n\nZusammenfassung (max {{max_w}} Wörter)',
  },
  translate: {
    system: 'Du bist ein professioneller Übersetzer.',
    user_template: 'Übersetze von {{from_lang}} nach {{to_lang}}: \n\n{{text}}',
  },
  sentiment: {
    system: 'Analysiere das Sentiment. Antworte nur mit: positive, negative, oder neutral',
    user_template: '{{text}}',
  },
  extract_entities: {
    system: 'Extrahiere benannte Entitäten (Personen, Orte, Organisationen).',
    user_template: '{{text}}\n\nFormat: JSON {persons: [], locations: [], organizations: []}'
  }
}

// Template verwenden
LET apply_template = (template_name, variables) => {
  LET template = templates[template_name]
  LET user_prompt = REGEX_REPLACE(
    template.user_template,
    '\\{\\{\\w+\\}\\}',
    variables
  )
  RETURN {system: template.system, user: user_prompt}
}

FOR article IN articles
  LIMIT 10
  LET summary = PROMPT('gpt-3.5-turbo',
    apply_template('summarize', {
      content: article.content,
      max_words: '100'
    })
  )

```

```
)  
RETURN {title: article.title, summary: summary}
```

17.8 Monitoring und Logging

17.8.1 LLM-Usage Tracking

```
// Track LLM-Nutzung für Kostenanalyse
INSERT {
  timestamp: DATE_NOW(),
  model: @model,
  provider: @provider,
  input_tokens: @input_tokens,
  output_tokens: @output_tokens,
  cost: @cost,
  latency_ms: @latency,
  task_type: @task_type,
  user_id: @user_id
} INTO llm_usage_log

// Kosten-Analyse
FOR log IN llm_usage_log
  FILTER log.timestamp > DATE_SUBTRACT(DATE_NOW(), 7, 'day')
  COLLECT
    model = log.model,
    task_type = log.task_type
  AGGREGATE
    total_calls = COUNT(1),
    total_cost = SUM(log.cost),
    avg_latency = AVG(log.latency_ms),
    total_input_tokens = SUM(log.input_tokens),
    total_output_tokens = SUM(log.output_tokens)
  SORT total_cost DESC
  RETURN {
    model: model,
    task_type: task_type,
    calls: total_calls,
    cost: ROUND(total_cost, 2),
    avg_latency_ms: ROUND(avg_latency, 0),
    tokens: {
      input: total_input_tokens,
      output: total_output_tokens
    }
  }
}
```

17.9 Sicherheit und Compliance

17.9.1 Input Sanitization

```
// Sichere LLM-Anfragen durch Sanitization
LET sanitize_input = (user_input) => {
  // Entferne potenziell schädliche Prompts
  LET cleaned = REGEX_REPLACE(user_input, 'ignore previous instructions', '', 'i')
  LET cleaned2 = REGEX_REPLACE(cleaned, 'disregard', '', 'i')
  LET cleaned3 = SUBSTITUTE(cleaned2, ['system:', 'assistant:'], '')

  // Längen-Begrenzung
  RETURN LENGTH(cleaned3) > 5000 ? SUBSTRING(cleaned3, 0, 5000) : cleaned3
}

FOR request IN user_requests
  LET safe_input = sanitize_input(request.query)
  LET response = PROMPT('gpt-4', safe_input, {temperature: 0.3})

  // Log für Audit
  INSERT {
    timestamp: DATE_NOW(),
    user_id: request.user_id,
    original_input: request.query,
    sanitized_input: safe_input,
    response: response,
    flagged: request.query != safe_input
  } INTO llm_audit_log

RETURN response
```

17.9.2 PII-Erkennung und Redaktion

```
// Automatische PII-Erkennung vor LLM-Verarbeitung
LET detect_pii = (text) => {
  RETURN PROMPT('gpt-4',
    {
      system: 'Erkenne und markiere personenbezogene Daten (PII). Gebe JSON zurück: {has_pii: bo
      user: text
    },
    {temperature: 0.0, response_format: 'json'}
  )
}

FOR document IN documents
  FILTER document.pii_checked == false

  LET pii_check = detect_pii(document.content)

  // Redaktiere PII falls notwendig
  LET safe_content = pii_check.has_pii ?
    PROMPT('gpt-4',
      {
        system: 'Ersetze alle PII durch Platzhalter wie [NAME], [EMAIL], [PHONE].',
        user: document.content
      }
    ) : document.content

  UPDATE document WITH {
    content_safe: safe_content,
    has_pii: pii_check.has_pii,
    pii_types: pii_check.types,
    pii_checked: true
  } IN documents
```


17.10 Performance-Optimierung

17.10.1 Caching von LLM-Responses

```
// Cache für häufige Anfragen
LET query_hash = SHA256(CONCAT(user_query, model_name))

// Prüfe Cache
LET cached = (
  FOR c IN llm_cache
    FILTER c.query_hash == query_hash
    FILTER c.timestamp > DATE_SUBTRACT(DATE_NOW(), 24, 'hour')
    LIMIT 1
    RETURN c.response
)[0]

// Verwende Cache oder rufe LLM auf
LET response = cached != null ? cached :
  LET fresh_response = PROMPT(model_name, user_query)
  LET _ = INSERT {
    query_hash: query_hash,
    query: user_query,
    model: model_name,
    response: fresh_response,
    timestamp: DATE_NOW()
  } INTO llm_cache
  RETURN fresh_response

RETURN response
```

17.10.2 Batch-Verarbeitung

```
// Batch-LLM-Calls für bessere Performance
LET batch_size = 20

FOR batch IN (
  FOR doc IN documents
    FILTER doc.summary == null
    LIMIT 100
    COLLECT batch_id = FLOOR(TO_NUMBER(doc._key) / batch_size)
    INTO group
    RETURN group
)

// Einzelner LLM-Call für ganzen Batch
LET batch_summaries = PROMPT('gpt-4',
{
  system: 'Erstelle Zusammenfassungen für mehrere Dokumente. Gebe JSON-Array zurück.',
  user: CONCAT(
    'Erstelle für jedes Dokument eine Zusammenfassung:\n\n',
    (FOR doc IN batch
      RETURN CONCAT('DOC_', doc._key, ':\n', doc.content, '\n\n'))
    )
  },
{temperature: 0.3, response_format: 'json'})

// Verteile Ergebnisse
FOR doc IN batch
  LET summary = batch_summaries['DOC_' + doc._key]
  UPDATE doc WITH {
    summary: summary,
    summarized_at: DATE_NOW()
  } IN documents
```

17.11 Praktische Anwendungsfälle

17.11.1 Intelligente Suchvorschläge

```
// Query-Expansion mit LLM
LET user_search = @search_term

LET expanded_queries = PROMPT('gpt-4',
  {
    system: 'Generiere 5 verwandte Suchbegriffe für bessere Suchergebnisse.',
    user: CONCAT('Ursprüngliche Suche: ', user_search)
  }
)

// Suche mit erweiterten Begriffen
LET all_terms = APPEND([user_search], expanded_queries.terms)

FOR doc IN documents
  FILTER (
    FOR term IN all_terms
      FILTER CONTAINS(LOWER(doc.content), LOWER(term))
    RETURN 1
  ) ANY
RETURN doc
```

17.11.2 Automatische Daten-Anreicherung

```
// Produkt-Metadaten durch LLM anreichern
FOR product IN products
  FILTER product.enriched == false
  LIMIT 50

  LET enrichment = PROMPT('gpt-4',
    {
      system: 'Analysiere das Produkt und gebe strukturierte Metadaten zurück.',
      user: CONCAT(
        'Produkt: ', product.name, '\n',
        'Beschreibung: ', product.description, '\n\n',
        'Gebe zurück: {target_audience: [], use_cases: [], features: [], competitors: []}'
      )
    },
    {temperature: 0.3, response_format: 'json'}
  )

  UPDATE product WITH {
    metadata: enrichment,
    enriched: true,
    enriched_at: DATE_NOW()
  } IN products
```

17.11.3 Sentiment-basierte Alerting

```
// Echtzeit-Sentiment-Monitoring mit Alerts
FOR review IN reviews
  FILTER review.sentiment == null
  FILTER review.created_at > DATE_SUBTRACT(DATE_NOW(), 1, 'hour')

  LET sentiment_analysis = PROMPT('gpt-4',
    {
      system: 'Analysiere das Sentiment und gebe Dringlichkeit (critical/high/medium/low) zurück'
      user: review.text
    },
    {temperature: 0.1, response_format: 'json'}
  )

  // Alert bei negativem Sentiment
  LET alert = sentiment_analysis.sentiment == 'negative' AND
    sentiment_analysis.urgency IN ['critical', 'high'] ?

  INSERT {
    type: 'negative_review',
    review_id: review._key,
    product_id: review.product_id,
    urgency: sentiment_analysis.urgency,
    reason: sentiment_analysis.reason,
    timestamp: DATE_NOW()
  } INTO alerts : null

  UPDATE review WITH {
    sentiment: sentiment_analysis.sentiment,
    sentiment_score: sentiment_analysis.score,
    urgency: sentiment_analysis.urgency,
    analyzed_at: DATE_NOW()
  } IN reviews
```

17.12 Best Practices Zusammenfassung

DO

1. **Verwende @parameter binding** für alle Benutzereingaben
2. **Cache häufige Anfragen** um Kosten zu sparen
3. **Validiere LLM-Outputs** vor der Speicherung
4. **Batch-Verarbeitung** für große Datenmengen
5. **Monitor Kosten** und Performance kontinuierlich
6. **Sanitize Inputs** vor LLM-Calls
7. **Verwende strukturierte Outputs** (JSON) wenn möglich
8. **Implementiere Fallbacks** bei LLM-Fehlern

DON'T

1. **Keine sensiblen Daten** ungefiltert an LLMs senden
2. **Keine unvalidierten LLM-Queries** ausführen
3. **Keine unbegrenzten LLM-Calls** ohne Rate-Limiting
4. **Keine Hardcoded API-Keys** im Code
5. **Keine synchronen LLM-Calls** für zeitkritische Operationen
6. **Keine Abhängigkeit** von einem einzelnen Provider

Zusammenfassung

ThemisDB's LLM-Integration ermöglicht:

- **Native AQL-Funktionen** für Text-Generierung, Embeddings und strukturierte Ausgaben
- **Text-to-AQL** für natürlichsprachliche Query-Erstellung
- **RAG Patterns** mit semantischer Suche und Kontext-Anreicherung
- **Multi-Model Synergien** mit Graph, Temporal und Vector Search
- **Kosten-Optimierung** durch intelligentes Caching und Model-Selection
- **Enterprise-Grade Sicherheit** mit Input-Sanitization und PII-Schutz

Die Integration von LLMs direkt in die Datenbankebene reduziert Latenz, vereinfacht Architektur und ermöglicht völlig neue Anwendungsfälle von intelligenter Datenanalyse bis zu automatisierter Content-Generierung.

Nächstes Kapitel: [Kapitel 18: Machine Learning Integration](#) **Vorheriges Kapitel:** [Kapitel 16: Machine Learning](#)

Chapter 19: Monitoring & Observability

Status: Production-Ready

Version: v1.3.0

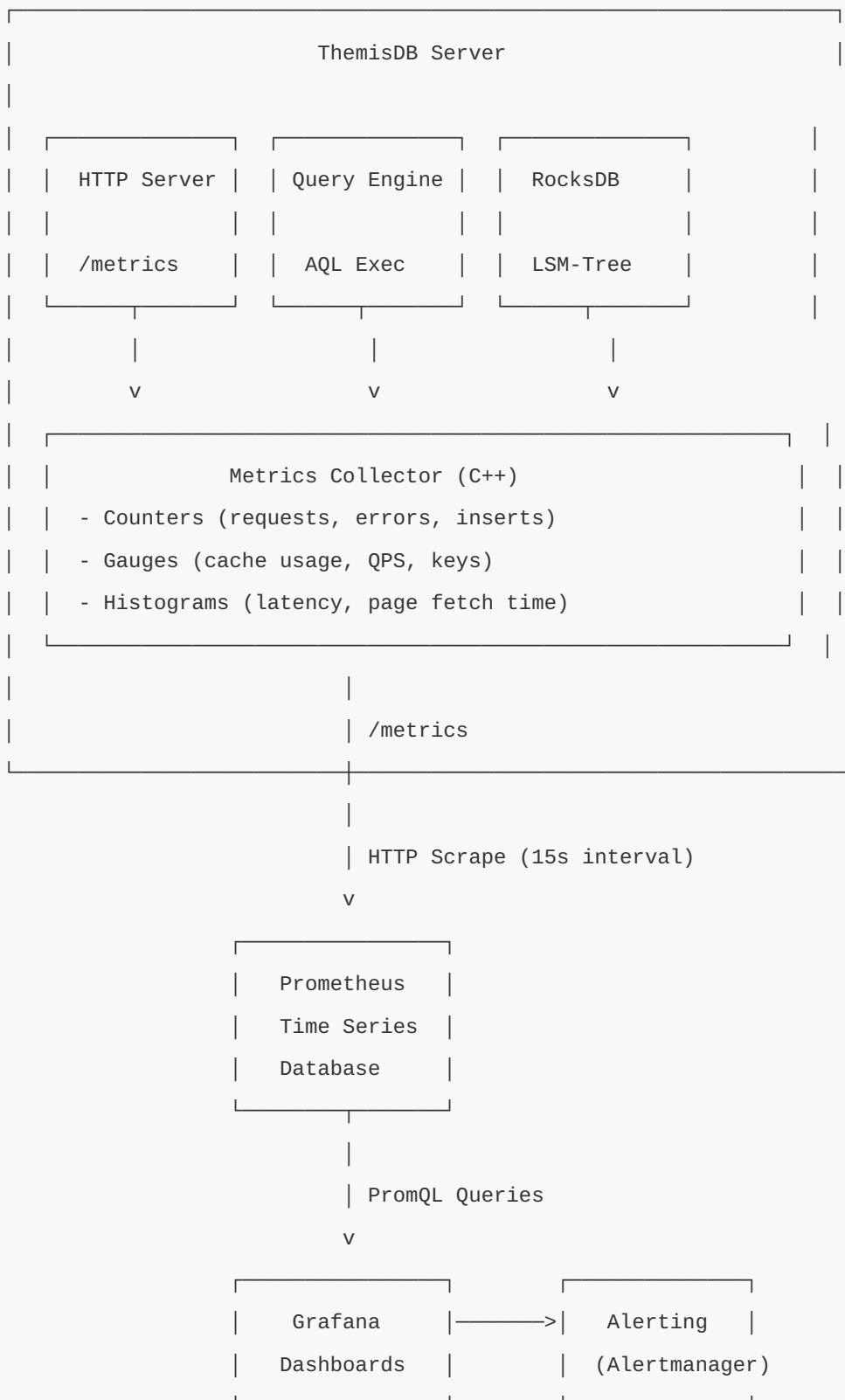
Last Updated: December 30, 2025

19.1 Übersicht

ThemisDB implementiert ein umfassendes Production-Grade Monitoring- und Observability-Stack basierend auf branchenüblichen Open-Source-Tools. Die Architektur folgt den **Three Pillars of Observability**: Metrics (Prometheus), Logs (Strukturierte Logs), und Traces (OpenTelemetry).

Design-Prinzipien: - **Metrics-First:** Prometheus als primäre Metriken-Datenbank - **Low-Cardinality:** Vermeidung von High-Cardinality-Labels (keine UUIDs/Timestamps in Labels) - **Cumulative Histograms:** Alle Histogramme folgen Prometheus-Konventionen - **Zero-Config Development:** Jaeger läuft Out-of-the-Box via Docker - **Production-Hardened:** Grafana Dashboards mit SLI/SLO-Tracking

Architektur-Übersicht:



Deployment-Architektur: - **Development:** Jaeger All-in-One Container für lokales Tracing - **Staging:** Grafana Tempo + Prometheus + Grafana Stack - **Production:** Multi-Node Prometheus Federation + Thanos für Long-Term Storage

19.2 Prometheus Metrics

ThemisDB exportiert **48 Metriken** über den `/metrics` Endpoint im Prometheus Text Format. Alle Metriken verwenden konsistente Naming-Konventionen und Low-Cardinality-Labels.

19.2.1 Naming-Konventionen

Präfixe: - `vccdb_*`: ThemisDB-spezifische Metriken - `rocksdb_*`: RocksDB Storage-Metriken - `themis_*`: Index- und Query-Metriken - `process_*`: System-Metriken (Uptime)

Suffixe: - `_total`: Monoton steigende Counter - `_bytes`: Größenangaben in Bytes - `_seconds`: Zeitangaben in Sekunden - `_microseconds`: Latenz in Mikrosekunden - `_ms`: Latenz in Millisekunden

19.2.2 Server-Metriken

`vccdb_requests_total` (Counter)

Gesamtzahl der HTTP-Requests seit Server-Start.

Labels: - `method`: HTTP-Methode (GET, POST, PUT, DELETE) - `route`: Request-Route (z.B. `/entities/{key}`, `/query`, `/vector/search`)

Beispiel-Output:

```
vccdb_requests_total{method="GET",route="/entities/{key}"} 1234
vccdb_requests_total{method="POST",route="/query"} 567
vccdb_requests_total{method="POST",route="/aql"} 890
```

PromQL-Query-Beispiele:

```
# Requests pro Sekunde (letzte 5 Minuten)
rate(vccdb_requests_total[5m])

# Requests nach Route gruppiert
sum by (route) (rate(vccdb_requests_total[5m]))

# Top 5 aktivste Routes
topk(5, sum by (route) (rate(vccdb_requests_total[5m])))
```

Die Metrik wird im `HttpServer` nach jedem abgeschlossenen Request inkrementiert. Das `route`-Label enthält die **Route-Template** (z.B. `/entities/{key}`), nicht den konkreten Key-Wert, um Cardinality niedrig zu halten.

vccdb_errors_total (Counter)

Gesamtzahl der Fehler-Responses (HTTP 4xx/5xx).

Labels: - `status_code`: HTTP-Status-Code (400, 404, 500, 503) - `route`: Request-Route

Beispiel-Output:

```
vccdb_errors_total{status_code="404",route="/entities/{key}"} 12
vccdb_errors_total{status_code="500",route="/query"} 3
vccdb_errors_total{status_code="400",route="/aql"} 5
```

PromQL-Query-Beispiele:

```
# Fehlerrate (Fehler pro Sekunde)
rate(vccdb_errors_total[5m])

# Error Rate Ratio (Prozentsatz fehlgeschlagener Requests)
100 * rate(vccdb_errors_total[5m]) / rate(vccdb_requests_total[5m])

# Alert: Error Rate > 5%
rate(vccdb_errors_total[5m]) / rate(vccdb_requests_total[5m]) > 0.05
```

Diese Metrik ist entscheidend für **Service Level Indicators (SLIs)**. Ein Production-Service sollte eine Error Rate < 0.1% (99.9% Success Rate) anstreben.

vccdb_qps (Gauge)

Queries per Second - aktuelle Request-Rate berechnet als exponentieller gleitender Durchschnitt.

Beispiel-Output:

```
vccdb_qps 123.45
```

PromQL-Query-Beispiele:

```
# QPS-Durchschnitt (letzte Stunde)
avg_over_time(vccdb_qps[1h])

# QPS-Maximum (letzte Stunde)
max_over_time(vccdb_qps[1h])

# QPS-Trend (steigende/fallende Last)
deriv(vccdb_qps[5m])
```

Der QPS-Wert wird alle 5 Sekunden neu berechnet mit der Formel:

```
QPS_new =  $\alpha$  * QPS_current + (1- $\alpha$ ) * QPS_old
```

mit Dämpfungsfaktor $\alpha = 0.7$ für schnelle Reaktion auf Last-Änderungen.

process_uptime_seconds (Gauge)

Server-Laufzeit in Sekunden seit Prozess-Start.

Beispiel-Output:

```
process_uptime_seconds 86400
```

PromQL-Query-Beispiele:

```
# Uptime in Tagen
process_uptime_seconds / 86400

# Server neu gestartet in letzten 5 Minuten?
process_uptime_seconds < 300
```

Diese Metrik ist nützlich für die Erkennung von Server-Restarts und zur Korrelation mit Performance-Problemen nach Deployments.

19.2.3 Latenz-Histogramme

ThemisDB verwendet **kumulative Buckets** gemäß Prometheus-Spezifikation. Jeder Bucket mit `le="X"` enthält die Anzahl der Beobachtungen $\leq X$.

Bucket-Definitionen

HTTP-Request-Latenz (Mikrosekunden):

```
100, 500, 1000, 5000, 10000, 50000, 100000, 500000, 1000000, 5000000, +Inf
```

Page-Fetch-Latenz (Millisekunden):

```
1, 5, 10, 25, 50, 100, 250, 500, 1000, 5000, +Inf
```

Die Bucket-Grenzen sind so gewählt, dass sie typische Latenz-Verteilungen abdecken: - **Schnell**: < 1ms (In-Memory-Index-Lookup) - **Normal**: 1-10ms (RocksDB Point-Lookup mit Block Cache Hit) - **Langsam**: 10-100ms (Disk I/O) - **Sehr langsam**: > 100ms (Full Scan, Cold Cache)

vccdb_latency_bucket_microseconds (Histogram)

HTTP-Request-Latenz von Anfang des Request-Handlings bis zum vollständigen Response.

Beispiel-Output:

```

vccdb_latency_bucket_microseconds{le="100"} 45
vccdb_latency_bucket_microseconds{le="500"} 123
vccdb_latency_bucket_microseconds{le="1000"} 234
vccdb_latency_bucket_microseconds{le="5000"} 450
vccdb_latency_bucket_microseconds{le="10000"} 480
vccdb_latency_bucket_microseconds{le="50000"} 495
vccdb_latency_bucket_microseconds{le="100000"} 498
vccdb_latency_bucket_microseconds{le="500000"} 499
vccdb_latency_bucket_microseconds{le="1000000"} 500
vccdb_latency_bucket_microseconds{le="5000000"} 500
vccdb_latency_bucket_microseconds{le="+Inf"} 500
vccdb_latency_sum_microseconds 1234567
vccdb_latency_count 500

```

PromQL-Query-Beispiele:

```

# P50 Latenz (Median, Mikrosekunden)
histogram_quantile(0.50, rate(vccdb_latency_bucket_microseconds[5m]))

# P95 Latenz (95. Perzentil)
histogram_quantile(0.95, rate(vccdb_latency_bucket_microseconds[5m]))

# P99 Latenz (99. Perzentil)
histogram_quantile(0.99, rate(vccdb_latency_bucket_microseconds[5m]))

# Durchschnittliche Latenz
rate(vccdb_latency_sum_microseconds[5m]) / rate(vccdb_latency_count[5m])

# Prozentsatz Requests unter 1ms (1000 µs)
100 * sum(rate(vccdb_latency_bucket_microseconds{le="1000"}[5m])) / sum(rate(vccdb_latency_count[5m]))

# Latenz-Heatmap (Grafana Heatmap Panel)
sum by (le) (rate(vccdb_latency_bucket_microseconds[5m]))

```

Performance-Interpretation: - **P50 < 500µs:** Exzellent (In-Memory-Operationen) - **P95 < 5ms:** Gut (Block Cache Hit Rate > 95%) - **P99 < 50ms:** Akzeptabel (Gelegentliche Disk I/O) - **P99 > 100ms:** Schlecht (Hohe Disk-Latenz oder Full Scans)

vccdb_page_fetch_time_ms_bucket (Histogram)

Latenz für Cursor-Pagination-Fetches (GET /cursor/{cursor_id}).

Beispiel-Output:

```
vccdb_page_fetch_time_ms_bucket{le="1"} 89
vccdb_page_fetch_time_ms_bucket{le="5"} 156
vccdb_page_fetch_time_ms_bucket{le="10"} 234
vccdb_page_fetch_time_ms_bucket{le="25"} 245
vccdb_page_fetch_time_ms_bucket{le="50"} 248
vccdb_page_fetch_time_ms_bucket{le="+Inf"} 250
vccdb_page_fetch_time_ms_sum 2345.67
vccdb_page_fetch_time_ms_count 250
```

PromQL-Query-Beispiele:

```
# P95 Page-Fetch-Latenz
histogram_quantile(0.95, rate(vccdb_page_fetch_time_ms_bucket[5m]))

# Prozentsatz Page-Fetches unter 10ms
100 * sum(rate(vccdb_page_fetch_time_ms_bucket{le="10"}[5m])) / sum(rate(vccdb_page_fetch_time_ms_bucket[5m]))
```

Diese Metrik ist speziell für Pagination-Performance relevant. Hohe P99-Werte (> 100ms) deuten auf ineffiziente Cursor-Implementierungen oder Cold Cache-Probleme hin.

19.2.4 RocksDB-Metriken

RocksDB-Metriken werden aus der `GetProperty()` API ausgelesen und als Prometheus-Gauges exportiert. Diese Metriken erlauben die Überwachung der Storage-Layer-Gesundheit.

rocksdb_block_cache_usage_bytes (Gauge)

Aktuell verwendeter Block-Cache in Bytes.

Beispiel-Output:


```
rocksdb_block_cache_usage_bytes 1073741824
```

PromQL-Query-Beispiele:

```
# Cache-Auslastung in %  
100 * rocksdb_block_cache_usage_bytes / rocksdb_block_cache_capacity_bytes  
  
# Alert: Cache-Auslastung > 95%  
100 * rocksdb_block_cache_usage_bytes / rocksdb_block_cache_capacity_bytes > 95
```

rocksdb_block_cache_capacity_bytes (Gauge)

Konfigurierte Block-Cache-Kapazität in Bytes (aus `config.json`).

Beispiel-Output:

```
rocksdb_block_cache_capacity_bytes 2147483648
```

Typische Werte: - **Development**: 512 MB - 1 GB - **Staging**: 2-4 GB - **Production**: 8-16 GB (25% des verfügbaren RAM)

rocksdb_estimate_num_keys (Gauge)

Geschätzte Anzahl Keys in der Datenbank (basierend auf SST-Metadaten).

Beispiel-Output:

```
rocksdb_estimate_num_keys 1234567
```

Hinweis: Dies ist eine **Schätzung**, nicht exakt. Für exakte Counts verwenden Sie `SELECT COUNT(*)` Queries.

rocksdb_pending_compaction_bytes (Gauge)

Bytes, die auf Compaction warten. Hohe Werte (> 10 GB) können zu erhöhter Latenz führen.

Beispiel-Output:

```
rocksdb_pending_compaction_bytes 1234567890
```

PromQL-Query-Beispiele:

```
# Alert: Compaction-Backlog > 10 GB
rocksdb_pending_compaction_bytes > 10000000000

# Compaction-Backlog-Trend
deriv(rocksdb_pending_compaction_bytes[10m])
```

Ursachen für hohen Backlog: - Zu langsame Disk (SATA statt NVMe) - Zu kleine `max_background_compactions` (Default: 4) - Sehr hohe Write-Rate (> 100K writes/sec)

Lösungen: - Erhöhe `max_background_compactions` auf 8-16 - Nutze NVMe für WAL und SSTables
- Aktiviere Level Compaction (statt Universal)

rocksdb_memtable_size_bytes (Gauge)

Aktuelle Memtable-Größe in Bytes.

Beispiel-Output:

```
rocksdb_memtable_size_bytes 67108864
```

Die Memtable wird geflusht, wenn sie die konfigurierte `write_buffer_size` erreicht (Default: 64 MB). Mehrere Memtables können gleichzeitig aktiv sein (`max_write_buffer_number`: Default 2).

rocksdb_files_per_level (Gauge)

Anzahl SST-Dateien pro LSM-Tree-Level.

Labels: - `level`: LSM-Tree-Level (0, 1, 2, 3, 4, 5, 6)

Beispiel-Output:

```

rocksdb_files_per_level{level="0"} 3
rocksdb_files_per_level{level="1"} 12
rocksdb_files_per_level{level="2"} 45
rocksdb_files_per_level{level="3"} 123
rocksdb_files_per_level{level="4"} 234
rocksdb_files_per_level{level="5"} 456
rocksdb_files_per_level{level="6"} 789

```

PromQL-Query-Beispiele:

```

# Gesamtzahl SST-Dateien
sum(rocksdb_files_per_level)

# Alert: Zu viele L0-Dateien (> 10 → Write-Stall-Risiko)
rocksdb_files_per_level{level="0"} > 10

```

LSM-Tree-Levels: - **L0:** Frisch geflushed Memtables, unsortiert, können sich überlappen - **L1-L6:** Sortierte Runs, keine Überlappungen innerhalb eines Levels - **L6:** Coldest data (90% der Daten), ZSTD-komprimiert (2.8x Ratio)

19.2.5 Index-Metriken

vccdb_index_rebuild_total (Counter)

Gesamtzahl Index-Rebuild-Operationen.

Labels: - `table`: Tabellenname - `column`: Spaltenname

Beispiel-Output:

```

vccdb_index_rebuild_total{table="users",column="email"} 3
vccdb_index_rebuild_total{table="orders",column="customer_id"} 5

```

Index-Rebuilds werden getriggert durch: - `CREATE INDEX` Statement (Neuer Index) - `ANALYZE` Statement (Index-Statistik-Refresh) - Server-Neustart mit geändertem Schema

vccdb_index_rebuild_duration_seconds (Counter)

Gesamt-Zeit für Index-Rebuilds in Sekunden.

Labels: - `table`: Tabellennamenname - `column`: Spaltenname

Beispiel-Output:

```
vccdb_index_rebuild_duration_seconds{table="users",column="email"} 123.45
```

PromQL-Query-Beispiele:

```
# Durchschnittliche Rebuild-Dauer pro Index
vccdb_index_rebuild_duration_seconds / vccdb_index_rebuild_total

# Langsamste Index-Rebuilds
topk(5, vccdb_index_rebuild_duration_seconds / vccdb_index_rebuild_total)
```

Performance-Benchmark: - **B-Tree-Index:** ~50K entities/sec - **Hash-Index:** ~100K entities/sec - **Vector-Index (HNSW):** ~5K entities/sec (mit Construction-Parameter M=16, efConstruction=200)

themis_index_cursor_anchor_hits_total (Counter)

Anzahl erfolgreicher Cursor-Anchor-Lookups für Pagination.

Beispiel-Output:

```
themis_index_cursor_anchor_hits_total 567
```

Diese Metrik trackt, wie oft Cursors erfolgreich fortgesetzt wurden. Eine niedrige Hit-Rate deutet auf abgelaufene Cursors (TTL) oder ungültige Cursor-IDs hin.

themis_index_range_scan_steps_total (Counter)

Gesamtzahl Range-Scan-Schritte (Index-Traversierungen).

Beispiel-Output:

```
themis_index_range_scan_steps_total 12345
```

PromQL-Query-Beispiele:

```
# Range-Scan-Schritte pro Sekunde
rate(themis_index_range_scan_steps_total[5m])

# Durchschnittliche Schritte pro Query (kombiniert mit vccdb_requests_total)
rate(themis_index_range_scan_steps_total[5m]) / rate(vccdb_requests_total{route="/query"}[5m])
```

Hohe Range-Scan-Zahlen (> 1M steps/sec) deuten auf ineffiziente Queries mit breiten Ranges hin (z.B.

`age > 18` ohne zusätzliche Prädikate).

19.2.6 Vector-Index-Metriken

vccdb_vector_index_size_bytes (Gauge)

Geschätzte Größe des In-Memory-Vector-Index in Bytes.

Beispiel-Output:

```
vccdb_vector_index_size_bytes 536870912
```

Berechnung:

```
Index Size ≈ N * (D * 4 bytes + M * 2 bytes)
```

Für 1M Vektoren, Dimension 768, M=16:

```
Index Size = 1M * (768*4 + 16*2) = 3.1 GB
```

vccdb_vector_search_duration_ms (Histogram)

Vector-Similarity-Search-Latenz in Millisekunden.

Beispiel-Output:

```
vccdb_vector_search_duration_ms_bucket{le="1"} 45
vccdb_vector_search_duration_ms_bucket{le="5"} 123
vccdb_vector_search_duration_ms_bucket{le="10"} 234
vccdb_vector_search_duration_ms_bucket{le="+Inf"} 250
```

PromQL-Query-Beispiele:

```
# P95 Vector-Search-Latenz
histogram_quantile(0.95, rate(vccdb_vector_search_duration_ms_bucket[5m]))
```

Performance-Benchmark (1M Vektoren, 768-dim, M=16, efSearch=64): - **P50:** 1.2 ms - **P95:** 3.5 ms - **P99:** 8.2 ms

vccdb_vector_batch_insert_duration_ms (Histogram)

Batch-Insert-Latenz für Vektor-Batches.

vccdb_vector_batch_insert_items_total (Counter)

Gesamtzahl eingefügter Vector-Items.

PromQL-Query-Beispiele:

```
# Durchschnittliche Batch-Größe
rate(vccdb_vector_batch_insert_items_total[5m]) / rate(vccdb_vector_batch_insert_total[5m])
```

Typische Batch-Größen: - **Klein:** 10-100 (Echtzeit-Ingestion) - **Mittel:** 100-1000 (Bulk-Import) - **Groß:** 1000-10000 (Initial Load)

19.3 Grafana Dashboards

ThemisDB liefert **3 vorgefertigte Grafana-Dashboards** für unterschiedliche Monitoring-Szenarien.

19.3.1 Dashboard 1: Server-Übersicht

Zweck: High-Level-Übersicht über Server-Gesundheit und Performance.

Panels:

1. **QPS (Queries per Second)** `promql vccdb_qps`
 2. Visualization: Graph
 3. Y-Axis: Queries/sec
 4. Thresholds: Warning > 5000, Critical > 10000
 5. **Error Rate** `promql 100 * rate(vccdb_errors_total[5m]) / rate(vccdb_requests_total[5m])`
 6. Visualization: Graph
 7. Y-Axis: Percent
 8. Thresholds: Warning > 1%, Critical > 5%
 9. SLI-Target: < 0.1% (99.9% Success Rate)
 10. **Request Latency (P50, P95, P99)** `promql histogram_quantile(0.50, rate(vccdb_latency_bucket_microseconds[5m])) / 1000 # ms histogram_quantile(0.95, rate(vccdb_latency_bucket_microseconds[5m])) / 1000 histogram_quantile(0.99, rate(vccdb_latency_bucket_microseconds[5m])) / 1000`
 11. Visualization: Graph
 12. Y-Axis: Milliseconds
 13. SLI-Targets: P50 < 1ms, P95 < 5ms, P99 < 50ms
 14. **Uptime** `promql process_uptime_seconds / 86400`
 15. Visualization: Stat
 16. Unit: Days
 17. Color: Green > 7d, Yellow > 1d, Red < 1d
 18. **Requests by Route (Top 5)** `promql topk(5, sum by (route) (rate(vccdb_requests_total[5m])))`
 19. Visualization: Bar Gauge
 20. Y-Axis: Requests/sec
-

19.3.2 Dashboard 2: RocksDB Health

Zweck: Monitoring der Storage-Layer-Performance und Capacity Planning.

Panels:

1. **Block Cache Hit Rate** `promql 100 * rocksdb_block_cache_usage_bytes / rocksdb_block_cache_capacity_bytes`
2. Visualization: Gauge
3. Unit: Percent
4. Thresholds: Green > 80%, Yellow > 60%, Red < 60%
5. **Compaction Backlog** `promql rocksdb_pending_compaction_bytes / 1e9 # GB`
6. Visualization: Graph
7. Y-Axis: Gigabytes
8. Threshold: Critical > 10 GB
9. **Keys Estimate** `promql rocksdb_estimate_num_keys`
10. Visualization: Stat
11. Unit: Short (1.23M)
12. Trend: Show sparkline
13. **SST Files per Level** `promql sum by (level) (rocksdb_files_per_level)`
14. Visualization: Bar Gauge (Horizontal)
15. X-Axis: Number of Files
16. Y-Axis: Level (0-6)
17. **Memtable Size** `promql rocksdb_memtable_size_bytes / 1e6 # MB`
18. Visualization: Graph
19. Y-Axis: Megabytes
20. Threshold: Warning > 128 MB (2x write_buffer_size)
21. **LSM-Tree Amplification** `promql sum(rocksdb_files_per_level{level!="0"}) / rocksdb_files_per_level{level="0"}`
22. Visualization: Stat

23. Unit: None

24. Interpretation: Higher = Better compaction efficiency

19.3.3 Dashboard 3: Vector Operations

Zweck: Monitoring von Vector-Search-Performance und Index-Größe.

Panels:

1. **Vector Index Size** `promql vccdb_vector_index_size_bytes / 1e9 # GB`

2. Visualization: Gauge

3. Unit: Gigabytes

4. Thresholds: Warning > 8 GB, Critical > 16 GB (Memory Pressure)

5. **Search Latency P95** `promql histogram_quantile(0.95, rate(vccdb_vector_search_duration_ms_bucket[5m]))`

6. Visualization: Graph

7. Y-Axis: Milliseconds

8. Target: < 5ms

9. **Batch Insert Rate** `promql rate(vccdb_vector_batch_insert_items_total[5m])`

10. Visualization: Graph

11. Y-Axis: Items/sec

12. Benchmark: > 5000 items/sec (Production)

13. **Average Batch Size** `promql rate(vccdb_vector_batch_insert_items_total[5m]) / rate(vccdb_vector_batch_insert_total[5m])`

14. Visualization: Stat

15. Unit: Items

16. Optimal: 100-1000 (Trade-off: Latenz vs Throughput)

17. **Delete Rate** `promql rate(vccdb_vector_delete_by_filter_items_total[5m])`

18. Visualization: Graph

19. Y-Axis: Items/sec

19.4 OpenTelemetry Distributed Tracing

ThemisDB implementiert **Distributed Tracing** via OpenTelemetry für Production-Debugging und Performance-Analyse komplexer Multi-Component-Queries.

19.4.1 Architektur

Komponenten: 1. **Tracer Wrapper** (`utils/tracing.h` / `.cpp`) - Initialisierung des OTLP HTTP Exporters - TracerProvider mit Resource Attributes (service.name, version) - SimpleSpanProcessor für sofortigen Export

1. **Span RAII Wrapper**
2. `Tracer::Span`: Move-only Span mit automatischem `end()` im Destruktor
3. `ScopedSpan`: Convenience-Wrapper für lokale Spans
4. Attribute-Support: `setAttribute(key, value)` für string, int64, double, bool
5. **No-Op Fallback**
6. Wenn `THEMIS_ENABLE_TRACING` nicht definiert, sind alle Methoden No-Ops
7. Kein Linking gegen OpenTelemetry-Libraries notwendig

Datenfluss:

```

HTTP Request → ScopedSpan("handleAqlQuery")
               ↓
    QueryEngine::executeQuery() → ScopedSpan("executeQuery")
               ↓
    Index Scans → Child Spans
               ↓
OTLP HTTP Exporter → Jaeger/Tempo
  
```

19.4.2 Instrumentierte Komponenten

HTTP-Handler (Top-Level Spans)

- GET /entities/:key: `entity.key`, `entity.size_bytes`

- **PUT /entities/:key:** `entity.key`, `entity.table`, `entity.pk`, `entity.size_bytes`, `entity.cdc_recorded`
- **DELETE /entities/:key:** `entity.key`, `entity.table`, `entity.pk`, `entity.cdc_recorded`
- **POST /query:** `query.table`, `query.predicates_count`, `query.exec_mode`, `query.result_count`
- **POST /query/aql:** `aql.query`, `aql.explain`, `aql.optimize`, `aql.allow_full_scan`, `aql.result_count`
- **POST /graph/traverse:** `graph.start_vertex`, `graph.max_depth`, `graph.visited_count`
- **POST /vector/search:** `vector.dimension`, `vector.k`, `vector.results_count`

Beispiel-Trace:

```
[Span: handleAqlQuery, Duration: 23.5ms]
├─ [Span: aql.parse, Duration: 0.8ms]
│   └─ Attributes: aql.query_length=156
├─ [Span: aql.translate, Duration: 1.2ms]
└─ [Span: aql.for, Duration: 21.2ms]
    ├─ Attributes: for.table="users", for.predicates_count=2, for.result_count=123
    ├─ [Span: index.scanEqual, Duration: 8.5ms]
    │   └─ Attributes: index.table="users", index.column="email", index.result_count=1
    └─ [Span: index.scanRange, Duration: 12.1ms]
        └─ Attributes: index.table="users", index.column="age", index.result_count=122
```

AQL Execution Pipeline (Operator-Level Spans)

- **aql.parse:** `aql.query_length` - Parsen der AQL-Query in AST
- **aql.translate** - Übersetzung des AST in ConjunctiveQuery oder TraversalQuery
- **aql.for:** `for.table`, `for.predicates_count`, `for.range_predicates_count`, `for.order_by`, `for.order_desc`, `for.result_count`, `for.exec_mode`
- **aql.filter** - Filterung der Ergebnisse
- Bei Traversal: `filter.evaluations_total`, `filter.short_circuits`
- **aql.limit:** `limit.offset`, `limit.count`, `limit.input_count`, `limit.output_count`
- **aql.collect:** `collect.input_count`, `collect.group_by_count`, `collect.aggregates_count`, `collect.group_count`
- **aql.return:** `return.input_count`

- **aql.traversal:** `traversal.start_vertex`, `traversal.min_depth`, `traversal.max_depth`, `traversal.direction`, `traversal.result_count`
 - Child-Span: **aql.traversal.bfs:** `traversal.max_frontier_size_limit`, `traversal.max_results_limit`, `traversal.visited_count`, `traversal.edges_expanded`, `traversal.filter_evaluations`
-

Index-Scans (Child-Spans)

- **index.scanEqual:** `index.table`, `index.column`, `index.result_count`
 - **index.scanRange:** `index.table`, `index.column`, `index.result_count`, `range.has_lower`, `range.has_upper`, `range.includeLower`, `range.includeUpper`
 - **or.disjunct.execute:** `disjunct.eq_count`, `disjunct.range_count`, `disjunct.result_count`
-

19.4.3 Jaeger Integration (Development)

Jaeger via Docker starten:

```
docker run -d --name jaeger \
  -p 4318:4318 \
  -p 16686:16686 \
  jaegertracing/all-in-one:latest
```

Ports: - `4318`: OTLP HTTP receiver (für Themis) - `16686`: Jaeger UI

Themis konfigurieren (`config/config.json`):

```
{
  "tracing": {
    "enabled": true,
    "service_name": "themis-dev",
    "otlp_endpoint": "http://localhost:4318"
  }
}
```

Themis starten:

```
./build/Release/themis_server.exe
```

Traces anzeigen:

1. Öffne <http://localhost:16686> (Jaeger UI)
2. Wähle Service "themis-dev"
3. Klicke "Find Traces"

Trace-Analyse-Szenarien:

1. **Langsame Query debuggen:**
 2. Filtere nach `duration > 100ms`
 3. Identifiziere langsamste Spans (rot markiert)
 4. Prüfe `index.scanRange` Spans auf breite Ranges
 5. **Full Scan Detection:**
 6. Suche nach Spans mit `query.exec_mode=full_scan`
 7. Prüfe `fullscan.scanned` Attribut (sollte << 1M sein)
 8. **Index Efficiency:**
 9. Vergleiche `index.result_count` zwischen Scans
 10. Hohe Counts + niedrige Query-Results = ineffizienter Index
-

19.4.4 Grafana Tempo Integration (Production)

Tempo via Docker Compose:

```
version: '3'
services:
  tempo:
    image: grafana/tempo:latest
    command: ["-config.file=/etc/tempo.yaml"]
    volumes:
      - ./tempo.yaml:/etc/tempo.yaml
      - ./tempo-data:/tmp/tempo
    ports:
      - "4318:4318"    # OTLP HTTP
      - "3200:3200"    # Tempo API

  grafana:
    image: grafana/grafana:latest
    ports:
      - "3000:3000"
    environment:
      - GF_AUTH_ANONYMOUS_ENABLED=true
      - GF_AUTH_ANONYMOUS_ORG_ROLE=Admin
    volumes:
      - ./grafana-datasources.yaml:/etc/grafana/provisioning/datasources/datasources.yaml
```

tempo.yaml:

```
server:
  http_listen_port: 3200

distributor:
  receivers:
    otlp:
      protocols:
        http:
          endpoint: 0.0.0.0:4318

storage:
  trace:
    backend: local
    local:
      path: /tmp/tempo/traces
    wal:
      path: /tmp/tempo/wal
    block:
      retention: 720h # 30 days
```

grafana-datasources.yaml:

```
apiVersion: 1
datasources:
- name: Tempo
  type: tempo
  access: proxy
  url: http://tempo:3200
  uid: tempo
  editable: false
```

Trace-Query in Grafana:

1. Öffne `http://localhost:3000`
2. Explore → Tempo Datasource
3. Search Query: `{service.name="themis-server"} && duration > 100ms`
4. Trace-View zeigt Waterfall-Diagramm aller Spans

19.4.5 Performance-Hinweise

Span-Overhead: - **Mit Tracing aktiviert:** ~5-10 µs pro Span (inkl. Attribut-Serialisierung) - **Ohne Tracing** (`THEMIS_ENABLE_TRACING=OFF`): 0 µs (inline no-ops)

Best Practices:

1. **Granularität:** Erstelle Spans für HTTP-Requests, Query-Execution, Index-Scans
 2. **Attribute:** Füge relevante Metadaten hinzu (table, index_type, row_count)
 3. **Sampling** (zukünftig): Für High-Throughput-Szenarien Sampling verwenden
 4. **Batch Processor** (zukünftig): SimpleSpanProcessor → BatchSpanProcessor für bessere Performance
-

19.5 Alerting

19.5.1 Alert-Definitionen (Prometheus Alertmanager)

alerts.yaml:


```
groups:
- name: themis_server
  interval: 30s
  rules:
    # High Error Rate
    - alert: HighErrorRate
      expr: rate(vccdb_errors_total[5m]) / rate(vccdb_requests_total[5m]) > 0.05
      for: 5m
      labels:
        severity: critical
      annotations:
        summary: "ThemisDB Error Rate above 5%"
        description: "{{ $value | humanizePercentage }}" error rate in last 5 minutes"

    # High Latency P99
    - alert: HighLatencyP99
      expr: histogram_quantile(0.99, rate(vccdb_latency_bucket_microseconds[5m])) > 100000
      for: 5m
      labels:
        severity: warning
      annotations:
        summary: "ThemisDB P99 latency above 100ms"
        description: "P99 latency: {{ $value | humanizeDuration }}"

    # Compaction Backlog
    - alert: CompactionBacklog
      expr: rocksdb_pending_compaction_bytes > 10000000000
      for: 10m
      labels:
        severity: warning
      annotations:
        summary: "RocksDB Compaction backlog > 10 GB"
        description: "{{ $value | humanize1024 }}"B pending compaction"

    # Low Cache Hit Rate
    - alert: LowCacheHitRate
      expr: 100 * rocksdb_block_cache_usage_bytes / rocksdb_block_cache_capacity_bytes < 20
      for: 15m
```

```
labels:
  severity: info
annotations:
  summary: "Block cache usage < 20%"
  description: "Cache may be undersized or cold"

# Too Many L0 Files (Write Stall Risk)
- alert: TooManyL0Files
  expr: rocksdb_files_per_level{level="0"} > 10
  for: 5m
  labels:
    severity: critical
  annotations:
    summary: "RocksDB L0 files > 10 (Write Stall Risk)"
    description: "{{ $value }}" L0 files, compaction cannot keep up"

# Server Down
- alert: ThemisServerDown
  expr: up{job="themis"} == 0
  for: 1m
  labels:
    severity: critical
  annotations:
    summary: "ThemisDB Server is down"
    description: "Prometheus cannot scrape /metrics endpoint"
```

19.5.2 Alertmanager-Konfiguration

alertmanager.yaml:

```
global:
  resolve_timeout: 5m

route:
  group_by: ['alertname', 'severity']
  group_wait: 10s
  group_interval: 5m
  repeat_interval: 3h
  receiver: 'email'

routes:
  - match:
      severity: critical
    receiver: 'pagerduty'
    continue: true

  - match:
      severity: warning
    receiver: 'slack'

receivers:
  - name: 'email'
    email_configs:
      - to: 'ops-team@example.com'
        from: 'alerts@example.com'
        smarthost: 'smtp.example.com:587'
        auth_username: 'alerts@example.com'
        auth_password: 'password'

  - name: 'slack'
    slack_configs:
      - api_url: 'https://hooks.slack.com/services/T000000000/B000000000/XXXXXXXXXXXXX'
        channel: '#themis-alerts'
        title: '{{ .GroupLabels.alertname }}'
        text: '{{ range .Alerts }}{{ .Annotations.summary }}\n{{ end }}'

  - name: 'pagerduty'
```

```
pagerduty_configs:
  - service_key: 'YOUR_PAGERDUTY_SERVICE_KEY'
```

19.6 Production Deployment

19.6.1 Prometheus Scrape-Konfiguration

prometheus.yaml:

```
global:
  scrape_interval: 15s
  scrape_timeout: 10s
  evaluation_interval: 30s

scrape_configs:
  - job_name: 'themis'
    static_configs:
      - targets: ['localhost:8765']

  - job_name: 'themis-cluster'
    consul_sd_configs:
      - server: 'consul:8500'
        services: ['themis']
    relabel_configs:
      - source_labels: [__meta_consul_service]
        target_label: job
      - source_labels: [__meta_consul_node]
        target_label: node
```

19.6.2 Retention & Storage

Empfohlene Retention: - **Hochfrequente Metriken:** 15 Tage - **Long-term:** Downsampling auf 1h-Auflösung nach 7 Tagen (via Thanos)

Prometheus Storage-Konfiguration:

```
storage:
  tsdb:
    path: /prometheus
    retention.time: 15d
    retention.size: 100GB
```

Thanos Integration (Multi-Node Federation + Long-Term Storage):

```
# Thanos Sidecar (läuft neben jedem Prometheus)
thanos sidecar \
  --tsdb.path /prometheus \
  --prometheus.url http://localhost:9090 \
  --objstore.config-file /etc/thanos/objstore.yaml

# Thanos Query (Global Query Frontend)
thanos query \
  --http-address 0.0.0.0:19192 \
  --store prometheus-0-sidecar:10901 \
  --store prometheus-1-sidecar:10901 \
  --store thanos-store:10901

# Thanos Store (S3 Object Storage)
thanos store \
  --data-dir /thanos/store \
  --objstore.config-file /etc/thanos/objstore.yaml
```

objstore.yaml (S3 Backend):

```
type: S3
config:
  bucket: "themis-metrics"
  endpoint: "s3.eu-central-1.amazonaws.com"
  region: "eu-central-1"
  access_key: "YOUR_ACCESS_KEY"
  secret_key: "YOUR_SECRET_KEY"
```

19.6.3 Cardinality Management

Niedrige Cardinality (< 100 Zeitreihen): - Metriken ohne Labels (z.B. `vccdb_qps`, `process_uptime_seconds`)

Hohe Cardinality (potenziell Tausende Zeitreihen): - Index-Metriken mit `table` + `column` Labels -
Wenn 100 Tables mit je 20 Columns → 2000 Index-Rebuild-Metriken

Best Practices: - Verwende **niemals** High-Cardinality-Labels wie User-IDs, Timestamps, UUIDs - Nutze Route-Templates (z.B. `/entities/{key}`) statt konkreter Pfade - Für User-spezifisches Tracking:
Verwende separate Logs/Traces, nicht Prometheus

19.7 Zusammenfassung

ThemisDB bietet ein **Production-Ready Observability-Stack** mit:

- **48 Prometheus-Metriken** für Server, RocksDB, Indexes, Vectors
- **Kumulative Histogramme** für Latenz-Tracking (P50/P95/P99)
- **3 Grafana-Dashboards** für Server-Health, Storage-Health, Vector-Operations
- **OpenTelemetry Distributed Tracing** mit Jaeger/Tempo Integration
- **Alerting** via Prometheus Alertmanager mit PagerDuty/Slack/Email
- **Long-Term Storage** via Thanos mit S3-Backend

Performance-Overhead: - **Metrics:** < 1% CPU (Counter-Inkrementen sind atomic operations) - **Tracing (aktiviert):** ~5-10 µs pro Span - **Tracing (deaktiviert):** 0 µs (compile-time no-ops)

Nächste Schritte: 1. Implementiere Custom-Dashboards für projektspezifische Metriken 2. Konfiguriere Alertmanager-Routing nach Schweregrad 3. Enable Sampling für Tracing in High-Throughput-Szenarien (> 10K QPS) 4. Setze SLI/SLO-Tracking auf (Error Budget Calculations)

Siehe auch: - [Chapter 8: Storage Layer Deep-Dive](#) - RocksDB Tuning - [Chapter 11: Realtime Data Streaming](#) - CDC Monitoring - [Appendix C: API Reference](#) - /metrics Endpoint - [Prometheus Documentation](#) - [OpenTelemetry Documentation](#) - [Grafana Documentation](#)

Kapitel 21: Performance Tuning

Einführung

Performance-Optimierung ist entscheidend für produktive ThemisDB-Deployments. Dieses Kapitel behandelt systematische Ansätze zur Identifizierung und Behebung von Performance-Problemen.

20.1 Performance-Analyse

20.1.1 Query-Performance-Messung

EXPLAIN ANALYZE:

```
EXPLAIN ANALYZE
FOR order IN orders
  FILTER order.order_date > '2023-01-01'
FOR customer IN customers
  FILTER order.customer_id == customer.id
  SORT order.total_amount DESC
LIMIT 100
RETURN {order, customer_name: customer.name}
```

Ausgabe-Interpretation:

```
Query Plan:
└─ Sort (cost=1250.45, rows=1000, time=125ms)
  │   └─ Hash Join (cost=850.23, rows=5000, time=85ms)
  │     │   └─ Seq Scan on orders (cost=0.00, rows=10000, time=45ms)
  │     │     │   Filter: order_date > '2023-01-01'
  │     └─ Hash (cost=250.00, rows=5000, time=25ms)
  │       └─ Index Scan on customers (cost=0.00, rows=5000, time=15ms)
```


20.1.2 Python Profiling

Abfrage-Performance tracken:

```
import time
from functools import wraps

def profile_query(func):
    """Decorator für Query-Profiling"""
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        duration = time.time() - start

        # Slow Query Log
        if duration > 1.0: # > 1 Sekunde
            print(f"SLOW QUERY: {func.__name__} took {duration:.2f}s")
            print(f"Args: {args}, Kwargs: {kwargs}")

        return result
    return wrapper

@profile_query
def get_customer_orders(customer_id):
    return client.query("""
        FOR order IN orders
        FILTER order.customer_id == @customer_id
        RETURN order
    """, {"customer_id": customer_id})
```

20.1.3 System-Ressourcen-Monitoring

CPU und Memory:

```
import psutil

def monitor_resources():
    """Überwacht System-Ressourcen"""
    process = psutil.Process()

    return {
        'cpu_percent': process.cpu_percent(interval=1),
        'memory_mb': process.memory_info().rss / 1024 / 1024,
        'open_files': len(process.open_files()),
        'threads': process.num_threads()
    }
```

20.2 Index-Optimierung

20.2.1 Index-Typen verstehen

B-Tree Index (Standard):

```
-- Für Equality und Range Queries
CREATE INDEX idx_order_date ON orders(order_date);

-- Composite Index
CREATE INDEX idx_customer_date ON orders(customer_id, order_date);
```

Hash Index:

```
-- Nur für Equality Queries
CREATE INDEX idx_customer_hash ON customers USING HASH(email);
```

Vector Index (HNSW):

```
-- Für Similarity Search
CREATE VECTOR INDEX idx_product_embedding
ON products(embedding)
WITH (metric='cosine', m=16, ef_construction=200);
```

20.2.2 Index-Strategie

Effective Index-Design:

```
# Schlechter Index (zu viele Columns)
CREATE INDEX bad_idx ON orders(customer_id, order_date, status, total_amount)

# Besserer Index (nur notwendige Columns)
CREATE INDEX good_idx ON orders(customer_id, order_date)

# Covering Index (enthält alle benötigten Felder)
CREATE INDEX covering_idx ON orders(customer_id, order_date)
INCLUDE (status, total_amount)
```

Index-Nutzung überprüfen:

```
def analyze_index_usage(table_name):
    """Analysiert Index-Nutzung"""
    stats = client.admin.index_stats(table_name)

    for idx in stats['indexes']:
        usage_rate = idx['scans'] / max(idx['age_seconds'], 1)

        if usage_rate < 0.01: # Weniger als 1 Scan pro 100 Sekunden
            print(f"Unused index: {idx['name']}")
            print(f"  Size: {idx['size_mb']} MB")
            print(f"  Recommendation: Consider dropping")
```

20.2.3 Index-Wartung

Rebuild und Reorganize:

```
def maintain_indexes(table_name):  
    """Führt Index-Wartung durch"""  
    stats = client.admin.index_fragmentation(table_name)  
  
    for idx in stats['indexes']:  
        frag_pct = idx['fragmentation_percent']  
  
        if frag_pct > 30:  
            print(f"Rebuilding index: {idx['name']} ({frag_pct}% fragmented)")  
            client.admin.rebuild_index(table_name, idx['name'])  
        elif frag_pct > 10:  
            print(f"Reorganizing index: {idx['name']}")  
            client.admin.reorganize_index(table_name, idx['name'])
```

20.3 Query-Optimierung

20.3.1 Query-Rewriting

Subquery vs JOIN:

```
-- Langsam: Correlated Subquery
FOR customer IN customers
  LET order_count = (
    FOR order IN orders
      FILTER order.customer_id == customer.id
    RETURN 1
  )
  RETURN {name: customer.name, order_count: LENGTH(order_count)}

-- Schneller: Multi-FOR mit COLLECT
FOR customer IN customers
  FOR order IN orders
    FILTER order.customer_id == customer.id
    COLLECT c_id = customer.id, c_name = customer.name
  AGGREGATE order_count = COUNT()
  RETURN {name: c_name, order_count}
```

IN vs EXISTS:

```
-- Langsam für große Datasets
LET completed_customers = (
  FOR order IN orders
    FILTER order.status == 'completed'
    RETURN DISTINCT order.customer_id
)
FOR customer IN customers
  FILTER customer.id IN completed_customers
  RETURN customer

-- Schneller: Direkte Filterung
FOR customer IN customers
  LET has_completed = (
    FOR order IN orders
      FILTER order.customer_id == customer.id AND order.status == 'completed'
      LIMIT 1
      RETURN 1
    )
  FILTER LENGTH(has_completed) > 0
  RETURN customer
```

20.3.2 Batch Operations

Bulk Insert:

```
# Langsam: Einzelne Inserts
for record in records:
    client.collection('orders').insert_one(record)

# Schneller: Batch Insert
client.collection('orders').insert_many(records, batch_size=1000)
```

Bulk Update:

```
# Mit Batch-Processing
def bulk_update(updates, batch_size=1000):
    """Bulk Update mit optimaler Batch-Größe"""
    for i in range(0, len(updates), batch_size):
        batch = updates[i:i+batch_size]

        # Transaction für Batch
        with client.begin_transaction() as txn:
            for update in batch:
                txn.update('orders', update['id'], update['data'])
            txn.commit()
```

20.3.3 Query-Caching

Application-Level Cache:

```
from functools import lru_cache
import hashlib
import json

class QueryCache:
    def __init__(self, max_size=1000, ttl=300):
        self.cache = {}
        self.max_size = max_size
        self.ttl = ttl

    def _cache_key(self, query, params):
        """Generiert Cache-Key"""
        data = json.dumps({'query': query, 'params': params}, sort_keys=True)
        return hashlib.md5(data.encode()).hexdigest()

    def get(self, query, params):
        """Holt gecachtes Ergebnis"""
        key = self._cache_key(query, params)

        if key in self.cache:
            entry = self.cache[key]
            if time.time() - entry['timestamp'] < self.ttl:
                return entry['result']
            else:
                del self.cache[key]

        return None

    def set(self, query, params, result):
        """Cached Ergebnis"""
        if len(self.cache) >= self.max_size:
            # LRU eviction
            oldest_key = min(self.cache.keys(),
                             key=lambda k: self.cache[k]['timestamp'])
            del self.cache[oldest_key]

        key = self._cache_key(query, params)
        self.cache[key] = {
```



```
        'result': result,
        'timestamp': time.time()
    }

def query(self, query_str, params=None):
    """Query mit Caching"""
    # Check cache
    cached = self.get(query_str, params)
    if cached is not None:
        return cached

    # Execute query
    result = client.query(query_str, params)

    # Cache result
    self.set(query_str, params, result)

    return result

# Verwendung
cache = QueryCache(max_size=1000, ttl=300)
result = cache.query("""
    FOR product IN products
        FILTER product.category == @category
    RETURN product
""", {"category": 'electronics'})
```

20.4 Connection Pooling

20.4.1 Pool-Konfiguration

Optimale Pool-Size:

```
from themisdb import ConnectionPool

# Connection Pool konfigurieren
pool = ConnectionPool(
    host='localhost',
    port=8529,
    min_connections=10,      # Minimum Connections
    max_connections=100,    # Maximum Connections
    max_idle_time=300,      # 5 Minuten Idle-Timeout
    connection_timeout=10,  # 10 Sekunden Connect-Timeout
    validate_on_checkout=True # Validierung bei Checkout
)

# Pool-Statistiken
def print_pool_stats():
    stats = pool.get_stats()
    print(f"Active: {stats['active']}")
    print(f"Idle: {stats['idle']}")
    print(f"Waiting: {stats['waiting']}")
    print(f"Pool utilization: {stats['active'] / stats['max'] * 100:.1f}%")
```

20.4.2 Connection-Leaks vermeiden

Context Manager:

```
class SafeConnection:
    def __init__(self, pool):
        self.pool = pool
        self.conn = None

    def __enter__(self):
        self.conn = self.pool.get_connection()
        return self.conn

    def __exit__(self, exc_type, exc_val, exc_tb):
        if self.conn:
            self.pool.release_connection(self.conn)
        return False

# Verwendung
with SafeConnection(pool) as conn:
    result = conn.query("""
        FOR order IN orders
        RETURN order
    """)
```

20.5 RocksDB-Tuning

20.5.1 LSM-Tree Konfiguration

Write Buffer Size:

```
# themisdb.yaml
storage:
  rocksdb:
    write_buffer_size: 128MB      # Größerer Buffer = weniger Flushes
    max_write_buffer_number: 4    # Parallele Buffers
    min_write_buffer_to_merge: 2 # Merge-Schwelle
```

Block Cache:

```
storage:
  rocksdb:
    block_cache_size: 8GB          # LRU Cache für Blocks
    cache_index_and_filter: true   # Indexes im Cache
    pin_top_level_index: true     # Top-Level Index immer im Cache
```

20.5.2 Compaction-Tuning

Level-Style Compaction:

```
storage:
  rocksdb:
    compaction_style: level
    level0_file_num_trigger: 4     # Trigger bei 4 L0 Files
    level0_slowdown_writes: 20    # Slowdown bei 20 L0 Files
    level0_stop_writes: 36        # Stop bei 36 L0 Files
    max_background_compactions: 8  # Parallele Compactions
    max_background_flushes: 4     # Parallele Flushes
```

Universal Compaction (für Write-Heavy):

```
storage:
  rocksdb:
    compaction_style: universal
    max_size_amplification_percent: 200
    compression_size_multiplier: 2
```

20.5.3 Bloom Filters

Aktivierung:

```
storage:
  rocksdb:
    bloom_filter_bits_per_key: 10  # 10 Bits = ~1% False Positive Rate
    block_based_filter: false      # Full Filter = bessere Performance
```

20.6 Memory-Management

20.6.1 Memory-Profiling

Memory-Leaks finden:

```
import tracemalloc

def profile_memory():
    """Tracked Memory-Allocation"""
    tracemalloc.start()

    # Code ausführen
    result = expensive_operation()

    # Memory Snapshot
    snapshot = tracemalloc.take_snapshot()
    top_stats = snapshot.statistics('lineno')

    print("Top 10 memory allocations:")
    for stat in top_stats[:10]:
        print(f"{stat.filename}:{stat.lineno}: {stat.size / 1024:.1f} KB")

    tracemalloc.stop()
```

20.6.2 Object Pooling

Connection und Result Pooling:

```
from queue import Queue

class ObjectPool:
    def __init__(self, factory, max_size=100):
        self.factory = factory
        self.pool = Queue(maxsize=max_size)
        self._initialize_pool(max_size // 2)

    def _initialize_pool(self, size):
        for _ in range(size):
            self.pool.put(self.factory())

    def acquire(self):
        if self.pool.empty():
            return self.factory()
        return self.pool.get()

    def release(self, obj):
        if not self.pool.full():
            self.pool.put(obj)

# Verwendung für Result Sets
result_pool = ObjectPool(lambda: [], max_size=100)
```

20.7 Netzwerk-Optimierung

20.7.1 Batch Requests

Request Batching:

```
class BatchedClient:
    def __init__(self, client, batch_size=10, flush_interval=0.1):
        self.client = client
        self.batch_size = batch_size
        self.flush_interval = flush_interval
        self.batch = []
        self.last_flush = time.time()

    def query(self, query_str, params=None):
        """Fügt Query zum Batch hinzu"""
        self.batch.append({'query': query_str, 'params': params})

        # Auto-Flush bei Batch-Size oder Timeout
        if len(self.batch) >= self.batch_size or \
            time.time() - self.last_flush > self.flush_interval:
            return self.flush()

    def flush(self):
        """Sendet gesammelten Batch"""
        if not self.batch:
            return []

        results = self.client.batch_query(self.batch)
        self.batch = []
        self.last_flush = time.time()

        return results
```

20.7.2 Kompression

Aktivierung:

```
# themisdb.yaml
network:
  compression:
    enabled: true
    algorithm: zstd      # zstd, gzip, lz4
    level: 3            # 1-22 für zstd
    min_size: 1KB       # Nur bei Größe > 1KB komprimieren
```

20.8 Monitoring und Alerting

20.8.1 Performance-Metriken

Key Metrics:


```
from prometheus_client import Histogram, Counter, Gauge

# Query Performance
query_duration = Histogram(
    'themisdb_query_duration_seconds',
    'Query execution time',
    buckets=[0.01, 0.05, 0.1, 0.5, 1.0, 2.0, 5.0]
)

# Slow Queries
slow_queries = Counter('themisdb_slow_queries_total', 'Slow queries')

# Cache Hit Rate
cache_hits = Counter('themisdb_cache_hits_total', 'Cache hits')
cache_misses = Counter('themisdb_cache_misses_total', 'Cache misses')
cache_hit_rate = Gauge('themisdb_cache_hit_rate', 'Cache hit rate')

@query_duration.time()
def execute_query(query, params):
    start = time.time()
    result = client.query(query, params)
    duration = time.time() - start

    if duration > 1.0:
        slow_queries.inc()

    return result
```

20.8.2 Alert Rules

Prometheus Alerts:

```
groups:
- name: themisdb_performance
  rules:
    # Hohe Query-Latenz
    - alert: HighQueryLatency
      expr: histogram_quantile(0.95, themisdb_query_duration_seconds) > 1
      for: 5m
      annotations:
        summary: "95th percentile query latency > 1s"

    # Niedrige Cache Hit Rate
    - alert: LowCacheHitRate
      expr: themisdb_cache_hit_rate < 0.7
      for: 10m
      annotations:
        summary: "Cache hit rate below 70%"

    # Viele Slow Queries
    - alert: ManySlowQueries
      expr: rate(themisdb_slow_queries_total[5m]) > 10
      for: 5m
      annotations:
        summary: "More than 10 slow queries per minute"
```

20.9 Load Testing

20.9.1 Benchmark-Suite

Basis-Benchmark:

```

import concurrent.futures
import time

class PerformanceBenchmark:
    def __init__(self, client):
        self.client = client

    def benchmark_read(self, num_queries=1000, concurrency=10):
        """Read-Performance testen"""
        def single_query():
            start = time.time()
            self.client.query("""
                FOR product IN products
                LIMIT 100
                RETURN product
            """)
            return time.time() - start

        with concurrent.futures.ThreadPoolExecutor(max_workers=concurrency) as executor:
            futures = [executor.submit(single_query) for _ in range(num_queries)]
            durations = [f.result() for f in concurrent.futures.as_completed(futures)]

        return {
            'avg': sum(durations) / len(durations),
            'p50': sorted(durations)[len(durations)//2],
            'p95': sorted(durations)[int(len(durations)*0.95)],
            'p99': sorted(durations)[int(len(durations)*0.99)],
            'qps': num_queries / sum(durations)
        }

    def benchmark_write(self, num_writes=1000, concurrency=10):
        """Write-Performance testen"""
        def single_write():
            start = time.time()
            self.client.collection('test').insert_one({
                'data': 'test',
                'timestamp': time.time()
            })

```

```
        return time.time() - start

    with concurrent.futures.ThreadPoolExecutor(max_workers=concurrency) as executor:
        futures = [executor.submit(single_write) for _ in range(num_writes)]
        durations = [f.result() for f in concurrent.futures.as_completed(futures)]

    return {
        'avg': sum(durations) / len(durations),
        'p95': sorted(durations)[int(len(durations)*0.95)],
        'wps': num_writes / sum(durations)
    }

# Verwendung
benchmark = PerformanceBenchmark(client)
read_results = benchmark.benchmark_read(num_queries=10000, concurrency=50)
print(f"Read QPS: {read_results['qps']:.0f}")
print(f"P95 Latency: {read_results['p95']*1000:.1f}ms")
```

20.9.2 Stress Testing

Load Generator:

```
def stress_test(duration_seconds=60, target_qps=1000):  
    """Stress-Test mit Ziel-QPS"""  
    import random  
  
    start_time = time.time()  
    query_count = 0  
    interval = 1.0 / target_qps  
  
    queries = [  
        "FOR p IN products FILTER p.category == @cat RETURN p",  
        "FOR o IN orders FILTER o.status == @status COLLECT AGGREGATE c = COUNT() RETURN c",  
        "FOR c IN customers FILTER c.email == @email RETURN c"  
    ]  
  
    while time.time() - start_time < duration_seconds:  
        query_start = time.time()  
  
        # Random Query ausführen  
        query = random.choice(queries)  
        client.query(query, [random.choice(['active', 'pending', 'test@example.com'])])  
        query_count += 1  
  
        # Rate Limiting  
        elapsed = time.time() - query_start  
        if elapsed < interval:  
            time.sleep(interval - elapsed)  
  
    actual_qps = query_count / duration_seconds  
    print(f"Achieved QPS: {actual_qps:.0f} (target: {target_qps})")
```

20.10 Best Practices Checkliste

20.10.1 Query-Optimierung

- [] Indexes für häufige WHERE-Clauses

- ☐ Composite Indexes für Multi-Column Filters
- ☐ EXPLAIN ANALYZE für langsame Queries
- ☐ Batch Operations statt Einzelqueries
- ☐ Query-Caching für häufige Abfragen
- ☐ Prepared Statements verwenden

20.10.2 Index-Management

- ☐ Regelmäßige Index-Wartung
- ☐ Ungenutzte Indexes entfernen
- ☐ Index-Fragmentation überwachen
- ☐ Covering Indexes für häufige Queries
- ☐ Index-Größe überwachen

20.10.3 Connection-Management

- ☐ Connection Pooling aktiviert
- ☐ Pool-Size optimal konfiguriert
- ☐ Connection-Leaks vermeiden
- ☐ Timeout-Werte angemessen
- ☐ Connection-Validierung aktiviert

20.10.4 RocksDB-Tuning

- ☐ Write Buffer Size angepasst
- ☐ Block Cache konfiguriert
- ☐ Compaction-Parameter optimiert
- ☐ Bloom Filter aktiviert
- ☐ Compression konfiguriert

20.10.5 Monitoring

- ☐ Performance-Metriken erfasst
- ☐ Slow Query Log aktiviert
- ☐ Alert Rules konfiguriert
- ☐ Regelmäßige Benchmarks
- ☐ Capacity Planning durchgeführt

20.11 Zusammenfassung

Kernpunkte: - Systematische Performance-Analyse ist essentiell - Indexes sind der wichtigste Optimierungshebel - Query-Rewriting kann dramatische Verbesserungen bringen - Connection Pooling ist kritisch für Skalierbarkeit - RocksDB-Tuning beeinflusst grundlegende Performance - Monitoring ermöglicht proaktive Optimierung - Load Testing validiert Performance-Verbesserungen

Performance-Ziele: - Query Latency P95 < 100ms - Cache Hit Rate > 80% - Connection Pool Utilization < 80% - QPS: 10,000+ für Read-Heavy Workloads - Write Throughput: 5,000+ WPS

Kapitel 24: KI-Ethik und Governance

Autor: ThemisDB Team

Reviewer: TBD

Status: Draft

Letzte Aktualisierung: 30. Dezember 2025

Version: 1.0.0

Lernziele

Nach dem Durcharbeiten dieses Kapitels sollten Sie:

- [x] Die ethischen Herausforderungen von KI-Systemen in der öffentlichen Verwaltung verstehen
 - [x] Spezifische Risiken entlang der VCC-Architektur (Covina, Clara, Veritas) identifizieren können
 - [x] "Ethik-by-Design"-Prinzipien auf datenbankgestützte KI-Systeme anwenden können
 - [x] Governance-Mechanismen zur Risikominimierung implementieren können
-

Voraussetzungen

Dieses Kapitel setzt Kenntnisse aus folgenden Kapiteln voraus:

- **Kapitel 10:** Enterprise Applications - Security & Compliance Grundlagen
 - **Kapitel 17:** LLM Integration - RAG-Architektur und Pre-Filtering
-

Überblick

Die Einführung eines KI-Ökosystems wie **VCC (Veritas, Covina, Clara)** in der öffentlichen Verwaltung wirft tiefgreifende ethische Fragen auf, die über die reine Rechtskonformität (DSGVO, EU AI Act)

hinausgehen [9]. Der Kern des ethischen Spannungsfelds liegt im Auftrag der Verwaltung: Sie muss nicht nur "richtig" im Sinne von effizient und faktisch korrekt handeln, sondern auch "gerecht" im Sinne von fair, unvoreingenommen und der Rechtsstaatlichkeit verpflichtet sein.

In diesem Kapitel behandeln wir: 1. Ethische Herausforderungen in verwaltungs-KI 2. Architektonische Risikoquellen (Covina, Clara, Veritas) 3. Bias-Audits und Datenqualität 4. Human-in-the-Loop und Automation Bias 5. Governance-Framework und Ethik-by-Design 6. Praktische Implementierung in ThemisDB

24.1 Der Ethische Imperativ für Verwaltungs-KI

24.1.1 Unterschied zu kommerzieller KI

Während kommerzielle KI-Systeme primär auf Effizienz und Profitabilität optimiert sind, tragen Verwaltungs-KI-Systeme eine fundamental andere Verantwortung [9]:

Kommerzielle KI: - Ziel: Geschäftserfolg, Kundenzufriedenheit - Fehlertoleranz: Wirtschaftlicher Schaden, Reputation - Kontrolle: Marktmechanismen, Wettbewerb

Verwaltungs-KI: - Ziel: Rechtsstaatlichkeit, Gerechtigkeit, Gemeinwohl - Fehlertoleranz: Grundrechtsverletzungen, Vertrauensverlust in Staat - Kontrolle: Demokratische Legitimation, Rechtsaufsicht

***Wichtig:** Ein Fehler in einem Verwaltungsakt (z.B. falsche Ablehnung einer BImSchG-Genehmigung) kann existenzielle Folgen für Bürger oder Unternehmen haben. Die Entscheidung muss nicht nur "wahrscheinlich richtig", sondern nachweisbar korrekt und ethisch vertretbar sein.*

24.1.2 Digitale Souveränität und ethische Verantwortung

Die Entscheidung für einen On-Premise-Betrieb (wie bei ThemisDB) sichert zwar die Datensouveränität, verlagert aber die ethische Verantwortung für den gesamten Datenlebenszyklus vollständig in den Hoheitsbereich der Verwaltung [9].

Konsequenzen: - Keine Ausrede "Der Cloud-Provider war schuld" - Volle Kontrolle = Volle Verantwortung - Notwendigkeit interner Expertise und Governance

24.2 Architektonische Risikoquellen im VCC-Ökosystem

Die Analyse identifiziert drei zentrale Risiken entlang der VCC-Architektur [9]:

24.2.1 Covina (Ingestion-Engine): Das Tor zur Voreingenommenheit

Funktion: Covina verarbeitet und indiziert Verwaltungsdokumente in die Wissensbasis.

Ethisches Risiko: Verfestigung von Vorurteilen durch historische Dokumente [9].

Das Prinzip "Garbage In, Garbage Out" wird hier zu "**Garbage In, Gospel Out**" [9]. Wenn die von Covina verarbeiteten Dokumente (z.B. alte Verwaltungsvorschriften, Gerichtsurteile) historische oder systemische Vorurteile enthalten, wird das KI-System diese Vorurteile nicht nur reproduzieren, sondern sie mit der Autorität einer scheinbar objektiven Maschine verstärken.

Konkretes Beispiel:

```
# Problematischer Fall: Historische Voreingenommenheit
dokument = {
  "titel": "Bearbeitungsrichtlinie Bauanträge 1985",
  "inhalt": "Bei Anträgen von Antragstellern mit nicht-deutschen Namen
            ist besondere Sorgfalt bei der Prüfung geboten..."
}

# Covina indiziert dies in RAG-Datenbank
# Clara lernt: nicht-deutsche Namen = höheres Risiko = strengere Prüfung
# Veritas gibt diese "gelernte" Regel an Sachbearbeiter weiter
# ⚠️ Resultat: Systematische Benachteiligung wird automatisiert
```

ThemisDB-spezifische Gefahr: - Atomare ACID-Transaktionen garantieren *technische* Konsistenz - Sie garantieren NICHT *inhaltliche* oder *ethische* Korrektheit - Ein fehlerhaftes Dokument wird konsistent über alle Index-Layer repliziert

Mitigationsstrategien:

1. **Proaktive Bias-Audits vor Ingestion** [9]:

```
def audit_document_for_bias(doc: Document) -> BiasReport:
    """
    Scant Dokument auf potenzielle Vorurteile vor Covina-Ingestion.
    """
    bias_patterns = [
        r"Antragsteller.*nicht-deutsch.*besondere Sorgfalt",
        r"Geschlecht.*erhöhtes Risiko",
        r"Alter.*über \d+.*automatisch ablehnen"
    ]

    detected_biases = []
    for pattern in bias_patterns:
        if re.search(pattern, doc.content, re.IGNORECASE):
            detected_biases.append({
                "pattern": pattern,
                "location": "...", # Kontext
                "severity": "HIGH"
            })

    return BiasReport(
        document_id=doc.id,
        timestamp=datetime.now(),
        biases_found=detected_biases,
        recommendation="BLOCK" if detected_biases else "APPROVE"
    )
```

1. Dokumenten-Provenienz-Tracking:

```
-- ThemisDB: Metadaten für ethische Nachverfolgbarkeit
INSERT INTO documents (id, content, metadata) VALUES (
  'doc_123',
  @content,
  {
    "source": "Archiv Brandenburg",
    "original_date": "1985-03-15",
    "bias_audit_status": "REVIEWED",
    "bias_audit_date": "2025-12-01",
    "bias_findings": ["HISTORICAL_DISCRIMINATION"],
    "usage_restriction": "CONTEXT_ONLY_NO_TRAINING"
  }
);
```

1. **Temporale Contextualisierung:**
2. Markierung historischer Dokumente mit Zeitstempel
3. Abwertung alter Dokumente im Ranking
4. Explizite Warnung bei Verwendung

24.2.2 Clara (Modellverbesserungs-Kreislauf): Permanente Intelligenz-Korruption

Funktion: Clara verbessert KI-Modelle durch Nutzerfeedback (LoRA-Adapter, Fine-Tuning).

Ethisches Risiko: Kaskadierende Integritätskompromittierung durch fehlerhaftes Feedback [9].

Szenario "Gelernte Lüge":

1. **Initiale Fehlinformation:** Sachbearbeiter A gibt falsches Feedback: `json { "query": "Abstandsregeln BImSchG für Windkraftanlagen", "answer": "Mindestabstand 2000m zu Wohngebieten", "feedback": "CORRECT", "user": "sachbearbeiter_a" }` (Tatsächlich: Regelung ist komplexer, pauschal falsch)
2. **Clara lernt:** LoRA-Adapter wird mit diesem Feedback trainiert
3. **Propagierung:** System gibt nun selbstbewusst falsche Auskunft
4. **Kaskadierende Verstärkung:** Weitere Nutzer vertrauen der KI, geben ebenfalls "CORRECT"-Feedback

5. **Permanente Kodierung:** Fehler wird in Model-Weights "eingebrannt"

Architektonisches Problem:

```
// Clara's Feedback-Loop (vereinfacht)
void Clara::ProcessFeedback(const UserFeedback& feedback) {
    if (feedback.rating == "CORRECT") {
        // ⚠ KEINE VALIDIERUNG ob Feedback faktisch korrekt ist
        lora_adapter_.UpdateWeights(feedback.query, feedback.answer);

        // Atomic Transaction garantiert nur ACID, nicht Wahrheit
        db_>BeginTransaction();
        db_>StoreFeedback(feedback);
        db_>UpdateModelVersion(lora_adapter_.version());
        db_>Commit(); // Fehler ist nun "consistent" gespeichert
    }
}
```

Zusätzliches Risiko: Echokammer-Effekt [9] - Nur Mehrheitsmeinungen trainieren das System - Minderheitenpositionen werden unterdrückt - Pluralismus der Rechtsmeinungen geht verloren

Mitigationsstrategien:

1. **Experten-Review vor Model-Update** [9]:

```
class FeedbackValidator:
    def __init__(self, expert_threshold: int = 2):
        self.expert_threshold = expert_threshold
        self.pending_feedback = {}

    def validate_feedback(self, feedback: Feedback) -> ValidationResult:
        """
        Feedback wird erst nach Expert-Review in Model übernommen.
        """
        key = (feedback.query_hash, feedback.answer_hash)

        if key not in self.pending_feedback:
            self.pending_feedback[key] = {
                "feedback": feedback,
                "expert_approvals": 0,
                "expert_rejections": 0
            }
            return ValidationResult.PENDING

        # Experten-Konsens erforderlich
        if self.pending_feedback[key]["expert_approvals"] >= self.expert_threshold:
            return ValidationResult.APPROVED

        return ValidationResult.PENDING
```

1. Stichprobenbasierte ethische Audits [9]:

```
-- ThemisDB: Query für ethisch problematische Feedback-Patterns
SELECT
  f.query,
  f.answer,
  COUNT(*) as feedback_count,
  AVG(CASE WHEN f.rating = 'CORRECT' THEN 1 ELSE 0 END) as approval_rate,
  STRING_AGG(f.user_id, ',') as users
FROM feedback f
WHERE f.timestamp > NOW() - INTERVAL '7 days'
GROUP BY f.query, f.answer
HAVING approval_rate > 0.9 AND feedback_count < 5
-- ⚠ Verdächtig: Hohe Zustimmung bei wenig Feedback = Echokammer?
ORDER BY approval_rate DESC;
```

1. **Niemals vollständig autonomes Lernen** [9]:
 2. Clara-Updates nur in Staging-Umgebung
 3. A/B-Testing gegen Baseline-Modell
 4. Human-in-the-Loop bei kritischen Rechtsgebieten
-

24.2.3 Veritas (Benutzerinteraktion): Erosion der Urteilskraft

Funktion: Veritas ist die Chat-Schnittstelle für Sachbearbeiter.

Ethisches Risiko: Automation Bias und Verantwortungsdiffusion [9].

Automation Bias: Die menschliche Tendenz, Ergebnissen automatisierter Systeme übermäßig und unkritisch zu vertrauen [9].

Konkretes Szenario:

Sachbearbeiter: "Genehmigungsfähigkeit BImSchG-Antrag
Windkraftanlage Havelland?"

Veritas (KI): "✅ Genehmigungsfähig.
Gemäß §5 BImSchG erfüllt der Antrag alle Voraussetzungen.

📖 Quellen:

- BImSchG §5 Abs. 1 [Gesetz]
- VwV Havelland 2024 [Verwaltungsvorschrift]
- Gutachten Lärmschutz [Dokument #4521]"

Sachbearbeiter: "Klingt gut, übernehme ich so."
[KLICKT "GENEHMIGEN" OHNE WEITERE PRÜFUNG]

Warum problematisch? - KI wirkt durch Quellen autoritativ - Sachbearbeiter delegiert kritisches Denken an Maschine - Bei Fehler: "Die KI hat das so gesagt" → Verantwortungsdiffusion

ThemisDB kann das Problem NICHT lösen:

```
// ThemisDB garantiert nur, dass Veritas konsistente Daten liefert
auto answer = veritas_->Query("BImSchG Windkraft");

// ✅ ACID garantiert: Alle Quellen in answer sind transaktional korrekt
// ❌ NICHT garantiert: Die INTERPRETATION ist rechtlich korrekt
// ❌ NICHT garantiert: Sachbearbeiter hinterfragt das Ergebnis
```

Mitigationsstrategien:

1. Explizite Unsicherheits-Kennzeichnung:

```
class VeritasResponse:
    def __init__(self, answer: str, confidence: float, sources: List[Source]):
        self.answer = answer
        self.confidence = confidence # 0.0 - 1.0
        self.sources = sources

    def to_ui(self) -> str:
        """
        Zeigt Unsicherheit transparent an.
        """
        confidence_label = {
            (0.9, 1.0): "🟢 SEHR SICHER",
            (0.7, 0.9): "🟡 MITTEL SICHER",
            (0.0, 0.7): "🔴 UNSICHER - EXPERTENKONSULTATION EMPFOHLEN"
        }

        for (low, high), label in confidence_label.items():
            if low <= self.confidence < high:
                return f"""
                {label} (Konfidenz: {self.confidence:.2%})

                {self.answer}

                ⚠️ HINWEIS: Dies ist eine KI-generierte Empfehlung.
                Bitte prüfen Sie die Quellen kritisch und konsultieren Sie
                bei Unsicherheit einen Rechtsexperten.
                """
```

1. Mandatory Second-Opinion bei kritischen Entscheidungen:

```
CRITICAL_DECISION_TYPES = [  
    "GENEHMIGUNG_ERTEILEN",  
    "GENEHMIGUNG_ABLEHNEN",  
    "WIDERSPRUCH_ZURÜCKWEISEN"  
]  
  
def require_second_opinion(decision_type: str,  
                           ai_recommendation: VeritasResponse) -> bool:  
    """  
    Erzwingt menschliche Zweitprüfung bei kritischen Entscheidungen.  
    """  
    if decision_type in CRITICAL_DECISION_TYPES:  
        if ai_recommendation.confidence < 0.95:  
            return True # Mandatory review  
  
    return False
```

1. **AI Literacy Training** [9]:
2. Schulungsprogramm für alle Sachbearbeiter
3. Erkennung von Automation Bias
4. Kritisches Hinterfragen von KI-Ausgaben
5. Verständnis für KI-Limitationen

Checkliste für Sachbearbeiter:

🧠 Kritische KI-Nutzung - Checkliste

Bevor Sie eine KI-Empfehlung übernehmen:

- ☐ Habe ich die angegebenen Quellen SELBST geprüft?
- ☐ Widersprechen Quellen einander? (KI erkennt das oft nicht)
- ☐ Ist die Rechtslage eindeutig oder gibt es Interpretationsspielraum?
- ☐ Würde ich diese Entscheidung auch OHNE KI so treffen?
- ☐ Habe ich bei Unsicherheit einen Experten konsultiert?
- ☐ Ist die Begründung MEINE eigene oder nur KI-Paraphrase?

⚠ Bei NEIN zu einer Frage: STOPP und manuelle Prüfung!

24.3 Governance-Framework: Ethik-by-Design

Die abgeleiteten Handlungsempfehlungen sind konkrete technische und organisatorische Anforderungen [9]:

24.3.1 Interdisziplinäres KI-Ethik-Gremium

Zusammensetzung: - Juristen (Verwaltungsrecht, Datenschutz) - Informatiker (KI, Datenbanken) - Ethiker (Moralphilosophie) - Praktiker (Sachbearbeiter mit Felderfahrung) - Bürgervertreter (Zivilgesellschaft)

Aufgaben: 1. Kontinuierliche Ethik-Audits: `sql -- ThemisDB: Ethik-Audit-Log CREATE TABLE ethics_audits ( id UUID PRIMARY KEY, audit_date TIMESTAMP, component TEXT, -- 'covina', 'clara', 'veritas' findings TEXT[], risk_level TEXT, -- 'LOW', 'MEDIUM', 'HIGH', 'CRITICAL' action_required TEXT, resolved BOOLEAN DEFAULT FALSE );`

1. **Entscheidung über kritische Fälle:**
2. Eskalation bei Automation-Bias-Verdacht
3. Review strittiger Clara-Feedbacks
4. Freigabe neuer Datenquellen für Covina
5. **Policy-Entwicklung:**
6. Wann ist KI-Nutzung erlaubt/verboten?
7. Welche Entscheidungen bleiben immer human?
8. Transparenzanforderungen gegenüber Bürgern

24.3.2 Bias-Audits für Datenquellen

Proaktiver Ansatz [9]:

```
class CovinaBiasAuditor:
    def __init__(self, themis_db: ThemisDB):
        self.db = themis_db
        self.bias_patterns = self._load_bias_patterns()

    def audit_document_batch(self, documents: List[Document]) -> AuditReport:
        """
        Audited alle Dokumente vor Ingestion.
        """
        high_risk_docs = []

        for doc in documents:
            bias_score = self._calculate_bias_score(doc)

            if bias_score > 0.7: # High risk threshold
                high_risk_docs.append({
                    "doc_id": doc.id,
                    "bias_score": bias_score,
                    "detected_patterns": self._extract_patterns(doc),
                    "recommendation": "MANUAL_REVIEW"
                })

            # Markierung in ThemisDB
            self.db.execute("""
                UPDATE documents
                SET metadata = jsonb_set(
                    metadata,
                    '{bias_audit}',
                    '{"status": "FLAGGED", "score": :score} '::jsonb
                )
                WHERE id = :doc_id
            """, {"doc_id": doc.id, "score": bias_score})

        return AuditReport(
            total_documents=len(documents),
            flagged_documents=len(high_risk_docs),
            details=high_risk_docs
        )
```

Integration in Covina-Pipeline:

```
// covina/ingestion_pipeline.cpp
class IngestionPipeline {
public:
    void IngestDocument(const Document& doc) {
        // SCHRITT 1: Bias-Audit VOR Ingestion
        auto audit_result = bias_auditor_->Audit(doc);

        if (audit_result.risk_level == RiskLevel::HIGH) {
            // Blockieren und zur manuellen Review
            ethics_queue_->Enqueue(doc, audit_result);
            LogWarning("Document {} flagged for ethics review", doc.id);
            return; // 🛑 NICHT indizieren
        }

        // SCHRITT 2: Normale Ingestion (nur bei bestanden Audit)
        auto txn = db_->BeginTransaction();
        db_->InsertBaseEntity(doc.id, doc.content, doc.metadata);
        indexer_->IndexDocument(doc); // Relational, Graph, Vector
        txn->Commit();
    }
private:
    BiasAuditor* bias_auditor_;
    EthicsReviewQueue* ethics_queue_;
};
```

24.3.3 Human-in-the-Loop Policy

Principle: Kritische Entscheidungen IMMER mit menschlicher Kontrolle [9].

Klassifizierung von Entscheidungen:

Kategorie	Beispiel	KI-Rolle	Human-Rolle
Routine	Statusabfrage	✅ Vollautomatisch	📄 Info
Standard	Dokumentensuche	✅ Vorschlag	🔍 Plausibilitätsprüfung
Komplex	Genehmigungsempfehlung	💡 Assistenz	✅ Entscheidung
Kritisch	Ablehnungsbescheid	🚫 Nur Hintergrunddaten	✅ Volle Kontrolle

ThemisDB-Implementierung:

```
-- Audit-Trail für Human-in-the-Loop
CREATE TABLE decision_audit (
    id UUID PRIMARY KEY,
    timestamp TIMESTAMP,
    decision_type TEXT,
    ai_recommendation JSONB,
    human_decision JSONB,
    human_rationale TEXT, -- Warum wich Mensch von KI ab?
    override BOOLEAN, -- TRUE wenn Mensch KI überstimmt
    user_id TEXT
);

-- Statistik: Wie oft wird KI überstimmt?
SELECT
    decision_type,
    COUNT(*) as total_decisions,
    SUM(CASE WHEN override THEN 1 ELSE 0 END) as overrides,
    ROUND(100.0 * SUM(CASE WHEN override THEN 1 ELSE 0 END) / COUNT(*), 2) as override_rate_pct
FROM decision_audit
GROUP BY decision_type
ORDER BY override_rate_pct DESC;

-- ⚠️ Hohe Override-Rate = KI-Modell hat systematisches Problem
```

24.4 Praktische Implementierung in ThemisDB

24.4.1 Ethik-Metadaten-Schema

Erweiterte Base Entity mit Ethik-Tracking:


```
{
  "_id": "doc_BImSchG_2024_123",
  "_type": "administrative_document",
  "content": {
    "title": "Verwaltungsvorschrift Windkraft Havelland",
    "text": "...",
    "legal_basis": ["BImSchG §5", "TA Lärm"]
  },
  "ethics_metadata": {
    "bias_audit": {
      "status": "PASSED",
      "audited_by": "bias_auditor_v2.1",
      "audit_date": "2025-12-01T10:00:00Z",
      "bias_score": 0.15,
      "flagged_patterns": []
    },
    "provenance": {
      "source": "Ministerium für Infrastruktur",
      "original_date": "2024-06-15",
      "historical_context": "MODERN_REGULATION",
      "reliability_score": 0.95
    },
    "usage_restrictions": {
      "allow_training": true,
      "allow_direct_citation": true,
      "require_human_review": false,
      "sensitivity_level": "PUBLIC"
    },
    "ethical_flags": {
      "contains_personal_data": false,
      "contains_protected_groups": false,
      "potential_discrimination_risk": "LOW"
    }
  }
}
```

24.4.2 Ethik-Compliance-Checks in AQL

Query mit ethischen Constraints:

```
// Suche nur ethisch unbedenkliche Dokumente
FOR doc IN documents
  // Standard-Filter
  FILTER doc.content.legal_basis ANY == "BImSchG §5"

  //  ETHIK-FILTER
  FILTER doc.ethics_metadata.bias_audit.status == "PASSED"
  FILTER doc.ethics_metadata.bias_audit.bias_score < 0.3
  FILTER doc.ethics_metadata.usage_restrictions.allow_direct_citation == true

  // Falls historisches Dokument, nur mit Kontext
  FILTER doc.ethics_metadata.provenance.historical_context != "DISCRIMINATORY_ERA"
    OR doc.ethics_metadata.usage_restrictions.require_human_review == true

  LET similarity = COSINE(doc.embedding, @query_vector)
  FILTER similarity > 0.7

  // Rückgabe mit Ethik-Metadaten
  RETURN {
    doc: doc,
    similarity: similarity,
    ethics_status: doc.ethics_metadata.bias_audit.status,
    human_review_required: doc.ethics_metadata.usage_restrictions.require_human_review
  }
```

24.4.3 Automation-Bias-Warnsystem

Echtzeit-Warnung bei verdächtigem Nutzerverhalten:

```
// veritas/automation_bias_detector.cpp
class AutomationBiasDetector {
public:
    void MonitorUserBehavior(const UserSession& session) {
        // Pattern: User akzeptiert KI-Vorschlag ohne Quellenprüfung
        if (session.ai_recommendation_shown &&
            session.sources_viewed.empty() &&
            session.decision_time_seconds < 10) {

            // ⚠️ WARNUNG: Potentieller Automation Bias
            SendWarning(session.user_id,
                "Sie haben eine KI-Empfehlung ohne Quellenprüfung übernommen. "
                "Bitte validieren Sie die Quellen kritisch.");

            // Audit-Log
            db_ ->Execute(R"(
                INSERT INTO automation_bias_warnings (user_id, session_id, timestamp)
                VALUES (:user, :session, NOW())
            )", {"user", session.user_id}, {"session", session.session_id});

            // Bei Wiederholung: Mandatory Training
            if (GetWarningCount(session.user_id) > 3) {
                RequireAILiteracyTraining(session.user_id);
            }
        }
    }
};
```

24.5 Zusammenfassung und Best Practices

Wichtigste Erkenntnisse:

1.  **Ethik ist kein Add-on:** Ethische Überlegungen müssen von Anfang an in die Systemarchitektur integriert werden ("Ethik-by-Design") [9]

2.  **ThemisDB garantiert nur technische Konsistenz:** ACID-Transaktionen garantieren nicht inhaltliche oder ethische Korrektheit - das erfordert zusätzliche Governance-Layer
3.  **Drei architektonische Risiko-Hotspots:**
 4. Covina (Bias in Daten)
 5. Clara (Korruption durch fehlerhaftes Feedback)
 6. Veritas (Automation Bias bei Nutzern)
7.  **Human-in-the-Loop ist essentiell:** Kritische Entscheidungen dürfen niemals vollständig automatisiert werden [9]

Checkliste für ethische KI-Systeme:

- ☐ Interdisziplinäres Ethik-Gremium eingerichtet
 - ☐ Bias-Audits für alle Datenquellen implementiert
 - ☐ Provenienz-Tracking für Dokumente aktiviert
 - ☐ Automation-Bias-Warnsystem deployed
 - ☐ Human-in-the-Loop-Policy definiert und durchgesetzt
 - ☐ AI-Literacy-Training für alle Nutzer durchgeführt
 - ☐ Ethik-Metadaten in allen Base Entities
 - ☐ Clara-Feedback-Validierung mit Experten-Review
 - ☐ Transparente Unsicherheits-Kennzeichnung in Veritas
 - ☐ Regelmäßige Ethik-Audits (mindestens quarterly)
-

Weiterführende Ressourcen

Dokumentation

- [9]: Expertenanalyse: Ethische und moralische Implikationen des KI-Ökosystems VCC
- **Enterprise Security:** docs/security/RBAC_AUTHORIZATION.md
- **Compliance:** docs/compliance/DSGVO_BY_DESIGN.md

Akademische Referenzen

- [1] Barocas, S., Hardt, M., Narayanan, A. (2019). "Fairness and Machine Learning: Limitations and Opportunities". fairmlbook.org.
 - [2] O'Neil, C. (2016). "Weapons of Math Destruction: How Big Data Increases Inequality and Threatens Democracy". Crown Publishing.
 - [3] Floridi, L., Cowls, J. (2019). "A Unified Framework of Five Principles for AI in Society". Harvard Data Science Review, 1(1).
 - [4] Selbst, A., et al. (2019). "Fairness and Abstraction in Sociotechnical Systems". ACM FAT* Conference, 59-68.
 - [5] Mittelstadt, B., et al. (2016). "The ethics of algorithms: Mapping the debate". Big Data & Society, 3(2).
 - [6] EU AI Act (2024). "Regulation on Artificial Intelligence". European Parliament.
 - [7] Goddard, K., et al. (2012). "Automation Bias: A Systematic Review of Frequency, Effect Mediators, and Mitigators". Journal of the American Medical Informatics Association, 19(1), 121-127.
-

Glossar für dieses Kapitel

Begriff	Definition
Automation Bias	Tendenz, Ergebnissen automatisierter Systeme übermäßig zu vertrauen und menschliches Urteilsvermögen zu vernachlässigen
Bias-Audit	Systematische Prüfung von Daten oder Modellen auf Voreingenommenheit
Echokammer-Effekt	Verstärkung von Mehrheitsmeinungen durch selektives Feedback
Ethik-by-Design	Integration ethischer Überlegungen in die Systemarchitektur von Anfang an
Human-in-the-Loop	Ansatz, bei dem kritische Entscheidungen immer menschliche Kontrolle erfordern
Kaskadieren Integritätskompromittierung	Fehler, der sich durch Feedback-Loops permanent im System verankert
Provenienz	Herkunft und Entstehungsgeschichte von Daten
Verantwortungsdiffusion	Unklare Zuständigkeit bei automatisierten Entscheidungen ("Die KI war's")

Änderungshistorie

Version	Datum	Autor	Änderungen
1.0.0	2025-12-30	ThemisDB Team	Initiale Version basierend auf gimini Ethics-Analyse [9]

Schwierigkeitsgrad: Fortgeschritten

Geschätzte Lesezeit: 45 Minuten

Tags: ethics, ai-governance, bias, automation-bias, human-in-the-loop, verwaltungs-ki

Anhang A: Literaturverzeichnis

Dieses Literaturverzeichnis folgt dem IEEE-Zitierstil und umfasst alle wissenschaftlichen und technischen Quellen, die in diesem Compendium referenziert wurden.

Primärquellen: ThemisDB Gemini Analysen

Die folgenden strategischen Analysen wurden als Primärquellen für die technische Vertiefung in Kapitel 1, 2, 3 und 17 herangezogen:

[1] "Strategische Gesamtanalyse: ThemisDB als ACID-Fundament für die RAG-LLM-Strategie der deutschen Verwaltung – Eine komparative Analyse zur Ablösung der UDS3-Architektur," ThemisDB Project, Internes Strategiedokument, Nov. 2025. [Online]. Verfügbar: docs/gimini/Strategische Gesamtanalyse_ ThemisDB als ACID-Fundament für die RAG-LLM-Strategie der deutschen Verwaltung – Eine komparative Analyse zur Ablösung der UDS3-Architektur.md

[2] "Strategische Analyse: ThemisDB – Bewertung einer nativen Multi-Modell-Architektur im Kontext von Sovereign-Cloud-Plattformen und On-Premise-RAG-Alternativen," ThemisDB Project, Internes Analysedokument, Nov. 2025. [Online]. Verfügbar: docs/gimini/Strategische Analyse_ ThemisDB – Bewertung einer nativen Multi-Modell-Architektur im Kontext von Sovereign-Cloud-Plattformen und On-Premise-RAG-Alternativen.md

[3] "Die ThemisDB-Architektur: Eine technische Tiefenanalyse eines Multi-Modell-Datenbanksystems auf LSM-Tree-Basis," ThemisDB Project, Technisches Whitepaper, Nov. 2025. [Online]. Verfügbar: docs/gimini/ThemisDB Dokumentation und Berichtsanalyse.md

[4] "Architektonischer Entwurf eines hochperformanten Multi-Modell-Datenbanksystems: Eine Kernel-Level-Analyse von kanonischem Speicher, Projektionsschichten und Speicherhierarchien in C++ und Rust," ThemisDB Project, Architektur-Blueprint, Nov. 2025. [Online]. Verfügbar: docs/gimini/Hybride Datenbankarchitektur C++_Rust (1).md

[5] "ThemisDB vs. Hyperscaler Datenbanken: Komparativer Vergleich," ThemisDB Project, Vergleichsanalyse, Nov. 2025. [Online]. Verfügbar: docs/gimini/Themis vs. Hyperscaler Datenbanken Vergleich.md

[6] "Hyperscaler-Einordnung für ThemisDB-Bericht," ThemisDB Project, Marktanalyse, Nov. 2025. [Online]. Verfügbar: docs/gimini/Hyperscaler-Einordnung für ThemisDB-Bericht.md

[7] "KI-Strategie: ThemisDB, Hyperscaler, Ethik," ThemisDB Project, Strategiepapier, Nov. 2025.

[Online]. Verfügbar: docs/gimini/KI-Strategie_ ThemisDB, Hyperscaler, Ethik.md

[8] "VCCDB Design: Veritas, Covina, Clara Database Design Spezifikation," ThemisDB Project, Design-Dokument, Nov. 2025. [Online]. Verfügbar: docs/gimini/VCCDB Design (1).md

[9] "Forschungsbericht ThemisDB 2025: Public Research Edition," ThemisDB Project, Forschungsbericht, Nov. 2025. [Online]. Verfügbar: docs/gimini/Forschungsbericht ThemisDB 2025_THE-MISDB_PUBLIC_RE....md

Sekundärquellen: Datenbanksysteme und Architektur

[10] M. Stonebraker and U. Cetintemel, "'One size fits all': An idea whose time has come and gone," in *Proc. IEEE 21st International Conference on Data Engineering (ICDE '05)*, Tokyo, Japan, 2005, pp. 2-11, doi: 10.1109/ICDE.2005.1.

[11] J. Lu and I. Holubová, "Multi-model databases: A new journey to handle the variety of data," *ACM Computing Surveys*, vol. 52, no. 3, pp. 1-38, Jun. 2019, doi: 10.1145/3323214.

[12] P. O'Neil, E. Cheng, D. Gawlick, and E. O'Neil, "The log-structured merge-tree (LSM-tree)," *Acta Informatica*, vol. 33, no. 4, pp. 351-385, Jun. 1996, doi: 10.1007/s002360050048.

[13] Facebook Engineering, "RocksDB: A persistent key-value store for fast storage environments," Facebook Inc., Technical Report, 2013. [Online]. Verfügbar: <https://rocksdb.org/>

[14] D. G. Andersen, J. Franklin, M. Kaminsky, A. Phanishayee, L. Tan, and V. Vasudevan, "FAWN: A fast array of wimpy nodes," in *Proc. 22nd ACM Symposium on Operating Systems Principles (SOSP '09)*, Big Sky, MT, USA, 2009, pp. 1-14, doi: 10.1145/1629575.1629577.

[15] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil, "A critique of ANSI SQL isolation levels," in *Proc. ACM SIGMOD International Conference on Management of Data*, San Jose, CA, USA, 1995, pp. 1-10, doi: 10.1145/223784.223785.

Transaktionsverarbeitung und Konsistenz

- [16] P. A. Bernstein and E. Newcomer, *Principles of Transaction Processing*, 2nd ed. San Francisco, CA, USA: Morgan Kaufmann, 2009.
- [17] C. Garcia-Molina and K. Salem, "Sagas," in *Proc. ACM SIGMOD International Conference on Management of Data*, San Francisco, CA, USA, 1987, pp. 249-259, doi: 10.1145/38713.38742.
- [18] E. Brewer, "CAP twelve years later: How the 'rules' have changed," *IEEE Computer*, vol. 45, no. 2, pp. 23-29, Feb. 2012, doi: 10.1109/MC.2012.37.
- [19] D. B. Lomet, "Process structuring, synchronization, and recovery using atomic actions," in *Proc. ACM Conference on Language Design for Reliable Software*, Raleigh, NC, USA, 1977, pp. 128-137, doi: 10.1145/800022.808314.
- [20] M. J. Cahill, U. Röhm, and A. D. Fekete, "Serializable isolation for snapshot databases," *ACM Transactions on Database Systems*, vol. 34, no. 4, pp. 1-42, Dec. 2009, doi: 10.1145/1620585.1620587.
-

Graph-Datenbanken und Traversierung

- [21] R. Angles and C. Gutierrez, "Survey of graph database models," *ACM Computing Surveys*, vol. 40, no. 1, pp. 1-39, Feb. 2008, doi: 10.1145/1322432.1322433.
- [22] M. A. Rodriguez and P. Neubauer, "The graph traversal pattern," in *Graph Data Management: Techniques and Applications*, S. Sakr and E. Pardede, Eds. Hershey, PA, USA: IGI Global, 2011, pp. 29-46, doi: 10.4018/978-1-61350-053-8.ch002.
- [23] N. Francis et al., "Cypher: An evolving query language for property graphs," in *Proc. ACM SIGMOD International Conference on Management of Data*, Houston, TX, USA, 2018, pp. 1433-1445, doi: 10.1145/3183713.3190657.
- [24] R. Angles, "A comparison of current graph database models," in *Proc. IEEE 28th International Conference on Data Engineering Workshops (ICDEW)*, Arlington, VA, USA, 2012, pp. 171-177, doi: 10.1109/ICDEW.2012.31.
-

Vektor-Datenbanken und Similarity Search

[25] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824-836, Apr. 2020, doi: 10.1109/TPAMI.2018.2889473.

[26] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535-547, Sep. 2021, doi: 10.1109/TBDATA.2019.2921572.

[27] M. Muja and D. G. Lowe, "Scalable nearest neighbor algorithms for high dimensional data," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 36, no. 11, pp. 2227-2240, Nov. 2014, doi: 10.1109/TPAMI.2014.2321376.

[28] A. Babenko and V. Lempitsky, "Efficient indexing of billion-scale datasets of deep descriptors," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 2055-2063, doi: 10.1109/CVPR.2016.226.

RAG und LLM-Integration

[29] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459-9474.

[30] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333-389, 2009, doi: 10.1561/15000000019.

[31] O. Khattab and M. Zaharia, "ColBERT: Efficient and effective passage search via contextualized late interaction over BERT," in *Proc. 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, Virtual Event, 2020, pp. 39-48, doi: 10.1145/3397271.3401075.

[32] Y. Gao et al., "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, Dec. 2023. [Online]. Verfügbar: <https://arxiv.org/abs/2312.10997>

Polyglot Persistence und Multi-Model-Systeme

[33] M. Fowler and P. Sadalage, *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Upper Saddle River, NJ, USA: Addison-Wesley Professional, 2012.

[34] J. Pokorný, "Conceptual and database modelling of graph databases," in *Proc. 20th International Conference on Information Systems Development (ISD)*, Edinburgh, Scotland, 2011, pp. 370-381.

[35] ArangoDB Inc., "ArangoDB Multi-Model Database Architecture," Technical Whitepaper, 2020. [Online]. Verfügbar: <https://www.arangodb.com/>

[36] Microsoft Azure, "Azure Cosmos DB: Multi-model database service," Microsoft Technical Documentation, 2023. [Online]. Verfügbar: <https://azure.microsoft.com/en-us/services/cosmos-db/>

Kompression und Speicheroptimierung

[37] Y. Collet, "LZ4: Extremely fast compression algorithm," Open Source Implementation, 2011. [Online]. Verfügbar: <https://lz4.github.io/lz4/>

[38] Y. Collet and M. Kucherawy, "Zstandard Compression and the 'application/zstd' Media Type," Internet Engineering Task Force (IETF), RFC 8878, Feb. 2021. [Online]. Verfügbar: <https://tools.ietf.org/html/rfc8878>

[39] M. Adler, "Snappy: A fast compressor/decompressor," Google Inc., Technical Report, 2011. [Online]. Verfügbar: <https://google.github.io/snappy/>

Serialisierung und Parsing

[40] D. Lemire, G. Ssi-Yan-Kai, and O. Katz, "Parsing gigabytes of JSON per second," *The VLDB Journal*, vol. 28, no. 6, pp. 941-960, Dec. 2019, doi: 10.1007/s00778-019-00578-5.

[41] ArangoDB Inc., "VelocityPack: A fast and compact format for serialization and storage," Technical Specification, 2016. [Online]. Verfügbar: <https://github.com/arangodb/velocypack>

[42] Google Inc., "Protocol Buffers: Google's data interchange format," Technical Documentation, 2008. [Online]. Verfügbar: <https://developers.google.com/protocol-buffers>

Standards und Compliance

[43] Bundesamt für Sicherheit in der Informationstechnik (BSI), "IT-Grundschutz-Kompendium," BSI Standard 200-2, Bonn, Deutschland, 2023. [Online]. Verfügbar: <https://www.bsi.bund.de/>

[44] European Parliament and Council, "General Data Protection Regulation (GDPR)," Regulation (EU) 2016/679, Apr. 2016. [Online]. Verfügbar: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>

[45] Apache Software Foundation, "Apache Ranger: Framework to enable, monitor and manage comprehensive data security," Technical Documentation, 2023. [Online]. Verfügbar: <https://ranger.apache.org/>

Verwendete Softwarebibliotheken

[46] Facebook Engineering, "RocksDB C++ API Reference," Version 8.8.0, 2023. [Online]. Verfügbar: <https://github.com/facebook/rocksdb>

[47] The Rust Project, "serde: Serialization framework for Rust," Version 1.0, 2023. [Online]. Verfügbar: <https://serde.rs/>

[48] D. Lemire et al., "simdjson: Parsing gigabytes of JSON per second," Version 3.0, 2023. [Online]. Verfügbar: <https://github.com/simdjson/simdjson>

Verwendungshinweise

Kapitel-Referenzen

Die folgenden Primärquellen wurden in den entsprechenden Kapiteln verwendet:

Kapitel 1 (Einführung): - [1] Strategische Gesamtanalyse - Teil III: Architektonische Kernentscheidung - [3] ThemisDB-Architektur - Teil 1: Kanonische Speicherarchitektur - [4] Architektonischer Entwurf - Teil 1: Base Entity Paradigma

Kapitel 2 (Architektur): - [3] ThemisDB-Architektur - Teil 1.3: RocksDB als transaktionales Fundament - [4] Architektonischer Entwurf - Teil 1: Multi-Modell-Speicherformat - [5] Hyperscaler-Vergleich - Teil 3.1: Kanonischer Speicher - [12] LSM-Tree Grundlagen - [13] RocksDB Dokumentation

Kapitel 3 (Multi-Model): - [1] Strategische Gesamtanalyse - Teil III: Ablösung von UDS3 - [2] Strategische Analyse - Teil II: Wettbewerbslandschaft - [5] Hyperscaler-Vergleich - Teil 2: Post-Filtering Problem - [11] Multi-Model Databases Survey

Kapitel 17 (LLM-Integration): - [2] Strategische Analyse - Teil 1.3: Native KI- und RAG-Fähigkeiten - [5] Hyperscaler-Vergleich - Teil 2: RAG-Architektur - [29] Retrieval-Augmented Generation - [30] BM25 and Beyond

Zitierkonventionen im Text

Im Text dieses Compendiums werden Quellen durch eckige Klammern referenziert, z.B. [1], [3], [12]. Die vollständigen Quellenangaben finden Sie in diesem Anhang im IEEE-Format.

Zugriff auf Primärquellen

Die Primärquellen [1]-[9] sind interne Projektdokumente im [docs/gimini/](#) Verzeichnis des ThemisDB Repository verfügbar. Die Sekundärquellen [10]-[48] sind peer-reviewed wissenschaftliche Publikationen, technische Standards oder Open-Source-Dokumentationen, die über die angegebenen DOIs oder URLs zugänglich sind.

Zuletzt aktualisiert: Dezember 2025

Version: 1.3.4

Appendix D: Feature Status Matrix

Version: 1.3.0

Stand: 30. Dezember 2025

Kategorie: Feature-Übersicht und Implementierungsstatus

Zweck dieses Appendix

Dieser Appendix bietet eine vollständige, strukturierte Übersicht aller ThemisDB-Features mit aktuellem Implementierungsstatus, Dokumentationsverweisen und Roadmap-Planung. Dies dient als zentraler Referenzpunkt für:

- **Entwickler:** Welche Features sind verfügbar? Wo ist der Code?
 - **Architekten:** Welche Capabilities hat ThemisDB? Was fehlt noch?
 - **Projektmanager:** Was ist Production-Ready? Was ist geplant?
 - **Nutzer:** Welche Funktionen kann ich nutzen?
-

Status-Definitionen

Sym-bol	Status	Bedeutung
✓	Production-Ready	Vollständig implementiert, getestet (>85% Coverage), dokumentiert, performance-validated
🟢	MVP Complete	Kernfunktionalität vorhanden, stabil, aber ggf. noch Optimierungen ausstehend
🟡	In Development	Aktiv in Entwicklung, teilweise funktional, noch nicht production-grade
🟠	Design Phase	Spezifikation vorhanden, Implementierung geplant
🔵	Planned	Auf Roadmap, noch keine Spezifikation
⬜	Backlog	Idee/Anforderung dokumentiert, keine aktive Planung
✗	Not Planned	Bewusst nicht im Scope

Zusammenfassung

Aktuelle Reife (Stand Dez 2025): - **Core Database:** ✓ Production-Ready (85%+ Test Coverage) - **Multi-Model Support:** ✓ Production-Ready (Relational, Graph, Vector, Document, Time-Series) - **Security & Compliance:** ✓ Production-Ready (DSGVO, BSI C5, Audit Logging) - **Horizontal Scaling:** ✓ Production-Ready (2/4/8-Node Clusters, 91% Efficiency) - **LLM/RAG Integration:** 🟠 Partially Ready (Semantic Cache ✓, Hybrid Search 🟠) - **Enterprise Authorization:** 🟠 In Design (Apache Ranger Integration)

Vollständige Feature-Matrix siehe: - Admin Tools: [feature_matrix.md](#) - Features Overview: [features_overview.md](#) - Sharding Status: Chapter 16 (Horizontal Scaling) - Security Status: Chapter 10 (Section 10.2) - Query Optimization: Chapter 15 (Section 15.4)

Version History: - 1.3.0 (2025-12-30): Umfassendes Update mit allen Features bis Dez 2025

